



國立成功大學
National Cheng Kung University

Introduction to Emerging Memory Techniques

Lecture 1: Flash Memory

Prof. Chien-Chung Ho





Agenda

- **Flash Memory: Why and How**
- Flash Translation Layer
 - Address Mapping
 - Garbage Collection
 - Wear Leveling Strategies
- Error Correction Codes
- Flash File Systems
- Hybrid Flash (SLC & MLC) Architecture
- Other Issues and Conclusion
 - E.g., Data Retention, Data Security, and Temperature



國立成功大學
National Cheng Kung University

Outline

- Introduction
- NAND Flash Technology
- Inherent NAND Flash Challenges
- Summary



Flash Memory

- Flash memory is a widely used memory/storage technology in today's products
 - E.g., USB drives, solid-state drives and mobile devices
- Flash memory is a type of **non-volatile memory**
 - It holds data states even when the power is lost.

↕
DRAM
(volatile)





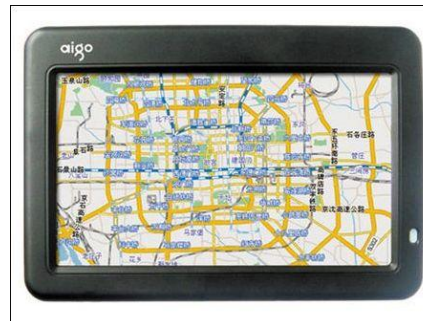
Why Flash Memory

- Diversified application domains

- Portable storage devices
- Consumer electronics
- Industrial applications
- Avionics / aerospace

- Nice features:

- Shock-resistance
- Non-volatility
- Low energy consumption
(Compared to HDD)
- Small feature size
- Low cost?



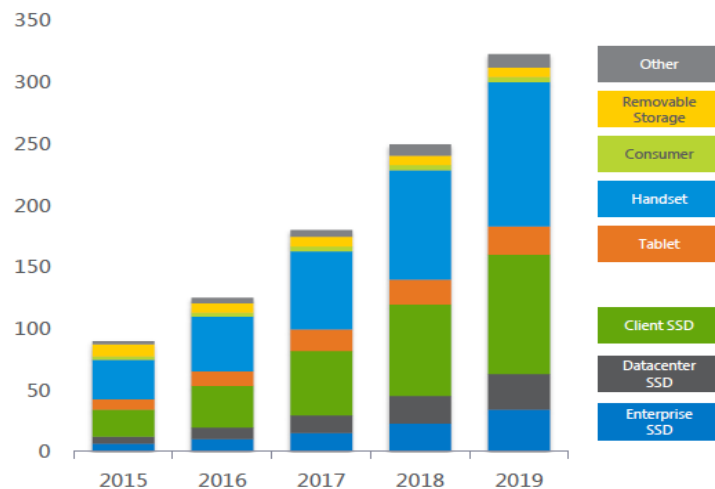


Trends in NAND Flash Memory

- Flash memory is widely adopted in various systems
- Price : **~40%** annual price reductions every year
- Revenue growth : **40%** every year



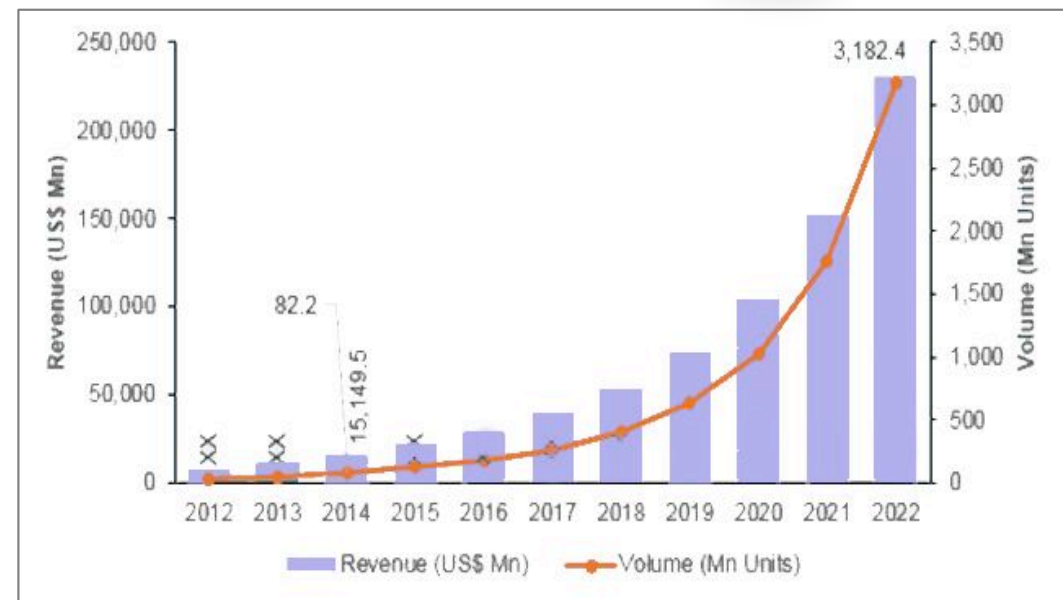
NAND Industry Bit Demand (B GB EU)



Tablets contain a mix of mobile DRAM, standard DRAM, and reduced-power solutions. Upgrade modules included with PC.



Sources : Micron, Intel And 3D NAND Post [\[link\]](#)



Sources : Global SSD Market to Post CAGR of 41% Until 2022, OriginStorage



Outline

- Introduction
- NAND Flash Technology
- Inherent NAND Flash Challenges
- Summary

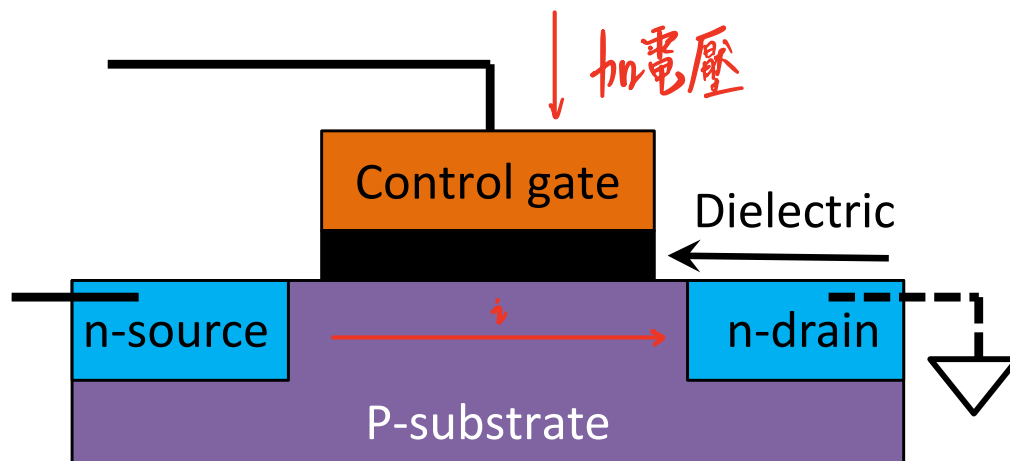


The Most Great Invention in 1990s

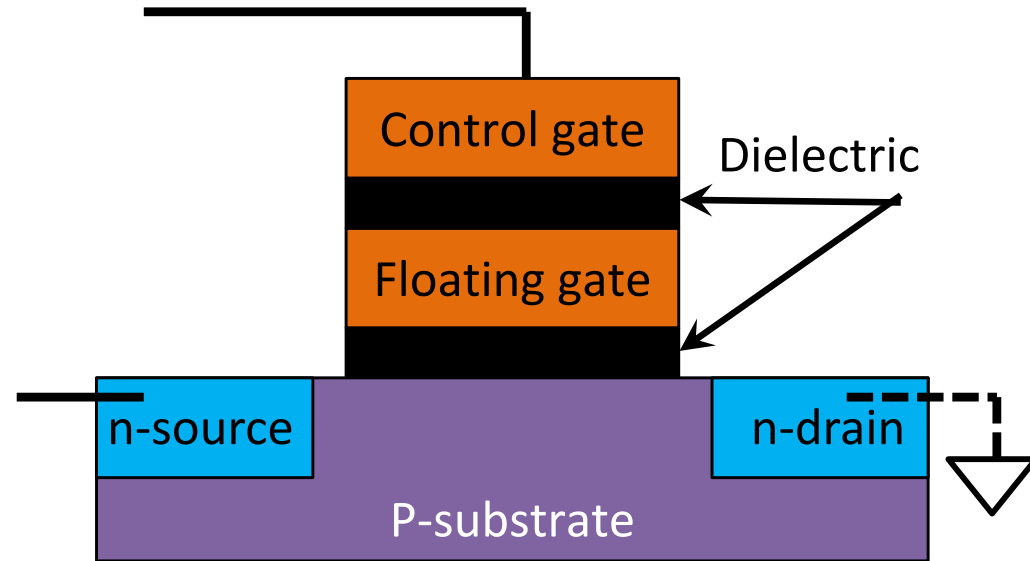
- Invented by **Dr. Fujio Masuoka** (舛岡 富士雄) born in 1943, while working for **Toshiba** around 1980.
- First presented in IEEE **International Electron Devices Meeting (IEDM)**, San Francisco, 1984.
- First commercialized by **Intel** as **NOR flash** in 1998 to replace ROM as BIOS and firmware.
- First **NAND flash** first introduced as SmartMedia storage in 1995. **(The focus of this tutorial)**



The Key Device: Floating-Gate Transistor



MOSFET

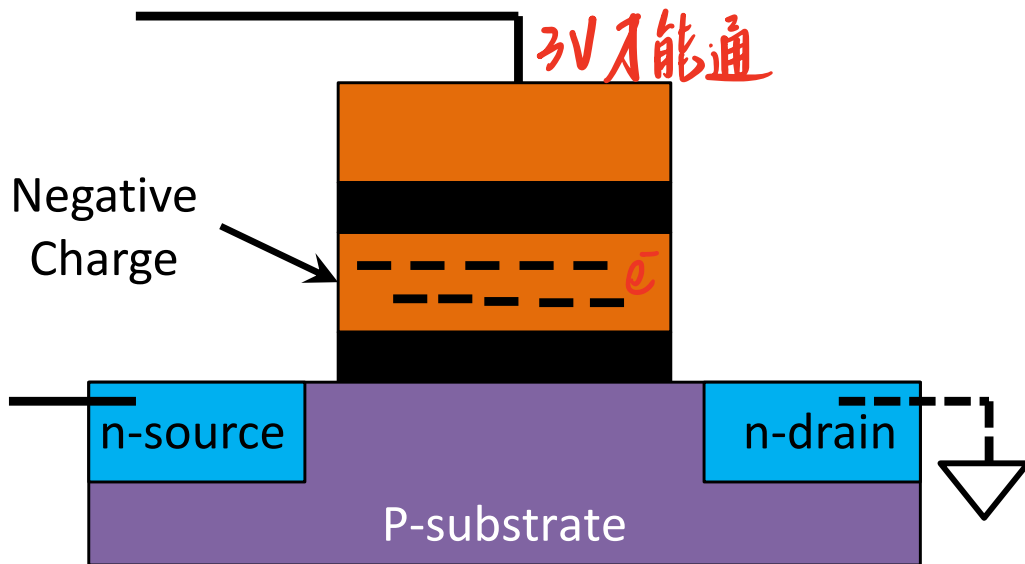


Floating-Gate Transistor

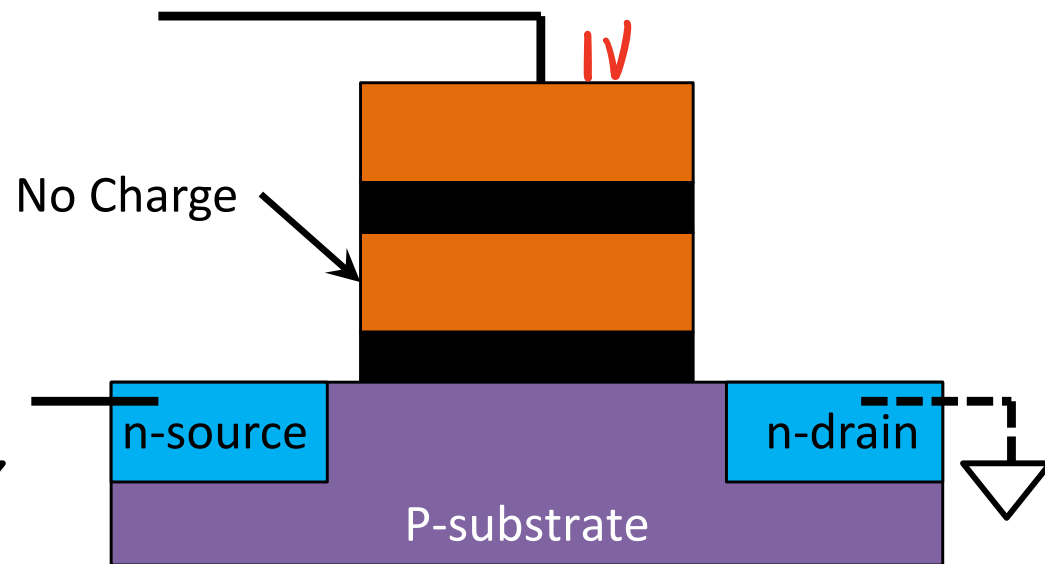
MOSFET: metal–oxide–semiconductor field-effect transistor

Single Level Cell (SLC)

Bit value = 0



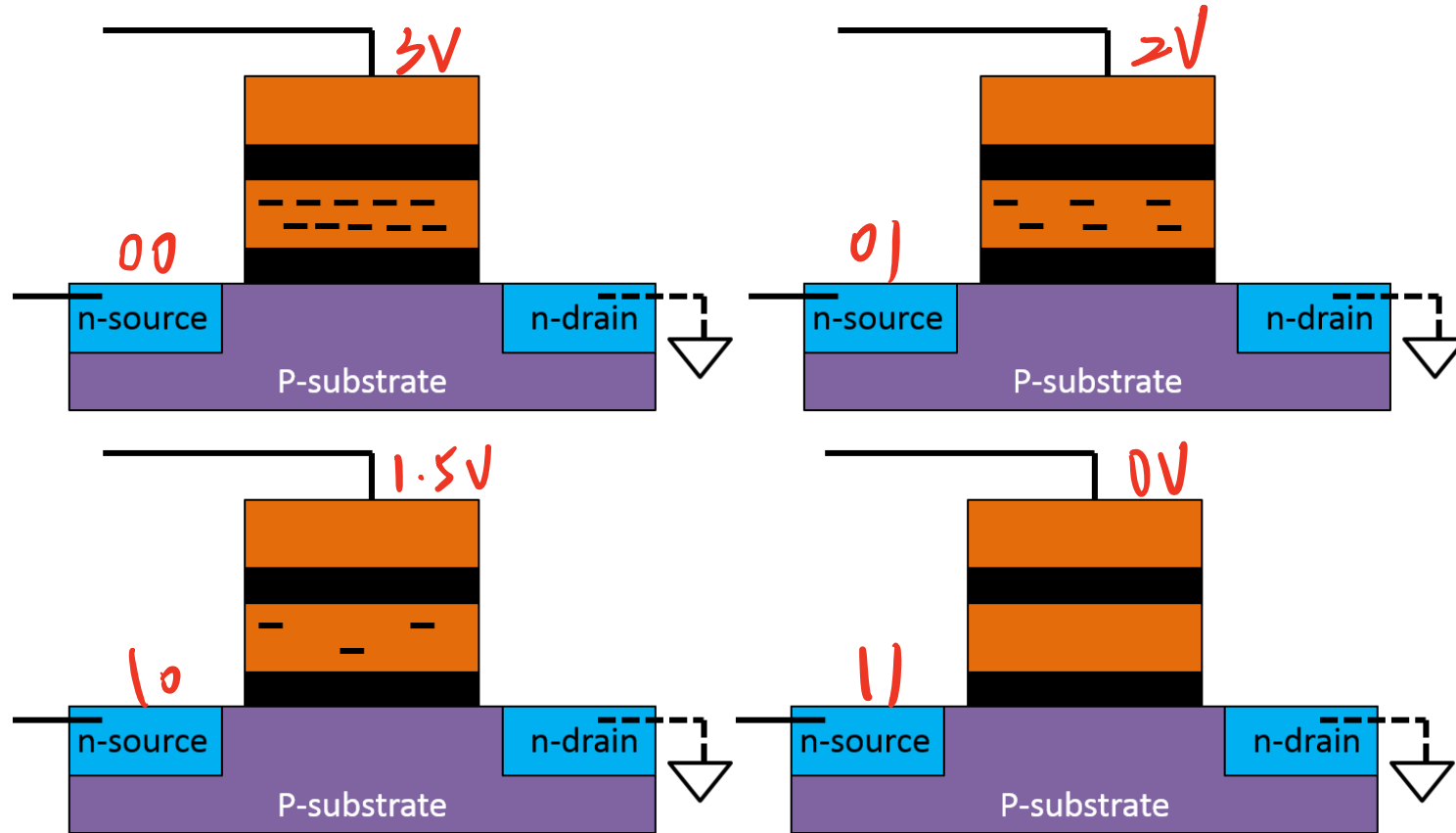
Bit value = 1



SLC (Single Level Cell)

- 2 charge states
- 1 bit per floating gate transistor

Multi Level Cell (MLC)



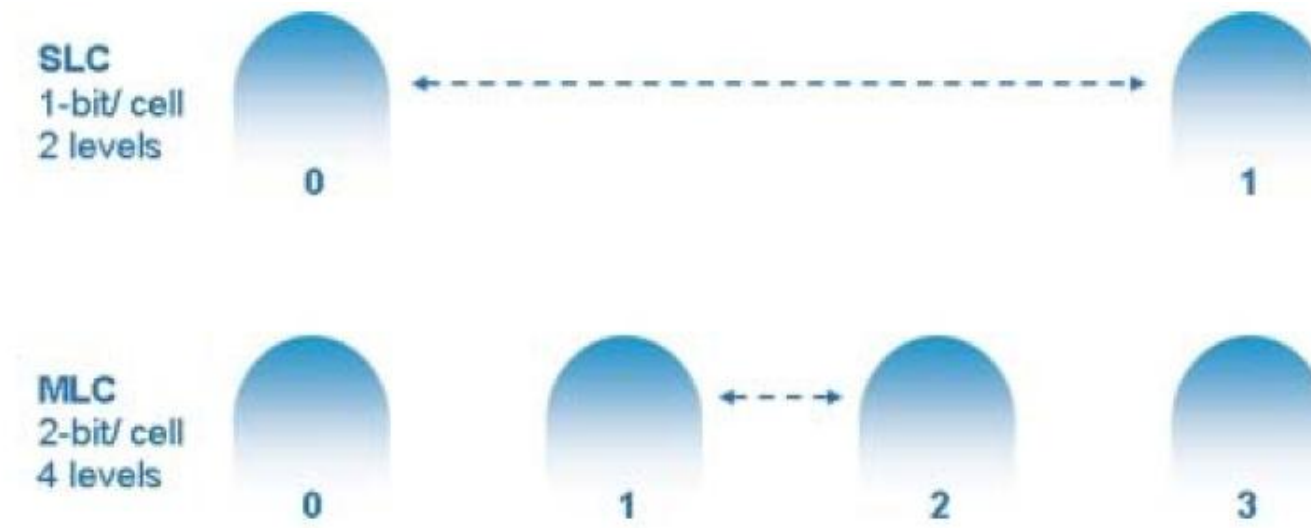
MLC (Multi Level Cell)

- > 2 charge states $\Rightarrow$ 1 cell 存 2 bit data
- > 1 bit per floating gate transistor



SLC vs. MLC

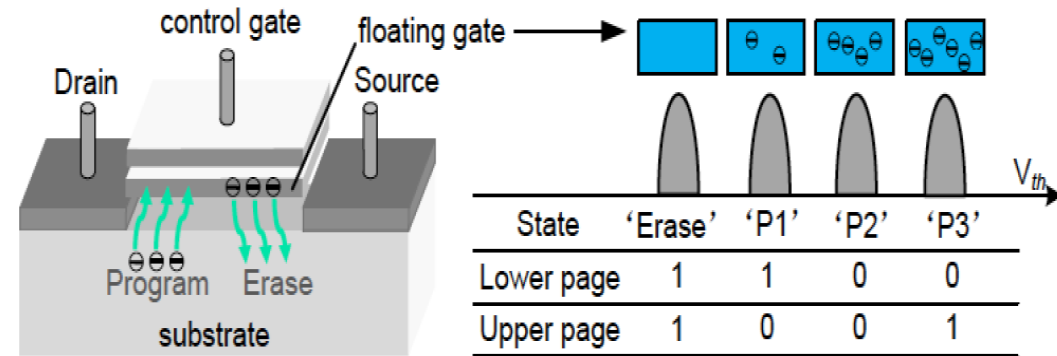
- Single-Level Cell (SLC) NAND $\Rightarrow$ *cost 較高*
 - SLC provides faster write speed, lower error rate and longer write endurance *可承受多少次 P/E cycle*
 - Usually used in the applications require high reliability, performance, and viability
- Multi-Level Cell (MLC) NAND
 - MLC allows each memory cell to store multiple bits of information
 - **Cost and reliability are relatively low** $\Rightarrow$ *reliability 低 $\Rightarrow$ 易壞*
 - Usually used in the cell phone, digital cameras, USB drives and memory cards



Flash Organization

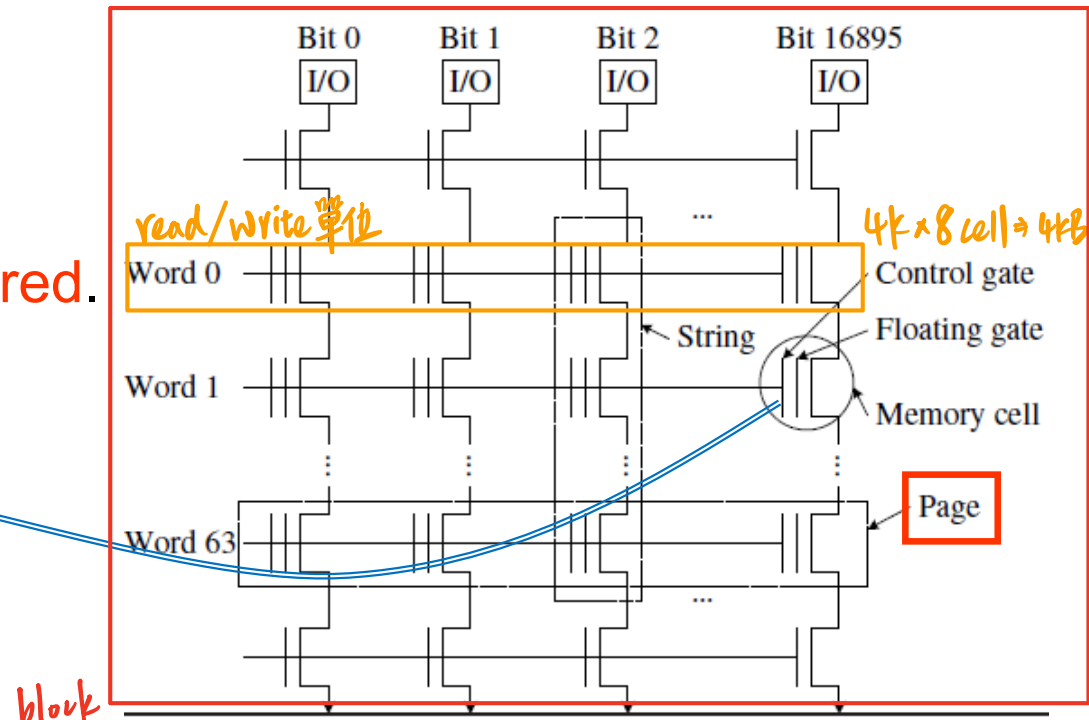
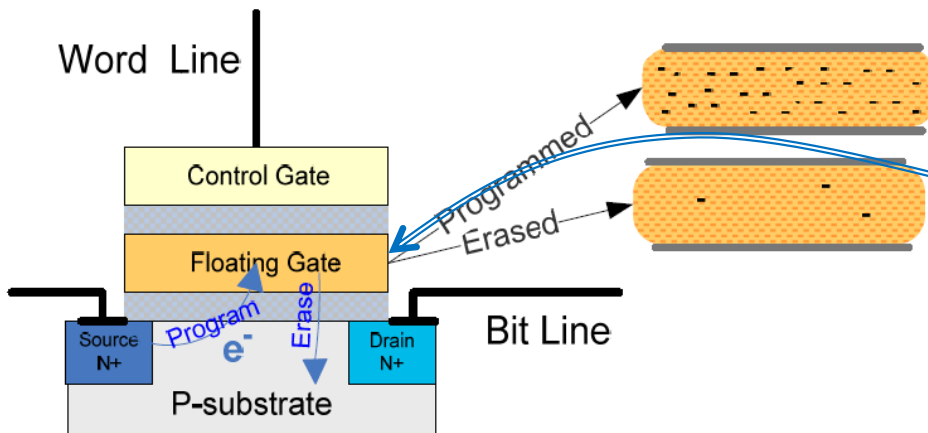
• Program $\Rightarrow$ 加 e^-

- Electron is moved **into** the floating gate (FG), and the threshold voltage is **raised**.



• Erase $\Rightarrow$ control gate 加電壓

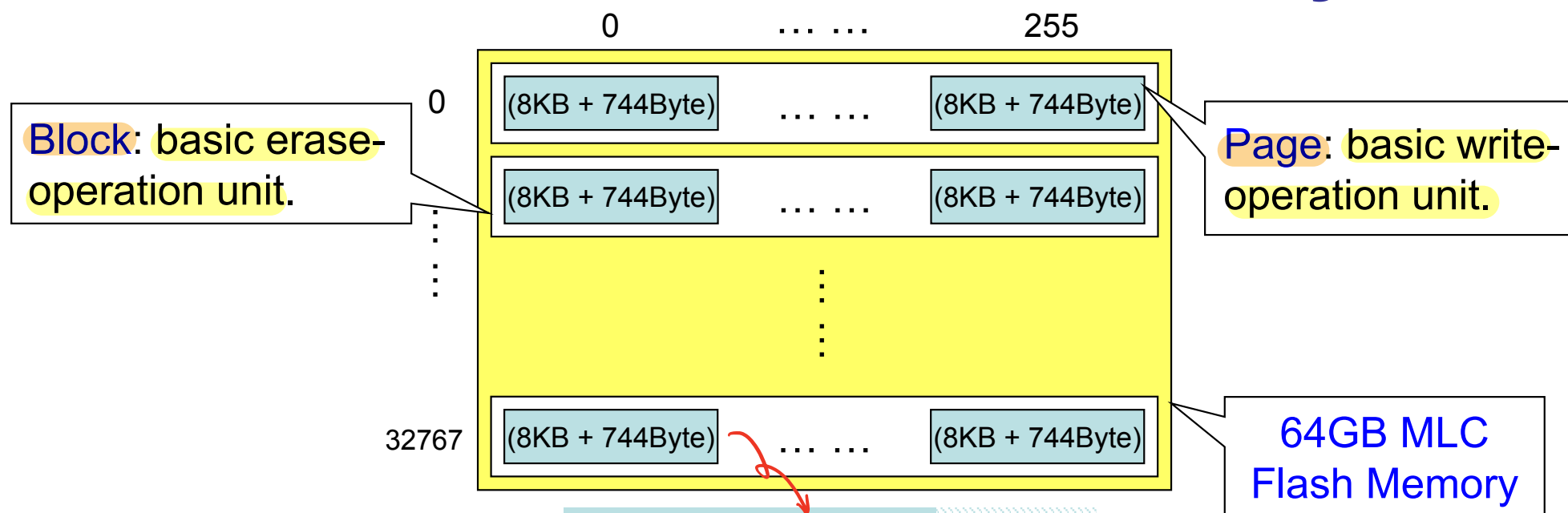
- Electron is **removed** from the floating gate (FG), and the threshold voltage is thus **lowered**.



Traditional 2D Flash – A **Block** Structure



Characteristics of Flash Memory



Features:

1. Write-once ⇒ 要 erase 才能再 write

2. Bulk-erasing

→ Out-place updating ⇒ 另找地方放新 data, 減少 erase

→ Garbage collection ⇒ 回收

3. Limited erase cycles

→ Wear leveling

⇒ 平均使用

Data

Spare

8KB

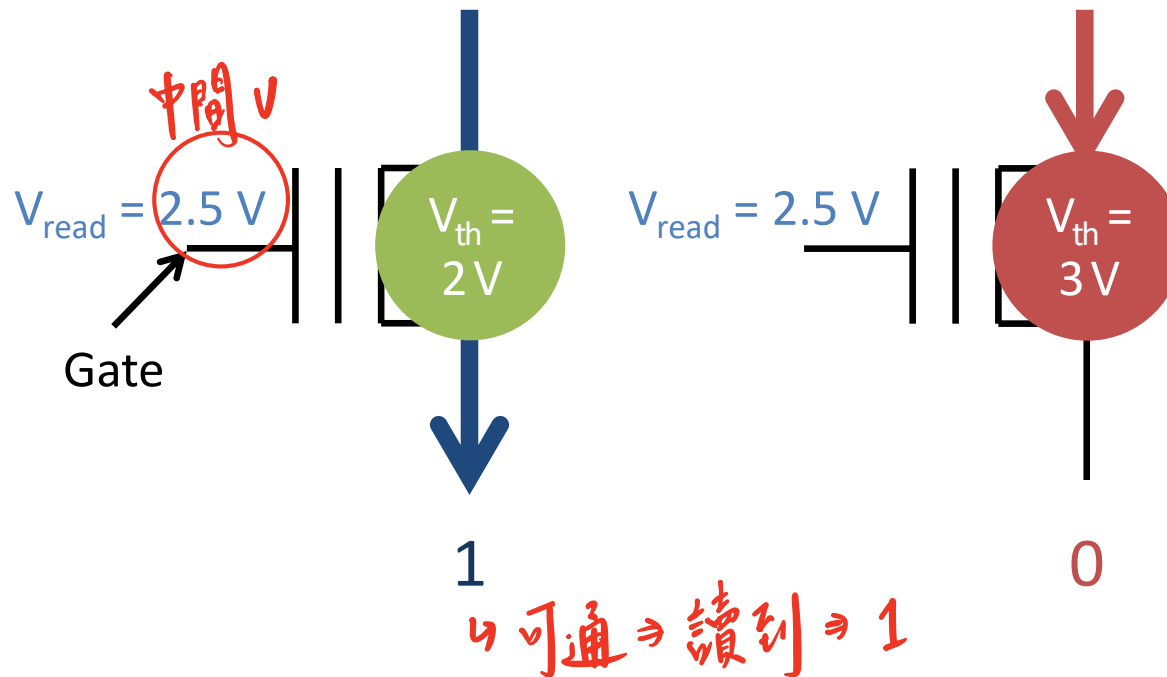
744Byte

(Source: Hynix Marketing)

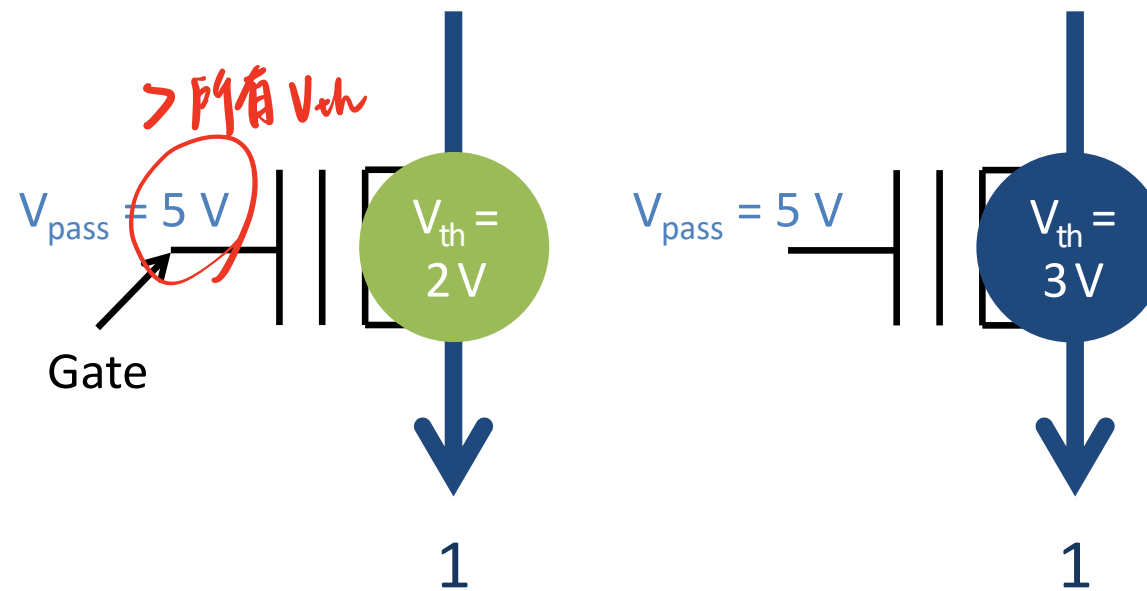
	SLC			MLC		
NAND Process	70/60nm	50nm	40/30nm	70/60nm	50nm	40/30nm
ECC required	1-bit	1-bit	4-bit	4-bit	4~8 bit	12~24 bit or more
Erase Cycle	100K	100K	TBD	10K	5K~10K	3K~5K
Data Retention	10 yrs	10 yrs	10 yrs	10 yrs	10 yrs	5 yrs

Flash Read

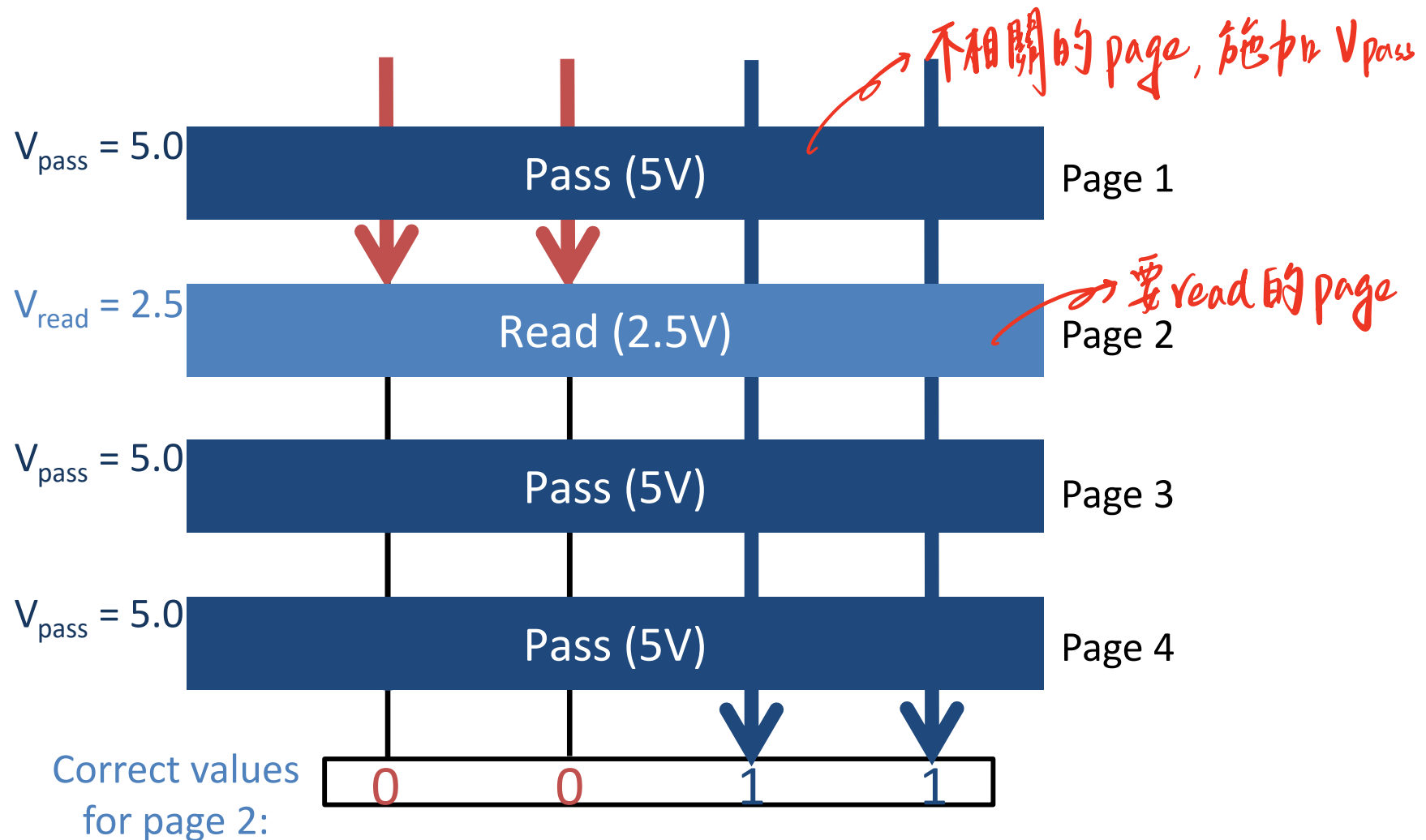
* V_{th} : threshold voltage



Flash Pass-Through

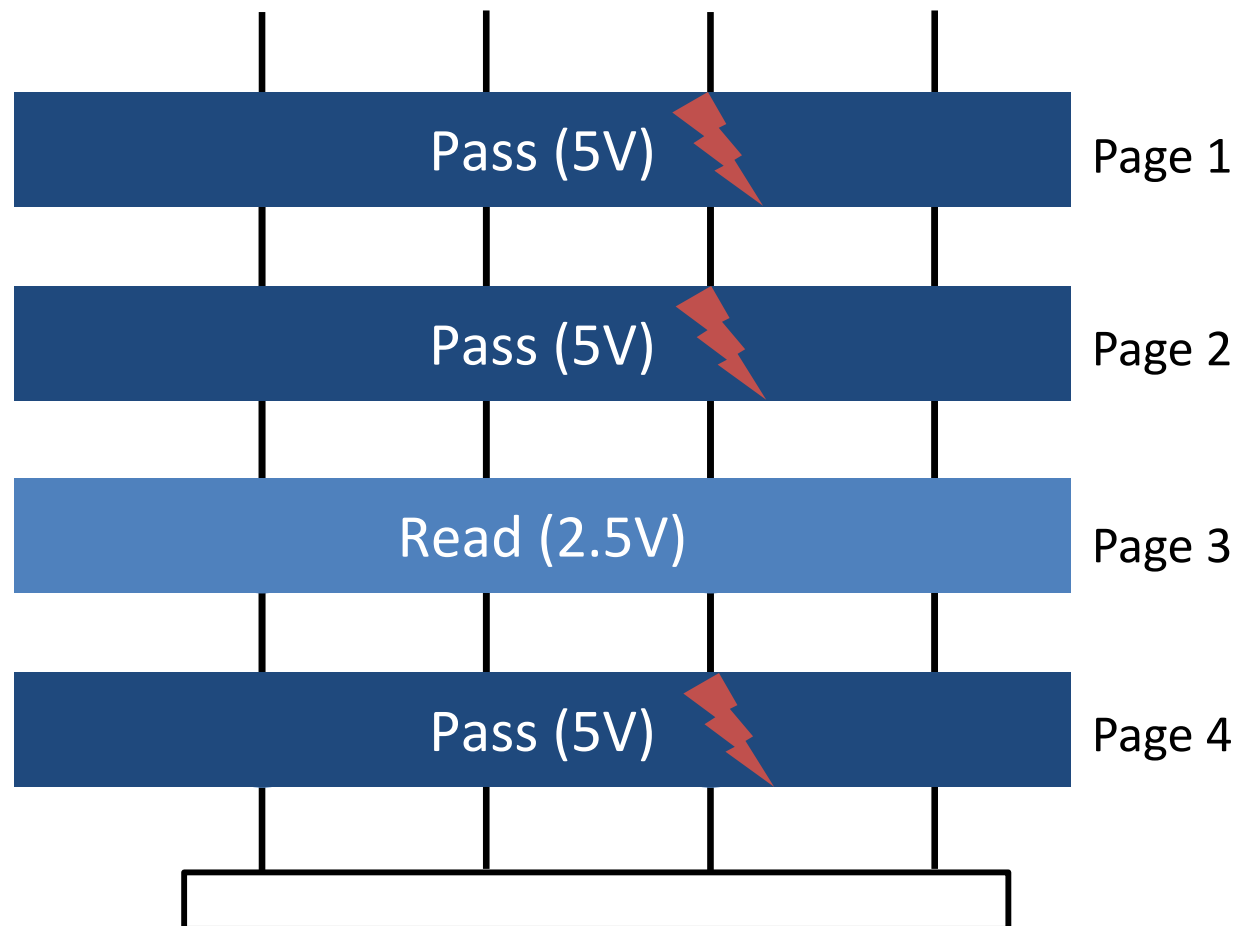


Read from Flash Cell Array

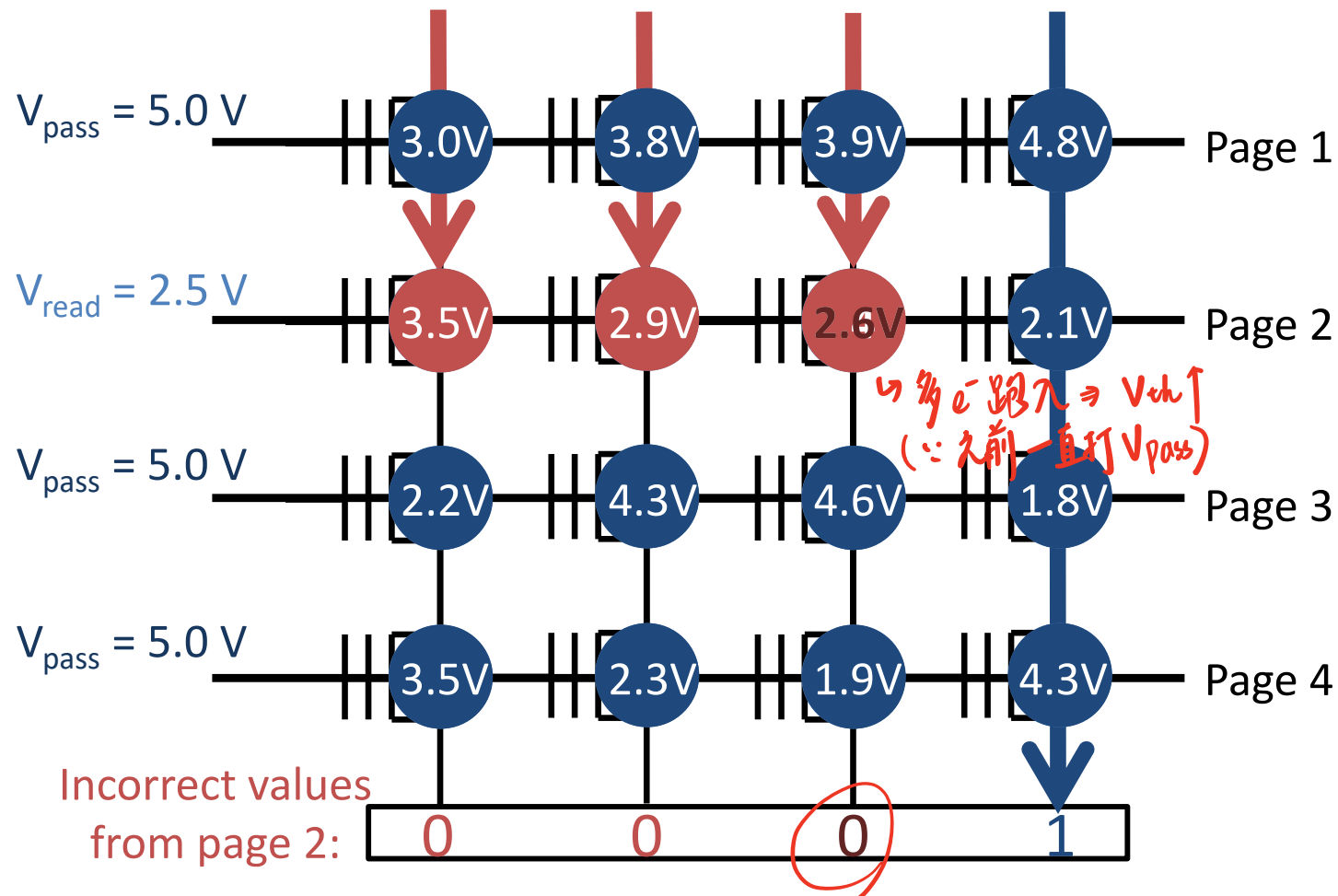




Read Disturb Problem : “Weak Programming” Effect



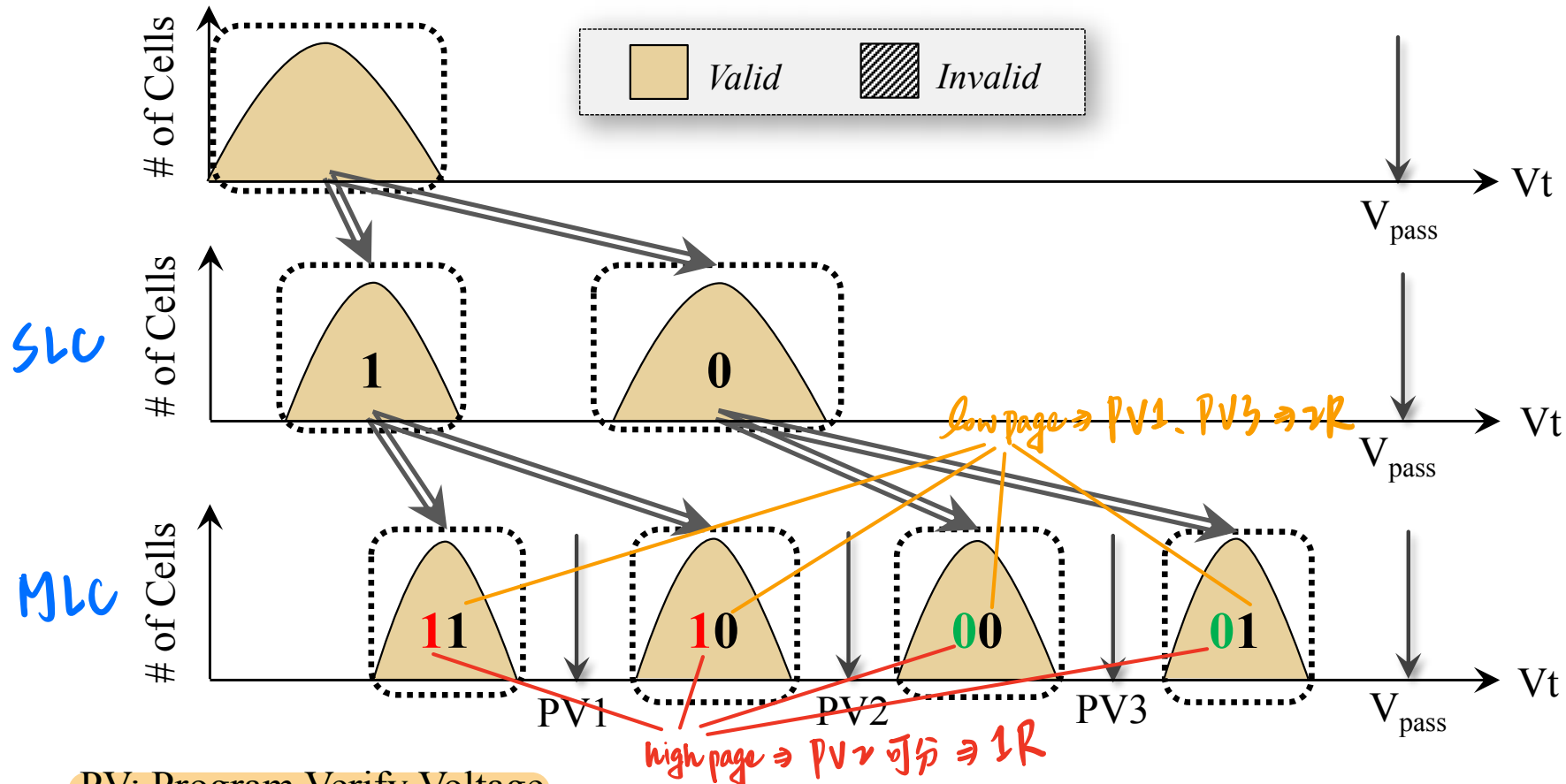
Read Disturb Problem: “Weak Programming” Effect



High pass-through voltage induces “weak-programming” effect



Program – MLC Page



PV: Program Verify Voltage

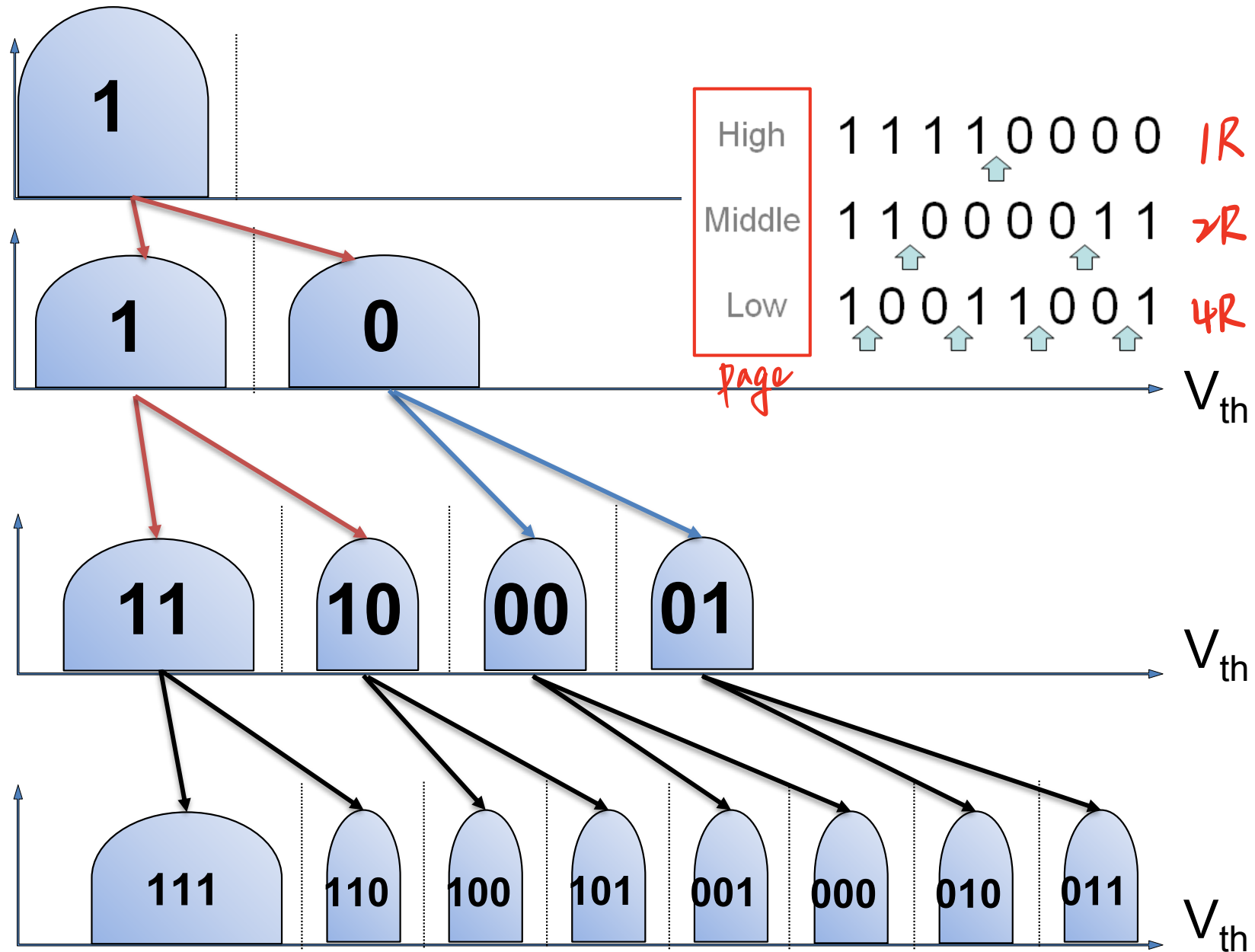
V_{pass} : A Pass-gate Voltage

V_t : Threshold Voltage

1. Low efficiency in programming low pages
2. High bit error rate in low pages



Program – TLC Page (1-2-4 Sensing)

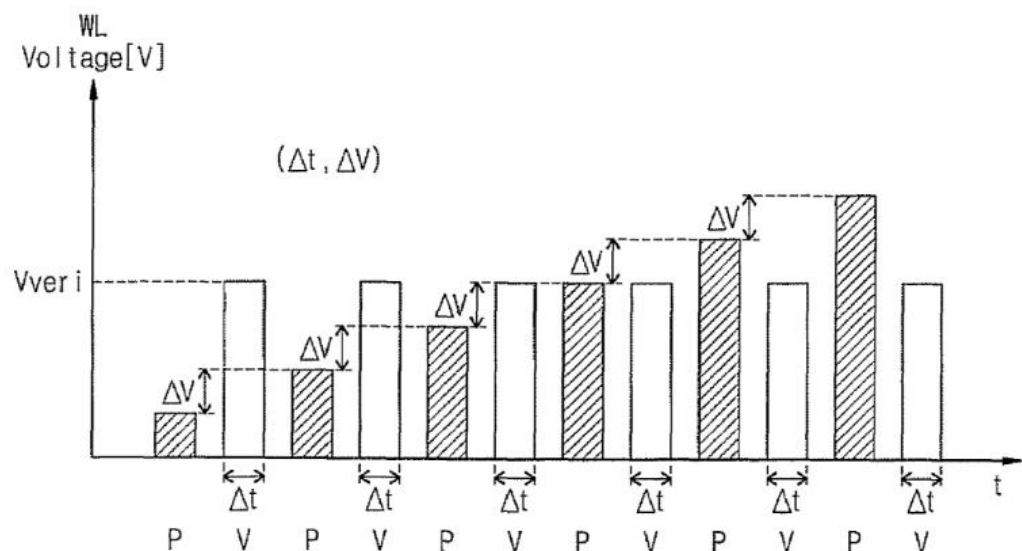




國立成功大學
National Cheng Kung University

Program: ISPP - Incremental Step Pulse Programming

↳ MLC, TLC 的 ISPP 時間長

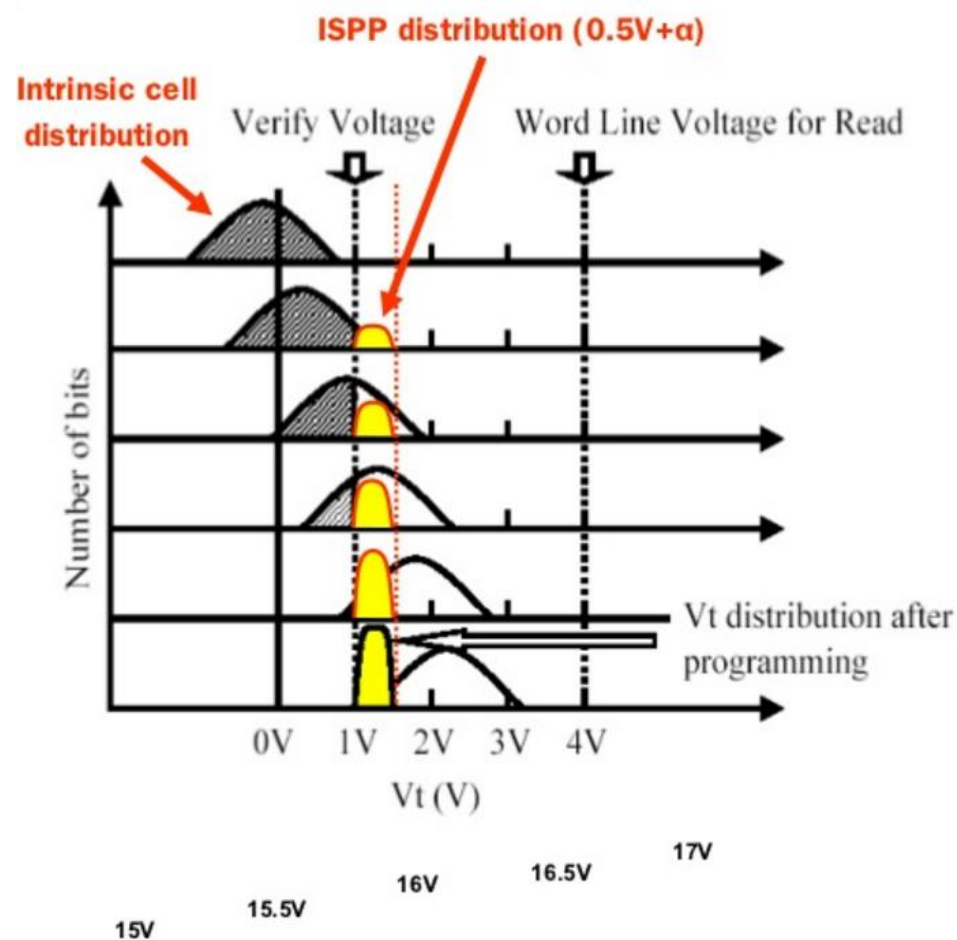


Slow cell
→ continuing program at higher bias

Fast cell
→ stop programming

B/L
0V → VCC

B/L
0V

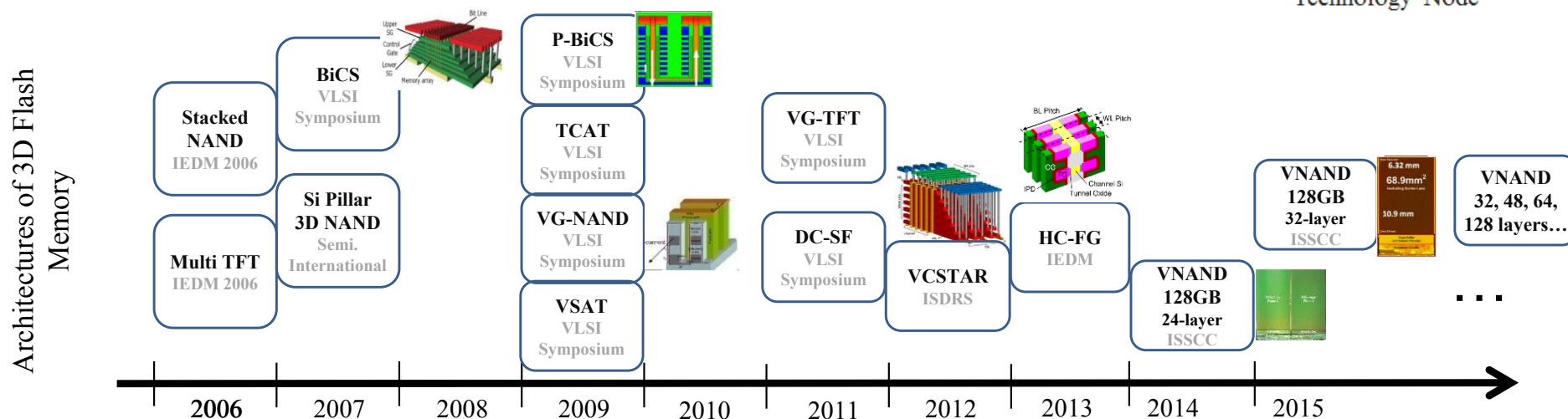
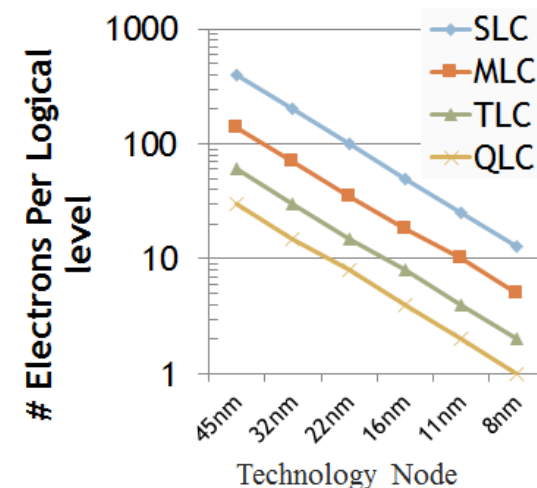




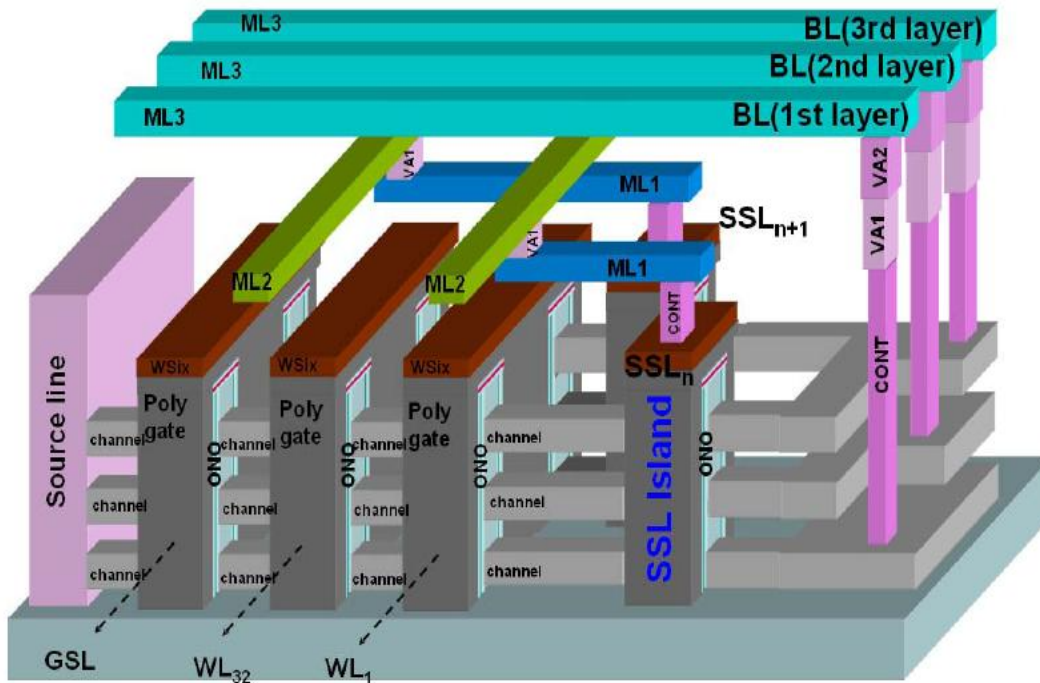
3D Flash Architecture Evolution

- 3D flash memory provides a good chance to further scale down the feature size and to reduce the bit cost.

- *Deliver very large storage space*
- *Worsen program disturbance*

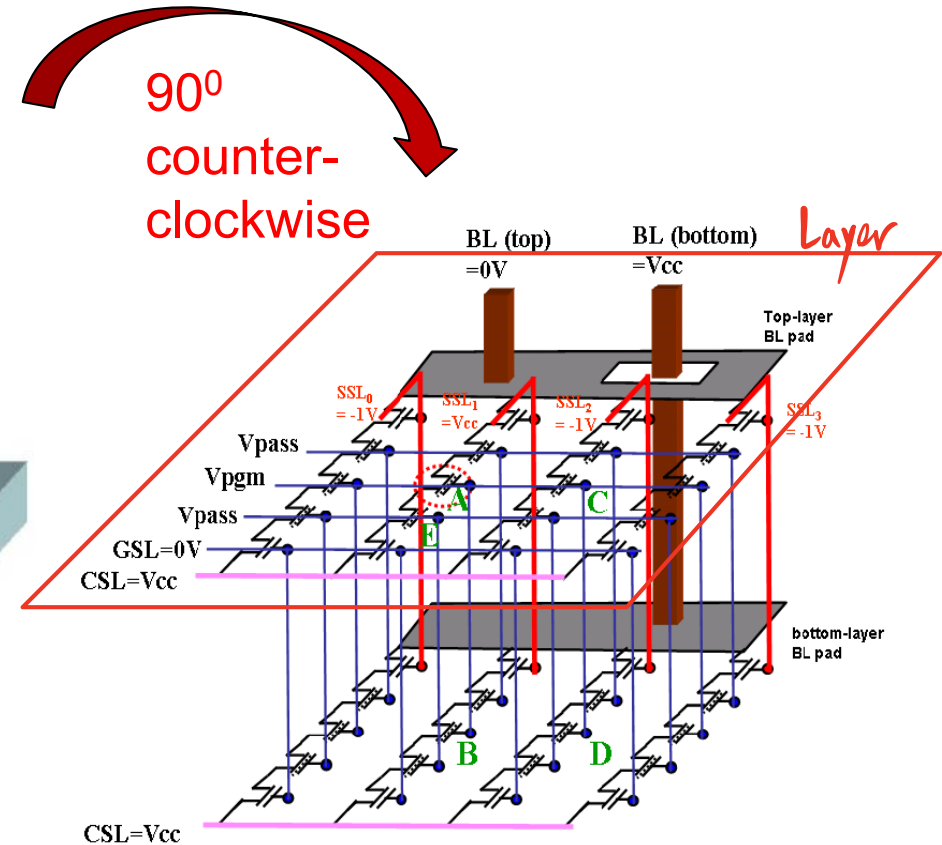


3D Flash Memory Architecture



Vertical Gate 3D Flash Memory

- WL= word line
- Channel = bit line
- SSL = string select line



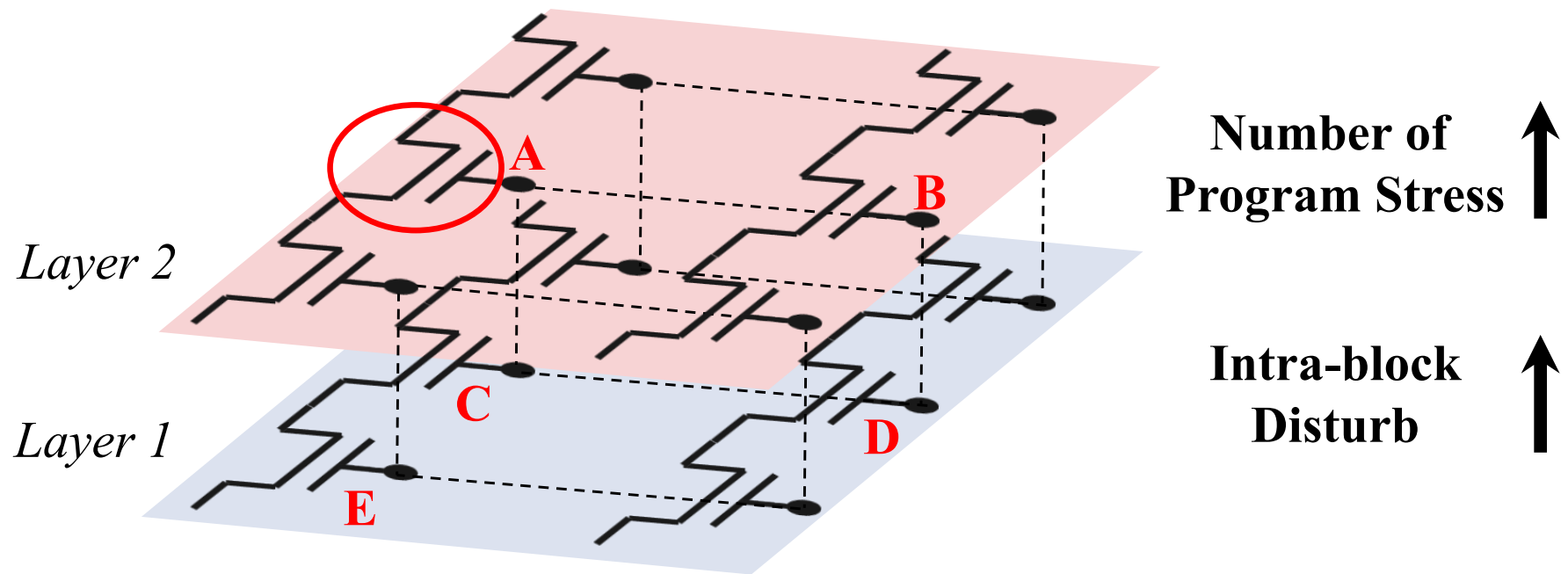
Equivalent Circuit Diagram

K.-P. Chang, H.-T. Lue, C.-P. Chen, C.-F. Chen, Y.-R. Chen, Y.-H. Hsiao, C.-C. Hsieh, Y.-H. Shih, T. Yang, K.-C. Chen, C.-H. Hung, and C.-Y. Lu. Memory architecture of 3d vertical gate (3dvg) nand flash using plural island-gate ssl decoding method and study of it's program inhibit characteristics. IMW '12.



Deteriorated Disturb on 3D Flash

- 3D flash memory introduces a new disturb, ***Z-direction Disturb***
→ Programming to Cell 'A' not only disturb Cell 'B', but also Cell 'C', 'D' and 'E'





Outline

- Introduction
- NAND Flash Technology
- Inherent NAND Flash Challenges
- Summary



Inherent Nature

- Challenges of using NAND flash include
 - Need to erase before writing (**write-once property**)
 - Wear out mechanism that limits service life (**Wear leveling**)
 - Data errors caused by **write and read disturb**
 - **Data retention** errors ⇒ 久放, data存放能力↓
 - Management of initial and runtime **bad blocks**
- We need sophisticated flash management techniques to make flash become powerful storage

問題



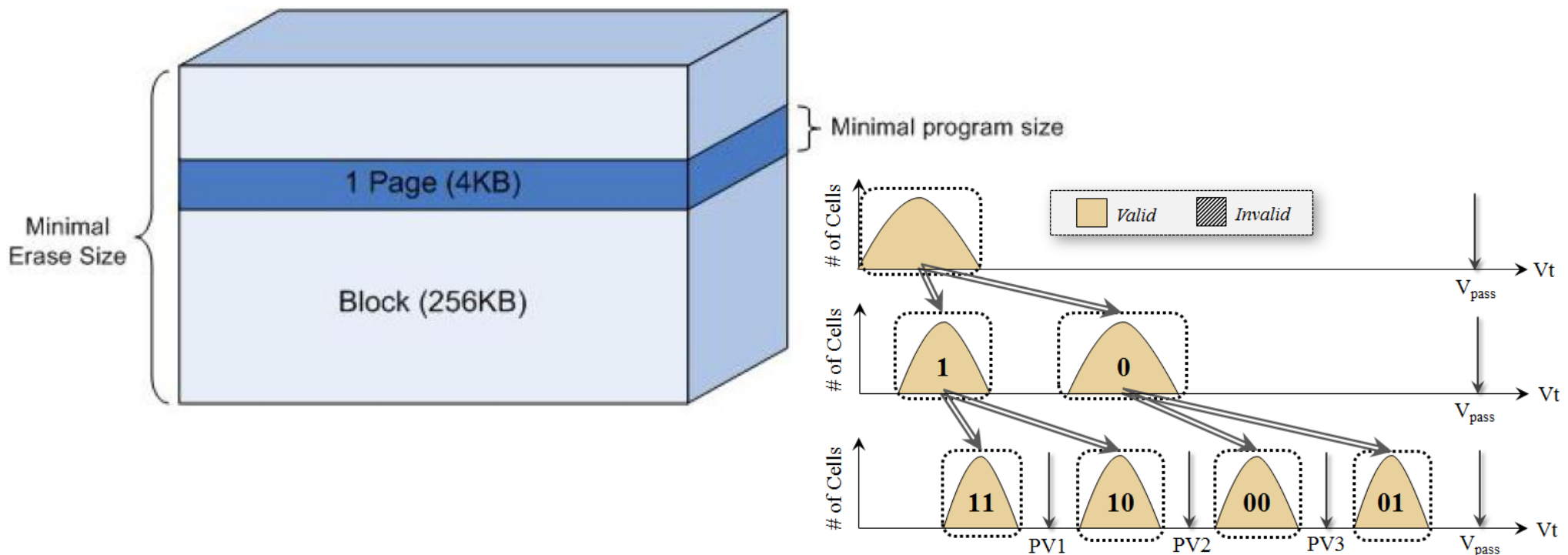
Research Issues

- Proper flash management makes flash memory a highly reliable data storage device
- Five significant **factors** affect reliability, performance and write endurance of SSD
 - SLC vs. MLC vs. TLC vs. 3D flash
 - Wear-leveling algorithms
 - Bad Block management techniques
 - Error detection and correction techniques
 - Write amplification



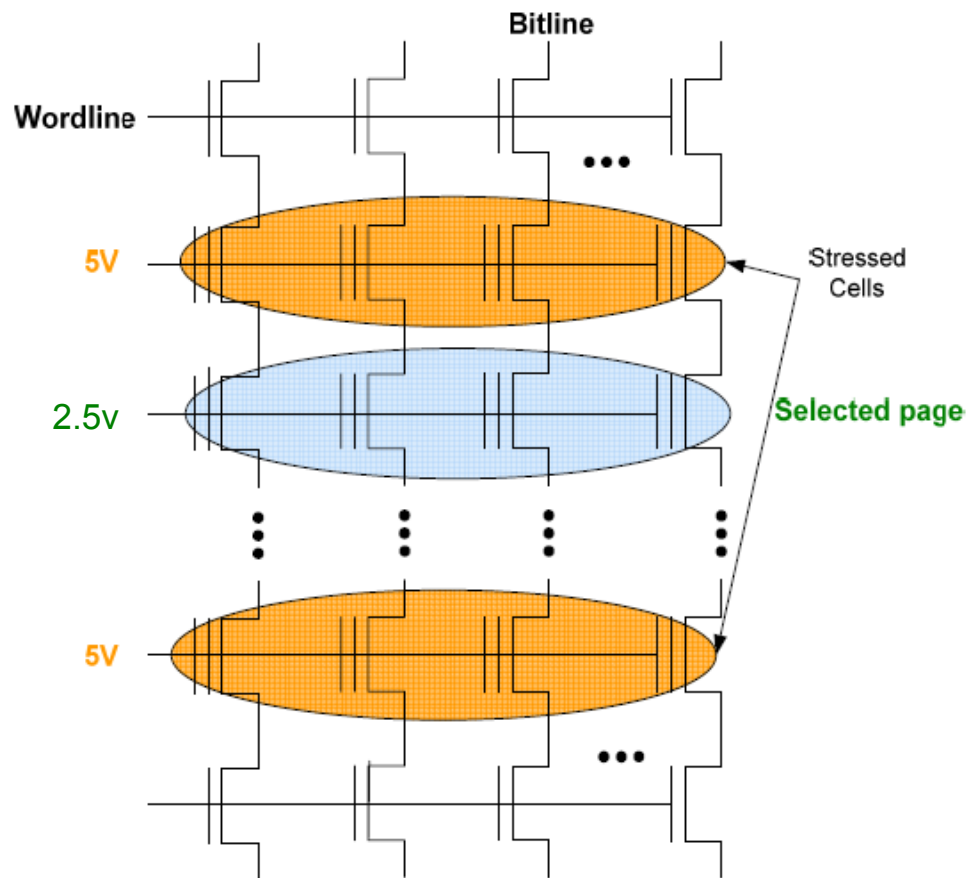
Erase Before Write

- The **Erase** operation sets all the bits in the block to a “1”.
- Once written to a “0”, the only way to reset a bit to a “1” is by erasing the entire block.

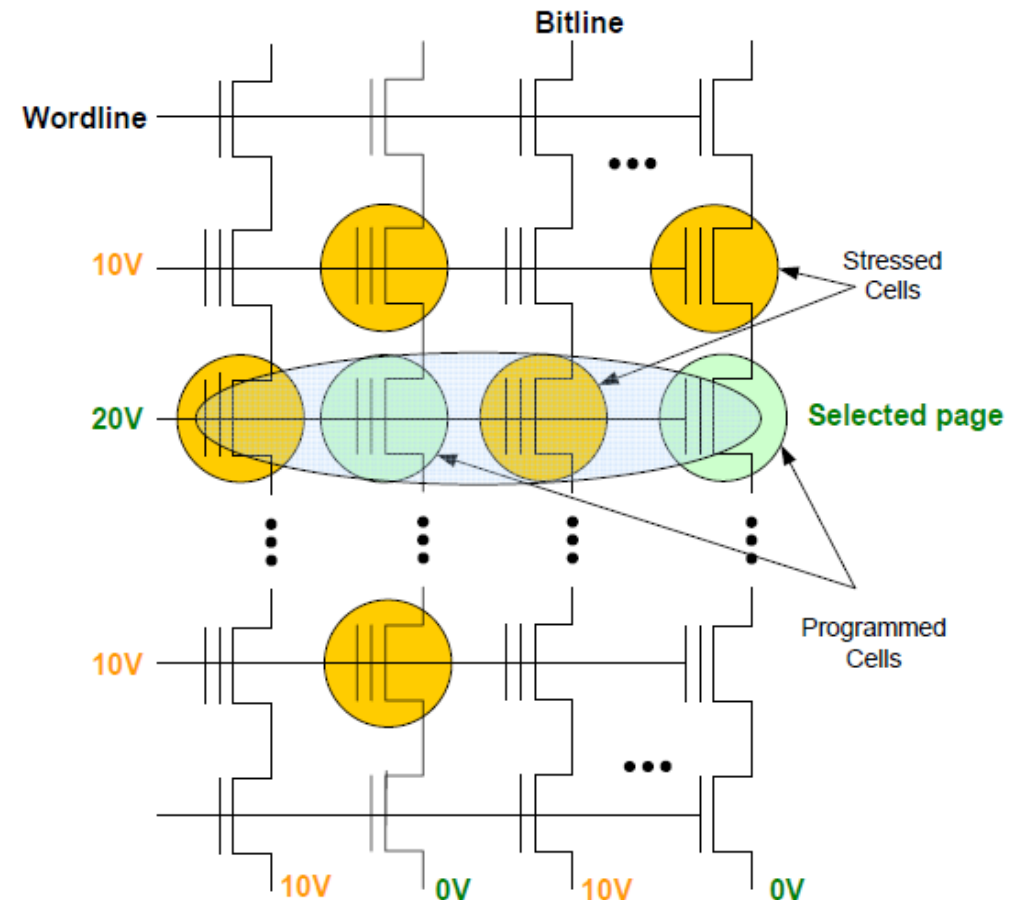


Read/Write Disturb

- Read disturb P_{RD}



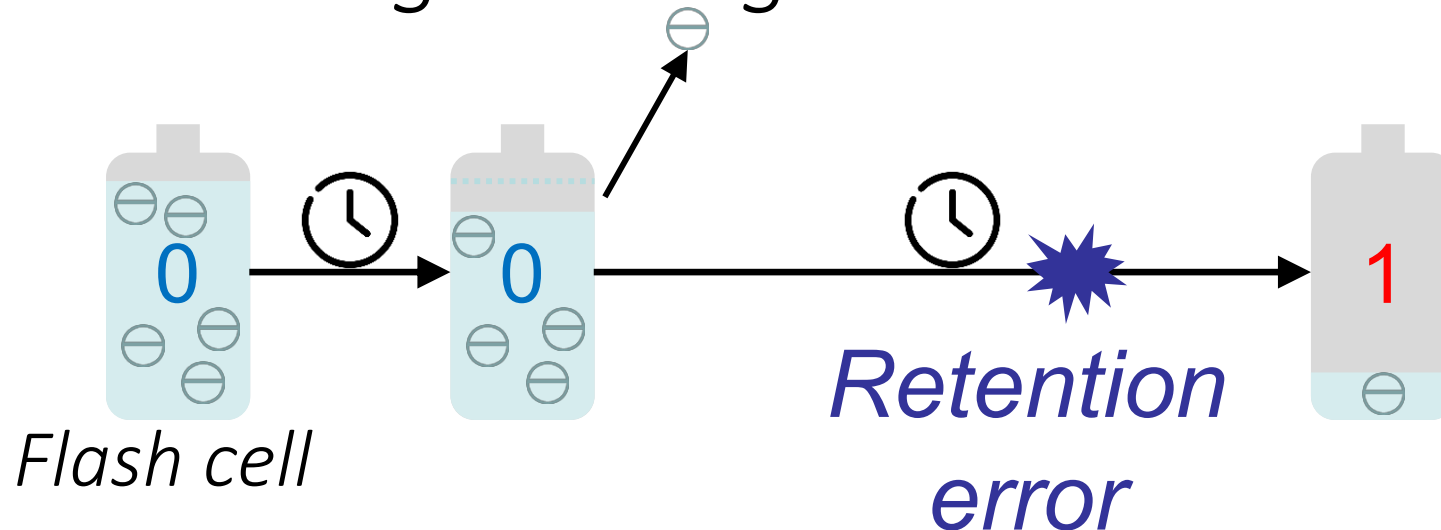
- Write disturb



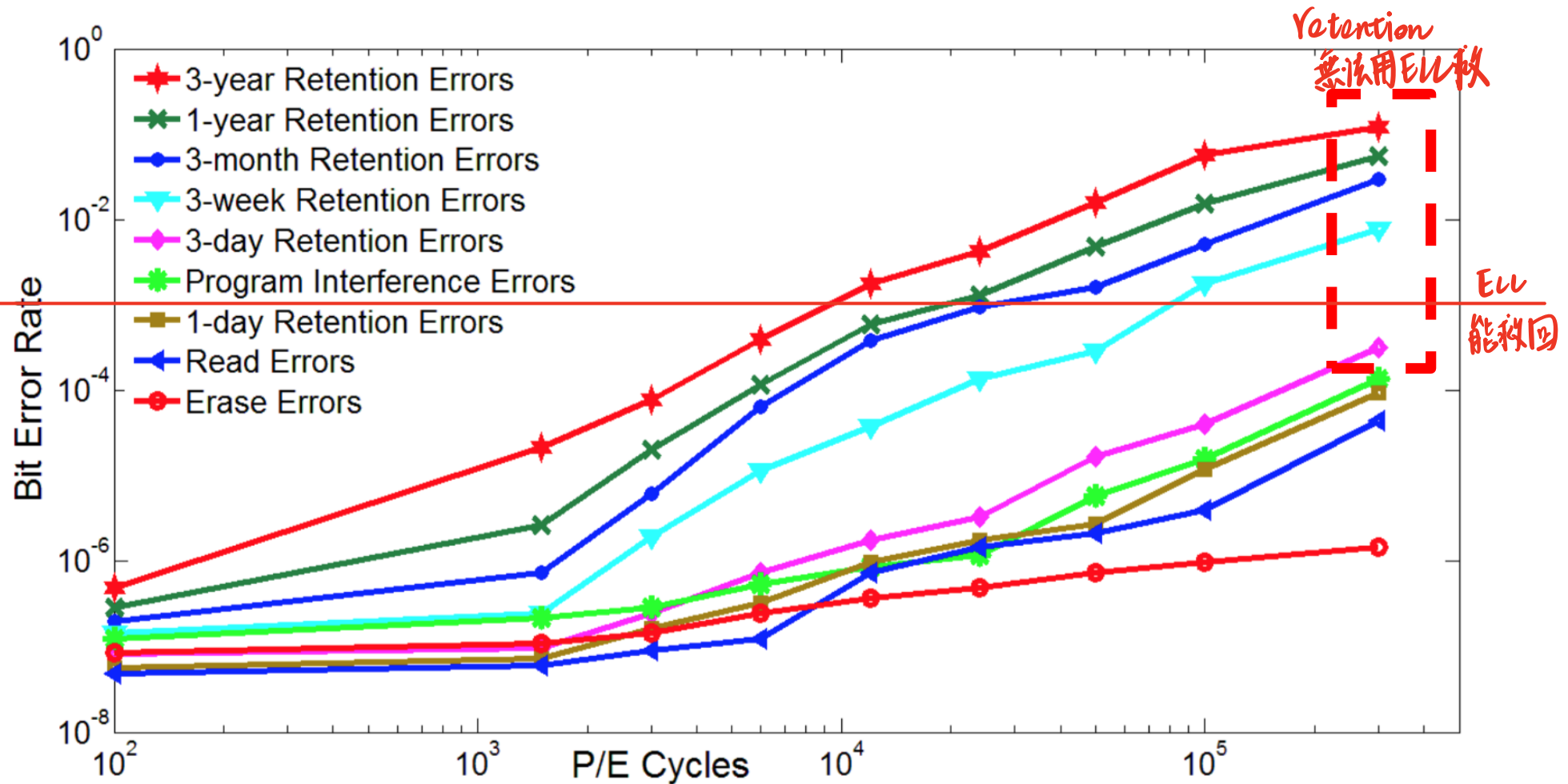
Data Retention Errors

- Data defines how long the written data remains valid within a memory device
- The data retention time is inversely related to the number of **Program/Erase cycles (P/E cycles)**

Charge leakage over time



P/E Cycles vs. Bit Error Rates



BER of various type of errors as P/E cycles increase

[1] Error Patterns in MLC NAND Flash Memory: Measurement, Characterization, and

Analysis. 978-3-9810801-8-6/DATE12/©2012 EDAA



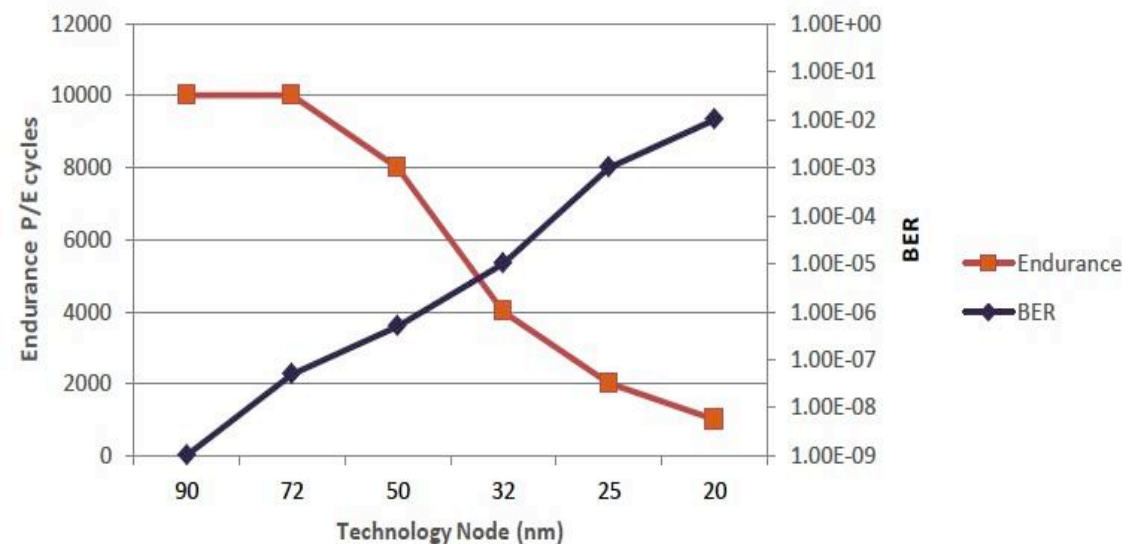
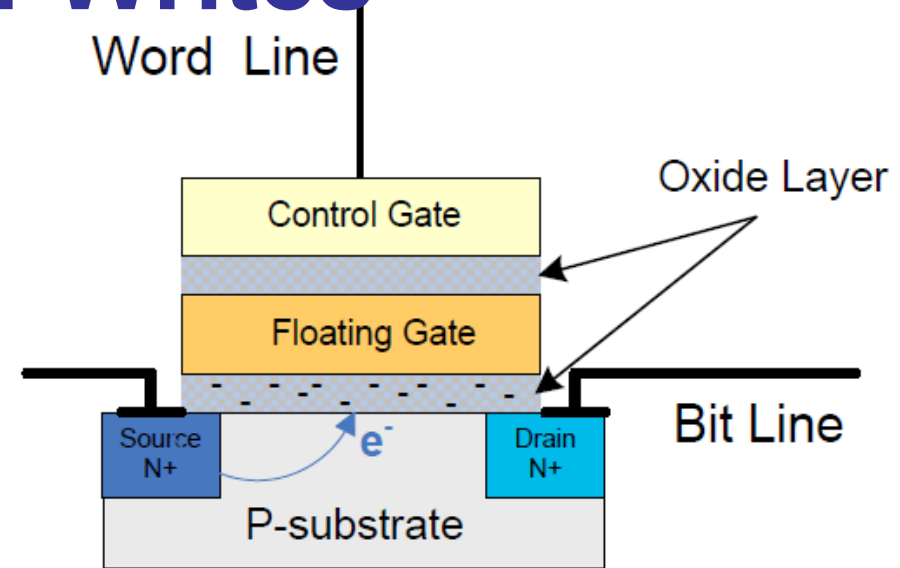
Bad Blocks

- Initial bad blocks ⇒ 製造良率問題
 - Due to the **production yield constraints** and the pressure to keep costs low
 - Up to **2%** of the SLC flash can contain bad blocks; the number for MLC flash is **about 5%**
- Accumulated bad blocks
 - Due to multiple write/erase cycles, **trapped electrons** in the **dielectric** cause a permanent shift in the voltage levels of the cell



Limited Number of Writes

- Writing damages the oxide layer
- NAND flash has finite number of Write/Erase cycles
 - SLC: ~100K P/E cycles
 - MLC: ~1K ~ 10K P/E cycles
- Wear leveling is necessary



Source: EDN Network



Outline

- Introduction
- NAND Flash Technology
- Inherent NAND Flash Challenges
- Summary



Summary

- Flash memory development trend and basics are introduced
- Flash memory issues are discussed
 - Management
 - Wear leveling
 - Disturbance and retention
 - Bad block

Agenda

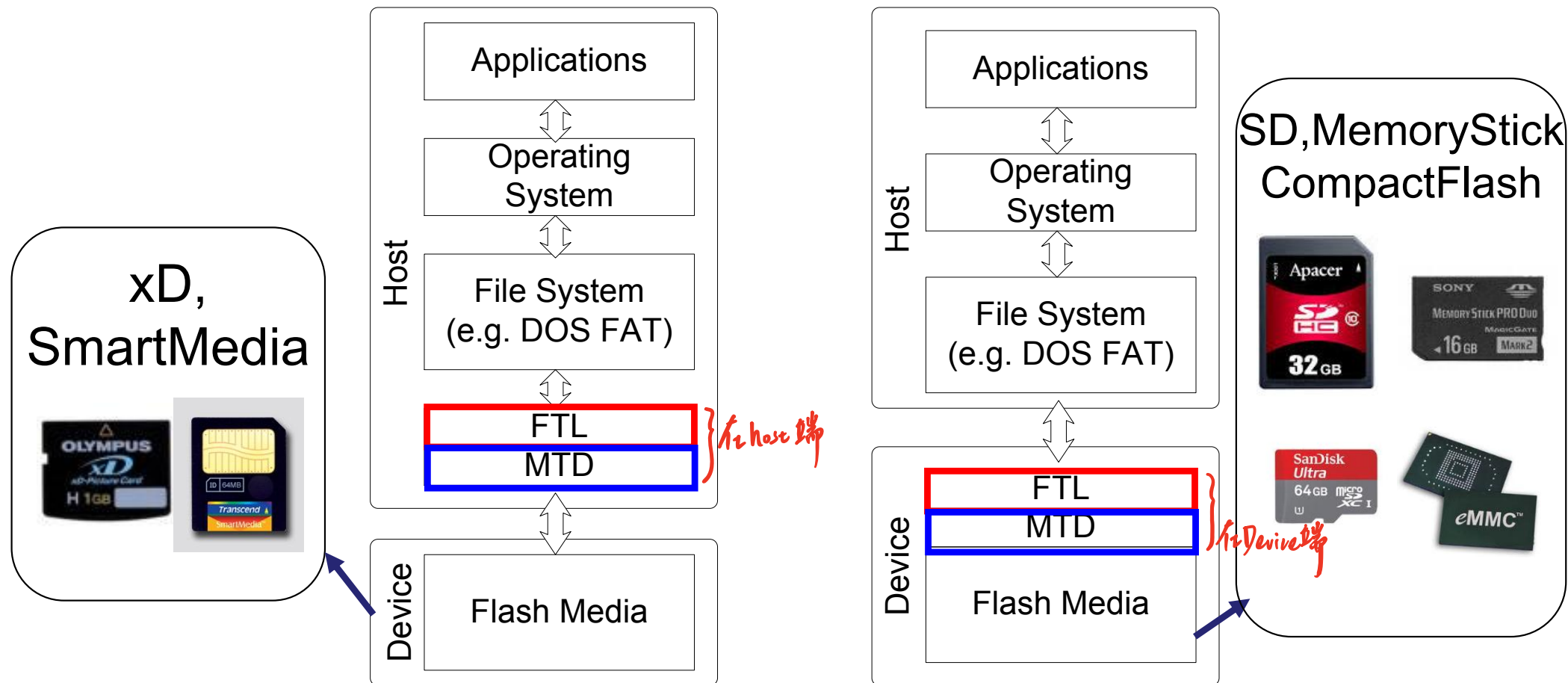
- Flash Memory: Why and How
- **Flash Translation Layer**
 - **Address Mapping**
 - Garbage Collection
 - Wear Leveling Strategies
- Error Correction Codes
- Flash File Systems
- Hybrid Flash (SLC & MLC) Architecture
- Other Issues and Conclusion



Outline

- Introduction
- Flash Translation Layer Designs
 - FTL
 - BL
 - NFTL
 - FAST
 - DFTL ⇒ 常用
- Summary

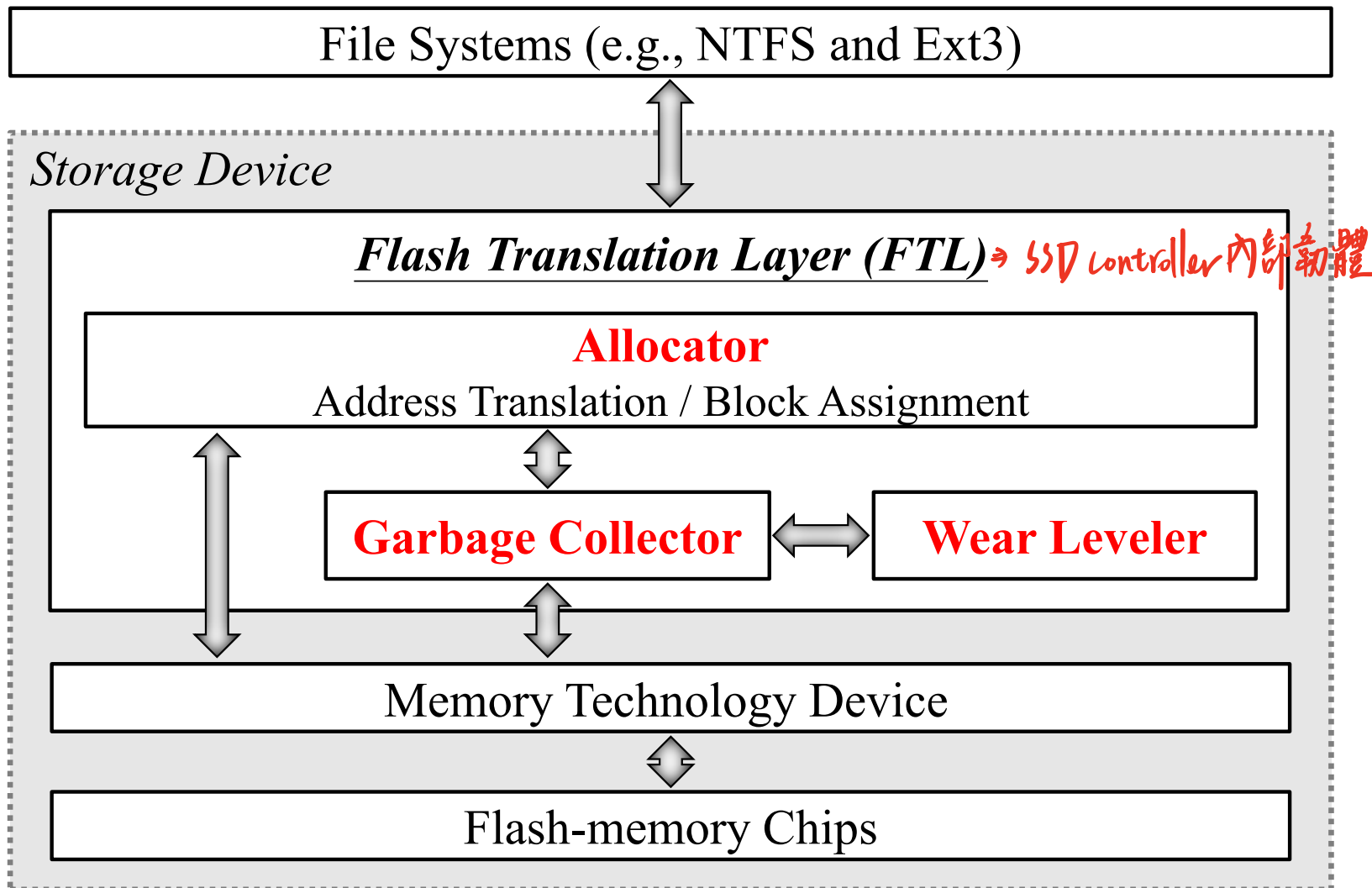
System Architecture



*FTL: Flash Translation Layer, MTD: Memory Technology Device



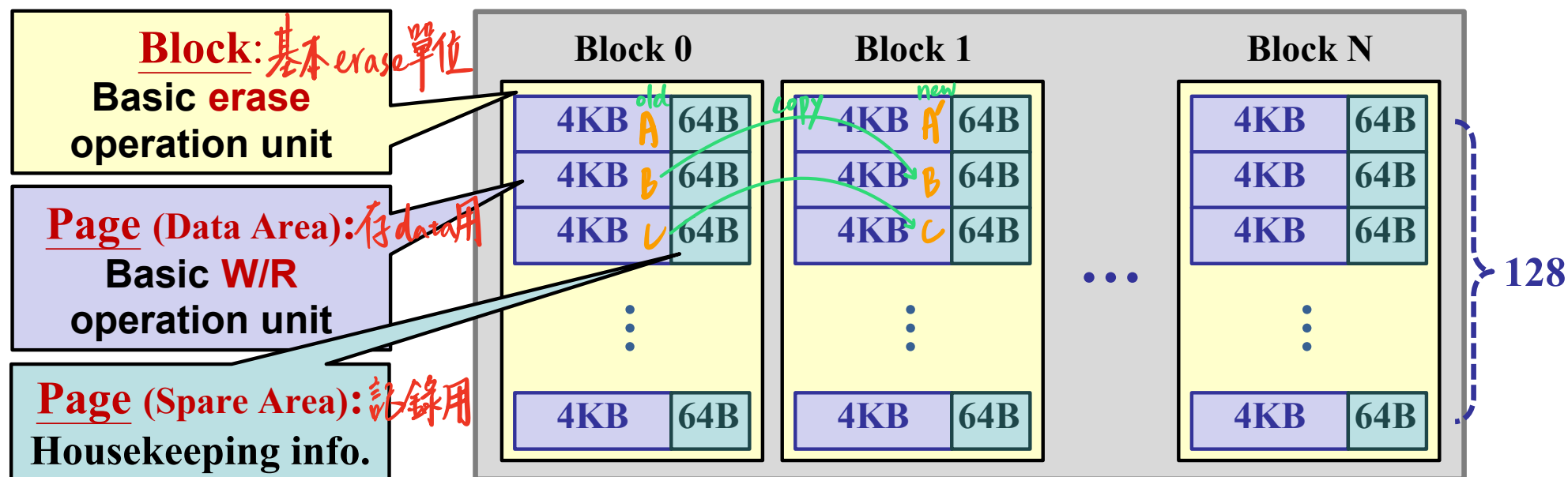
System Architecture

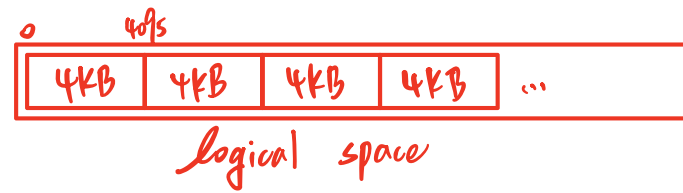


Flash Management Challenges

- Write-once property (page)
 - Out-place updating → 更新後的資料不在原值
- Limited erase cycles (block)
 - Wear leveling
- Bulk erasing (block)
 - Garbage collection
 - Live-page-copying → 需把其他有效data搬走, 再erase掉

Physical Layout of Flash Memory





ex: 100 byte

$$\Rightarrow 100 \div 4096 = 0 \dots 100$$

logical page offset

FTL (Page-level)

Address Translation Table (in main-memory)

↳ 記錄 logical 對到 physical 哪裡

Access LBA 3

9	(1,3)
8	(1,2)
7	(2,1)
6	(1,0)
5	(4,5)
4	(0,5)
3	(0,6)
2	(0,4)
1	(0,1)
0	(0,3)

* table 大小:

$$\frac{1 \text{ TB SSD}}{4 \text{ KB page}} = 256 \text{ M 個 page}$$

$$4 \text{ B} \times 256 \text{ M} = 1024 \text{ MB} = 1 \text{ GB table}$$

⇒ 佔太多 SSD 的 DRAM

Flash memory

PBA (block, page) Data Area Spare Area

1,3		
1,2		
1,1		
1,0		
0,6	Data	LBA = 3
0,5		
0,4		
0,3		
0,2		
0,1		
0,0		

LBA(array index)

logical block address

PBA(block,page)



Limited RAM Space of SSDs

- Spec of Samsung 990 PRO

規格

介面：PCIe 4.0 x4

NVMe：2.0

尺寸：M.2 2280

容量：1TB、2TB

控制器：三星Pascal

快閃記憶體：176層3D V-NAND TLC

DRAM 快取：LPDDR4-1866 1GB(1TB版)/2GB(2TB版)

尺寸：80 x 22 x 3.2mm

重量：9.0g

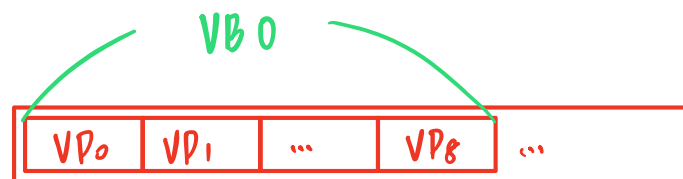
功率：55mW(待機)/5.5W(平均)/8.5W(突發模式)

軟體：三星管理魔術師

保固：5 年或1200TBW有限保固



Source: <https://news.xfastest.com/review/review-06/126619/samsung-990-pro/>

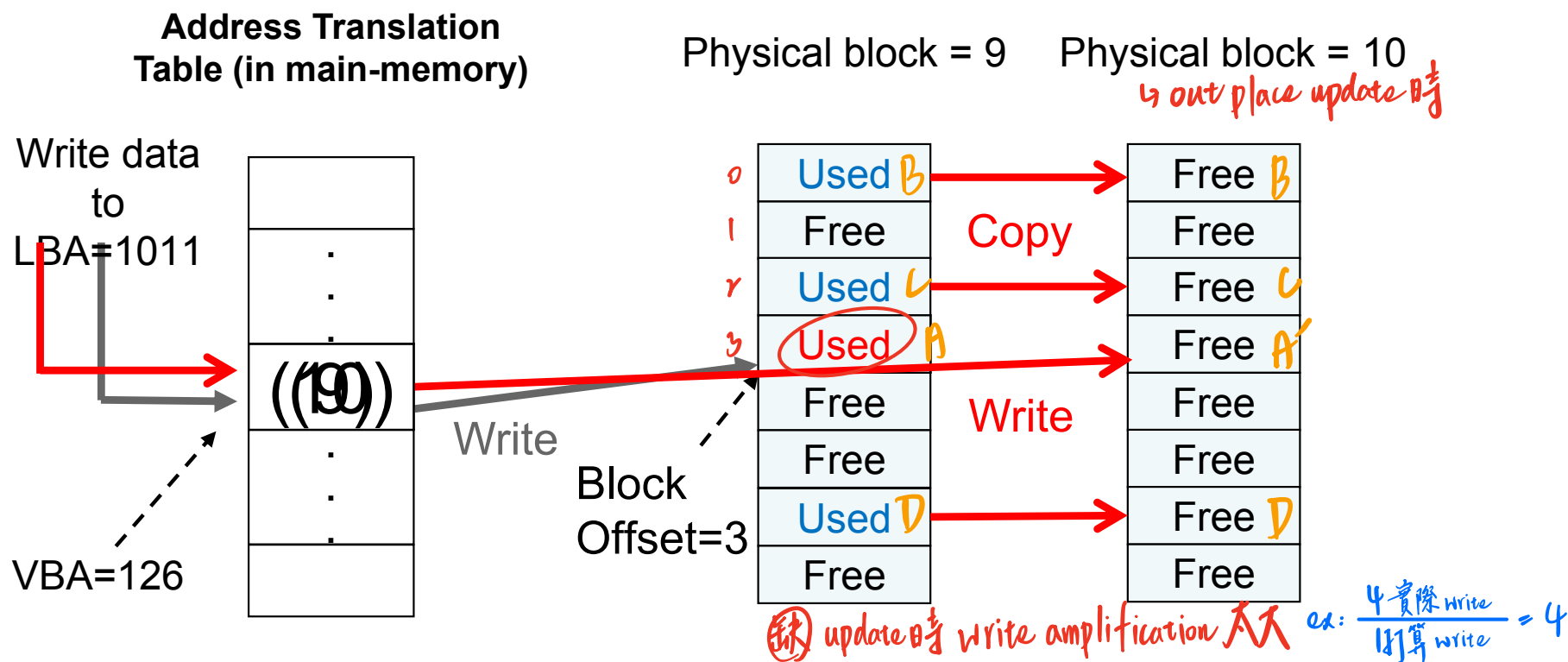


BL (Block-level)

Request LBA=1011

⇒ virtual block address(VBA) = $1011/8 = 126$ ⇒ 幾號 Block (VB 126)

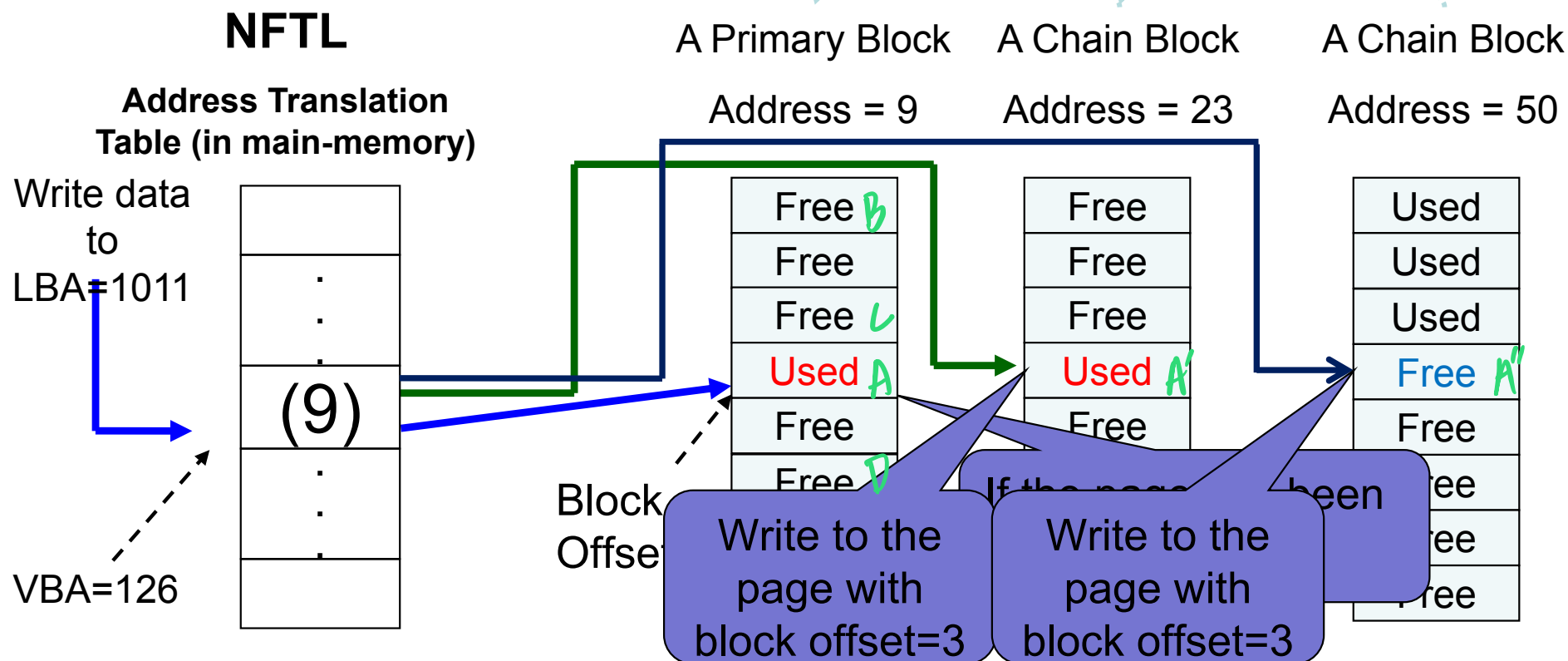
⇒ block offset = $1011 \% 8 = 3$ ⇒ 幾號 page (VP 3)





Request LBA=1011

$\Rightarrow$ block offset = $1011 \% 8 = 3$ (優) 解決寫入放大 (缺) 空間使用率差



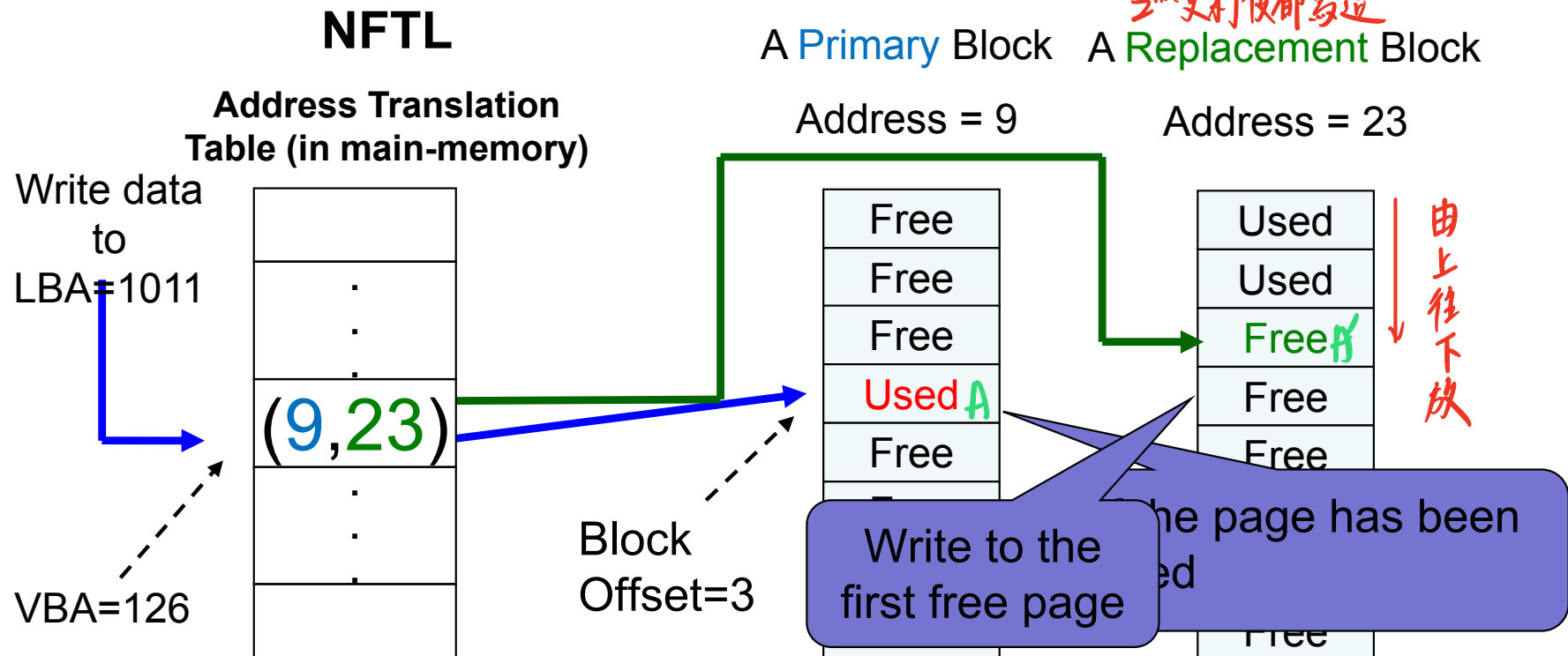


Request LBA=1011

⇒ block offset = $1011 \% 8 = 3$

③ read data 時間長

2nd更新後都寫這



FAST (Hybrid)

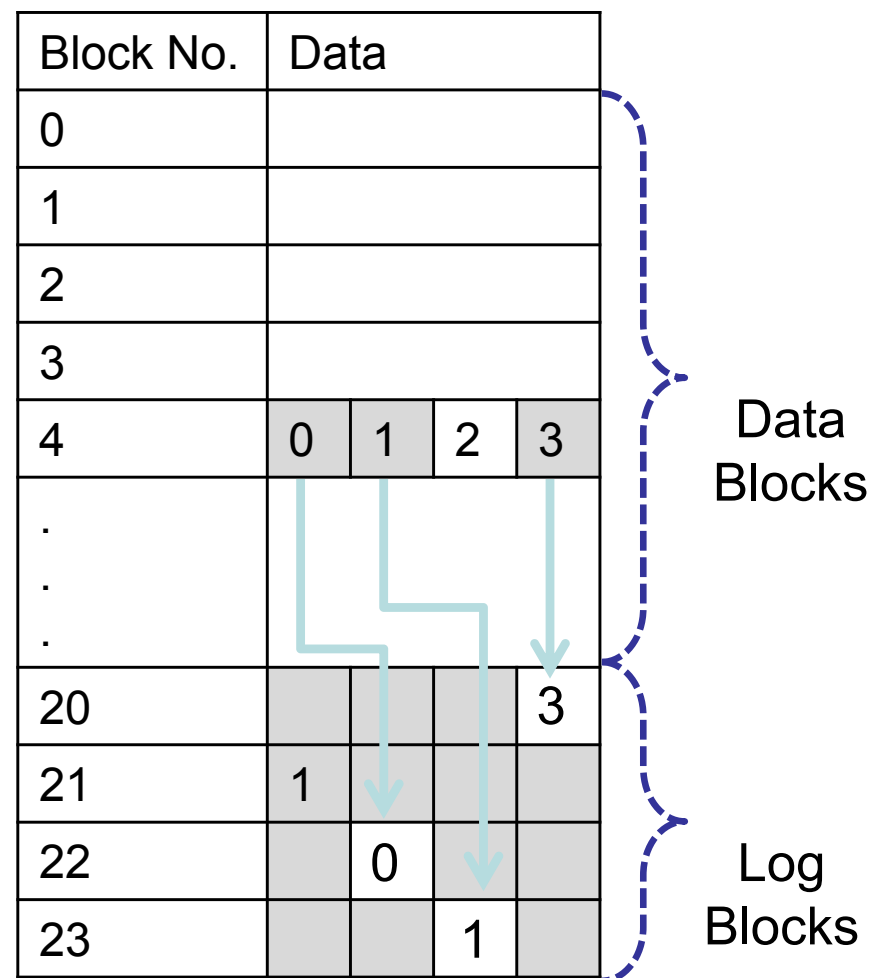
- Data blocks $\Rightarrow$ *primary*
 - Block-level mapping
- Log blocks $\Rightarrow$ *replacement*
 - Page-level mapping
- ALL data blocks share all log blocks $\Rightarrow$ *share replacement block*
- Updated data are written to log blocks and merged back to data blocks

Flash memory

Block No.	Data			
0				
1				
2				
3				
4	0	1	2	3
.				
.				
.				
20				3
21	1			
22		0		
23			1	

Data Blocks

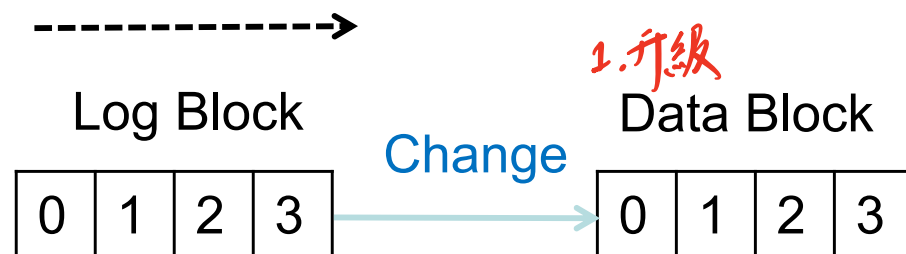
Log Blocks



FAST (Hybrid) - Merge Operations

(a) Switch Merge

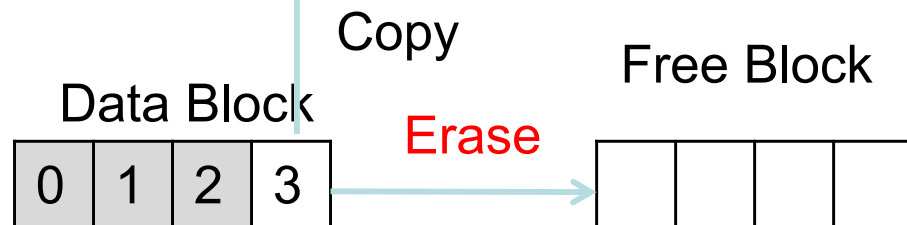
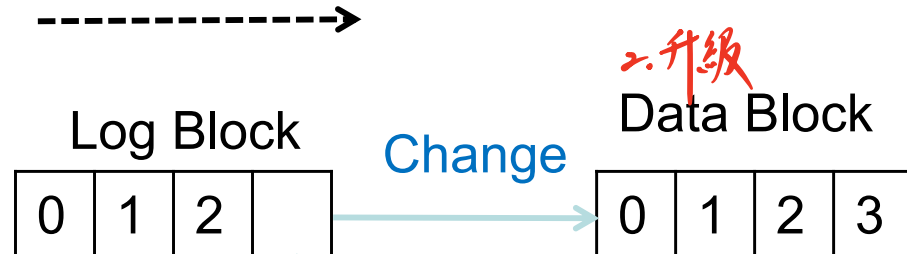
Sequential writes
(LPN): 0, 1, 2, 3



2. 原本的可erase掉

(b) Partial Merge

Semi-Sequential writes
(LPN): 0, 1, 2



1. 原本的copy走, 再erase

FAST (Hybrid) - Merge Operations

(c) Full Merge

Random writes

(LPN): 4, 0, 5, 1, 4, 2, 5, 4

Log Block 0(LB₀)

4	0	5	1
---	---	---	---

Log Block 0(LB₁)

4	2	5	4
---	---	---	---

Log Block 0(LB₀)

--	--	--	--

Log Block 0(LB₁)

4	2	5	4
---	---	---	---

1. Copy

Erase

--	--	--	--

0	1	2	3
---	---	---	---

4	5	6	7
---	---	---	---

Change

--	--	--	--

2. 升級

0	1	2	3
---	---	---	---

4	5	6	7
---	---	---	---

Data Block (DB₀)

Data Block (DB₁)

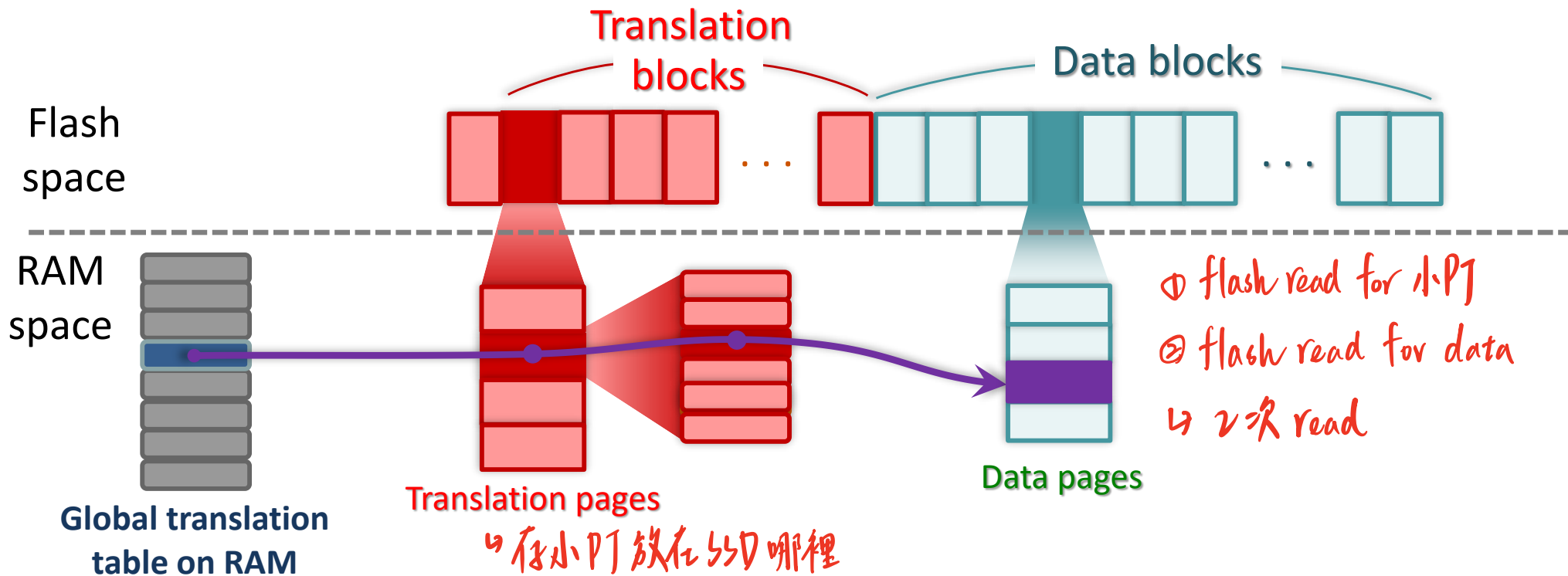
Data Block (DB₀)

Data Block (DB₁)



Demand-based FTL (DFTL) (Page-Level)

- Page-level address translation for all data
- On-demand caching/buffering
 - Addresses of **data pages** are saved in **translation pages** on **flash**
 - Addresses of **frequently-used data pages** are cached on **RAM**
 - Addresses of **translation pages** are cached on **RAM**



Summary

	<u>Page-grained</u>		<u>Block-grained</u>		<u>Hybrid</u>
Metrics	FTL	DFTL	BL	NFTL	FAST
RAM Consumption	1.9 GB	11 MB ↑	11 MB	22 MB	11 MB ↑
Read Time	0.1 us	0.1 us ↑	0.1 us	6.4 ms	0.1 us
Write Time	1 ms	1 ms ↑	128 ms	1.7 ms	1 ms
GC Overhead	Low	Median	N/A	High	High
System Overhead	Low	Median	Median	Low	High

Capacity	1 TB
Page / block size	4 KB / 0.5MB
Page read / write	100 us / 1ms

缺



Agenda

- Flash Memory: Why and How
- **Flash Translation Layer**
 - Address Mapping
 - **Garbage Collection**
 - Wear Leveling Strategies
- Error Correction Codes
- Flash File Systems
- Hybrid Flash (SLC & MLC) Architecture
- Other Issues and Conclusion



Outline

- Introduction
- State-of-the-art garbage collection (GC) schemes
- Design issues and discussion
- Summary

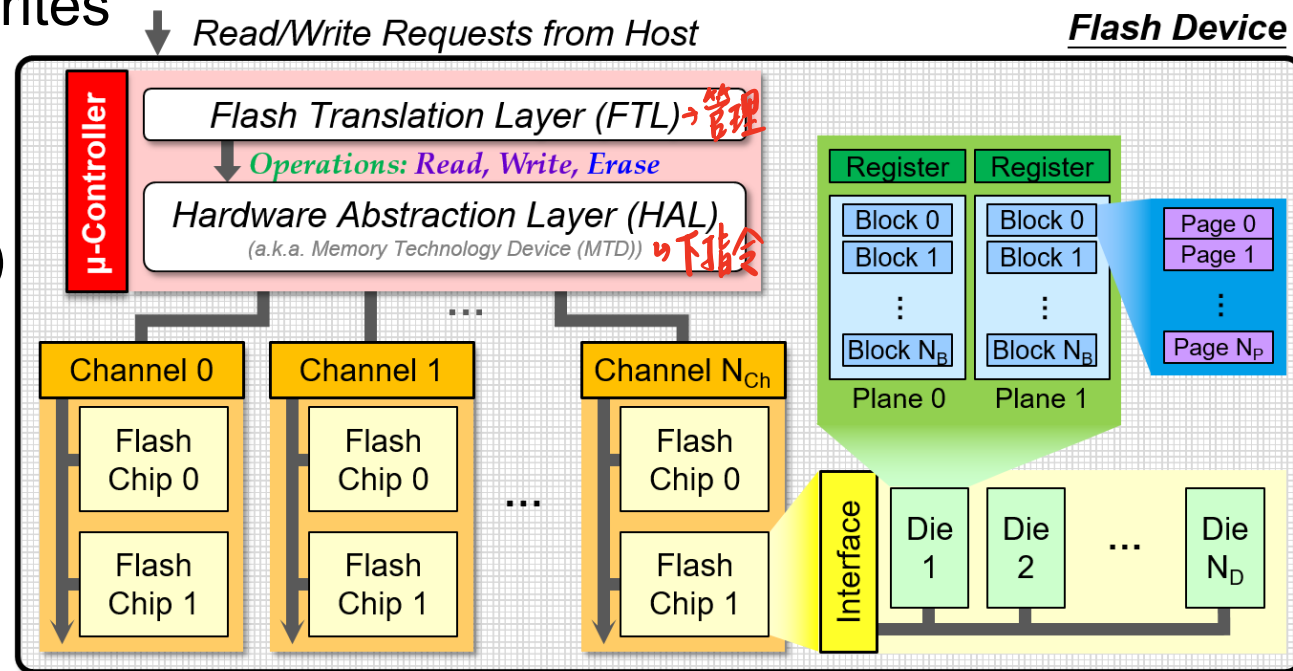
NAND Flash Memory: An Introduction

- Chip geometries

- Physical geometry: chip—die—plane—block—page $\Rightarrow$ *layout*
- Two areas of a page: data area & spare area

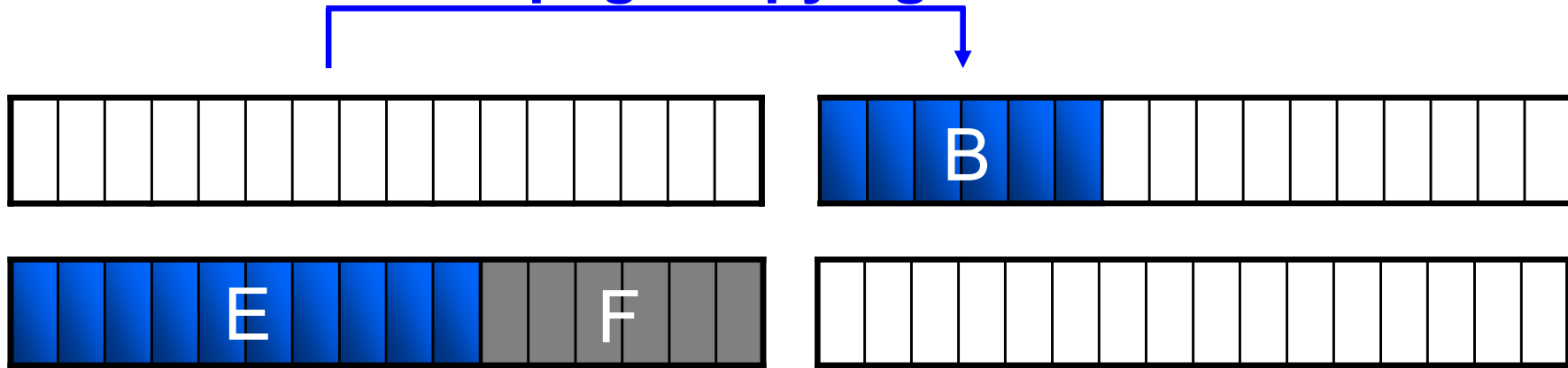
- Access properties

- Page-based reads/writes
- Write-once property
- Bulk erasure
(block-based erases)
- Wear-out effect



An Example of Garbage Collection

Live-page copying



 : page contains hot(/frequently accessed) data

 : page contains cold(/infrequently accessed) data

 : free page

 : dead page



Typical GC Procedure

- **Victim block selection**: Select one or more flash blocks (called **victim blocks**) for reclamation
選GC的對象 block
- **Data re-placement**: **Copy all live pages** of the victim blocks (if they exist) to other blocks (non-victim blocks at this time of GC)
- **Erase the victim blocks**
- **Return the erased victim blocks** to free block list (if the list exists)



Outline

- Introduction
- State-of-the-art garbage collection (GC) schemes
- Design issues and discussion
- Summary



State-of-the-art GC schemes

- The baseline approaches
 - **Random** (baseline)
 - **FIFO (RR)** (baseline)
 - **Greedy** (baseline): 1995
- Cost-benefit (**CB**): 1999
- Cost-age-time (**CAT**): 1999
- Cost-age-time with age-sort (**CATA**): 2006
- Hot-cold swapping (**HC**): 1999
- Adaptive GC (**AGC**) mechanism for N-log block flash memory: 2006
- Fast and endurant garbage collection (**FeGC**): 2011



Baseline Approaches

- The **Random** policy selects the victim block in a uniformly-random manner
 - The Random policy intends to yield a long-term average use over each flash block
- The **FIFO** policy (also called RR policy) cleans blocks in a Round-Robin (RR) manner
 - 優 – The FIFO policy forces that every block has an erase count differing by **at most 1** $\Rightarrow$ wear-leveling 很好
 - 缺 – Since most data on storage are static and most blocks contain few invalid pages, the FIFO policy (often) incurs a **considerable live page copying overhead**
- The **Greedy** policy cleans the block with **the largest number of invalid pages**
 - The Greedy policy minimizes the **live page copying** overhead
 - 缺 – As flash space is filled with static live data, a small number of blocks might be repetitively selected, erased, and worn-out quickly



The Cost-Benefit (CB) Policy

- **CB** cleans the block with **the largest benefit-cost value**: $\Rightarrow$ 挑大的做 GC

$$\frac{\text{benefit}}{\text{cost}} = \frac{\text{age} \times (1 - u)}{2u}$$

$2u$ A valid page reduces 2pts., 1pt to read the page and 1pt to write it to another block

- **age**: **invalidated time period** = current time – the time when a page of the block was lastly invalidated
 - *Blocks with old pages are cleaned with time $\rightarrow$ natural wear-leveling!*
- **u**: **percentage of valid pages of the block** $\Rightarrow$ 有效 data 比例

- **Assumptions**

- All considered blocks have no free pages



- Read-write time skew is not considered



The Cost-Age-Time (CAT) Policy

- **CAT** cleans the victim block(s) that **minimize** $\Rightarrow$ 排小的做比

$$[u/(1-u)] * (1/\text{age}) * \text{erase_count}$$
 where
 - u is ratio of live pages of the block
 - **age** is the time since last invalidation of any page of the block
 - **Number of cleaning** is the **erase count** of the block $\Rightarrow$ wear-leveling 考量
- **Comments**
 - The definitions of age in CB and in CAT have similar effects!
 - CB prevents a block to own invalid pages for long, which is not sufficient; CAT adds a safety facility to it—the erase count.



The Cost-Age-Time with Age-sort (CATA) Policy

- **CATA** presents the following ideas:
 - Victim block selection must cooperate with a suitable **data migration policy** to yield good results
 - CATA predicts future workloads by **probabilistic models for data migration**, and selects the victim block with **maximal**

$$\frac{1-u}{1+u} \cdot age \cdot \frac{1}{erase_count}, \text{ where}$$

- ***u*** is the utilization, i.e., ratio of live data, of the block
- ***age*** is the time since last invalidation of any page of the block
- ***erase_count*** is the normalized erase count of the block



The Hot-Cold (HC) Swapping Policy

- **HC** enhances Greedy by separating hot data from cold data, similar to a **data-clustering** work by Mei-Ling Chiang et al.
 - For every n times of GC (called a **collection period**), the **collector** (!= cleaner) is invoked to collect and append cold data from fragmented files to the log end → data clustering
 - This method also proposes the **wear-leveling degree**:
 $\Delta_{\epsilon} = \epsilon_{\max} - \epsilon_{\min}$, where $e_{\max}$ and $e_{\min}$ are the maximal and minimal erase counts among all flash blocks
 - **HC** cleans the block with **minimal Cleaning Index**:

This algorithm needs to re-compute the Cleaning Index of every block on every erase!

$$\text{Cleaning Index} = (1 - l) \times \mu_i + l \times \frac{\epsilon_i}{\epsilon_{\max} + 1}$$

l : normalized wear-leveling degree 考量用係數

u_i : utilization of block (ratio of live pages)

e_i : erase count of the block

$e_{\max}$: maximal erase count among all blocks



The Adaptive GC (AGC) Policy

- **Two-phase GC (for NFTL type 1)**
 - **Passive GC**: Blocks with all-invalid pages are added to the erase-candidate list
 - **Active GC** is triggered when the number of free blocks is low
 - Low system loading: Select the block with the **longest** log-block chain (to improve the efficacy of GC)
 - High system loading: Select the block with the **shortest** log-block chain (to reduce the GC blocking time)



The Fast and Endurant GC (FeGC) Policy

- **FeGC** cleans the block with the largest sum of ages of invalid pages:

$$CWA = \sum_{i=1}^n age_i$$

age_i: the time that the page has been invalidated

n: the number of invalid pages of the block

- Blocks with lots of invalid pages have a high probability to be cleaned
→ *Good GC performance*
- Blocks with some invalid pages that have been invalidated for a long time have a high probability to be cleaned → *Wear-leveling considerations*
- FeGC uses **hot-cold block swapping** to yield wear-leveling



Outline

- Introduction
- State-of-the-art garbage collection (GC) schemes
- **Design issues and discussion**
- Summary



Design Issues and Discussion

- **Scalability**: The large amount of information maintained for GC and wear-leveling is even less tolerable for ultra-large-scale flash storages in the future
- **Flexibility**: GC schemes should better adapt to
 - Shifting access patterns
 - Heterogeneous storage media such as SLC + MLC



Outline

- Introduction
- State-of-the-art garbage collection (GC) schemes
- Design issues and discussion
- Summary



Summary

- We have discussed various GC schemes that
 - Reduce the GC overheads
 - Expand the flash lifetime
- Existing approaches do not completely fulfill the more serious future requirements on
 - Scalability
 - Flexibility
- GC for next-generation flash storages like server storages is an important topic



Agenda

- Flash Memory: Why and How
- **Flash Translation Layer**
 - Address Mapping
 - Garbage Collection
 - **Wear Leveling Strategies**
- Error Correction Codes
- Flash File Systems
- Hybrid Flash (SLC & MLC) Architecture
- Other Issues



Outline

- **Introduction**
- **Static Wear Leveling**
 - Dual-pool
 - Block Erasing Table (BET)
 - Rejuvenator
 - Observational method
 - Progressive
 - Error Rate Aware
- **Summary**

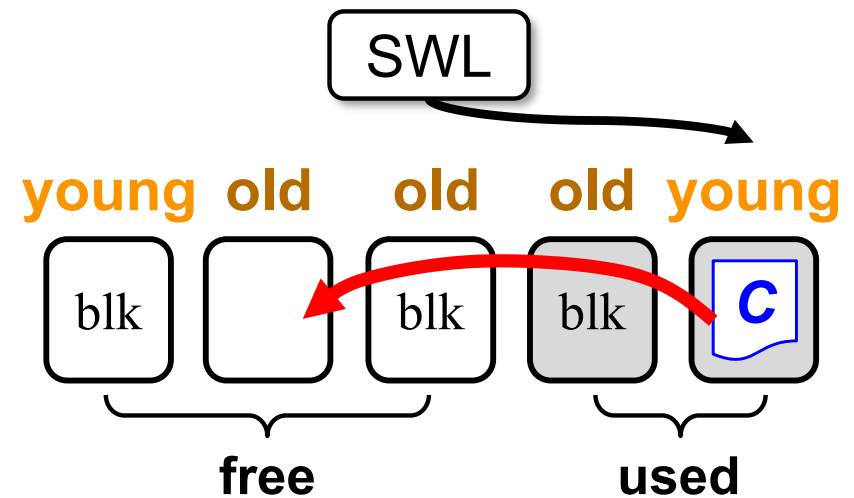
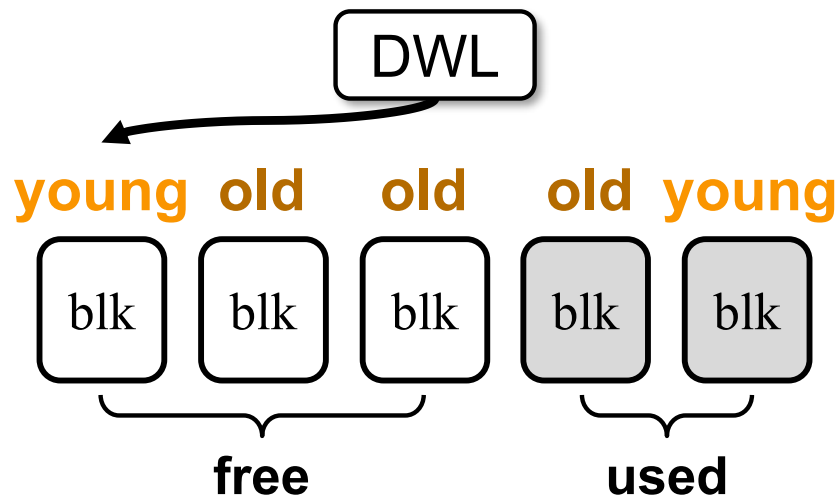


Introduction

- Endurance of flash memory cells ⇒ 壽命
 - Single Level Cell (SLC): < 100,000 P/E cycles
 - Multiple Level Cell (MLC): < 5,000 P/E cycles
- The objective of Wear Leveling
 - To prolong the endurance of flash memory by evenly distributing block erases

Introduction

- Wear Leveling can be categorized as either static or dynamic
 - Dynamic Wear Leveling $\Rightarrow$ 容易固定某些 block (缺)
 - Choose free block with lower erase count among free blocks
 - Static Wear Leveling
 - Let blocks have even erase counts by moving cold data to more worn blocks



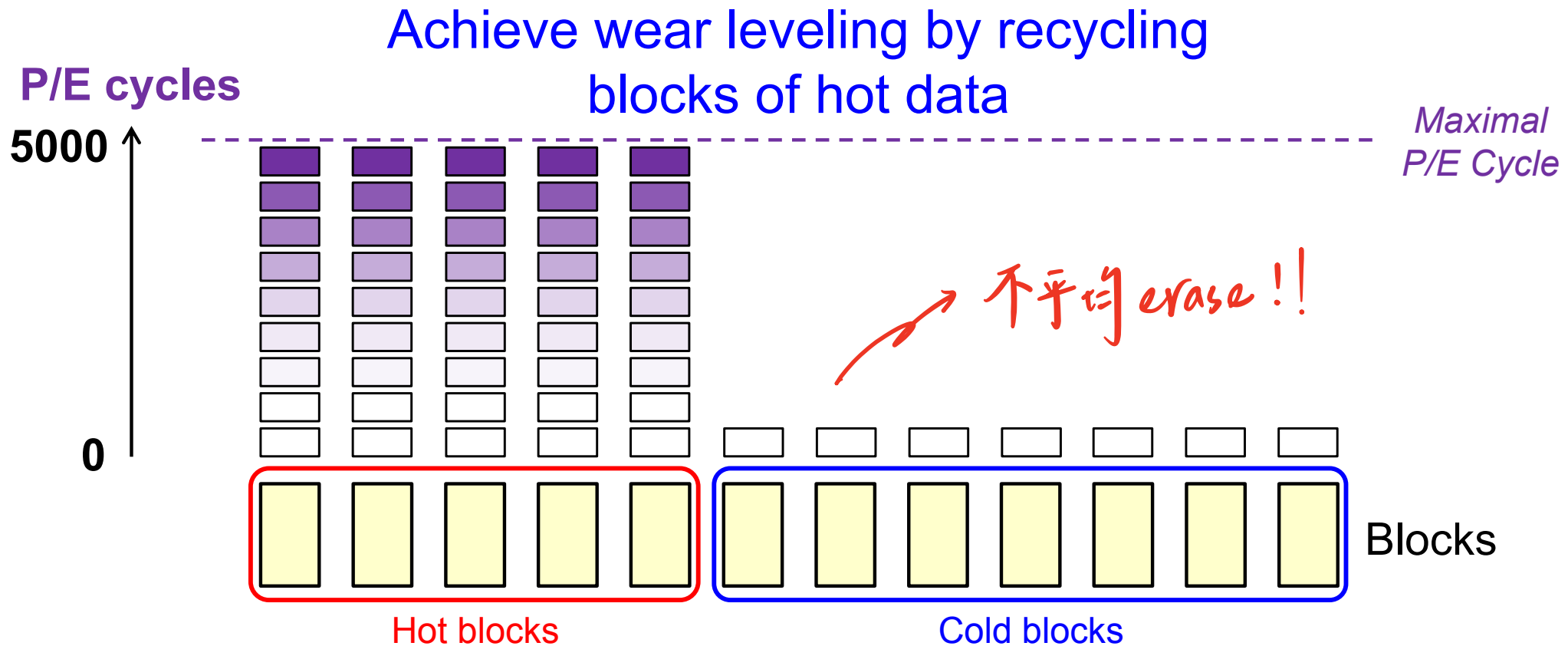


Wear Leveling

- Existent wear leveling designs prevent wearing out some blocks excessively, based on the **P/E cycles** of blocks

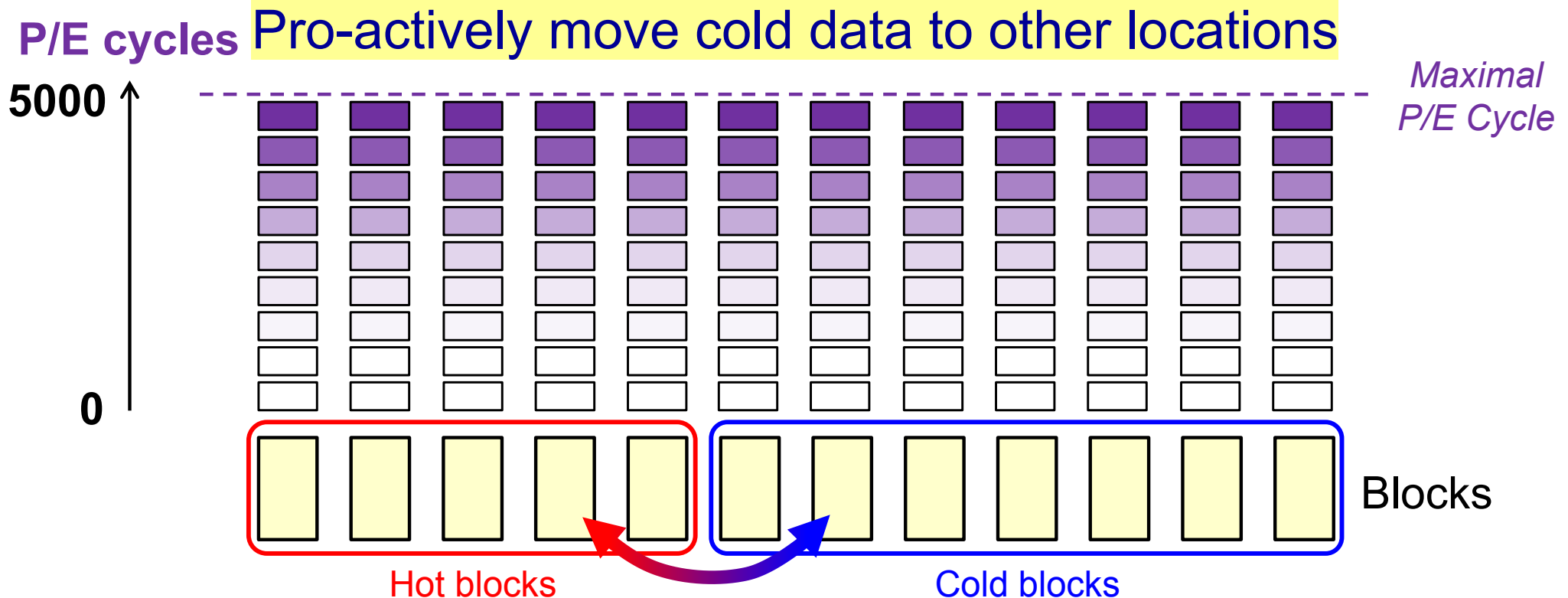
– Dynamic Wear Leveling

– Static Wear Leveling



Wear Leveling (Cont.)

- Existent wear leveling designs prevent wearing out some blocks excessively, based on the **P/E cycles** of blocks
 - Dynamic Wear Leveling
 - Static Wear Leveling





Outline

- Introduction
- **Static Wear-leveling**
 - Dual-pool
 - Bit Erase Table (BET)
 - Rejuvenator
 - Observational method
 - Progressive
 - Error Rate Aware
- Summary



A Category of Static Wear Leveling

- Static Wear Leveling
 - Even out erase count across **ALL** blocks
 - Classify into *Proactive* and *Passive* (depend on its [GC policy](#))

Algorithm	Type	Address Mapping
BET	Passive	Not constrained
Dual-pool	Passive	Not constrained
Rejuvenator 返老還童	Proactive + Passive	Page + Block mapping
Observational	Proactive	Hybrid mapping



Outline

- Introduction
- **Static Wear Leveling**
 - Dual-pool
 - Block Erasing Table (BET)
 - Rejuvenator
 - Observational method
 - Progressive
 - Error Rate Aware
- Summary



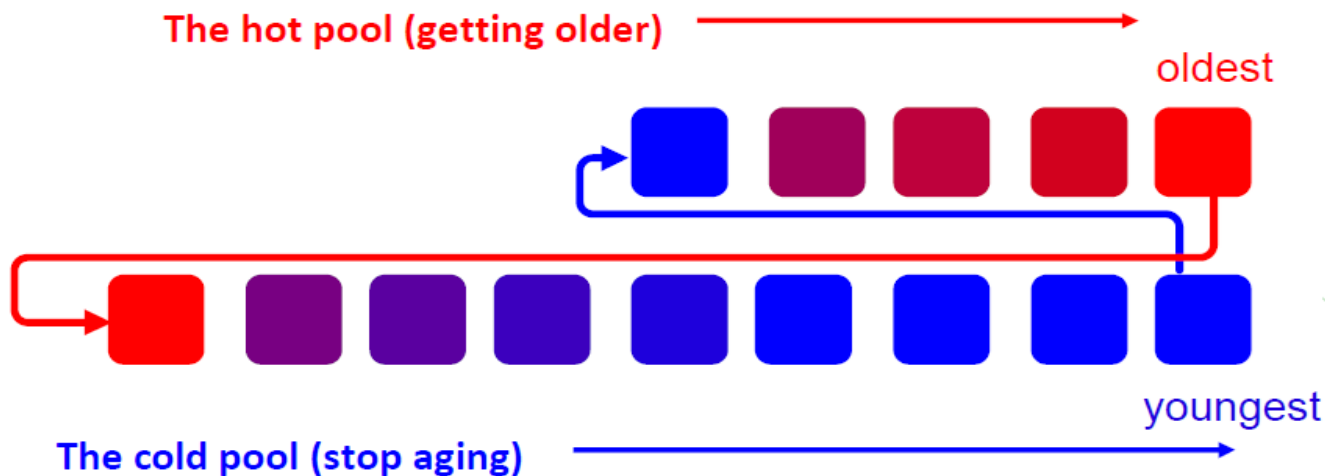
Dual Pool_[2]

- Motivation
 - The majority of data are cold
- Idea
 - To cease the wearing of blocks by storing cold data
 - Blocks just involved in wear leveling are left alone until wear leveling takes effect



Dual Pool

- Hot and cold data stay in hot and cold pools respectively
- Migrating **cold data** from **young blocks** to **old blocks**
 - When the difference on erase count between the head of hot pool and the rear of cold pool exceeds a predefined threshold
- Adaptive Pool Resizing
 - Spatial locality in workloads may change with time
 - Prevent some block never involved in wear leveling again





Outline

- Introduction
- **Static Wear Leveling**
 - Dual-pool
 - **Block Erasing Table (BET)**
 - Rejuvenator
 - Observational method
 - Progressive
 - Error Rate Aware
- Summary



Block Erase Table (BET)_[1]

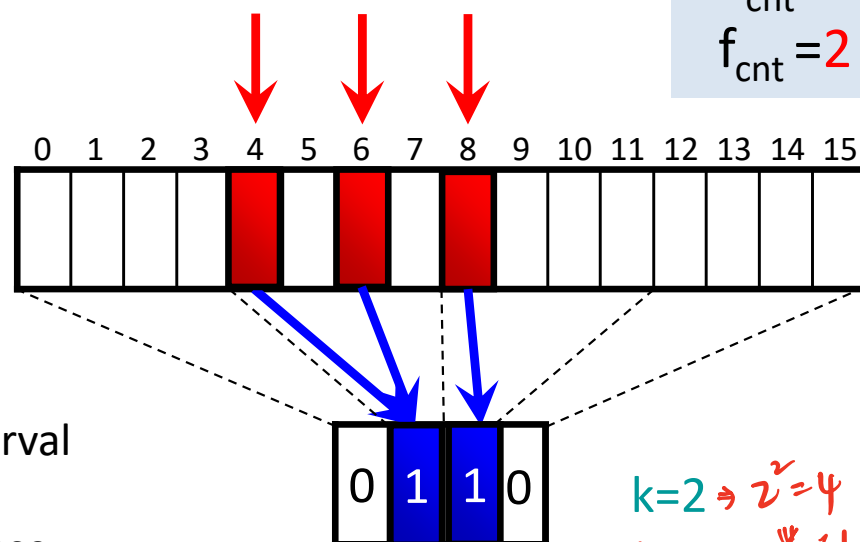
- Motivation
 - Use a bit array to remember block erase status
 - Trigger wear leveler by the **degree of diversity**
- Each bit in bit array can corresponds to 2^k contiguous blocks.

e_{cnt} : total number of block erases done since the BET is reset

f_{cnt} : number of 1's in the BET

 : block has been erased in current resetting interval

 : index to a block that the Cleaner wants to erase



$e_{cnt} = 3$
 $f_{cnt} = 2$

$k=2 \Rightarrow 2^2=4$
1 bit array 對應 4 block

Bit Erase Table (BET)

- *Unevenness level* (e_{cnt} / f_{cnt}) $\geq T \rightarrow$ Trigger SW Leveler
- Reset BET when all flags are set.

T: threshold, T=1000 in this example

e_{cnt} : total number of block erases since the BET is erased

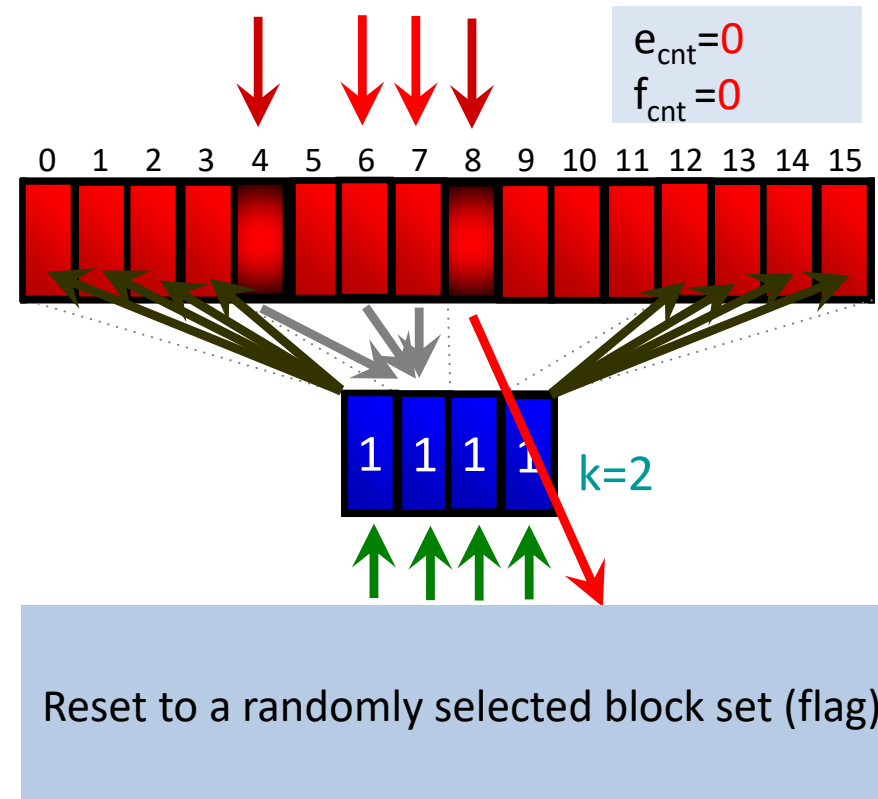
f_{cnt} : number of 1's in the BET

↑ : index that SW Leveler triggers the Cleaner to do garbage collection

 : index in the selection of a block set

 : index to a block that the Cleaner wants to erase

: block has been erased in current resetting interval





Outline

- Introduction
- **Static Wear Leveling**
 - Dual-pool
 - Block Erasing Table (BET)
 - **Rejuvenator**
 - Observational method
 - Progressive
 - Error Rate Aware
- Summary

Rejuvenator_[3] 返老還童

- Motivation
 - Achieve **wear leveling** with minimized performance overhead
 - Achieve efficient **garbage collection**
- Idea
 - Keep the variance in erase counts of the blocks to a minimum
- Separate hot/cold data
 - By using window based scheme with counters

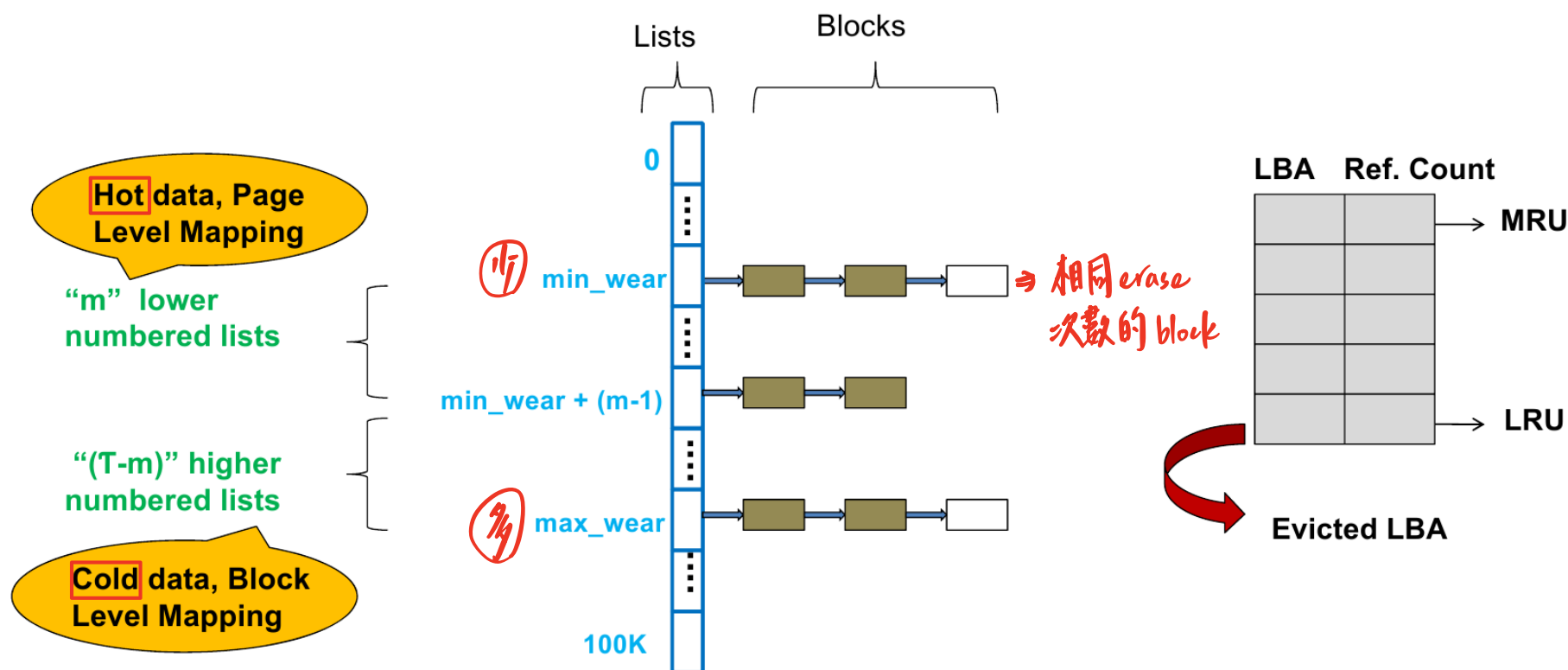


Rejuvenator

- Maintain τ lists of blocks
 - Page mapping for hot data
 - Hybrid mapping for cold data

$$\text{diff} = \text{max_wear} - \text{min_wear}$$

$$\text{diff} \leq T-1$$





Outline

- Introduction
- **Static Wear Leveling**
 - Dual-pool
 - Block Erasing Table (BET)
 - Rejuvenator
 - **Observational method** 参考
 - Progressive
 - Error Rate Aware
- Summary



Observational method – Motivation_[4] Only for FAST (Hybrid Method)

- Two scenarios will trigger block allocations
 - Updated data is receiving (intuitive)
 - Or in a merge procedure**

Table 2: Block Allocation Ratios in FAST

Trace	New Allocation	Merge Allocation
SPC1	3.90%	96.10%
TPC-C	33.76%	67.24%
MSR-hm_0	4.87%	95.13%
MSR-mds_0	13.02%	86.98%
MSR-prn_0	16.07%	83.93%
MSR-prxy_0	7.07%	92.93%
MSR-rsrch_0	18.42%	81.58%
MSR-stg_0	7.30%	92.70%
MSR-ts_0	8.29%	91.71%
MSR-web_0	6.75%	93.25%

**Most
parts**



Observational method

- Key idea

- Observe the **temporal locality** of write behaviors, and allocate blocks proactively
- Maintain a **block access table (BAT)**
 - Records the footprints of recent logical block accesses
 - Perform *locality-based block allocation* (LBA) when merge

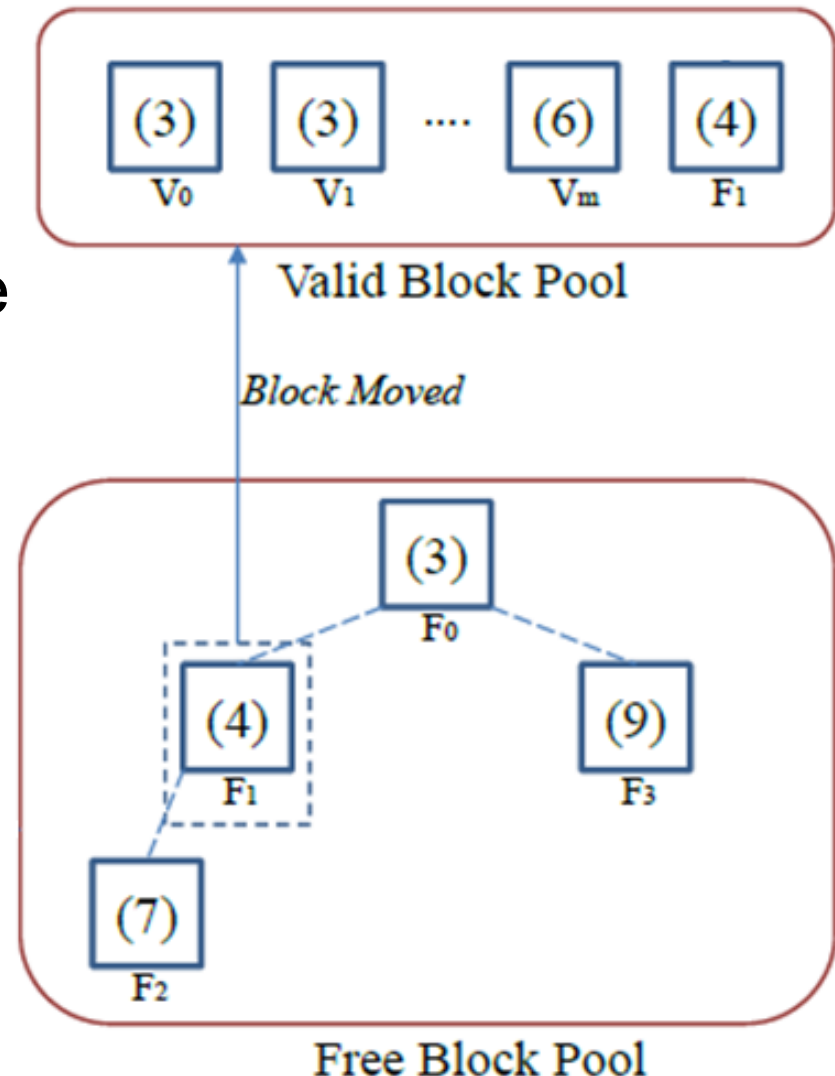
- Objective

- Reduce and evenly distribute erasures under hybrid mapping with log blocks in a **proactive** way

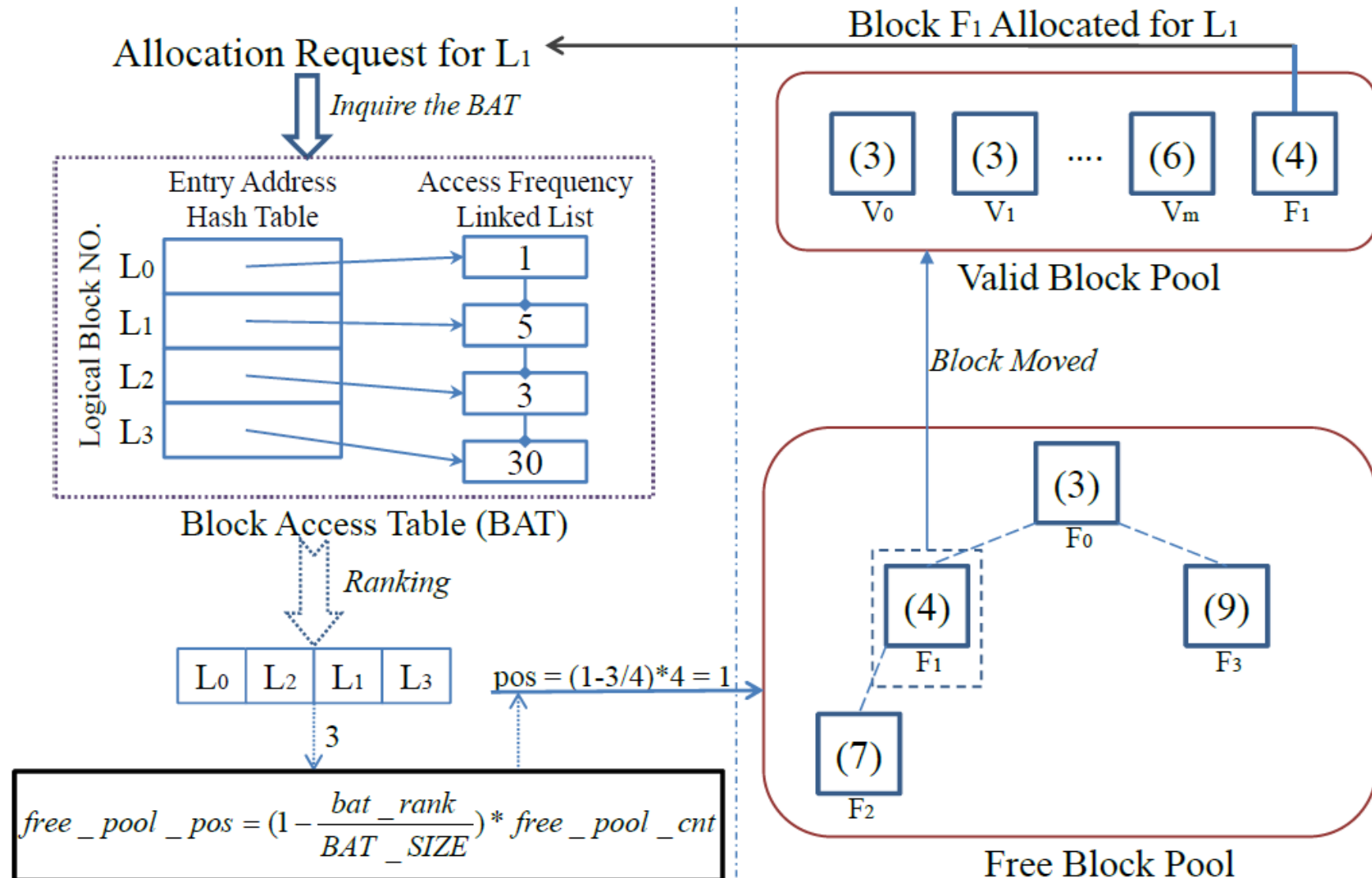


Observational method - Block Allocation

- Valid Block Pool
 - Ordered by their **updated time**
 - Just like an ordinary FIFO queue
- Free Block Pool
 - A min-heap sorted according to the **erase count** of each block
 - Enqueue an erased block:
 $O(\log(n))$



Observational method - Block Allocation



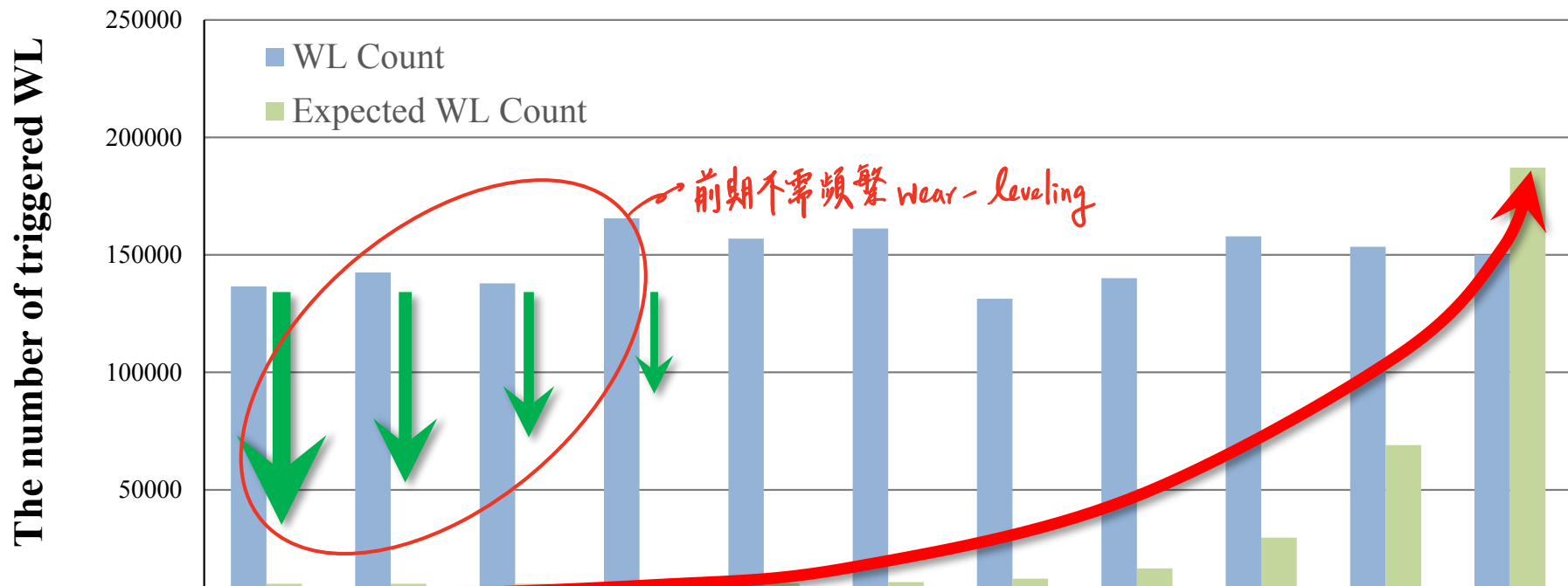


Outline

- Introduction
- **Static Wear Leveling**
 - Dual-pool
 - Block Erasing Table (BET)
 - Rejuvenator
 - Observational method
 - **Progressive**
 - Error Rate Aware
- Summary

Progressive Wear Leveling

- Prevent data migration (WL) in the early stages of the device lifetime
- Progressively increase the number of triggered wear leveling actions



Progressively trigger WL and prevent any block from being worn out prematurely

Progressive Threshold

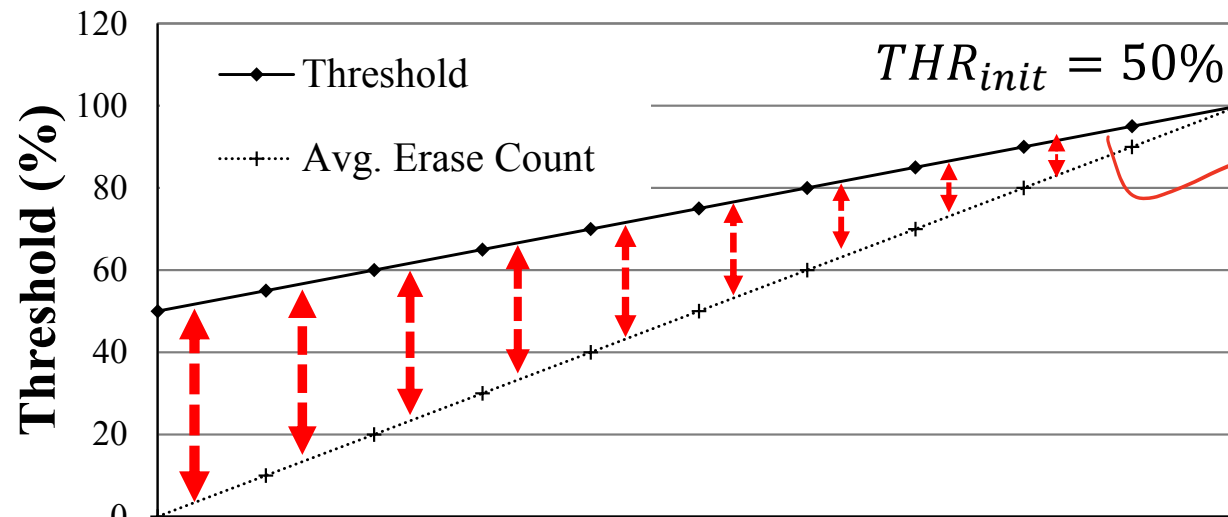
Triggering Condition: any block's erase count > *progressive threshold*

→ Prevent any block from being worn out prematurely!!!

- Determine a *progressive strategy* for *threshold* based on $erase_{avg}$

Average erase count

$$threshold = erase_{avg} * \frac{BLK_{endurance}^{1} - THR_{init}^{0.5}}{BLK_{endurance}^{1}} + THR_{init}^{0.5}$$



initial threshold:
the smallest value
to trigger PWL

threshold

from Fixed Threshold → to Progressive Threshold



Outline

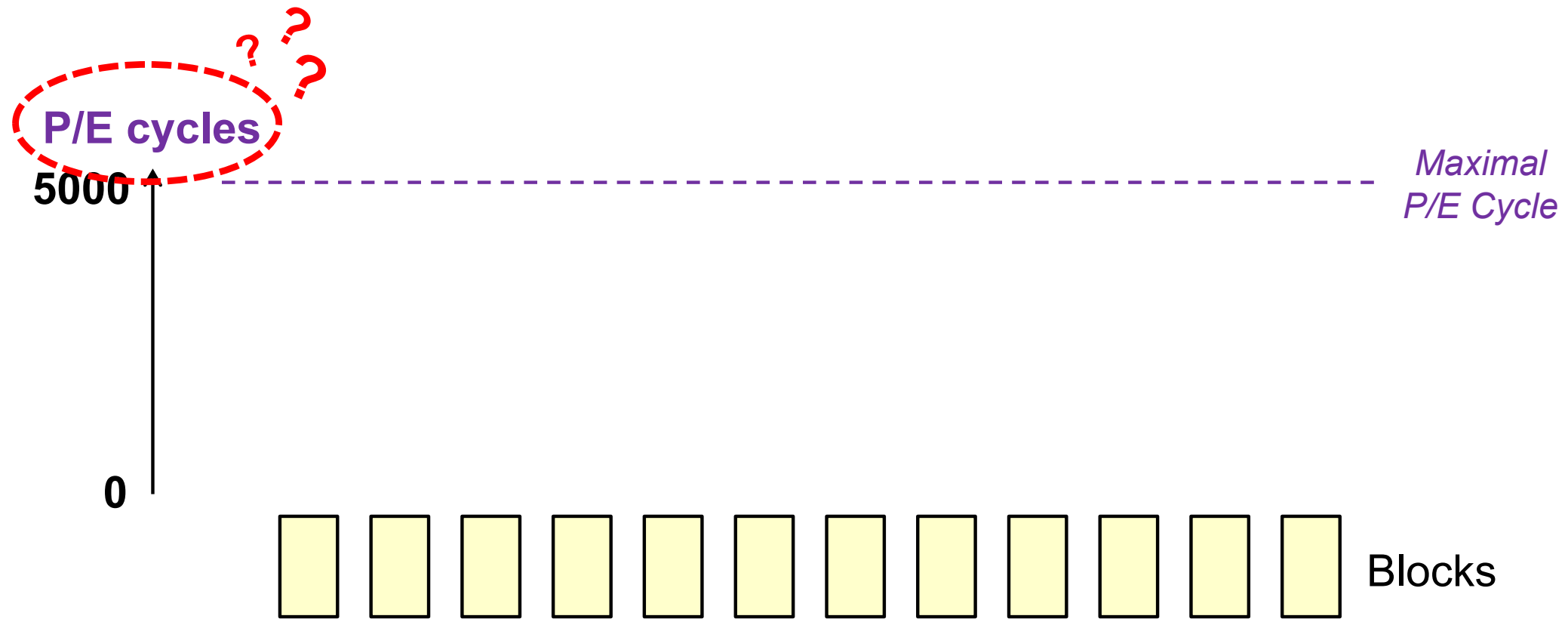
- Introduction
- **Static Wear Leveling**
 - Dual-pool
 - Block Erasing Table (BET)
 - Rejuvenator
 - Observational method
 - Progressive
 - **Error Rate Aware**
- Summary



Observation

- 1) P/E cycles might not accurately reflect the reliability of flash devices (compared to BER)

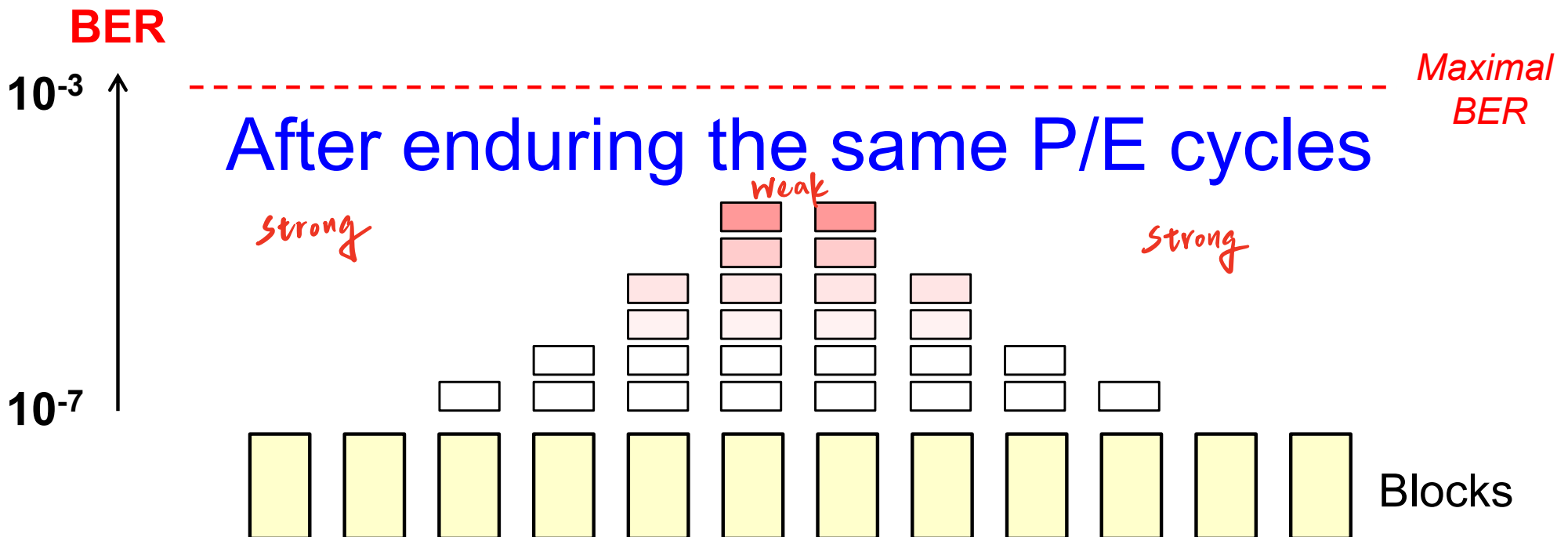
↳ bit error rate





Observation

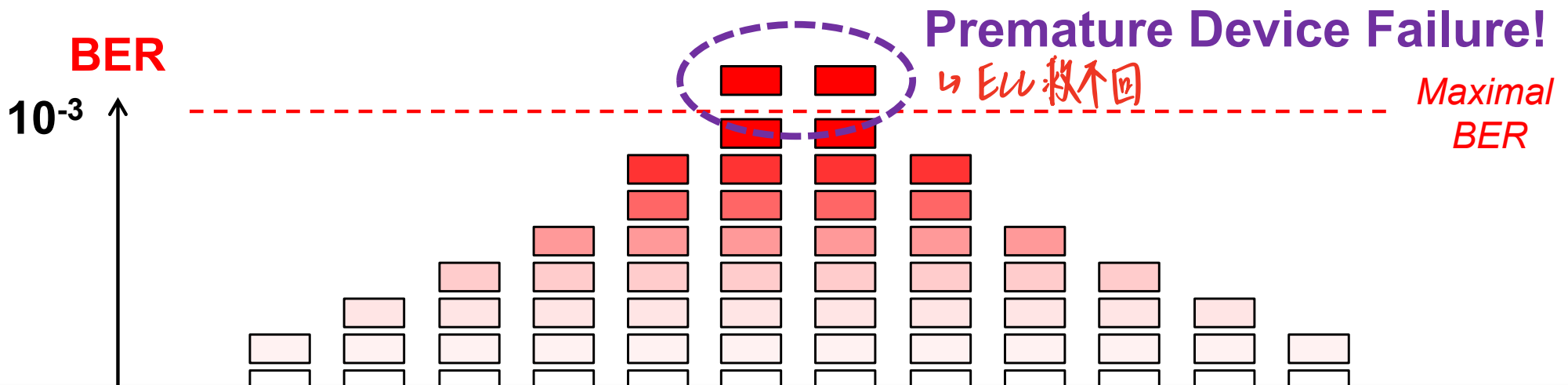
- 1) P/E cycles might not accurately reflect the reliability of flash devices (compared to BER)
- 2) Blocks might have different BERs when enduring the same P/E cycles due to the manufacturing process variation
BER 不一樣多





Observation

- 1) P/E cycles might not accurately reflect the reliability of device (compared to BER)
- 2) Blocks might have different BERs when endured the same P/E cycles due to the manufacturing process variation

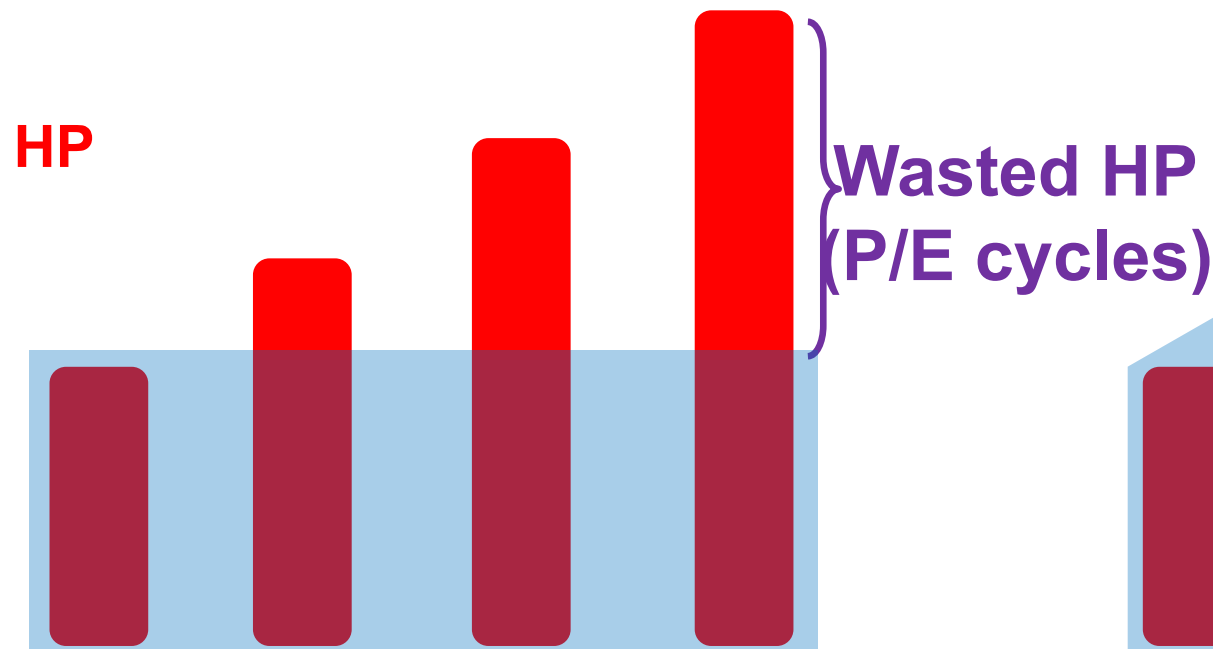


The BER or the number of error bits could be a better metric to design wear leveling schemes!



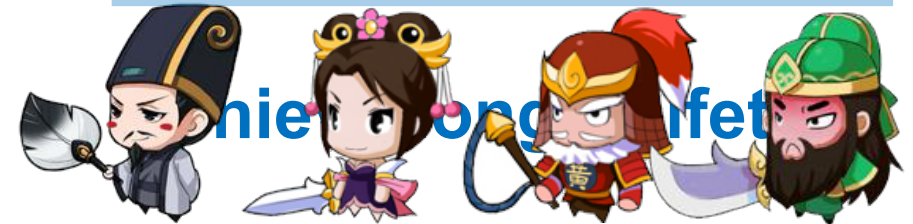
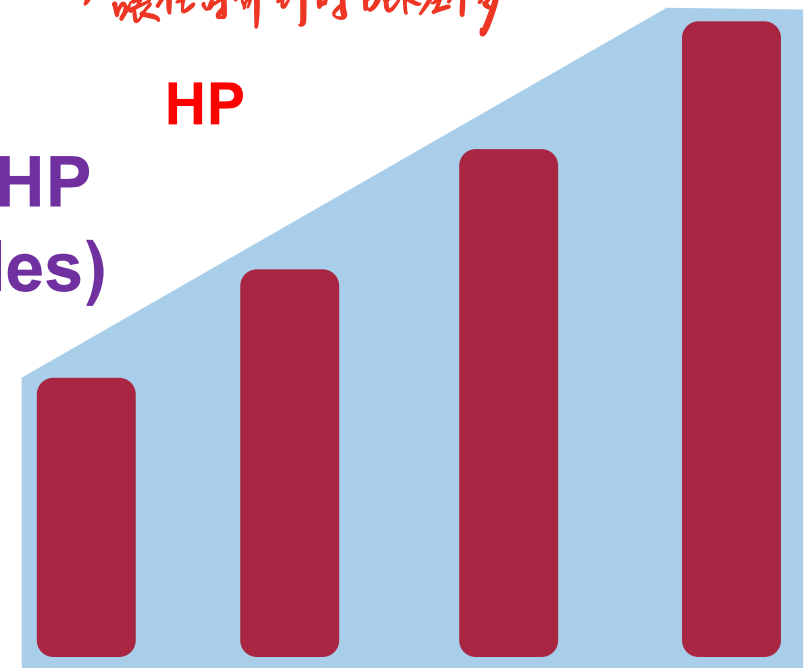
Error-Rate-Aware Wear Leveling

讓在壽命到時 BER 差不多



孔明 孫尚香 黃忠 關羽
Kong Ming Sun Shang Hsiang Huang Chung Kuan Yu

Conventional Wear Leveling



孔明 孫尚香 黃忠 關羽
Kong Ming Sun Shang Hsiang Huang Chung Kuan Yu

Error Rate Aware



Error-Rate-Aware Wear Leveling

Flash Translation Layer $\Rightarrow$ ECC 電路佔一半!!

Allocator

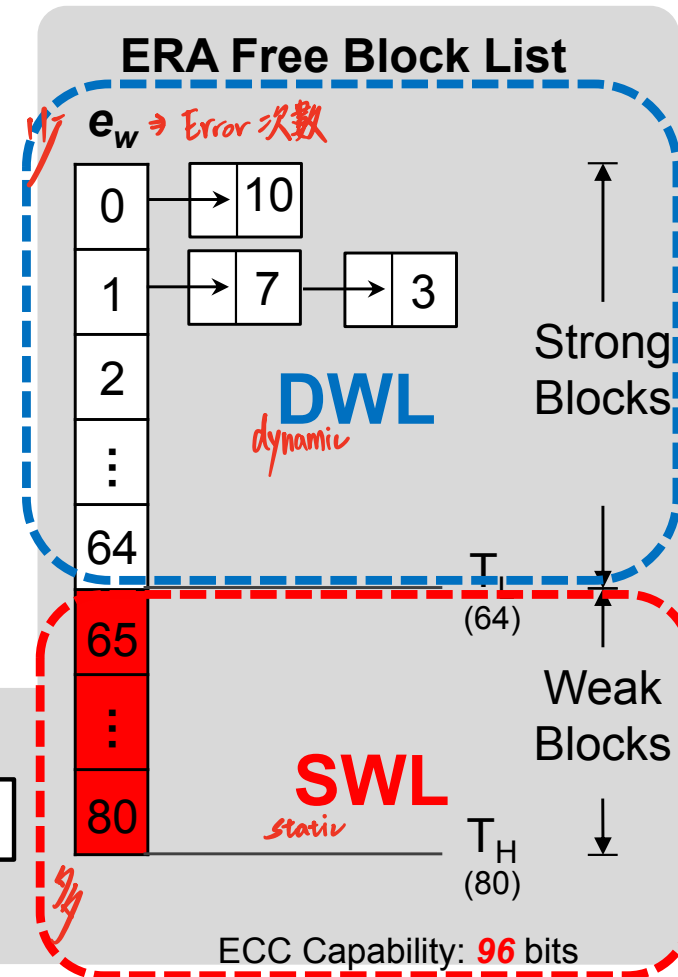
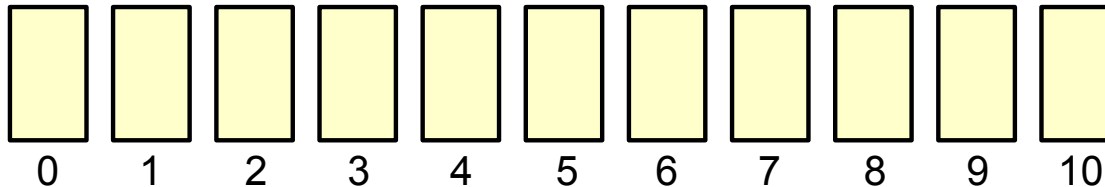
Garbage Collector

Remember the **max**
number of **permanent**
error bits among all
pages

read write erase
R / W / E

Write Error
Count

0 2 94 1 1 1 80 1 1 2 0



Blocks

ECC HW

New ERA Wear Leveling Design

Flash Translation Layer

Allocator

Garbage Collector

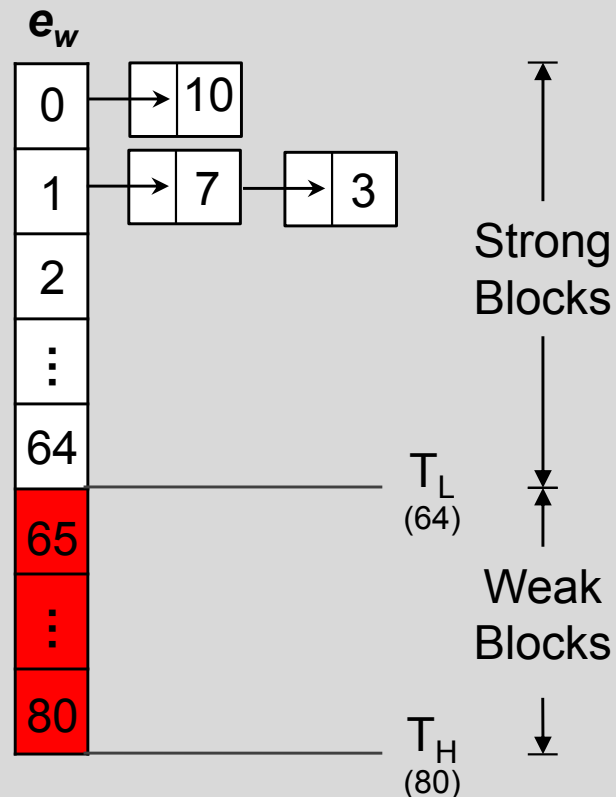
Write error count might be updated after write operations

write ^{block} blk, page
W (4, 20)

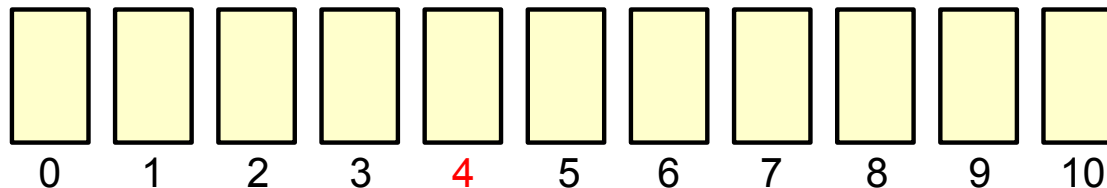
Write Error Count (WEC)

0	2	94	1	2	1	80	1	1	2	0
---	---	----	---	---	---	----	---	---	---	---

ERA Free Block List



ECC Capability: 96 bits



Blocks

New ERA Wear Leveling Design

Flash Translation Layer

Allocator

Collector

An erased block will be inserted into the free block list based on its WEC

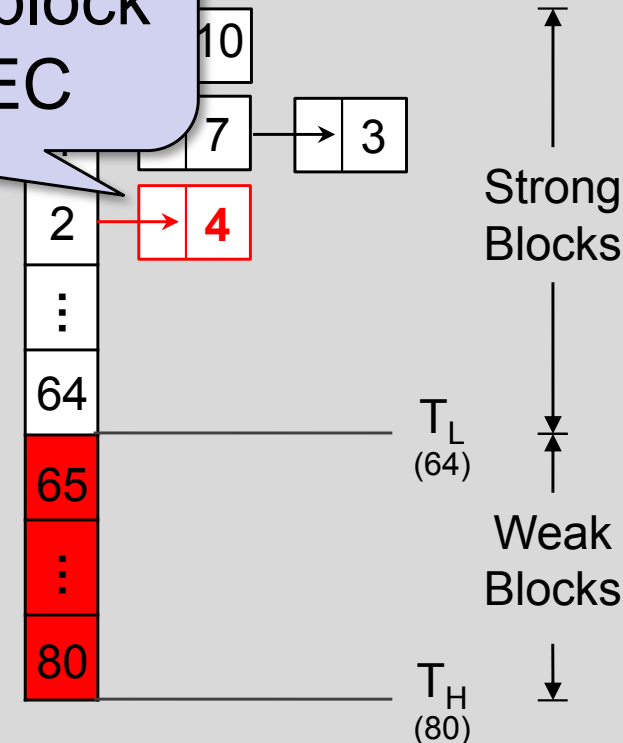
Free Block List

Block Erase

blk
E (4)

Write Error
Count (WEC)

0 2 94 1 2 1 80 1 1 2 0

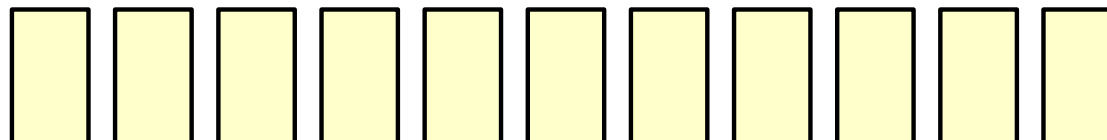


T_L
(64)

Weak
Blocks

T_H
(80)

ECC Capability: 96 bits



Blocks



New ERA Wear Leveling

Flash Translation Layer

Allocator

Garbage Collector

Allocate the most health block

Free Block List

weak : 放 cold data

strong : 放 hot data

blk, page
R (6, 30)

Insert

Contains too many error bits!

Perform SWL

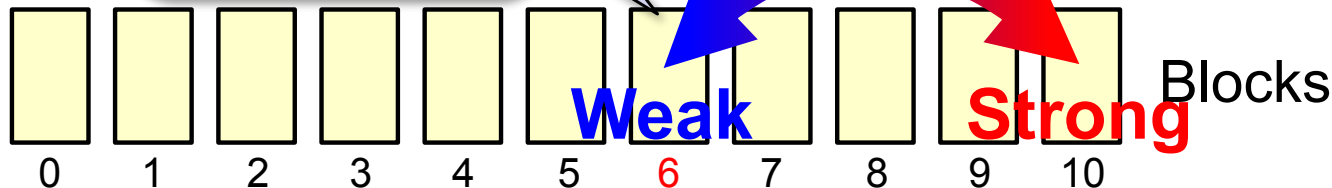
ECC Capability: 96 bits

Strong Blocks

Weak Blocks

T_L
(64)

T_H
(80)





Outline

- Introduction
- Static Wear Leveling
 - Dual-pool
 - Block Erasing Table (BET)
 - Rejuvenator
 - Observational method
- **Summary**



Summary

- We have discussed six well-known wear-leveling designs
 - Four traditional ones to keep erase count gap in a **fixed defined range**
 - **One aggressive method** to adaptively adjust the erase counter gap
 - One new approach that conducts wear leveling **based on error rate, instead of erase cycle**

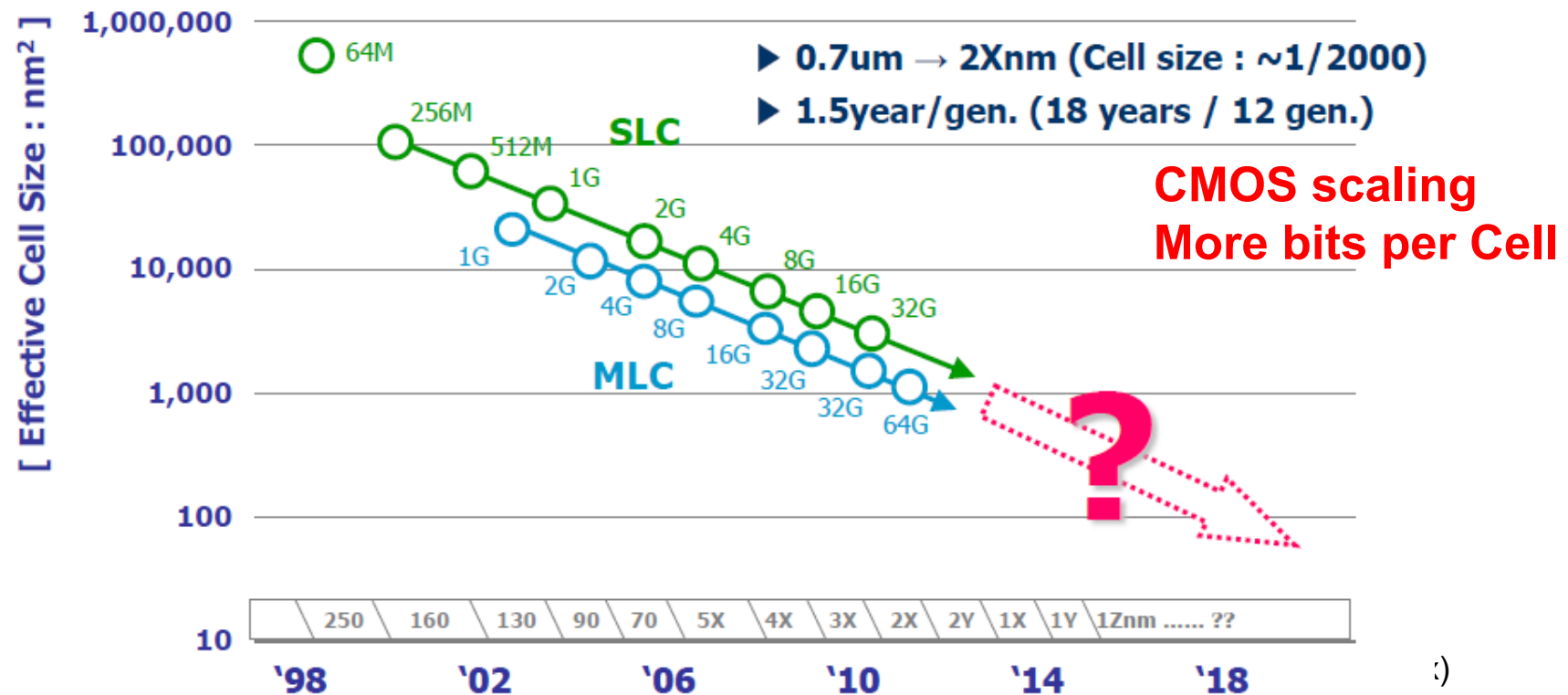


Agenda

- Flash Memory: Why and How
- Flash Translation Layer
 - Address Mapping
 - Garbage Collection
 - Wear Leveling Strategies
- **Error Correction Codes**
- Flash File Systems
- Hybrid Flash (SLC & MLC) Architecture
- Other Issues and Conclusion



Evolution of NAND Flash Memory

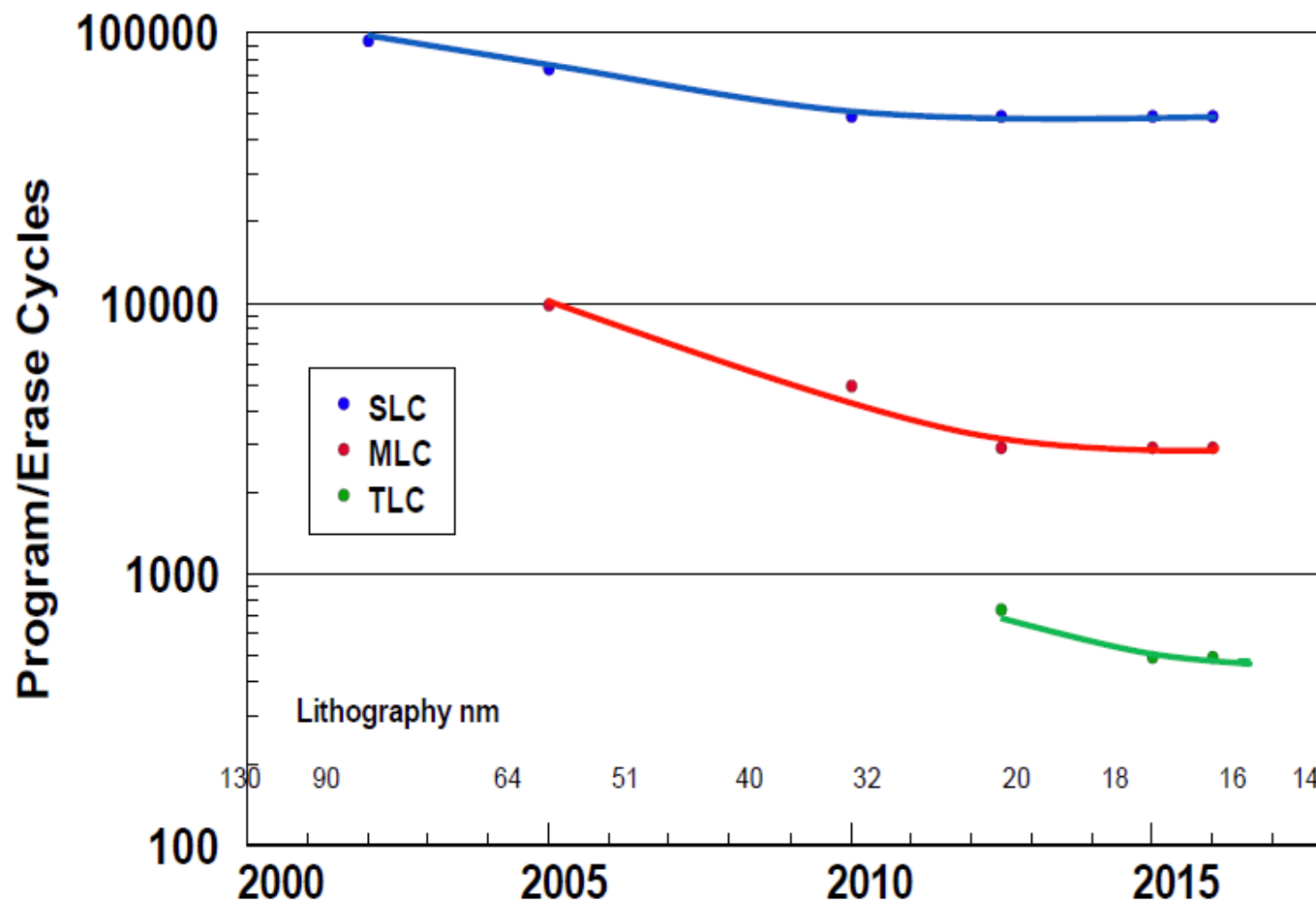


- Flash memory is widening its range of applications
 - Portable consumer devices, laptop PCs and enterprise servers



Flash Challenges: Reliability and Endurance

NAND Flash Memory Endurance Properties



■ P/E cycles
(provided)
A few thousands

■ P/E cycles
(required)
Writing
the full capacity
of the drive
10 times per day
for 5 years
(STEC)

> 50k P/E cycles



NAND Flash Memory is Increasingly Noisy

Write

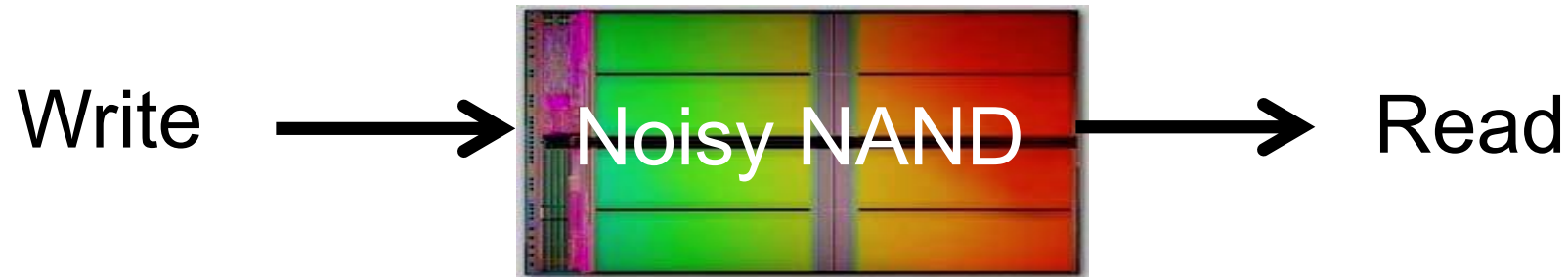


Noisy NAND



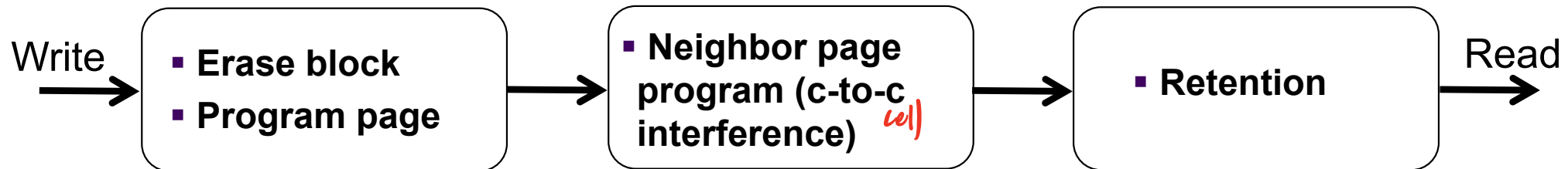
Read

NAND Flash Error Model



Experimentally characterize and model dominant errors

Cai et al., “Error Patterns in MLC NAND Flash Memory: Measurement, Characterization, and Analysis”, **DATE 2012**



Cai et al., “Threshold voltage distribution in MLC NAND Flash Memory: Characterization, Analysis, and Modeling”, **DATE 2013**

Cai et al., “Program Interference in MLC NAND Flash Memory: Characterization, Modeling, and Mitigation”, **ICCD 2013**

Cai et al., “Neighbor-Cell Assisted Error Correction in MLC NAND Flash Memories”, **SIGMETRICS 2014**

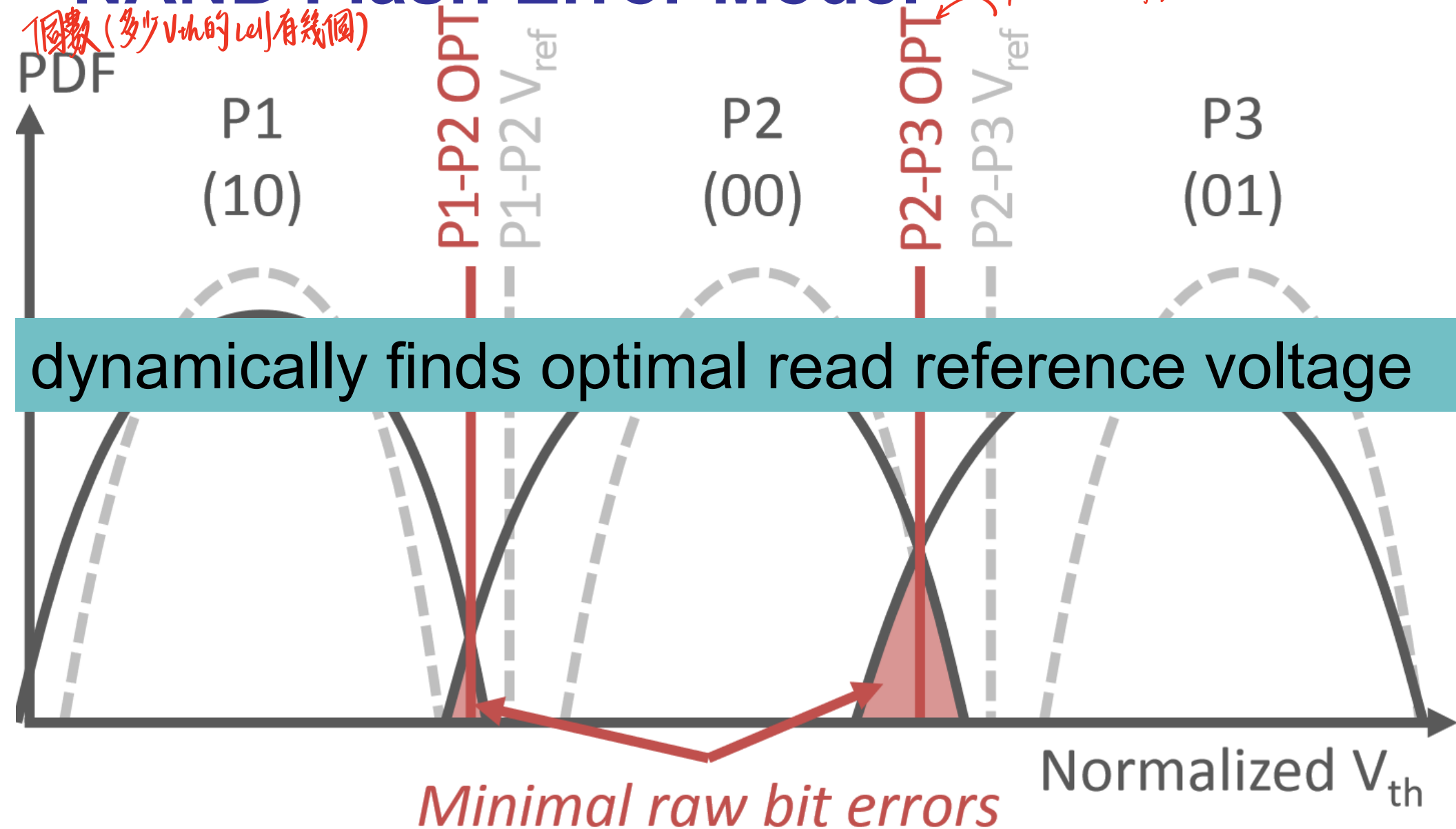
Cai et al., “Flash Correct-and-Refresh: Retention-aware error management for increased flash memory lifetime”, **ICCD 2012**

Cai et al., “Error Analysis and Retention-Aware Error Management for NAND Flash Memory”, **ITJ 2013**

NAND Flash Error Model

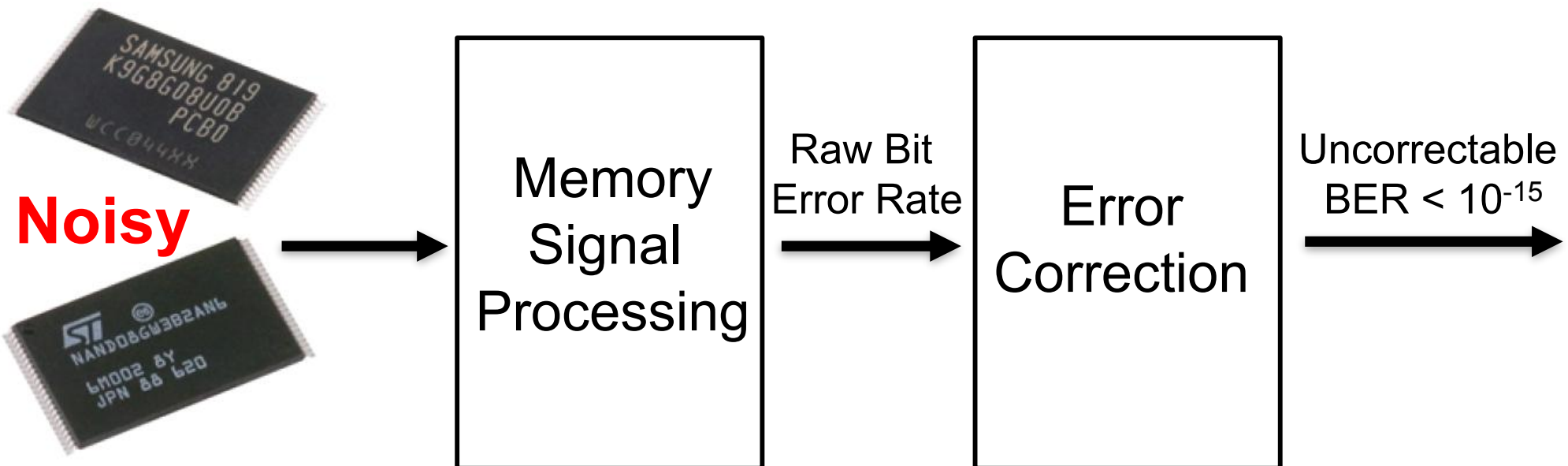
動態調整 V_{ref} , 使ECC較易找回

個數 (多少 V_{th} 的 cell 有幾個)





Future NAND Flash-based Storage Architecture





Hamming Code

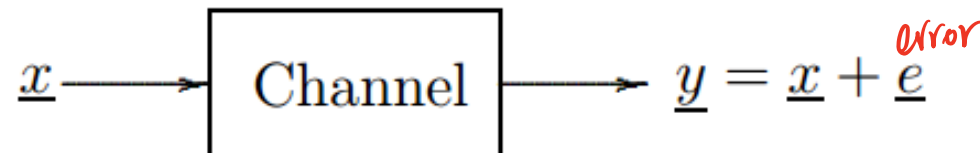
encode後7bit 原始資料4bit

- In a (7, 4) Hamming code, the parity equations are determined as follow:

$$\underline{H}\underline{x}^T = 0$$

$$\begin{cases} x_1 + x_2 + x_3 + x_5 = 0 \\ x_1 + x_2 + x_4 + x_6 = 0 \\ x_1 + x_3 + x_4 + x_7 = 0 \end{cases} \quad H = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$x_i \in \{0, 1\}$$

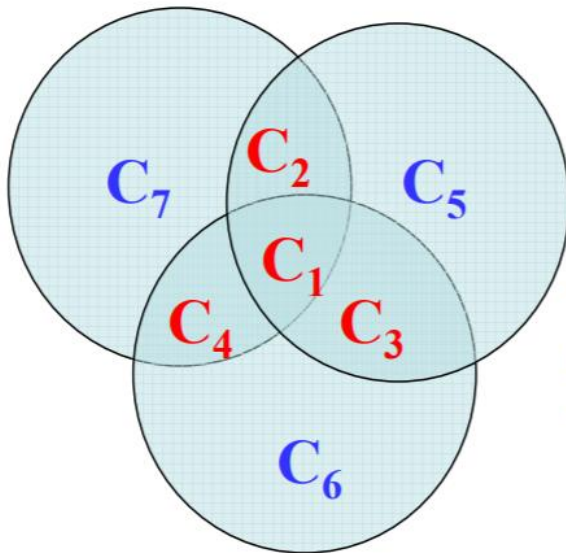


$\underline{x}$: Encoded code word $\underline{y}$: Noisy code word $\underline{e}$: Noise



Hamming Code

- Message digits: $C_1 C_2 C_3 C_4$
- Code Word: $C_1 C_2 C_3 C_4 C_5 C_6 C_7$
- Parity Check Equations:



$$C_1 \oplus C_2 \oplus C_3 \oplus C_5 = 0$$

$$C_1 \oplus C_3 \oplus C_4 \oplus C_6 = 0$$

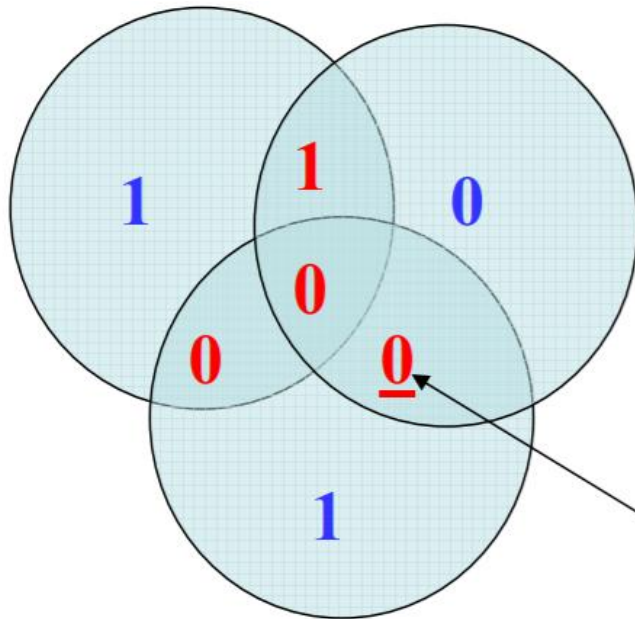
$$C_1 \oplus C_2 \oplus C_4 \oplus C_7 = 0$$

There is an **even** number
of 1's in each circle.



Hamming Code

- Transmitted code word: 0 1 1 0 0 1 1
- Received block with one error in a message bit: 0 1 **0** 0 **0** 1 1



$$C_1 \oplus C_2 \oplus C_3 \oplus C_5 = 0$$

$$C_1 \oplus C_3 \oplus C_4 \oplus C_6 = 0$$

$$C_1 \oplus C_2 \oplus C_4 \oplus C_7 = 0$$

Error is in 3rd digit

Hamming Code Decoding

- For Example:

$$y = (1101000) = \overset{x}{(1101010)} + \overset{e}{(0000010)}$$

$$\underline{s}^T = H\underline{y}^T$$

$$= \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$= \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \text{Indicate the 2nd bit error}$$



BCH (Bose, Ray-Chaudhuri and Hocquenghem)

- Correct Multiple Errors

$$(15, 11) \quad H = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

- Adding extra 4 bits properly can let the code correct two-bit errors.

$$H_{SEC} = (\underline{\alpha_1 \alpha_2} \quad H_{DEC} = \begin{pmatrix} \alpha_1 & \alpha_2 & \alpha_3 & \dots & \alpha_{15} \\ \beta_1 & \beta_2 & \beta_3 & \dots & \beta_{15} \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \dots, \text{etc.}$$



BCH (Bose, Ray-Chaudhuri and Hocquenghem)

×	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2	0	2	4	6	8	A	C	E	3	1	7	5	B	9	F	D
3	0	3	6	5	C	F	A	9	B	8	D	E	7	4	1	2
4	0	4	8	C	3	7	B	F	6	2	E	A	5	1	D	9
5	0	5	A	F	7	2	D	8	E	B	4	1	9	C	3	6
6	0	6	C	A	B	D	7	1	5	3	9	F	E	8	2	4
7	0	7	E	9	F	8	1	6	D	A	3	4	2	5	C	B
8	0	8	3	B	6	E	5	D	C	4	F	7	A	2	9	1
9	0	9	1	8	2	B	3	A	4	D	5	C	6	F	7	E
A	0	A	7	D	E	4	9	3	F	5	8	2	1	B	6	C
B	0	B	5	E	A	1	F	4	7	C	2	9	D	6	8	3
C	0	C	B	7	5	9	E	2	A	6	1	D	F	3	4	8
D	0	D	9	4	1	C	8	5	2	F	B	6	3	E	A	7
E	0	E	F	1	D	3	2	C	9	7	6	8	4	A	B	5
F	0	F	D	2	9	6	4	B	1	E	C	3	8	7	5	A

- Adding extra 4 bits properly can let the code correct two-bit errors.

$$H_{DEC} = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & A & B & C & D & E & F \\ 1 & 8 & F & C & A & 1 & 1 & A & F & F & C & 8 & A & 8 & C \end{pmatrix}$$



LDPC (Low-density parity-check codes)

- The name comes from their parity-check matrix which contains only a few 1's in comparison to the amount of 0's.

$C_1 C_2 C_3 C_4 C_5 C_6 C_7 C_8 C_9 C_{10} C_{11} C_{12}$

0	0	1	0	0	1	1	1	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	1	1	1	0
0	1	0	0	0	1	1	0	0	1	0	0
1	0	1	0	0	0	0	1	0	0	1	0
0	0	0	1	1	0	0	0	1	0	0	1
1	0	0	1	1	0	1	0	0	0	0	0
0	0	0	0	0	1	0	1	0	0	1	1
0	1	1	0	0	0	0	0	1	1	0	0

$$C_3 \oplus C_6 \oplus C_7 \oplus C_8 = 0$$

$$C_1 \oplus C_2 \oplus C_5 \oplus C_{12} = 0$$

$$C_4 \oplus C_9 \oplus C_{10} \oplus C_{11} = 0$$

$$C_2 \oplus C_6 \oplus C_7 \oplus C_{10} = 0$$

$$C_1 \oplus C_3 \oplus C_8 \oplus C_{11} = 0$$

$$C_4 \oplus C_5 \oplus C_9 \oplus C_{12} = 0$$

$$C_1 \oplus C_4 \oplus C_5 \oplus C_7 = 0$$

$$C_6 \oplus C_8 \oplus C_{11} \oplus C_{12} = 0$$

$$C_2 \oplus C_3 \oplus C_9 \oplus C_{10} = 0$$



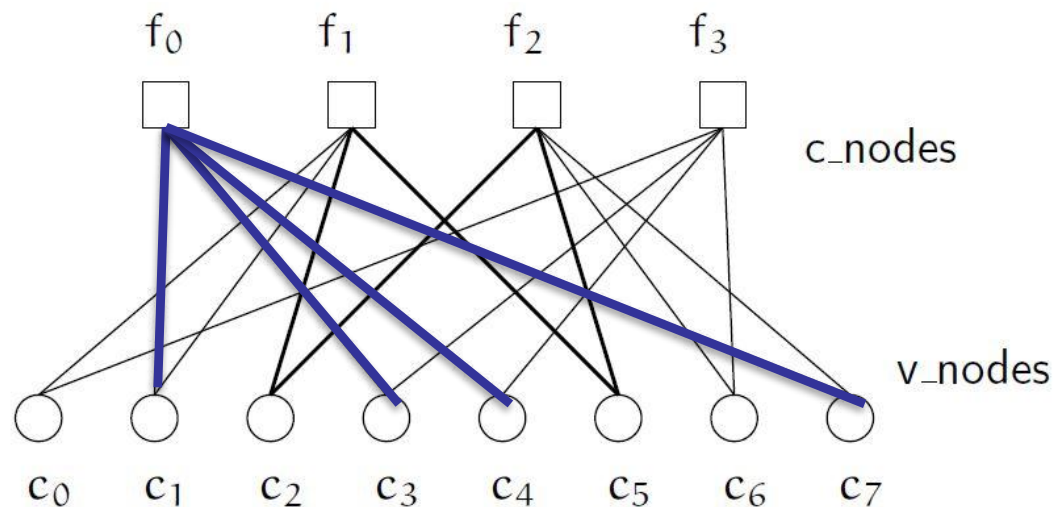
LDPC (Low-density parity-check codes)

- The name comes from their parity-check matrix which contains only a few 1's in comparison to the amount of 0's.
- Dimension $n \times m$ for a $(8, 4)$ code

$$H = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \end{bmatrix} \quad (1)$$

$$c = Gp$$

$$z = Hy$$



m : check nodes

n : variable nodes



LDPC Code and Soft Sensing

- Hard-decision decoding (BCH, LDPC)
 - Inputs are in binary form
 - Simple hardware implementation
 - Relatively weak error correction capability
- Soft-decision decoding (LDPC)
 - Inputs are quantized to integers
 - Complicated hardware design
 - Strong error correction capability
- For LDPC code, its error correction strength strongly depends on the accuracy of the input information which presents the probability information of the data stored in NAND flash memory



LDPC Code and Soft Sensing



1,0,1,1,0,1,0,0,0,1,1,0 (Hard-decision)

-4,+2,-1,-1,+3,-2,+2,+3,+2,-4,-3,+1 (Soft-decision)

$$\log \left(\frac{P(y_i | x_i = 0)}{P(y_i | x_i = 1)} \right)$$

Log-likelihood ratio (LLR)

Positive: 0
Negative: 1
Magnitude presents the probability

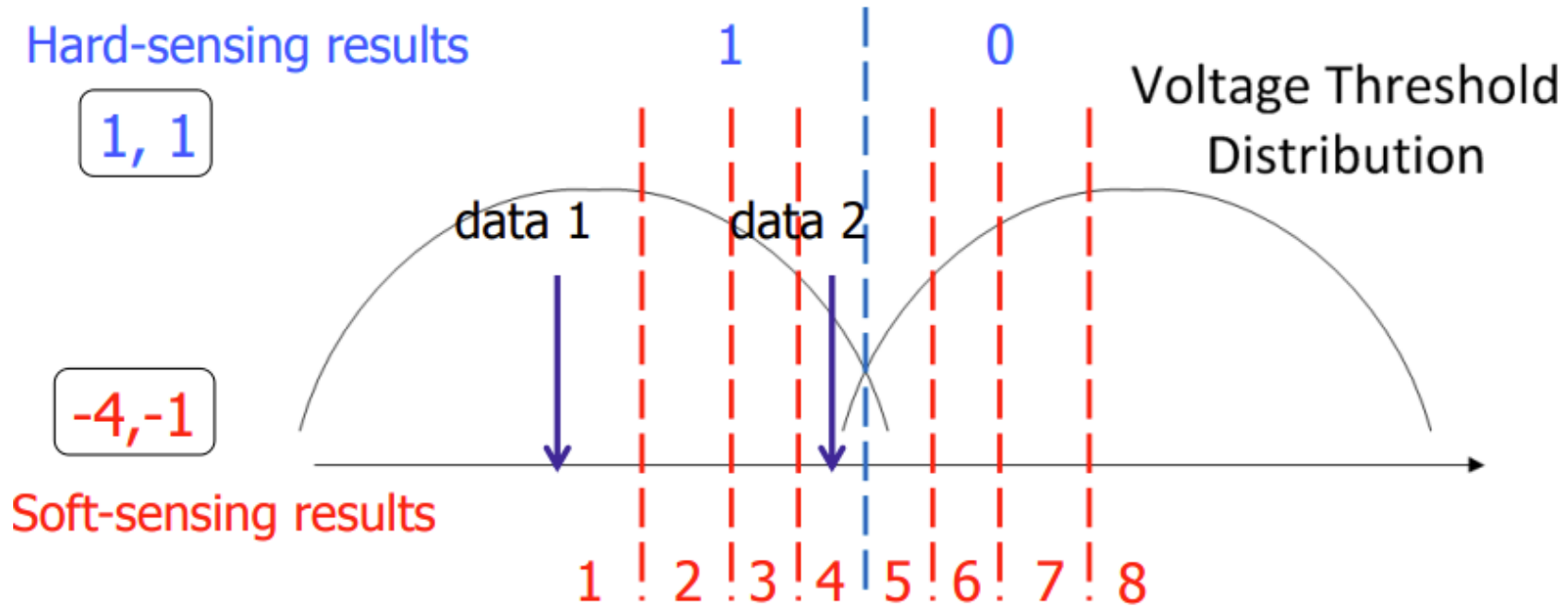
決定是0是1機率大小

Input Init.

LDPC Decoder

LDPC Code and Soft Sensing

Hard sensing vs. Soft sensing

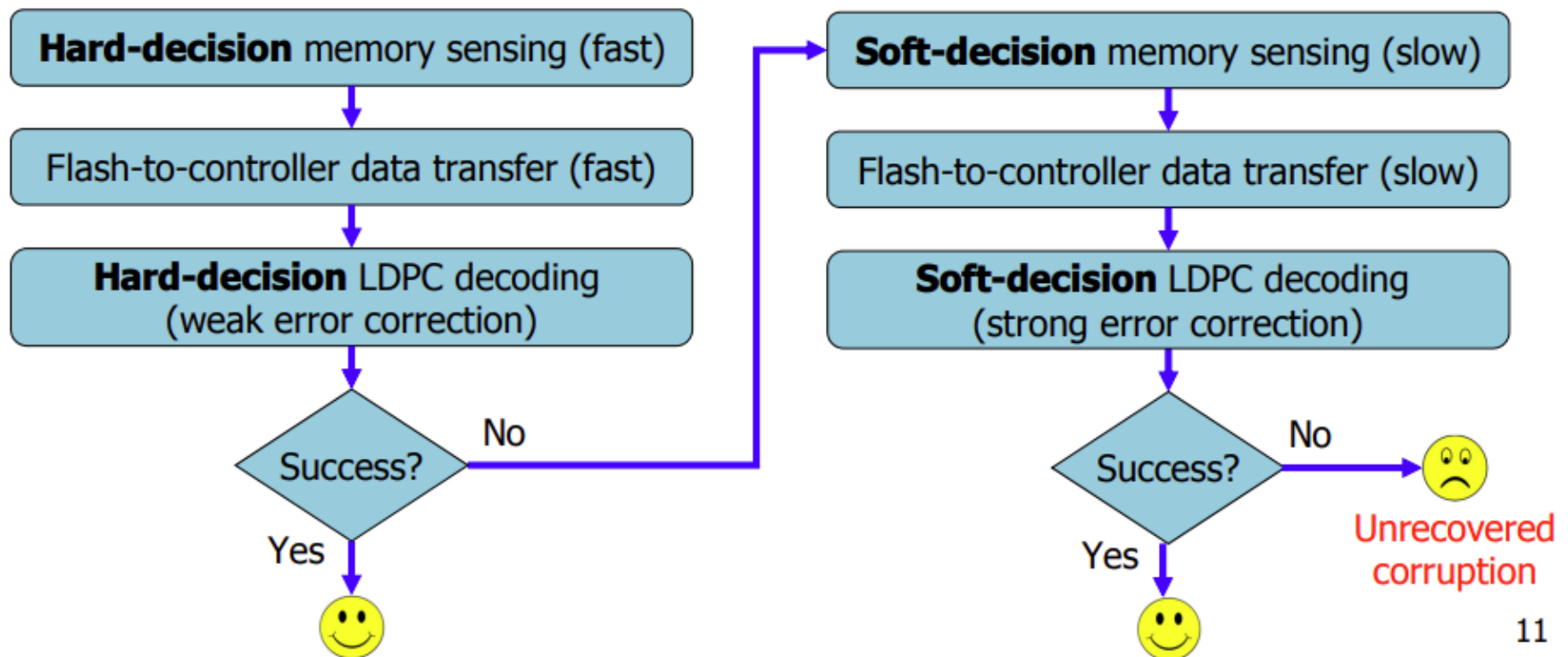


Quantization	1 bit	2 bits	3 bits
Times of Sensing	1	3	7

Soft-decision decoding suffers from a huge latency

LDPC Code and Soft Sensing

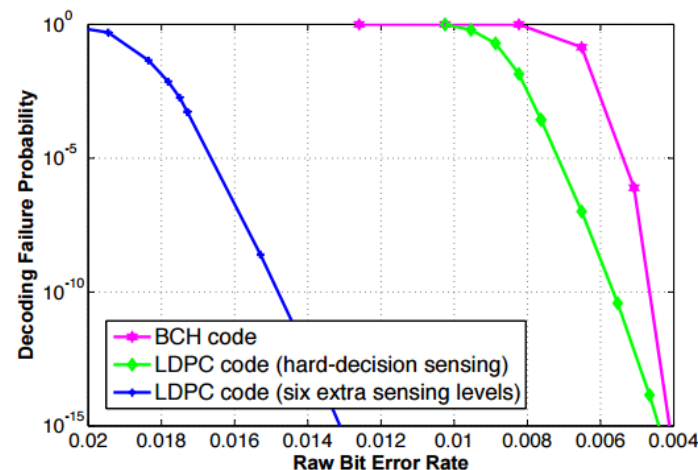
- Large latency of soft-decision sensing and corresponding data transfer to SSD controller is destructive to read performance
- A basic two-step read strategy
 - Hard-decision decoding still works in most of the times





Read Performance Bottleneck

- NAND flash chip I/O bandwidth: 200 MB/s (ONFI 2.1)
 - It takes **20us** to transfer 4KB page data.
- **Seven-level soft-decision sensing LDPC**: it takes about **80μs** to transfer sensing results of 4KB data to the controller.
- LDPC decoding throughput: 4Gbps
 - It takes **8us** to decode 4KB page data.
- **Soft-decision memory sensing and corresponding data transfer completely dominate the overall read latency.**





Summary

- ECC are used to protect data.
- The most popular ECCs are BCH and LDPC
 - Soft-sensing performance is a big performance issue
- Improve LDPC or any other alternatives (e.g., polar code?)



Agenda

- Flash Memory: Why and How
- Flash Translation Layer
 - Address Mapping
 - Garbage Collection
 - Wear Leveling Strategies
- Error Correction Codes
- **Flash File Systems** 参考
- Hybrid Flash (SLC & MLC) Architecture
- Other Issues



Outline

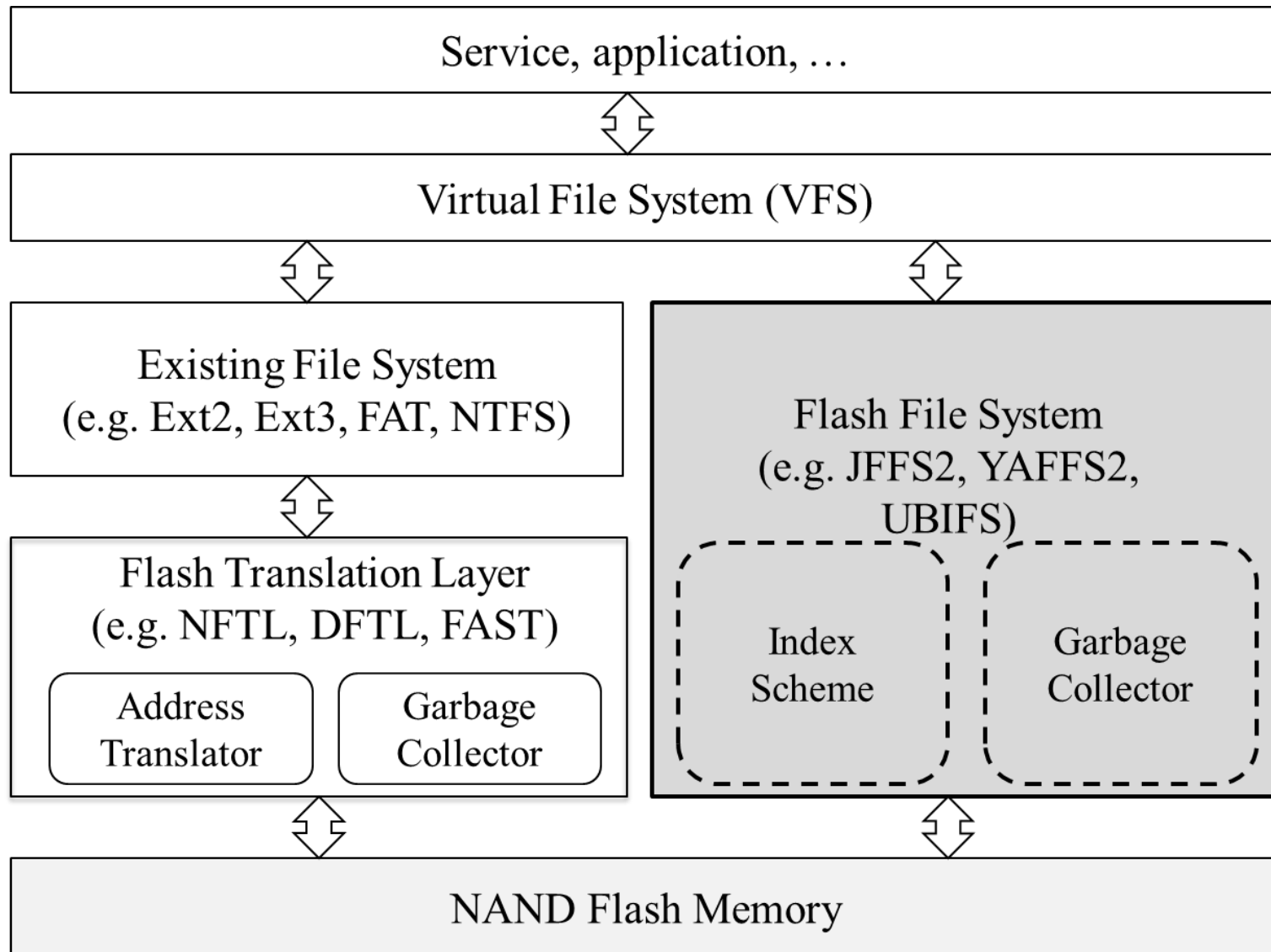
- Introduction
- Log-Structured File System
- Existing Work
- Summary



Introduction

- NAND flash memory characteristics
 - Erase-before-write
 - Limited number of writes
- Two major approaches are proposed to overcome these limitations
 - Flash translation layer (FTL)
 - e.g. FTL, NFTL, DFTL
 - Native file system
 - e.g. JFFS, YAFFS, UBIFS

Flash-based Storage System





Comparison

- Similarity
 - Mapping management
 - Garbage collection
 - Wear leveling
- Main difference between FTL and flash file system
 - **Viewpoint of file mapping**



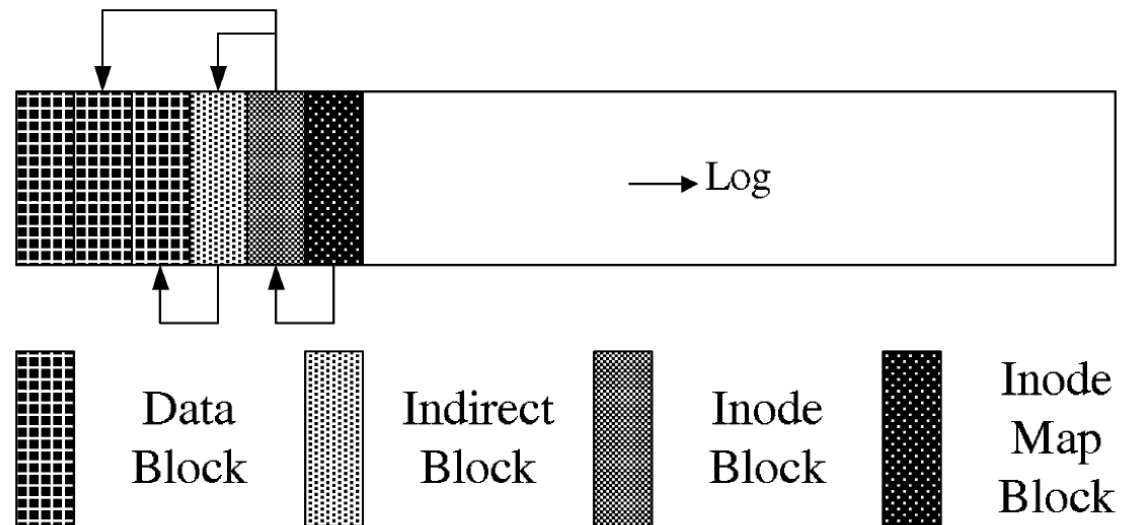
Outline

- Introduction
- Log-Structured File System
- Existing Work
- Summary



Log-Structured File System (1/3)

- Objective
 - Utilize a disk's raw write bandwidth efficiently
 - Provide fast crash recovery
- Basic concept
 - Store all file system data in a sequential structure called a **log**
 - Buffer all writing operations in memory and flush them to a log sequentially at once





Log-Structured File System (2/3)

- Data structure used in LFS
 - inode, indirect block and directory block
 - Data block and directory block
 - LFS uses “**inode map**” to maintain the current locations of all inodes
- Features
 - Log structure allows crash recovery
 - Increasing write performance by eliminating almost all disk seeks



Log-Structured File System (3/3)

- Design issues
 - How to maintain large free extents for writing new logs?
 - How to improve the overhead caused by the “segment clean” operation?
 - How to avoid the low-locality behavior happening?



Outline

- Introduction
- Log-Structured File System
- Existing Work
- Summary



JFFS & JFFS2 (1/3)

- Journaling Flash File System
 - JFFS2 is a log-structured flash file system designed for small-scale embedded system
 - Design for NOR flash, and extend to NAND flash later
 - Use **variable-sized unit called a node** to store file system metadata and data



國立成功大學
National Cheng Kung University

JFFS & JFFS2 (3/3)

- Challenges on JFFS2
 - Memory footprint problem
 - Mounting time



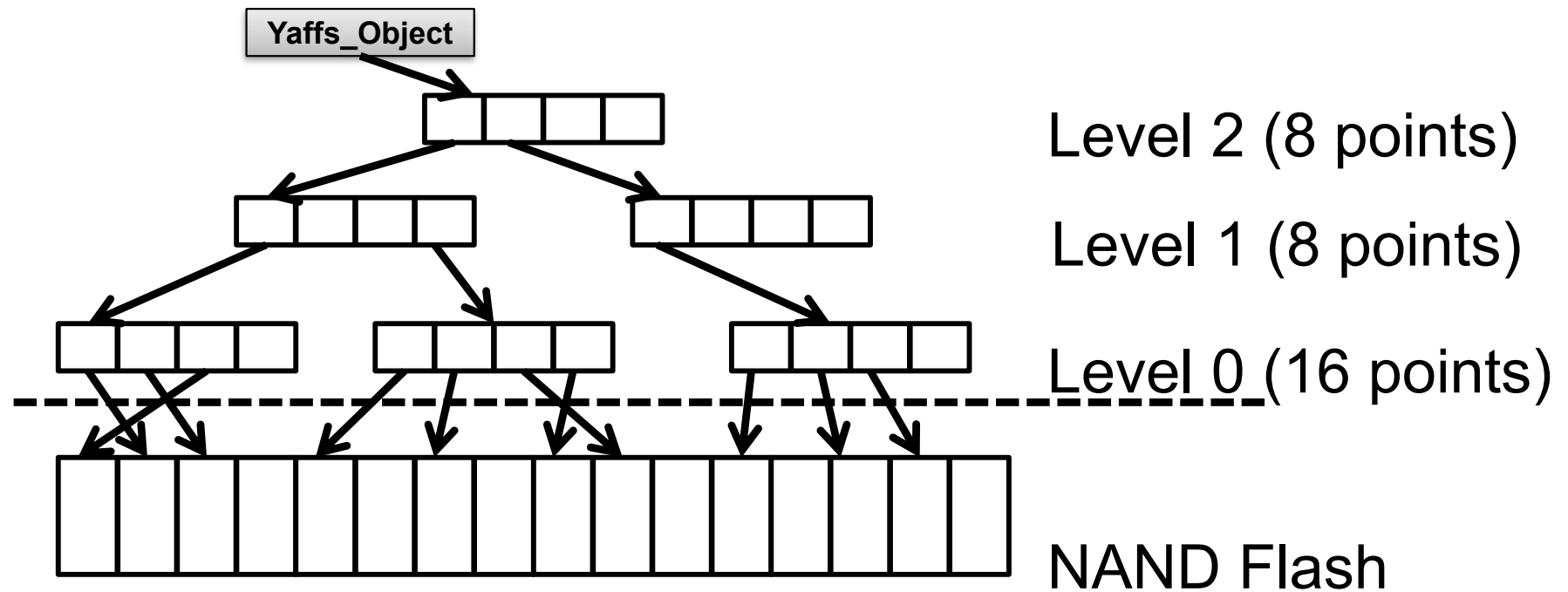
YAFFS & YAFFS2 (1/2)

- Yet Another Flash File System
 - YAFFS is another variant of a **log-structured file system** designed for NAND flash memory
 - The structure of YAFFS is similar to that of the original version of JFFS
- Main difference between JFFS and YAFFS
 - Node header information is moved to NAND's spare area, named **chunk**
 - YAFFS scans all **spare area** only



YAFFS & YAFFS2 (1/2)

- Tnode tree



- Issues

- Mounting time
- RAM consumption



Outline

- Introduction
- Log-Structured File System
- Existing Work
- Summary



Summary

- LFS is an appropriate approach for the designing of native file system
 - The behavior is fitted to the characteristic of flash memory
 - JFFS and YAFFS are the well-known examples
- With the growing of the NAND flash capacity, the existing works perform poor in
 - scalability
 - performance



Agenda

- Flash Memory: Why and How
- Flash Translation Layer
 - Address Mapping
 - Garbage Collection
 - Wear Leveling Strategies
- Error Correction Codes
- Flash File Systems
- **Hybrid Flash (SLC & MLC) Architecture**
- Other Issues



Outline

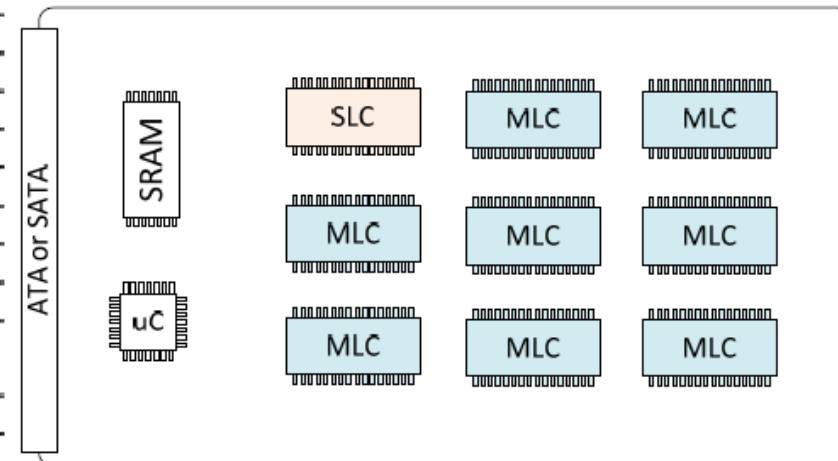
- **Introduction**
- Management of Heterogeneous Hybrid Architecture
- Introduction to Homogeneous Hybrid Architecture
 - Program Method for TLC (MLC_{x3}) NAND
 - Hard Partitions and Soft Partitions
- Adaptive Management
 - Phoenix: Reviving MLC Blocks as SLC to Extend NAND Flash Devices Lifetime
 - Utilization-aware Self-tuning
- Summary



Hybrid Architecture

- The hybrid flash architectures for SSDs, which combines MLC flash and SLC flash, are to **complement their drawbacks with each other's advantages**.
 - Heterogeneous hybrid architecture
 - It combines an SLC flash chip with many MLC flash chips
 - Homogeneous hybrid architecture
 - Managing MLC as SLC**: An MLC flash chip can also be used in the SLC mode

Type	SLC	MLC
Product No	K9F2G08UXA	K9GAG08B0M
Capacity	256M*8 bits	2G*8 bits
Page Size / Block Size	2K+64 / 128K+4K bytes	4K+128 / 512K+16K bytes
Page Read	25 us	60 us
Page Write	200 us	800 us
Block Erase	1.5 ms	1.5 ms
Block Endurance	100K cycles	10K cycles
Operating Characteristics of Read, Write, and Erase	3.3V / 15mA	3.3V / 15mA
Standby Power Dissipation	3.3V / 10uA	3.3V / 10uA
Cost per GB (mid 2007)	\$10.37	\$6.81





Outline

- Introduction
- **Management of Heterogeneous Hybrid Architecture**
- Introduction to Homogeneous Hybrid Architecture
 - Program Method for TLC (MLC_{x3}) NAND
 - Hard Partitions and Soft Partitions
- Adaptive Management
 - Phoenix: Reviving MLC Blocks as SLC to Extend NAND Flash Devices Lifetime
 - Utilization-aware Self-tuning
- Summary



Management of Heterogeneous Hybrid Architecture [1]

- Data placement
 - Data of what attributes should be written to SLC?
- Data-attribute identification
 - How to do data attribute identification?
- Data migration
 - How to phase out data from SLC?
- Wearing balance (Lifetime issue)
 - Unbalanced wear causes the device to fail before most of its cells deteriorate above their maximum wear level
 - How to balance the wear level of SLC and MLC?



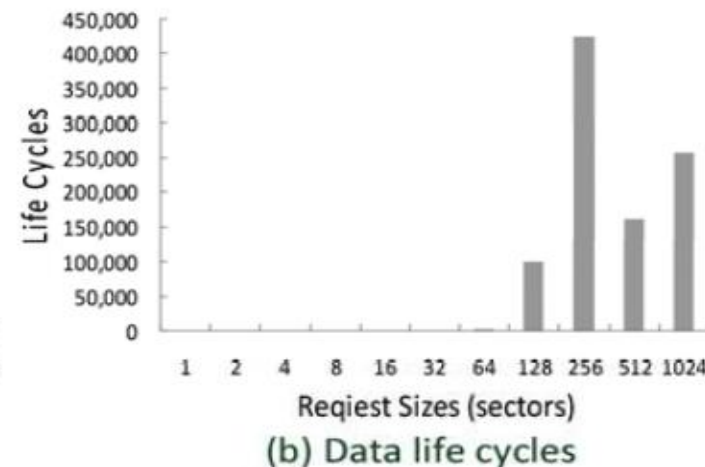
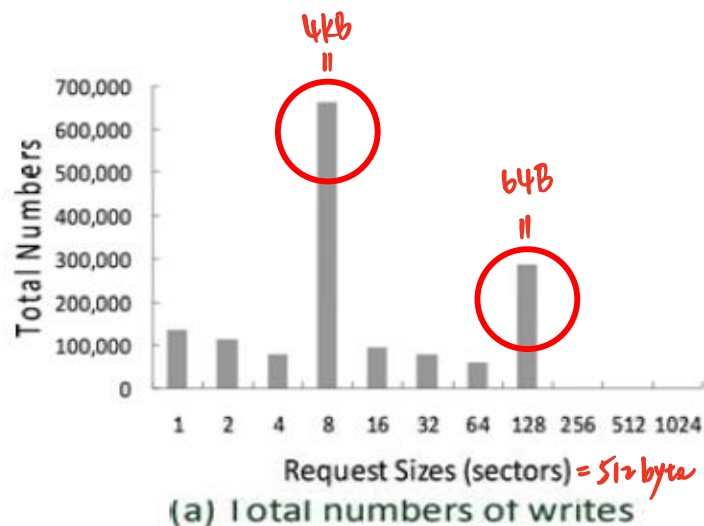
Data Placement

- Small writes are hot data that can be serviced by SLC flash
- Why?
 - SLC flash is much faster than MLC flash in terms of writes
 - SLC-flash blocks are more durable than MLC-flash blocks
 - The page size of SLC flash is finer than that of MLC flash



Data-Attribute Identification

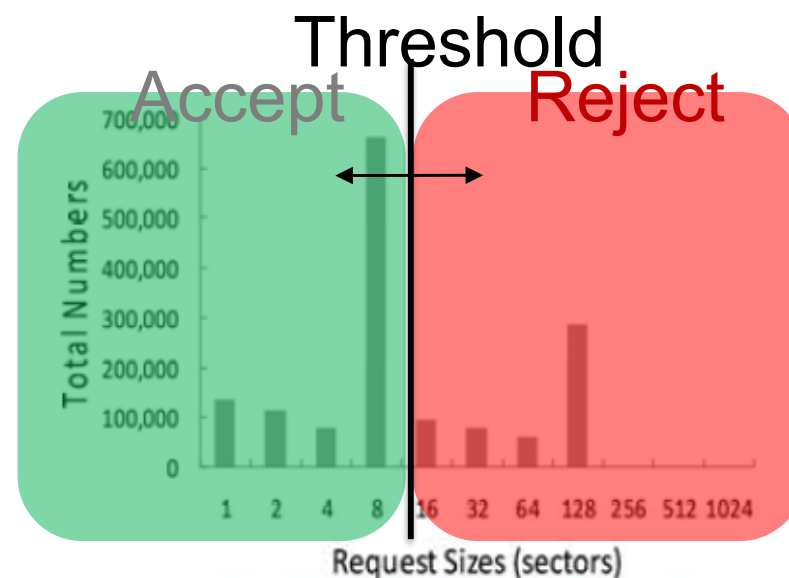
- By analyzing realistic access patterns, this paper found that
 - Many small writes go to hot data
 - Many large writes go to cold data
 - The distribution of write sizes is **bimodal**





Data-Attribute Identification – Hot-data Filter

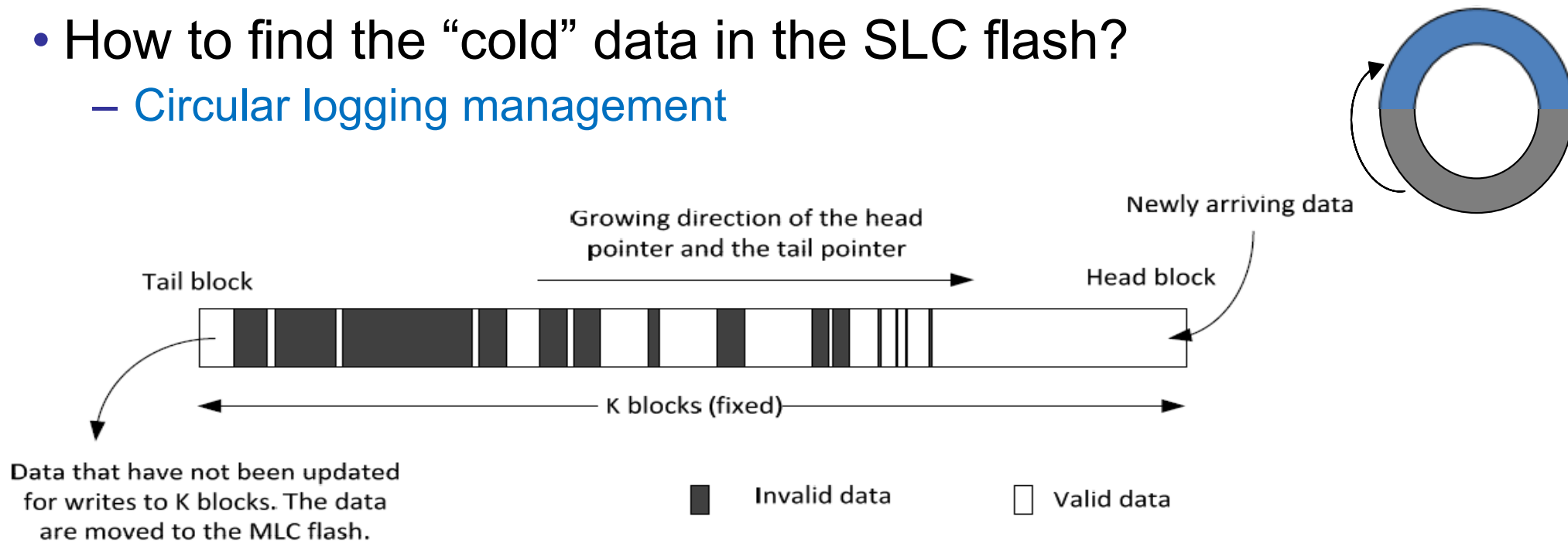
- How to define “small” size?
 - Periodically invoking a **2-means clustering algorithm** to identify the two peaks and redefine the threshold of small writes
- The hot-data filter simply accepts small writes and rejects the others, it doesn't need to keep track of data attributes





Data Migration

- Hot data might become cold
 - To leave free space for hot data, cold data should be phased out from SLC flash to MLC flash.
- It is possible to combine the overheads of data migration and garbage collection
- How to find the “cold” data in the SLC flash?
 - Circular logging management



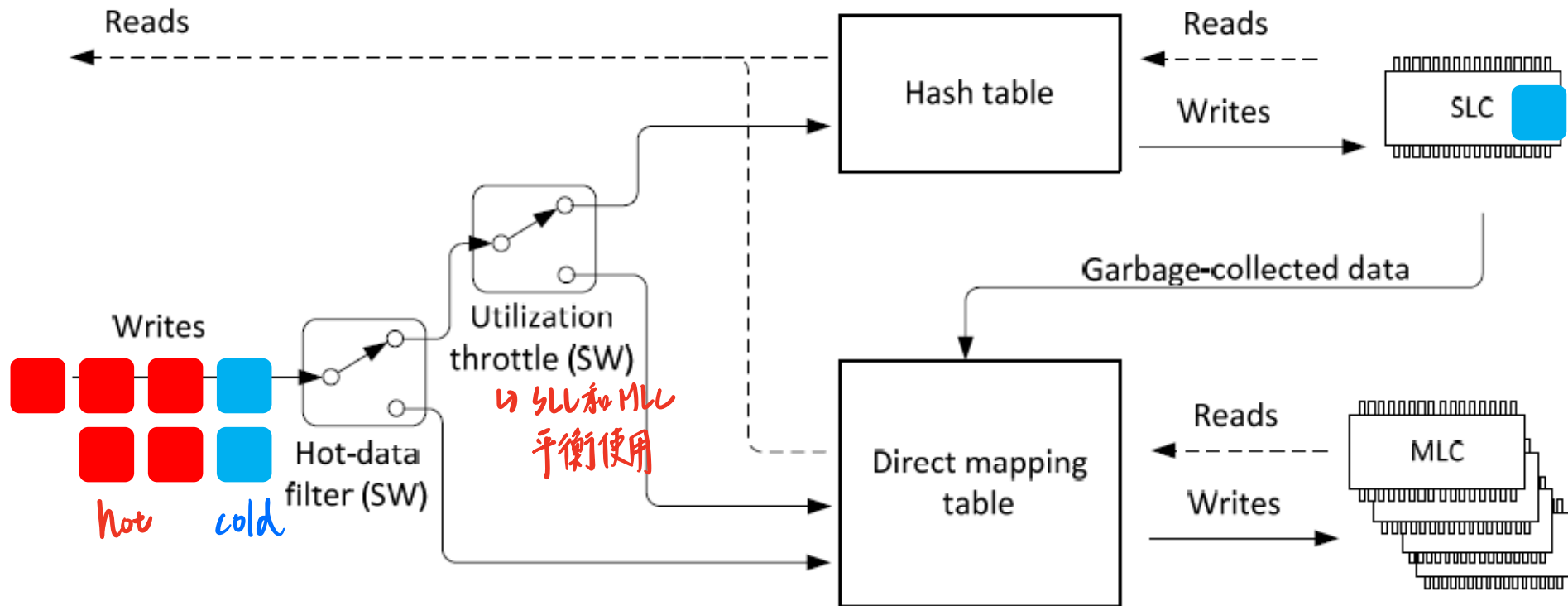


Wearing Balance (Lifetime Issue)

- SLC flash and MLC flash have different block endurances
- It is possible that the SLC flash reaches its lifetime before the MLC flash, and vice versa
- Utilization throttle
 - Reduce the write traffic to the SLC flash so that the wearing of the SLC flash can gradually be quenched



Architecture of heterogeneous hybrid SSDs – The Whole Picture



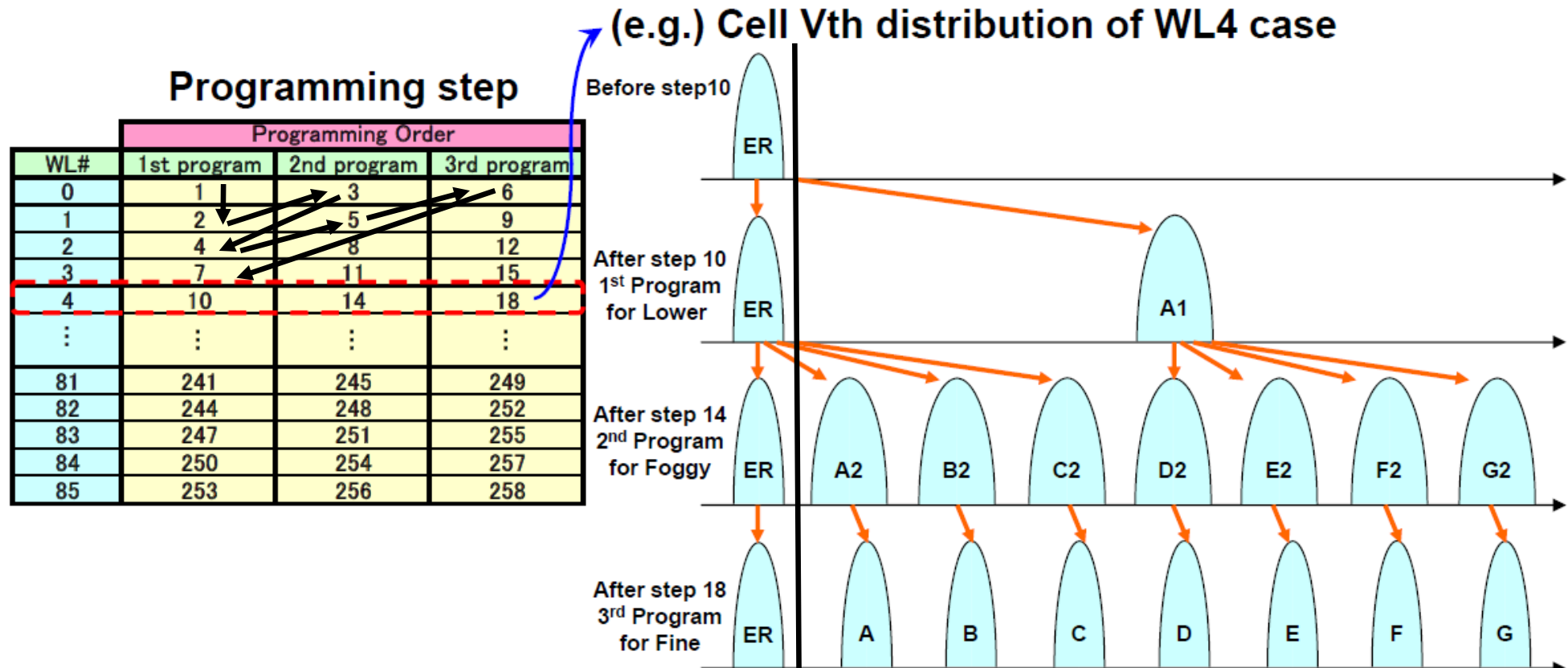


Outline

- Introduction
- Management of Heterogeneous Hybrid Architecture
- **Introduction to Homogeneous Hybrid Architecture**
 - **Program Method for TLC (MLC_{x3}) NAND**
 - Hard Partitions and Soft Partitions
- Adaptive Management
 - Phoenix: Reviving MLC Blocks as SLC to Extend NAND Flash Devices Lifetime
 - Utilization-aware Self-tuning
- Summary

Program Method for TLC NAND [2]

- Foggy & Fine program reduces the cell coupling effect (disturbance) and improves read and program performance



Note : Lower/Middle/Upper data of the WL can be read out only if Fine program is completed.



Outline

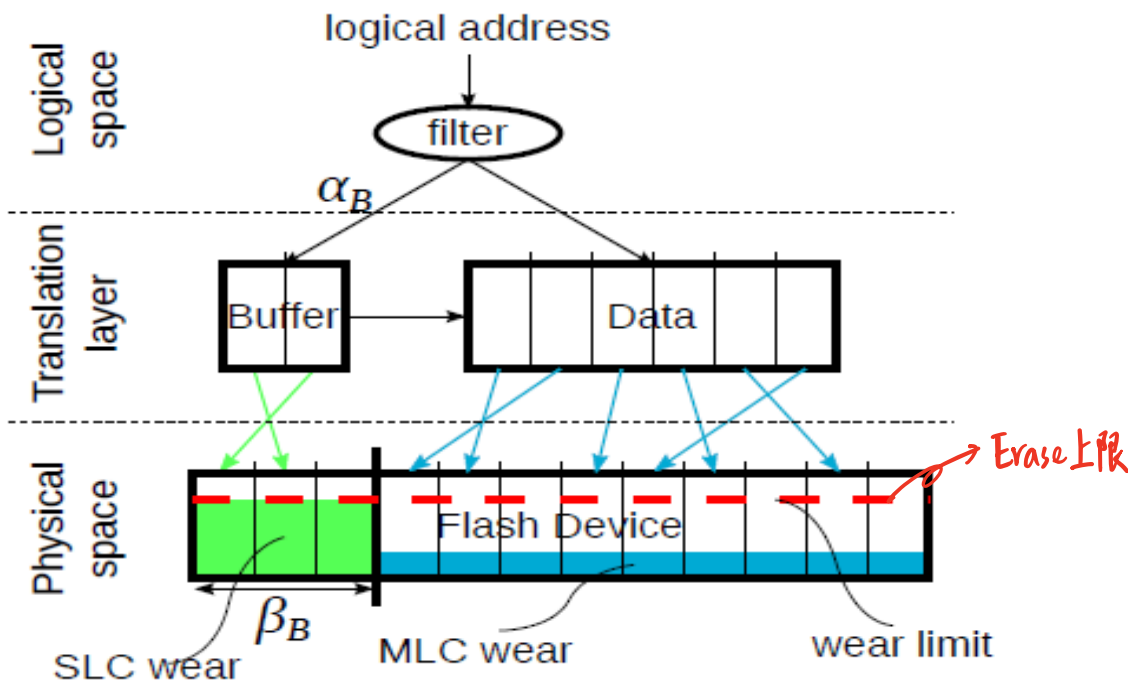
- Introduction
- Management of Heterogeneous Hybrid Architecture
- **Introduction to Homogeneous Hybrid Architecture**
 - Program Method for TLC (MLC_{x3}) NAND
 - **Hard Partitions and Soft Partitions**
- Adaptive Management
 - Phoenix: Reviving MLC Blocks as SLC to Extend NAND Flash Devices Lifetime
 - Utilization-aware Self-tuning
- Summary



Homogeneous Hybrid SSDs Architecture – Hard Partitioning [3]

固定MLC當SLC使用

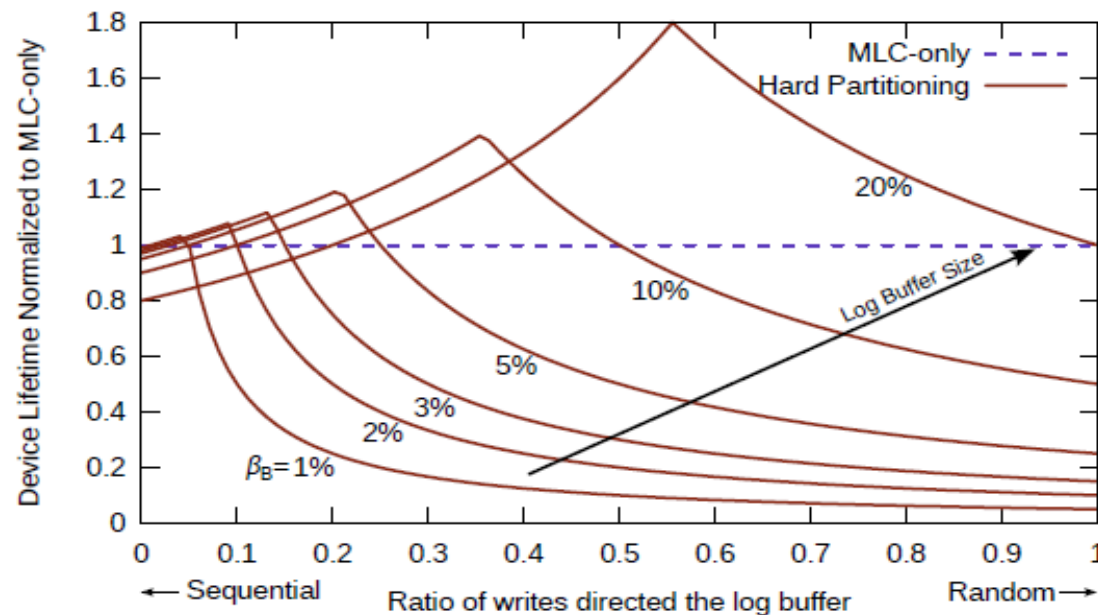
- Faster MLC: Managing MLC as SLC
 - An MLC-flash chip can also be used in the SLC mode
 - SSDs are made of only one type of MLC flash chips
- Hard partitions





Lifetime Issue of Hard Partitioning

- **Unbalanced wear** causes the device to fail well before most of its cells deteriorate above their maximum wear level (the large data partition is still healthy).
- The previous utilization throttle scheme causes the performance lost in most of the cases.

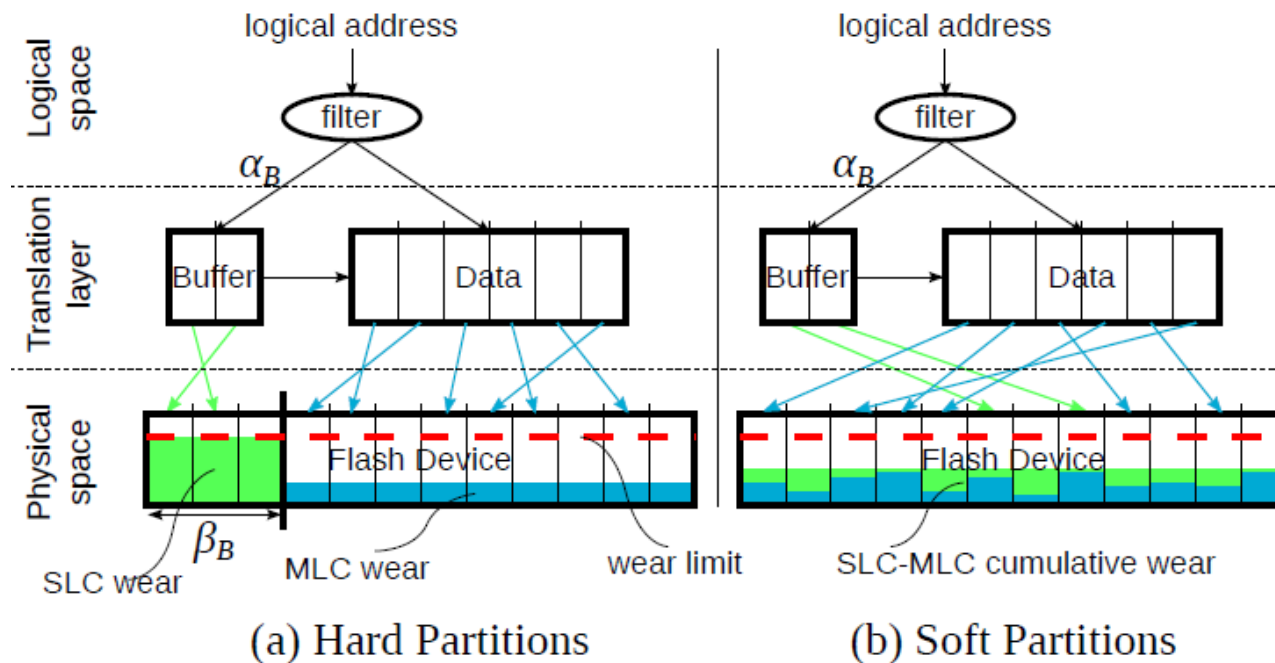




Homogeneous Hybrid SSDs Architecture – Soft Partitioning

以輪流來當SLC

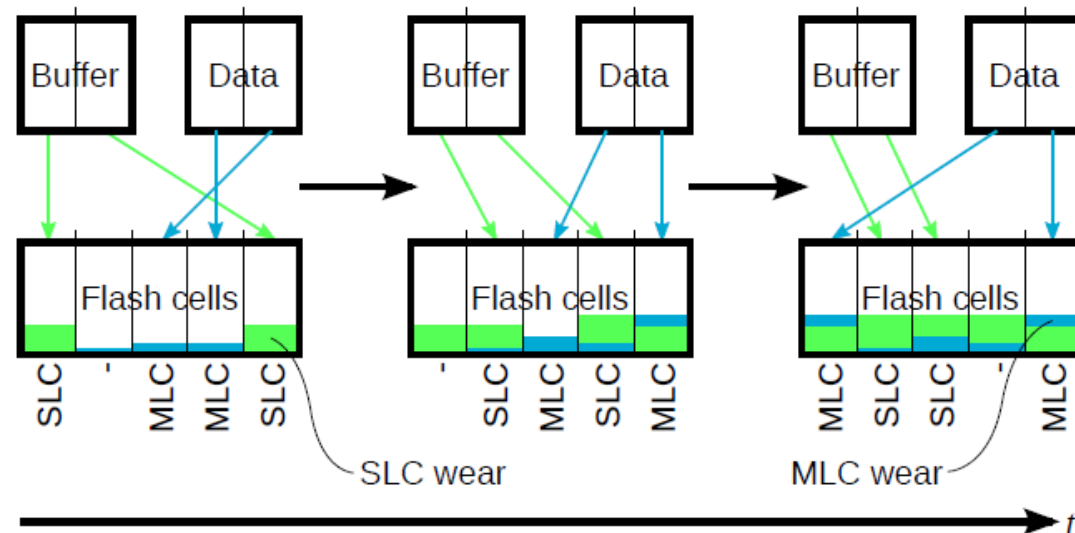
- A soft SLC-MLC partitioning breaks the rigidity of hard partitioning by changing the physical allocation of the buffer depending on the wear of the device





Software-Controlled Log Buffer

- A block is initially allocated to the **buffer**, thus managed as **SLC**, then is freed from both partitions to be later allocated to the **data partition**, managed as **MLC**.
- This soft partitioning scheme balances the device wear and obtains high performance without compromising device lifetime.





Outline

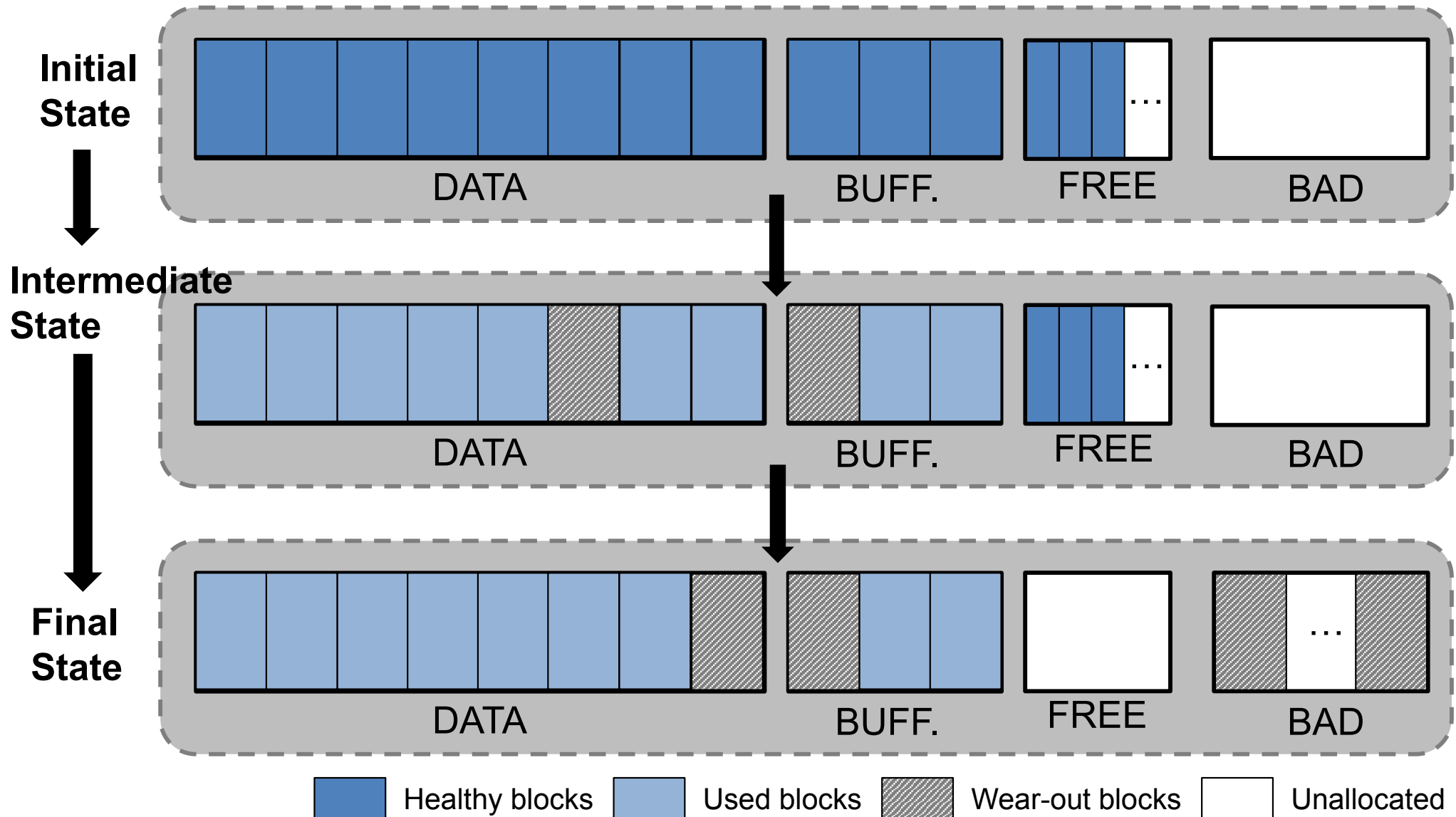
- Introduction
- Management of Heterogeneous Hybrid Architecture
- Introduction to Homogeneous Hybrid Architecture
 - Program Method for TLC (MLC_{x3}) NAND
 - Hard Partitions and Soft Partitions
- **Adaptive Management**
 - **Phoenix: Reviving MLC Blocks as SLC to Extend NAND Flash Devices Lifetime**
 - Utilization-aware Self-tuning
- Summary



Phoenix Scheme: Revived Blocks [4]

- An important observation:
 - When a cell becomes unreliable for MLC use, it can still be used in SLC-mode for a significant amount of extra P/E cycles in the homogeneous hybrid SSD architecture
↳ MLC復活成SLC
- A novel scheme to keep in using worn-out MLC blocks as SLC blocks (**revived blocks**)
- By “**reviving**” these blocks, which would be normally considered as useless, it mitigate the degradation in lifetime of MLC flash

Lifetime of the Baseline Device

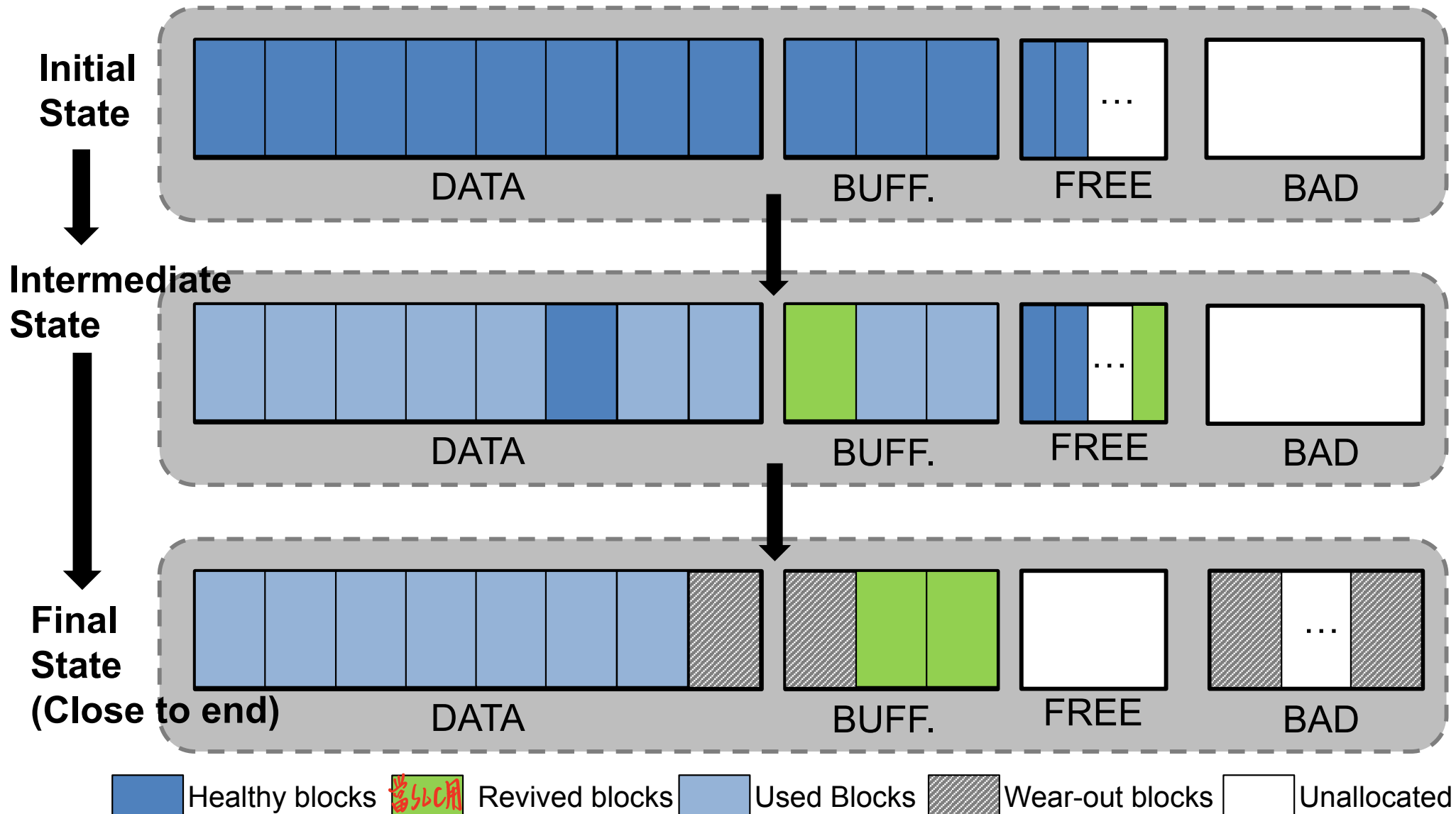




Reviving MLC blocks in SLC-mode

- Typical flash devices stop using a block whenever it permanently fails or when it becomes unreliable by showing a bit error rate too high to be handled by the implemented ECC
 - Such a device will die when running out of free blocks
- The Phoenix scheme is proposed to revive the unreliable blocks with high error rates by using in SLC-mode, and revived blocks will be allocated to the buffer partition

Lifetime with Phoenix Scheme





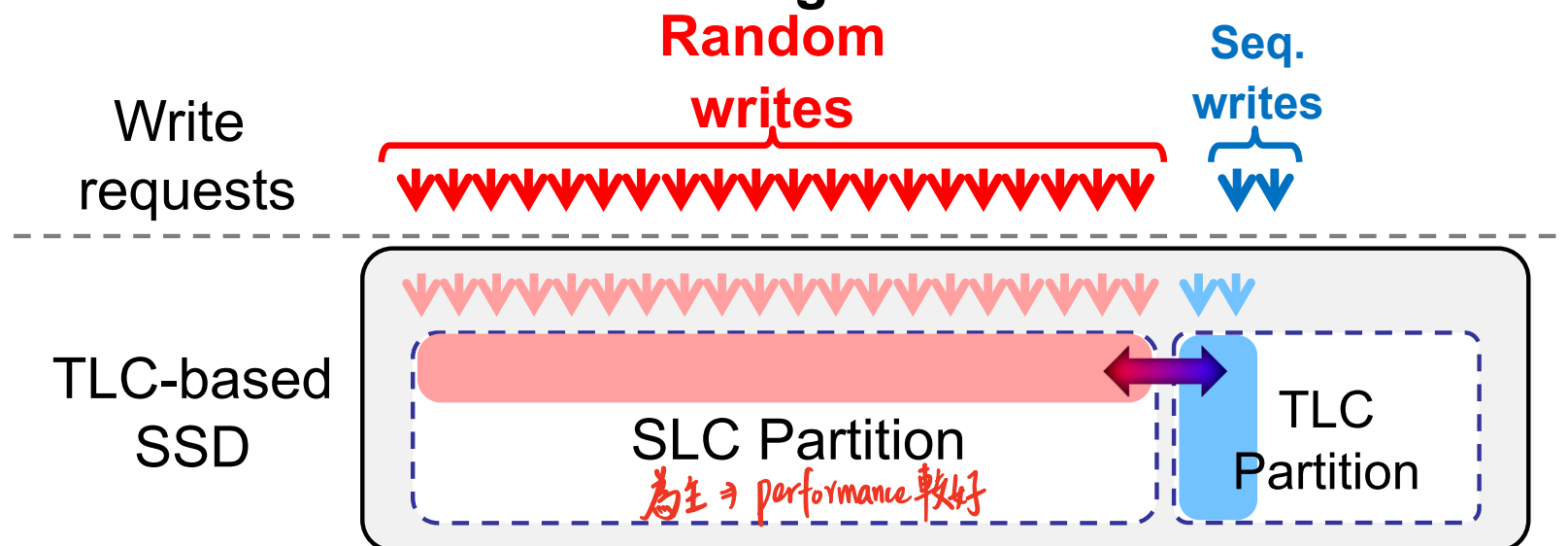
Outline

- Introduction
- Management of Heterogeneous Hybrid Architecture
- Introduction to Homogeneous Hybrid Architecture
 - Program Method for TLC (MLC_{x3}) NAND
 - Hard Partitions and Soft Partitions
- **Adaptive Management**
 - Phoenix: Reviving MLC Blocks as SLC to Extend NAND Flash Devices Lifetime
 - **Utilization-aware Self-tuning**
- Summary



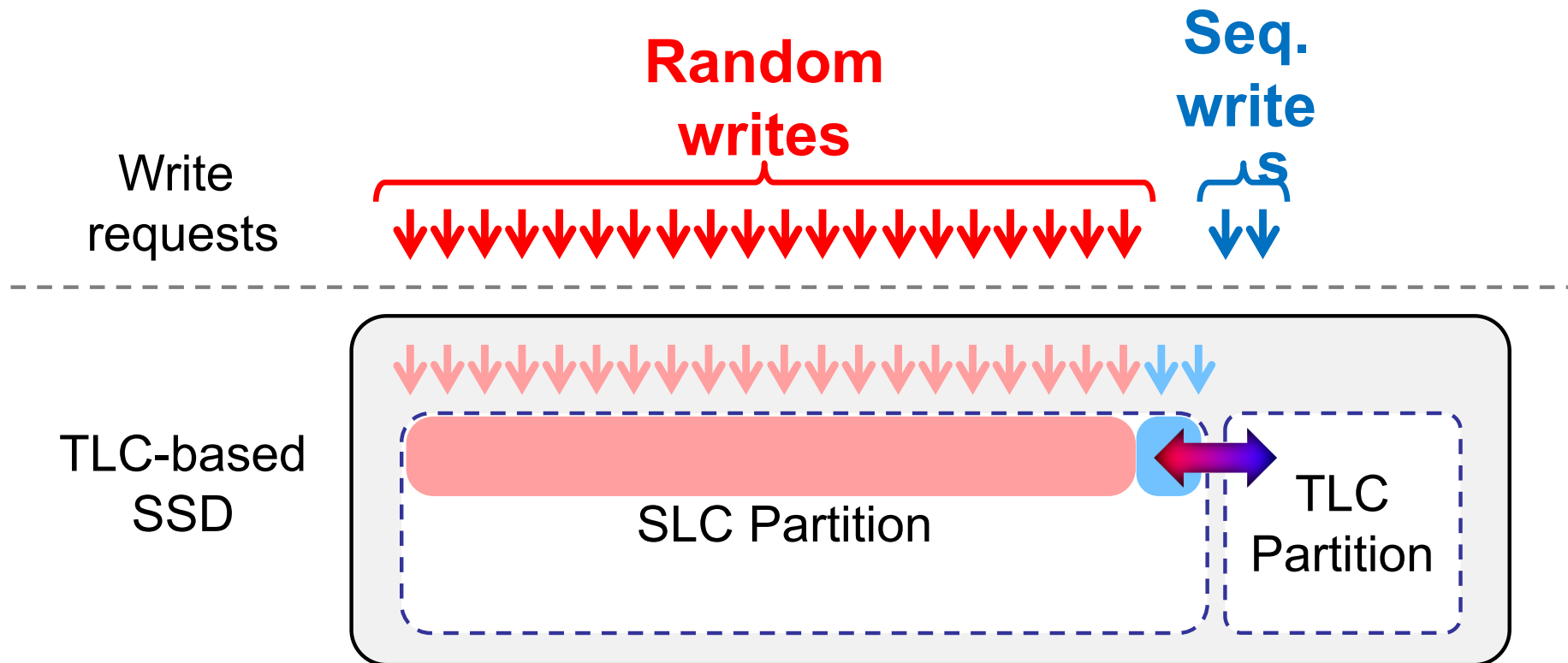
Fixed Partitions and Variable Partitions

- Fixed partitions (Hard partitions and Soft partitions)
 - Allocate a fixed number of blocks in the SLC mode to improve the storage performance
- Variable partition
 - Dynamically adjusting the number of blocks in the SLC mode according to the amount of data stored in the storage at the runtime
 - Utilization-aware Self-tuning for TLC Flash Devices [5]



Utilization-aware Self-tuning

- Exploiting the **mode-switching** capability of TLC flash blocks to **dynamically change** the partition sizes

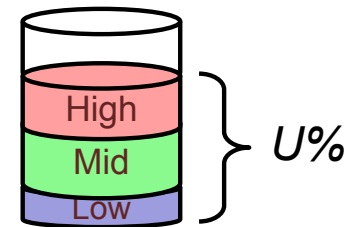


Trading non-used physical space for the performance improvement → 空間不足時才部分換成TLC

Self-tuning Garbage Collection

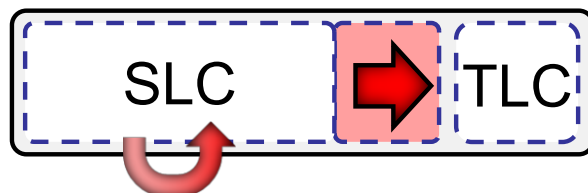
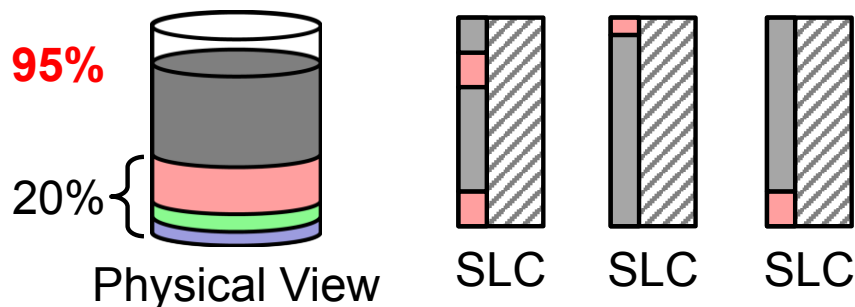
- Garbage collection is performed based on the current logical utilization $U\%$

$$\text{Logical Utilization} = \frac{\text{Valid Data}}{\text{Storage Capacity}}$$

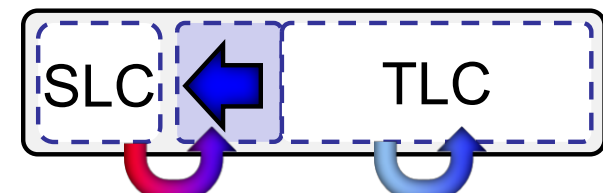
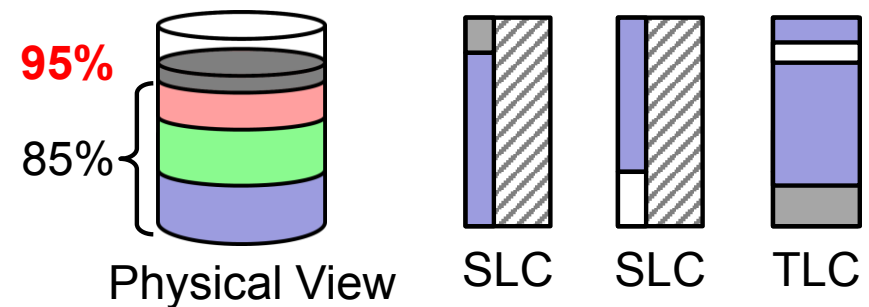


Logical Utilization

Lower Utilization



Higher Utilization





Outline

- Introduction
- Management of Heterogeneous Hybrid Architecture
- Introduction to Homogeneous Hybrid Architecture
 - Program Method for TLC (MLC_{x3}) NAND
 - Hard Partitions and Soft Partitions
- Adaptive Management
 - Phoenix: Reviving MLC Blocks as SLC to Extend NAND Flash Devices Lifetime
 - Utilization-aware Self-tuning
- **Summary**



Summary

- The **hybrid flash architecture** is to complement the drawbacks of the SLC and MLC of NAND flash with each other's advantages
- Management of hybrid architectures
 - Data replacement and migration (performance issue)
 - Wearing balance (lifetime issue)
- **Foggy & Fine TLC program (3-step programming)** reduces the disturbance and improves performance
- **Soft partitioning scheme** balances the device wear in the homogeneous hybrid architecture
- **Phoenix scheme** extends the device lifetime by reviving the worn-out MLC blocks as SLC blocks
- **A utilization-aware self-tuning** dynamically resizes the SLC/MLC partition sizes according to the data priority and the space utilization



Agenda

- Flash Memory: Why and How
- Flash Translation Layer
 - Address Mapping
 - Garbage Collection
 - Wear Leveling Strategies
- Error Correction Codes
- Flash File Systems
- Hybrid Flash (SLC & MLC) Architecture
- **Other Issues and Conclusion**

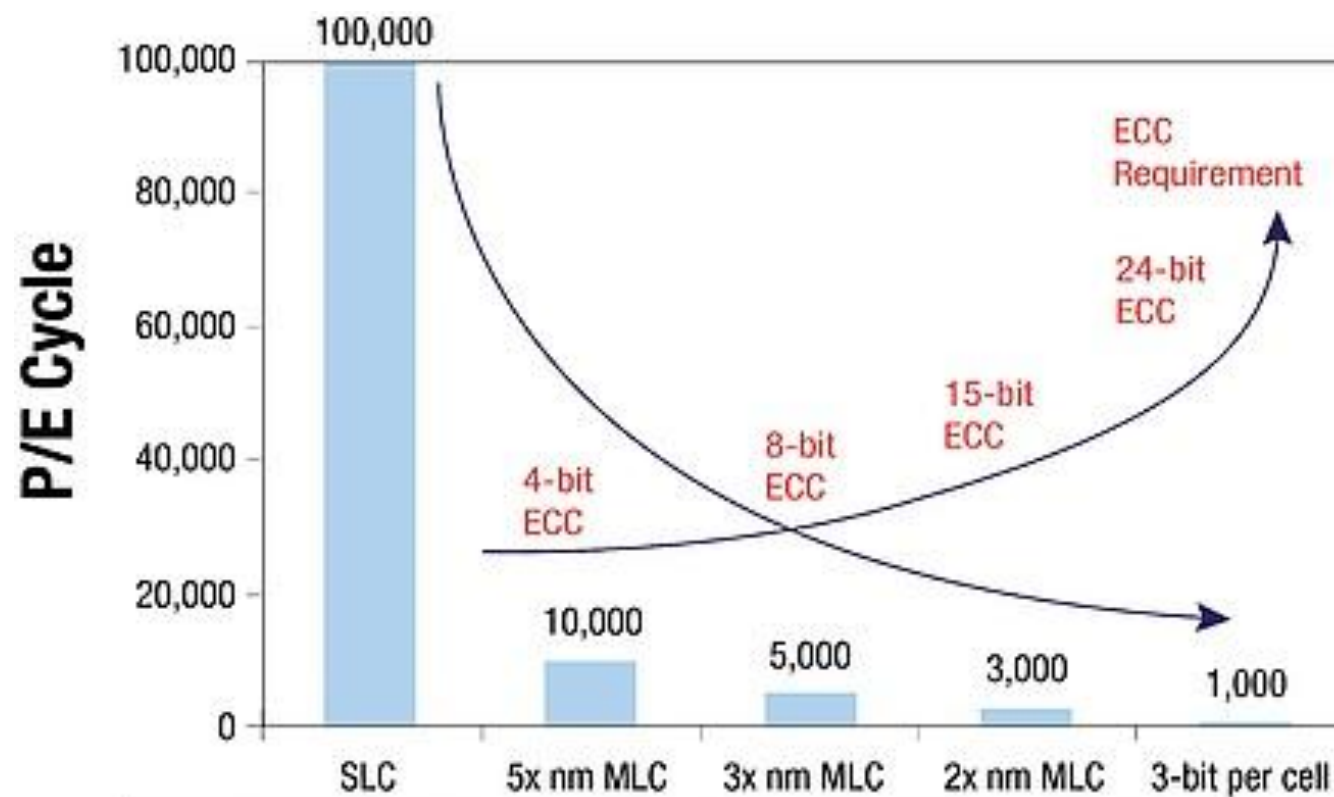


Outline

- **Data Retention Issue**
- Temperature Issue
- Data Security Issue
- Exploiting Adverse Effects of NAND Flash Memory
- Summary

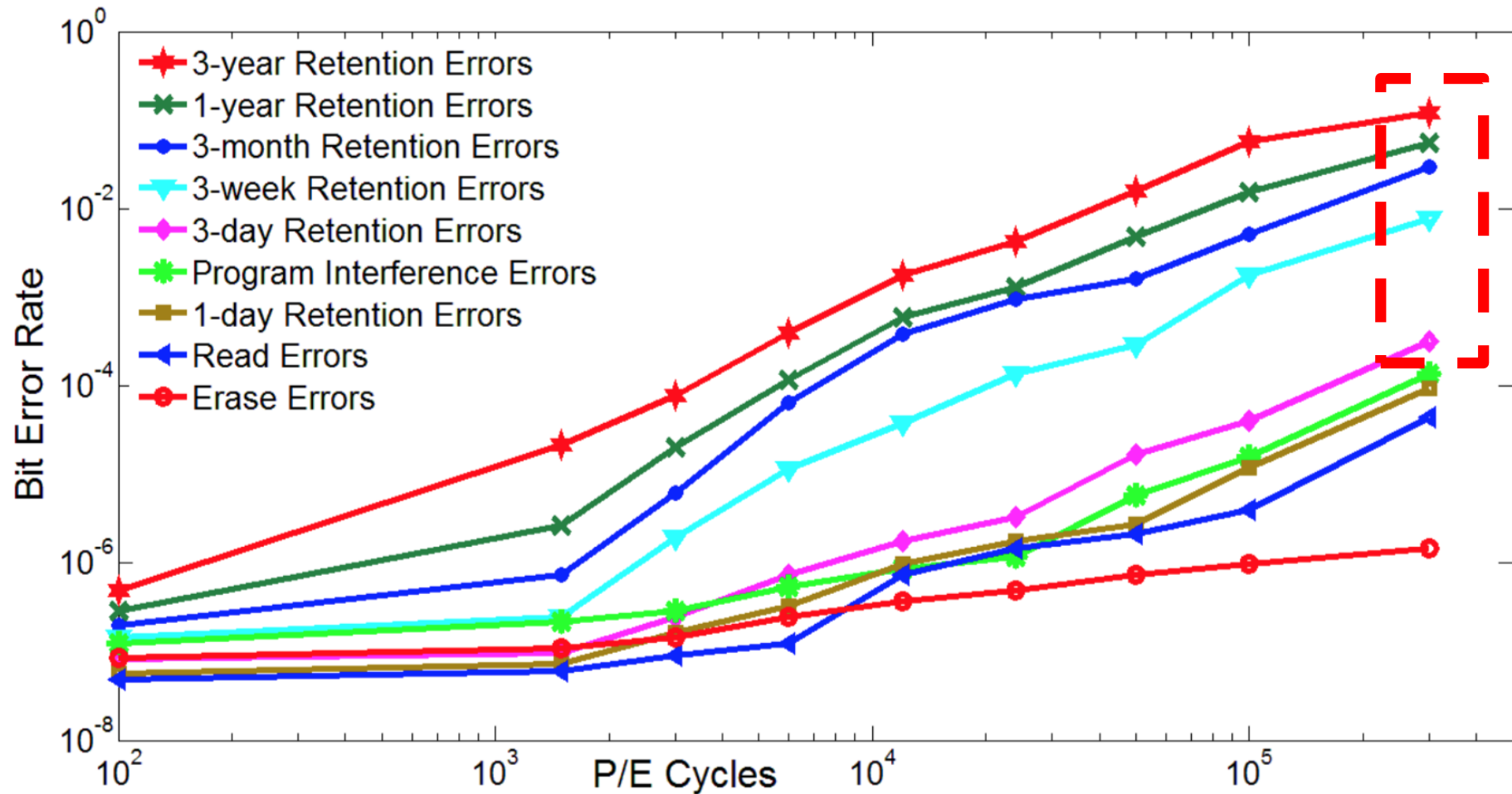
Trend of Flash Memory

- As NAND flash scales up and stores more bits per cell, the reliability and endurance of flash memory decrease.



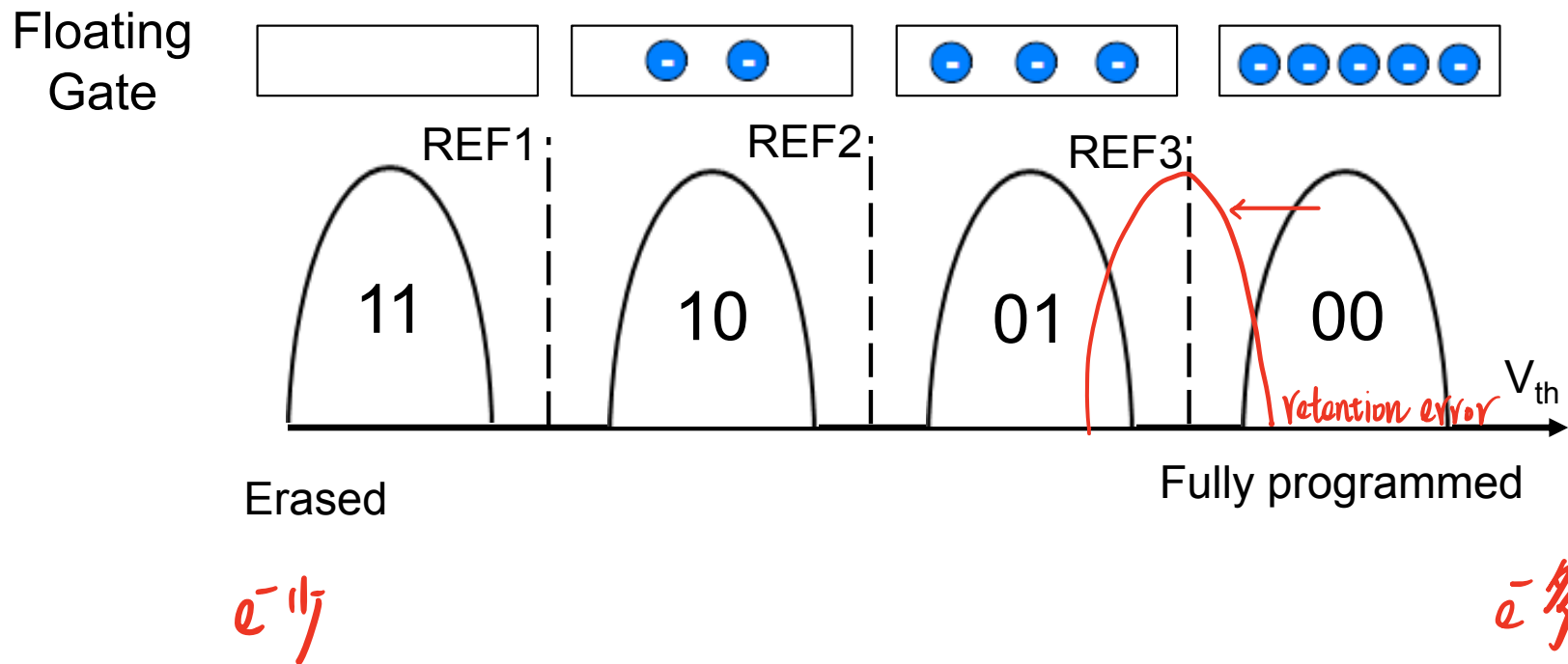
Source: JMicron, Western Digital, Morgan Stanley Research

P/E Cycles vs. Bit Error Rates



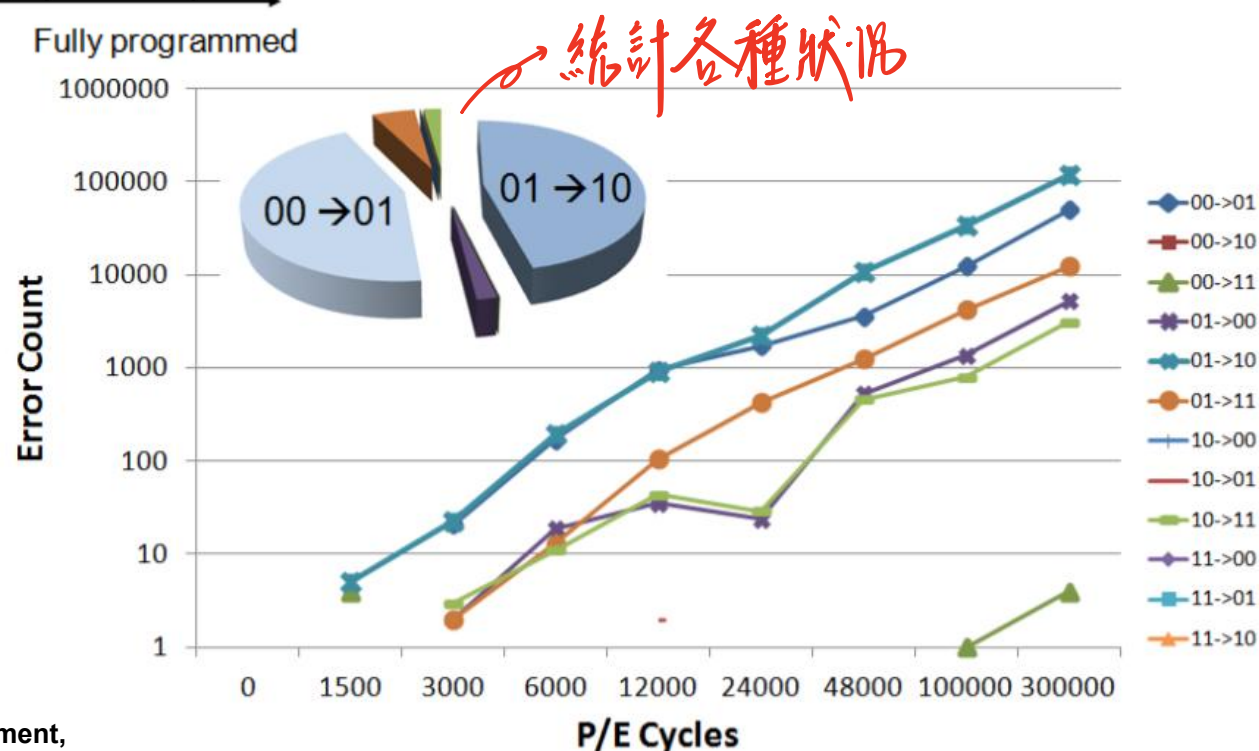
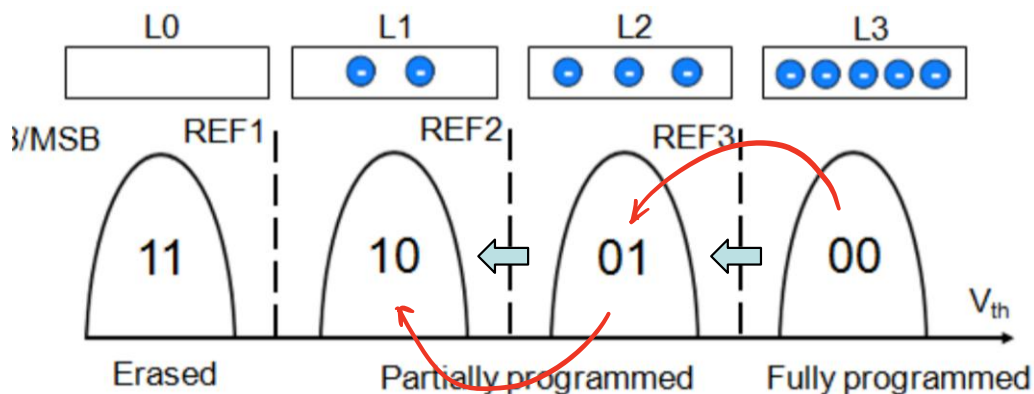
BER of various type of errors as P/E cycles increase

In-depth Retention Error Analysis





Error Patterns of Retention Errors

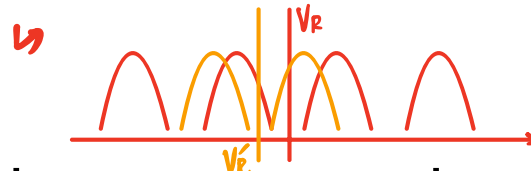


Value-dependence of retention error



Solutions to Retention Error

A. Find the **optimal read reference voltage** with read retries



B. **Refresh** data in programmed cells

↪ 定期把 data 讀出送往 ECC

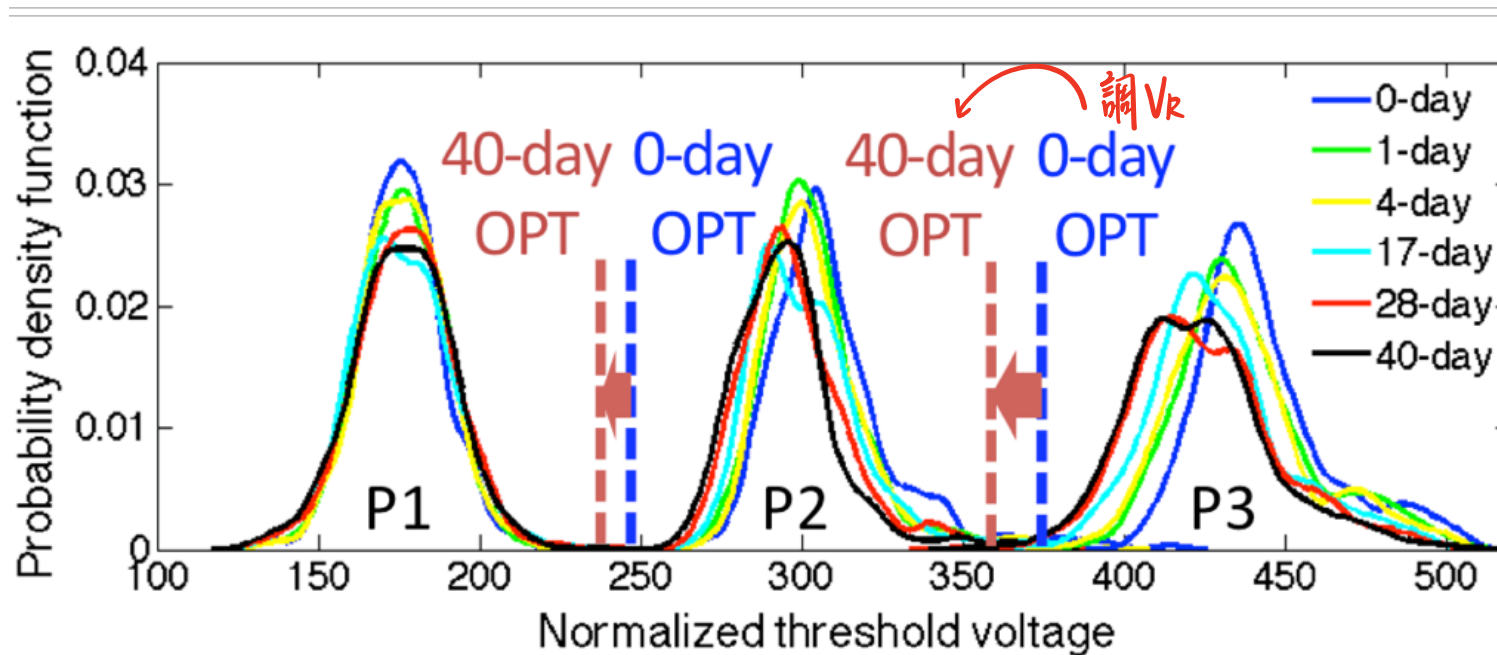
C. Enhance **ECC** capability

↪ 加強 ECC (晶片面積↑)



Optimal Read Reference Voltage (OPT)

- Threshold voltage distributions tend to shift to **lower voltage states with retention age**
- Different read reference voltages lead to different raw bit error rates

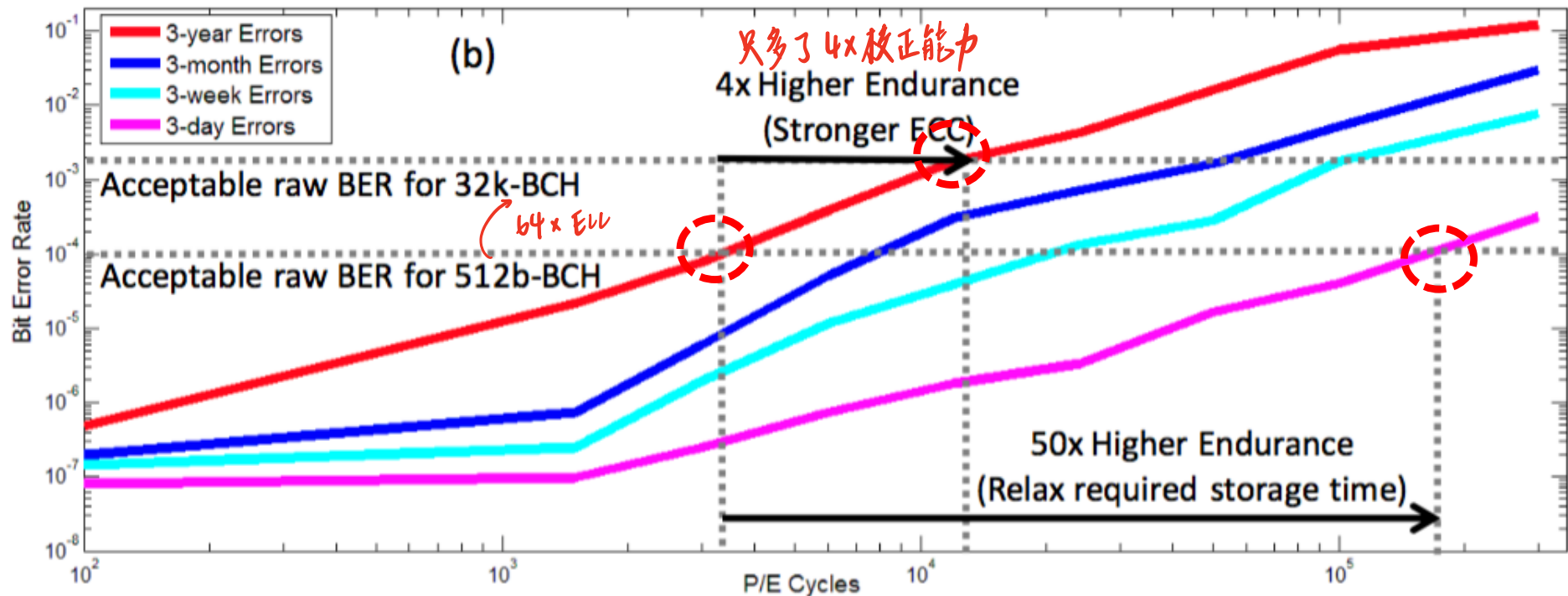


Threshold voltage distribution of 2y-nm MLC NAND v.s. retention age under room temperature



Stronger ECC vs. Relaxing Retention Time

- Increase the strength of ECC results in diminishing returns
- Relax the required retention time to achieve a longer lifetime
 - Use periodic refresh to retain the data integrity





Flash Correct-and-Refresh (FCR)

- Basic idea of FCR
 - Periodically read + correct + refresh
 - “Refresh” is done by either reprogramming or remapping
- Three versions of FCR
 1. Remapping-based FCR
 2. Hybrid reprogramming/remapping-based FCR
 3. Adaptive-rate FCR



Remapping-based FCR

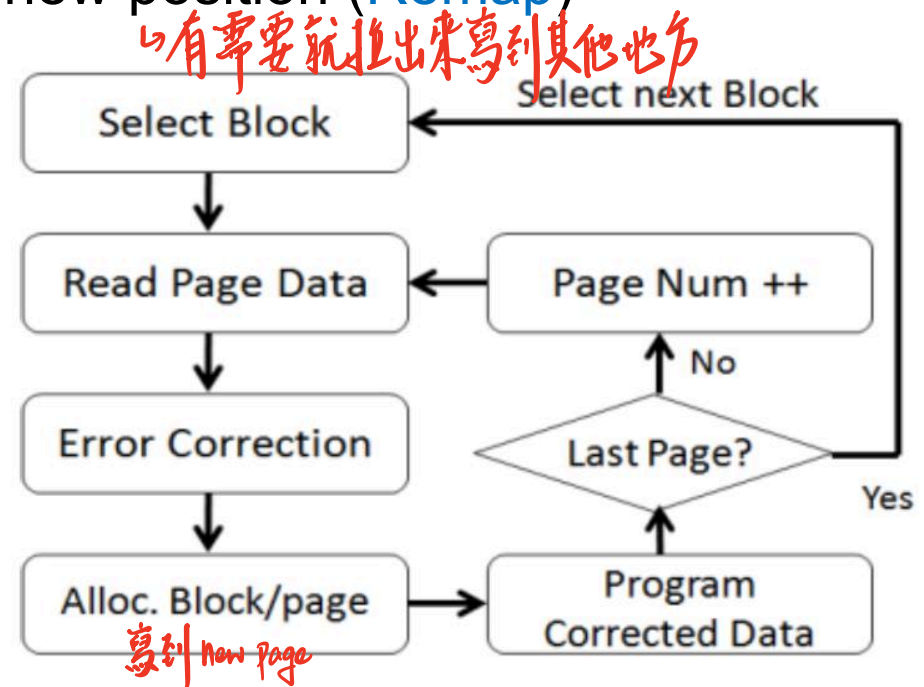
• Steps

1. Read out all data in a page
2. Perform ECC correction
3. Program the corrected data to new position (Remap)

• Fixed refresh rate

• Drawback

- Additional erase operations
- If refresh is too frequently
-> Reduce lifetime due to additional P/E cycles

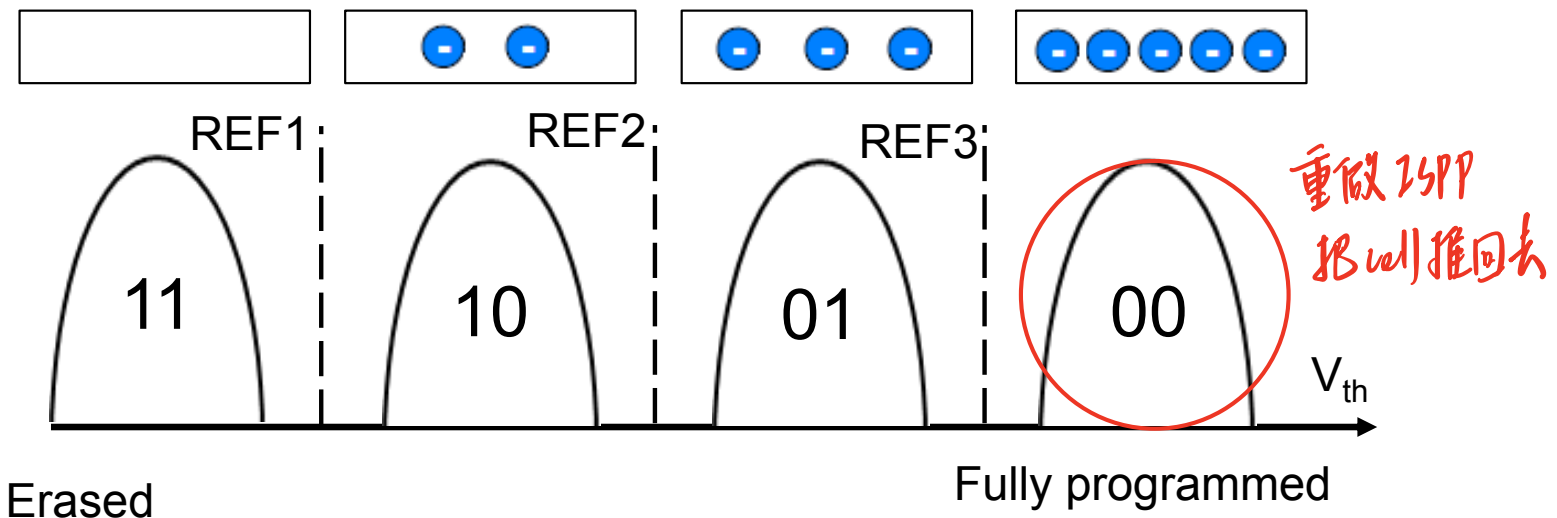




FCR: In-place Reprogram Operation

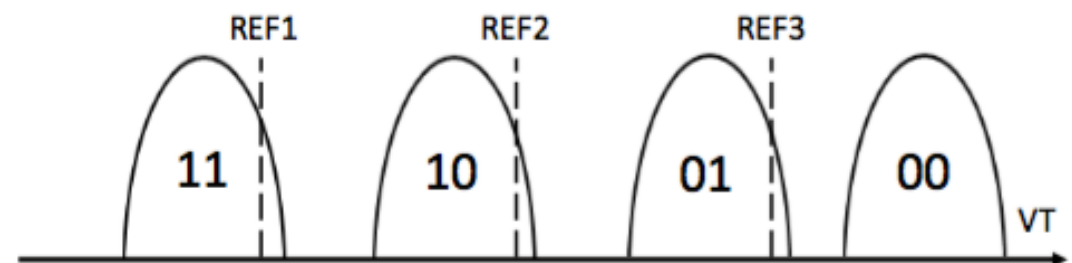
- In-Place Reprogram (by ISPP)

Refresh to prevent degradation



- Problem : Accumulated Program Errors

- Program (Right-shift) error cannot be fixed by in-place reprogram
- 大家都被推到





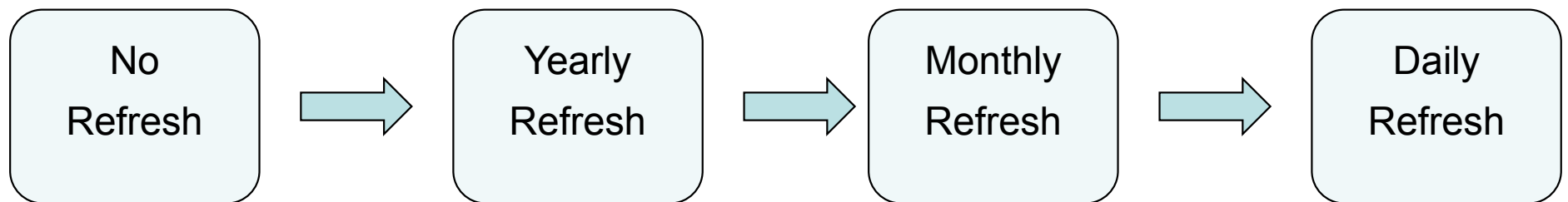
Hybrid Reprogramming / Remapping FCR

- Mitigate the errors accumulated due to periodically reprogramming
- If # of right shift errors exceeds a threshold $T \rightarrow$ **remap**
- Steps
 1. Read out all data in a page
 2. Perform ECC correction
 3. Compare cell threshold voltage
 4. (# of right shift errors $\leq T$) ? Reprogram : Remap
- **Fixed refresh rate**



Adaptive-Rate FCR

- Observation
 - Retention error rate is acceptable when P/E cycle < 1000
 - Retention error rate increases as the P/E cycle increases
- Main Idea : Adapt the refresh rate to P/E cycle a **block** has incurred



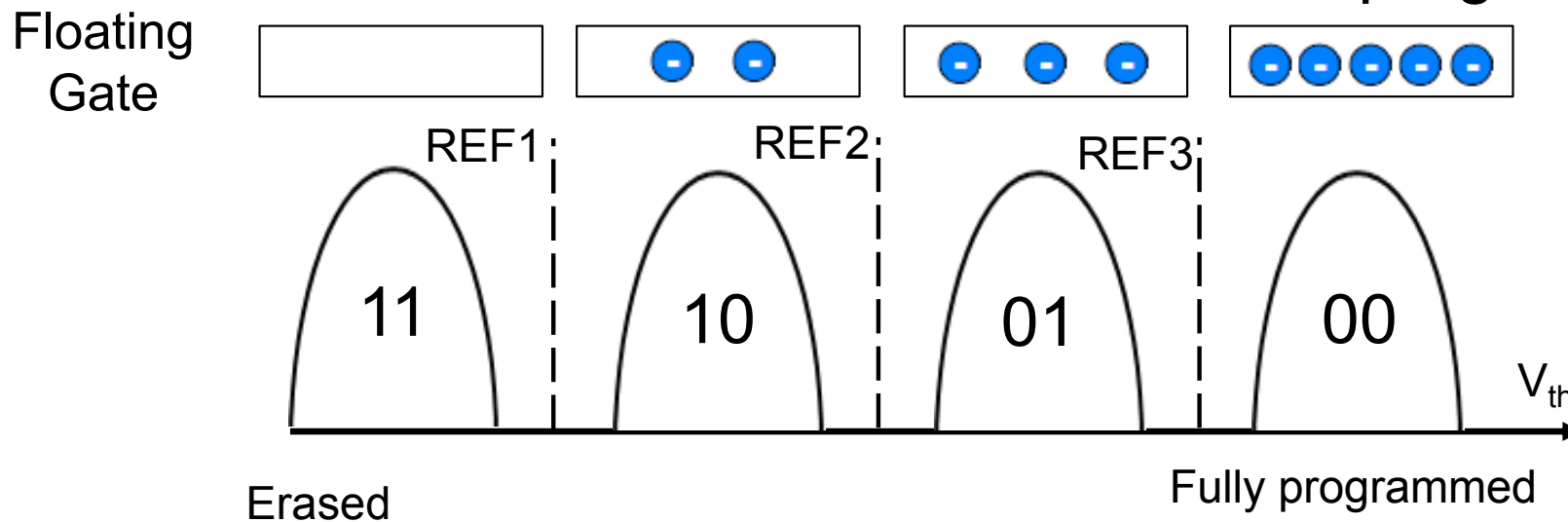
P/E cycle increases →



FCR: Refresh to Solve Retention Error

- Refresh data in programmed cells

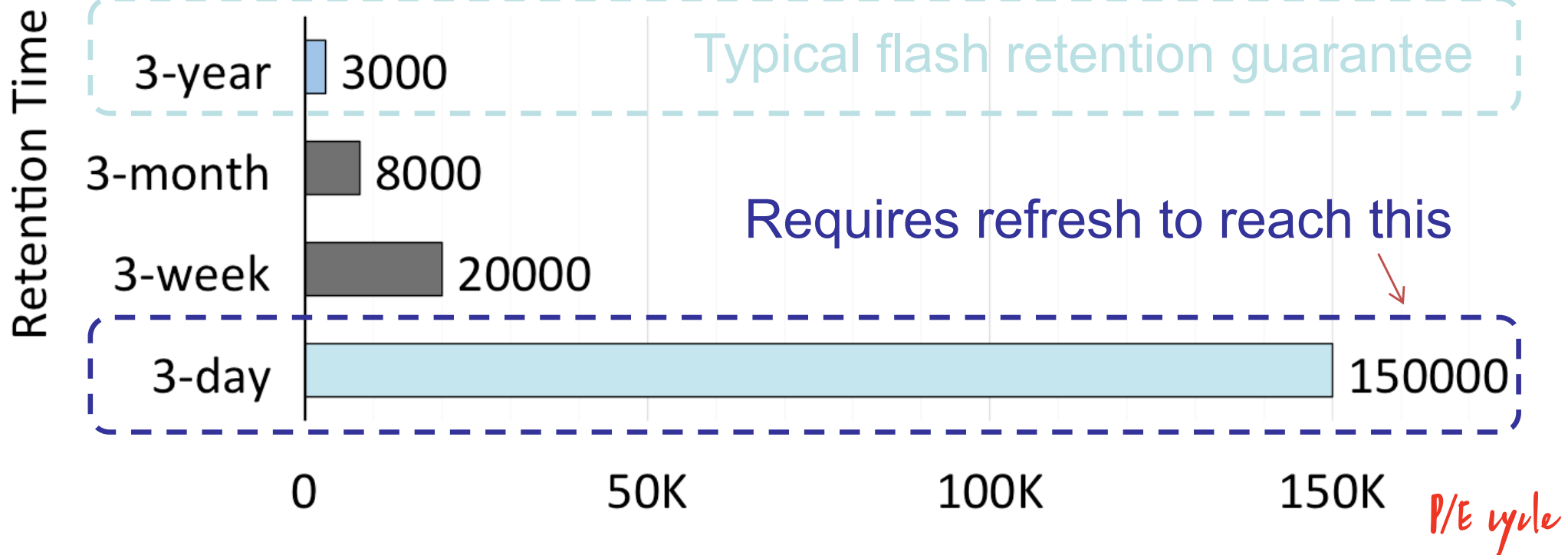
Refresh stops programmed data



Refresh operation in NAND flash memory ?

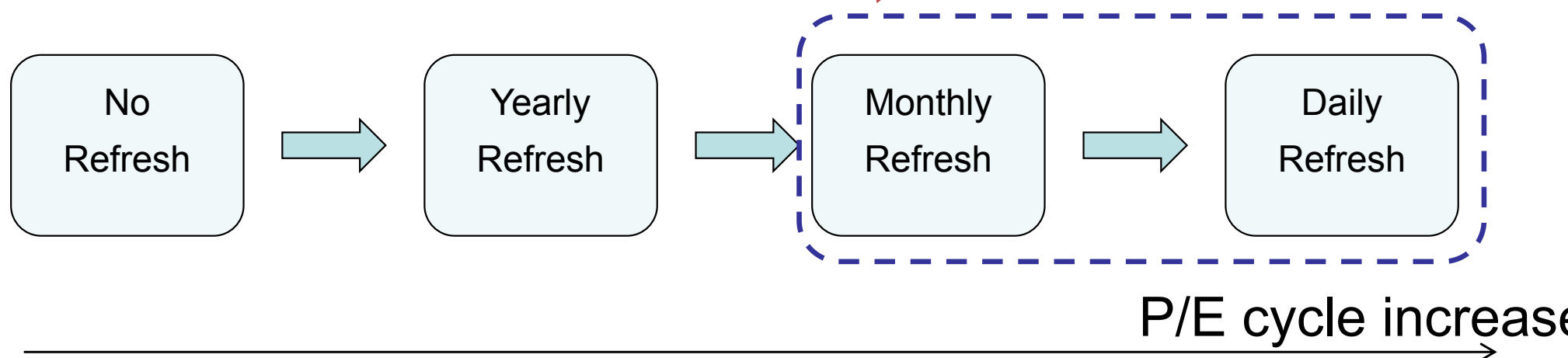


WARM: Retention Time Relaxation Through Refresh



WARM: Refresh Overhead

- Problem of FCR
 - Refresh overhead reduces extended lifetime
- Possible solution
 - Adaptive-rate refresh

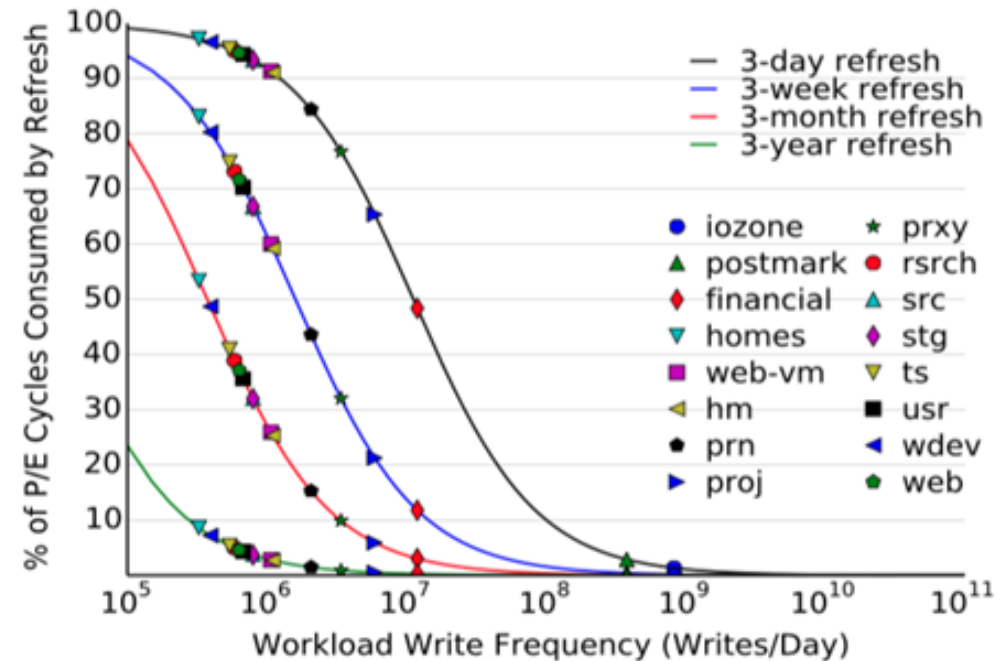


Goal: Reduce refresh overhead, improve flash lifetime



WARM: Key Observations

1. Refresh overhead with different refresh rates
2. Write-hot pages can skip refresh



Fraction of P/E cycles consumed by refresh operations.

Update

Invalid Page
Invalid Page
Write-Hot Page

Skip Refresh

Retention Effect

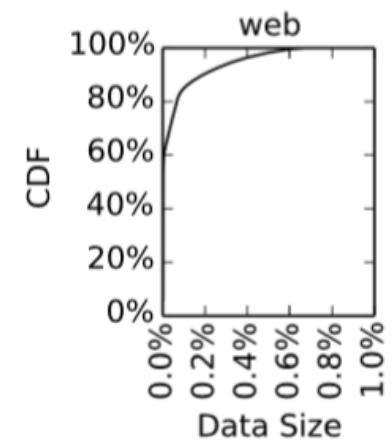
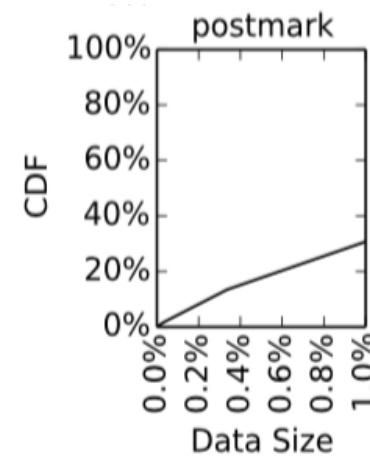
Invalid Page
Write-Cold Page

Need Refresh



WARM: Opportunities to Exploit Write-Hotness

- For most of the applications, only a very small fraction of all data pages are write-hot pages
- Write-hot pages account for a large portion of the write request



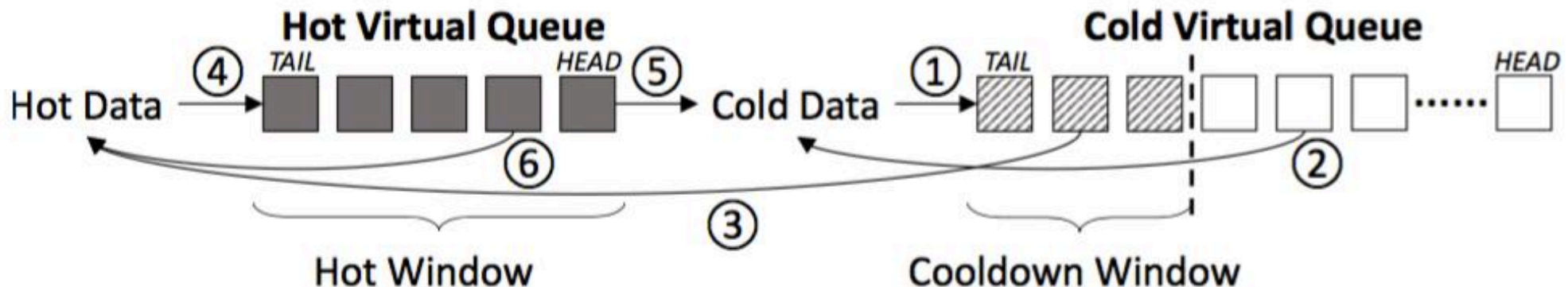
Problem : Refresh is at a block granularity

Solution : Physically separate write-hot from write-cold into different blocks



WARM: Partitioning Algorithm via a Size-Tunable Queue

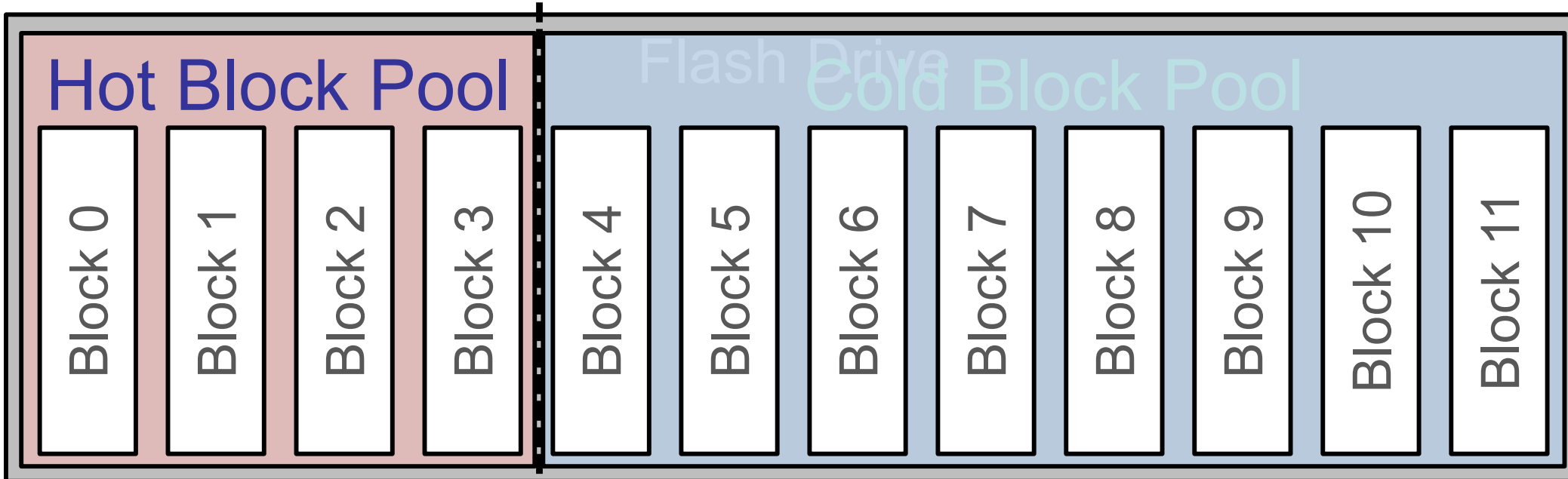
- Dynamically adjust the size of hot and cold pools as well as the cooldown window periodically
 - Prevent ping-ponging between hot and cold pools
 - Ensuring every page in the hot pool will be written more frequently than hot pool retention time



不做FLR



WARM: Write-Hotness Aware Flash Policies

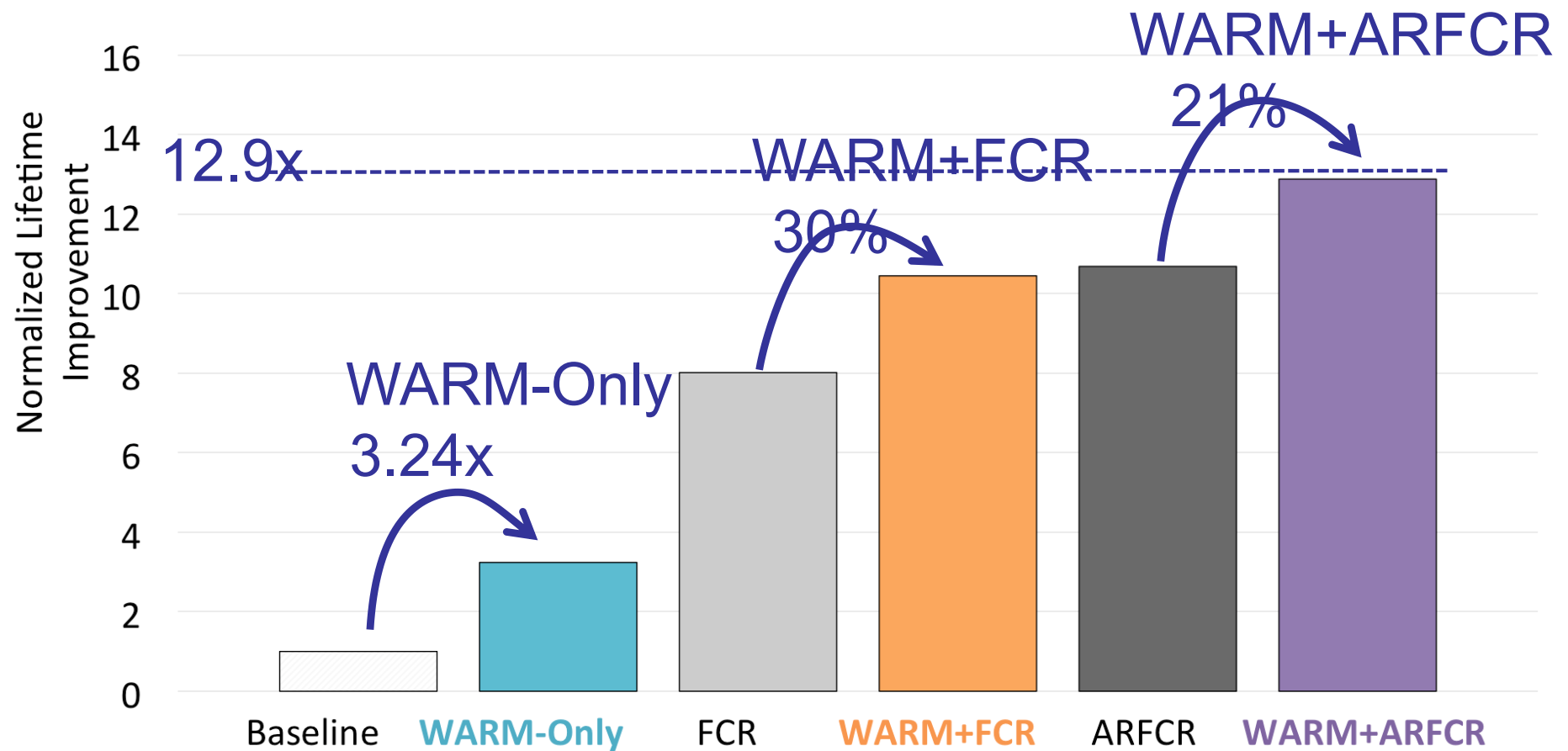


- Naturally relaxed retention time
- Program in block order
- Garbage collect in block order
- All blocks **naturally wear-leveled**

- Write-cold data → lower write frequency, less wear-out
- **Use refresh** to relax retention time
- Conventional garbage collection
- Conventional wear-leveling algorithm

Also global wear leveling

Evaluation of Lifetime Improvement



Outline

- Data Retention Issue
- Data Security Issue
- **Temperature Issue**
- Exploiting Adverse Effects of NAND Flash Memory
- Summary



Temperature Issue

- 3D NAND Flash Memory
 - Charge-Trap instead of floating gate
 - Vulnerable to high temperature
 - High temperature speeds up retention error
- Current physical space allocation strategies do not consider **temperature issue** of physical blocks
 - Consecutive writes are allocated to continuous physical space
 - Spatial locality results in the accumulation of heat
- Lower-temperature blocks are more reliable
 - Allocate **critical data** to these blocks



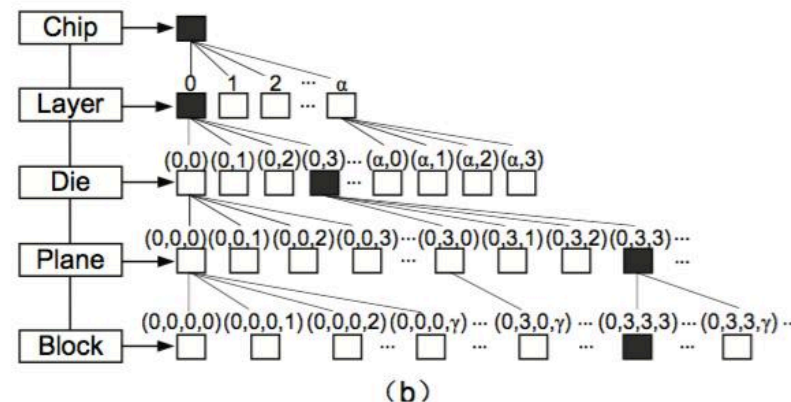
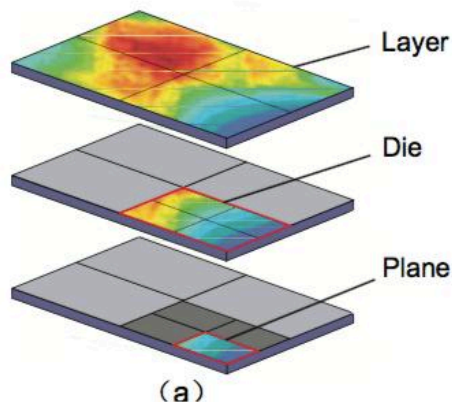
Block Selection with Temperature Mining

- Temperature of a block is also related to temperature of its neighbor blocks
- The selection strategy requirement:
 - **Fast enough** - the overhead is almost negligible
 - Unnecessary to be very accurate, but only have to identify **relatively low temperature**
- Due to the layout dependence in the 3D structure, **nearby blocks normally share similar temperature**
 - Offer opportunity to a fast and good enough solution



Block Selection with Temperature Mining (Cont'd)

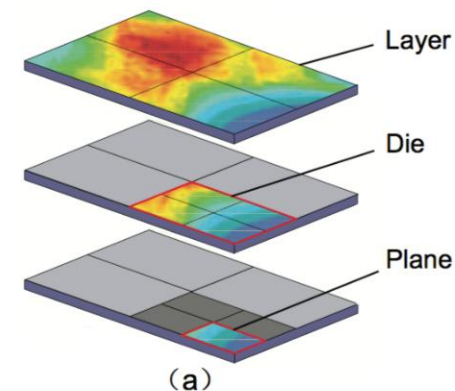
- Utilize **on-chip thermal sensor** or thermal image cameras
- View a 3D chip as **a cube with many layers**
- Modeling temperature using **a multi-level quad tree**
 - Doesn't require complex sorting algorithms
 - Can effectively find relatively low temperature blocks





Block Selection with Temperature Mining (Cont'd)

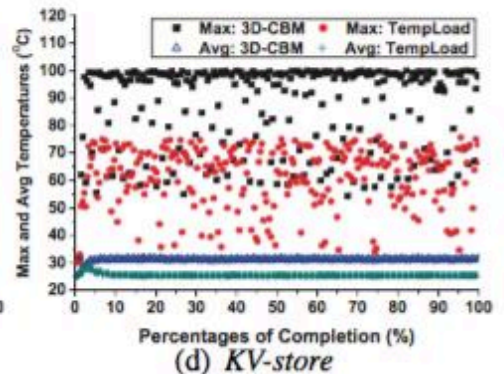
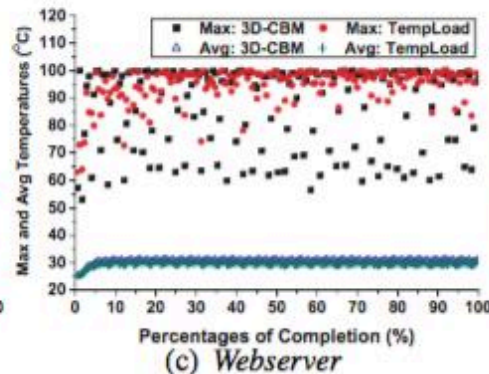
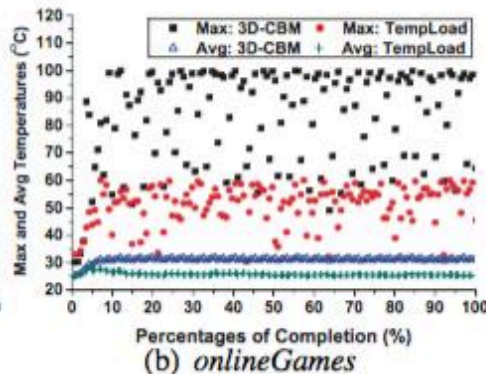
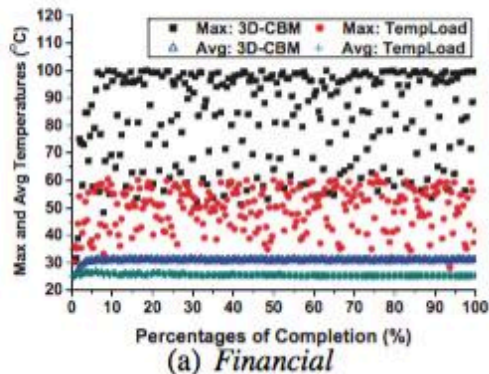
- With quad tree, only need to compare a fixed number of nodes at each level
- For the last stage of search, we have a **relatively low temperature plane**
 - Separate all blocks in a plane into many **chunks**
 - Use **round-robin** scheduling among chunks **in a plane**
- For consecutively allocation demands → Assign free blocks from different planes
 - Prevent heat accumulation
 - Improve wear-leveling





Experiment Result

- Both max and average temperatures are lowered
- For a read-dominant trace (c), the effect is not obvious
 - No re-allocation for a read operation





Outline

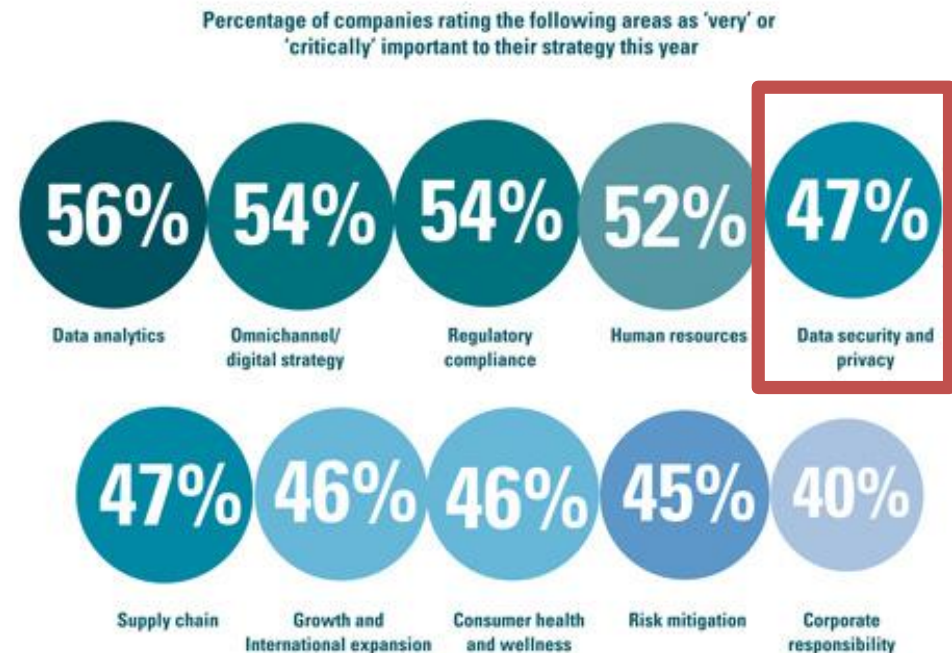
- Data Retention Issue
- Temperature Issue
- **Data Security Issue**
- Exploiting Adverse Effects of NAND Flash Memory
- Summary

Why Data Security is Important?

- Nowadays, data security has risen to be one of the most critical concerns of computer professionals

- To protect **sensitive data**

- Bank's data and accounts
- Enterprise's secret data
- User's credit card numbers
- User's web browsing history
- Anyone's temporary files



Source: KPMG and CGF Global Consumer Executive Top of Mind Survey 2014.

- However, there is a lack of **secure deletion** in most systems and devices

- E.g., file systems are not designed to offer security property



Traditional Deletion

storage
medium

Name	Deleted	Data
File_1	No	Some sensitive data
File_2	No	xxxxxxxxx
...	...	...

after delete "File_1"

storage
medium

Name	Deleted	Data
File_1	Yes	Some sensitive data
File_2	No	xxxxxxxxx
...	...	...

It's called "Unlinking"

Overwritten in HDD and SSD

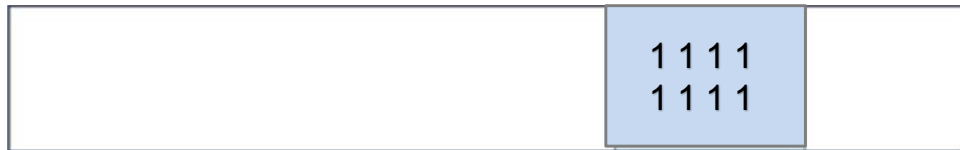
◆ Hard Disk Drive

- 1-1 logical to physical mapping
- It could be directly overwritten

1. Write "ABCD WXYZ" to LBA 8000



2. Write 1s to LBA 8000



3. Write 0s to LBA 8000



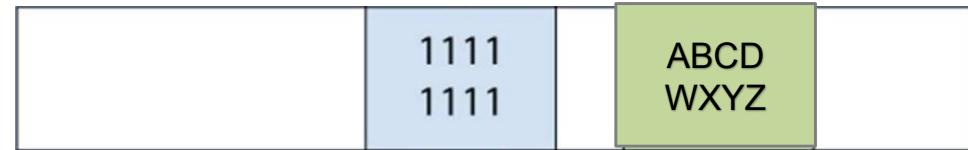
◆ Solid State Drive

- No 1-1 mapping
- It couldn't be directly overwritten

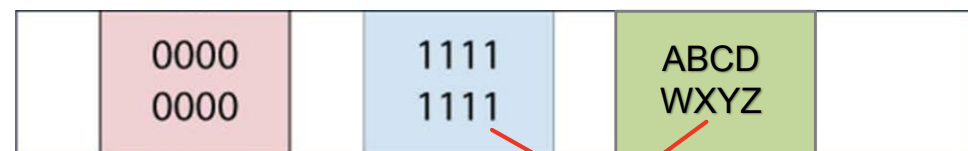
1. Write "ABCD WXYZ" to LBA 8000



2. Write 1s to LBA 8000



3. Write 0s to LBA 8000

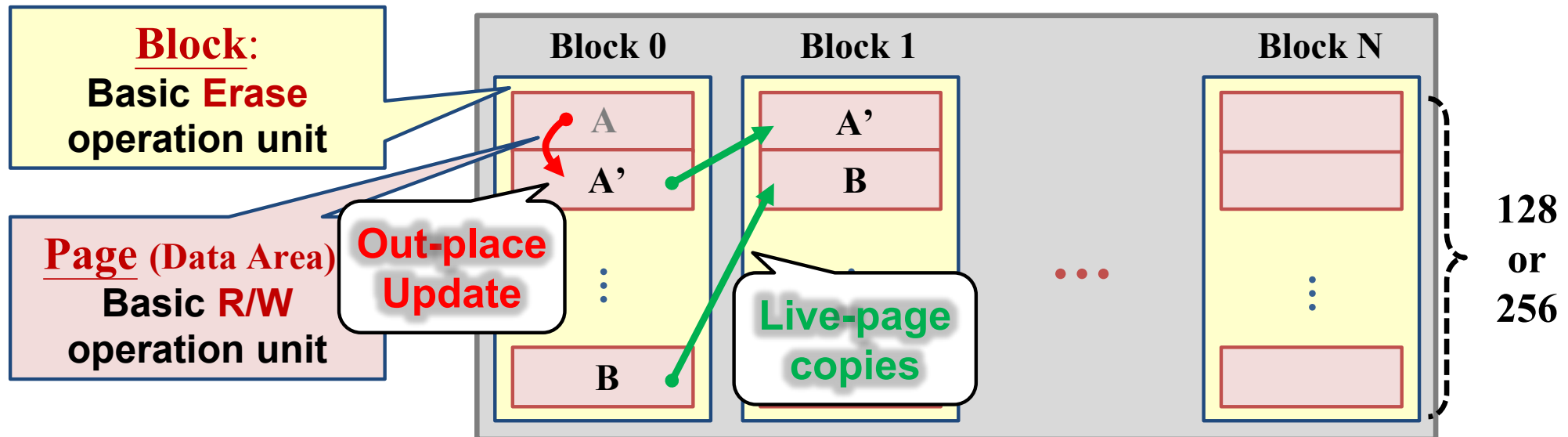


old version



How Data is Erased in NAND flash

- **Garbage Collection**
 - Write-once property (*page*)
 - Out-place updates
 - Bulk-erasing (*block*)
 - Garbage collection
 - Live-page-copying



Physical Layout of Flash Memory



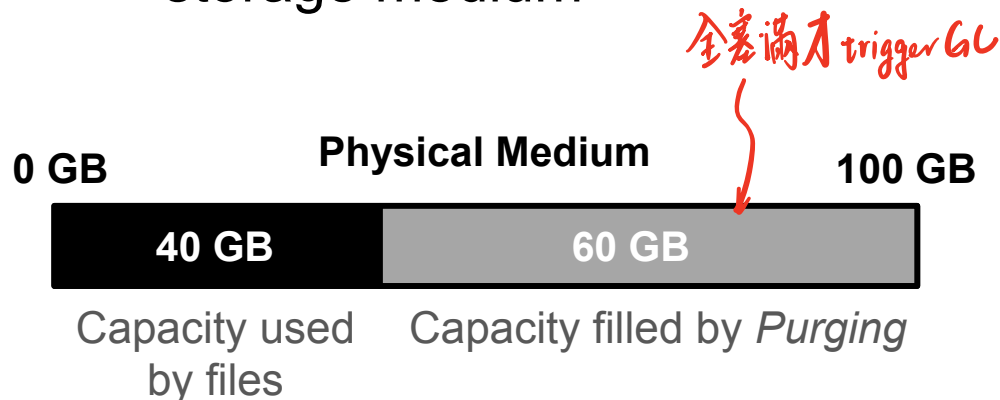
Purging & Ballooning

• Main idea

- It **creates junk files** to occupy empty space, then sanitize data by **triggering GC**

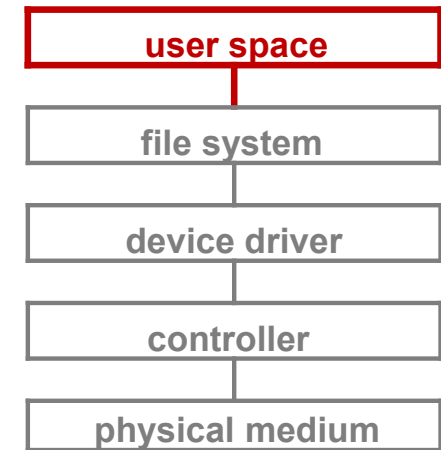
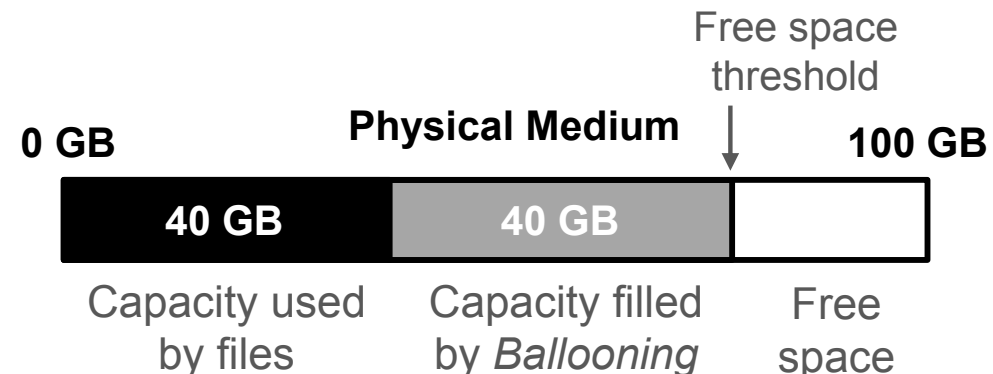
• Purging

- Fill up the capacity of the storage medium



• Ballooning

- Continually occupy empty space to keep it at a threshold





Data Node Encrypted File System

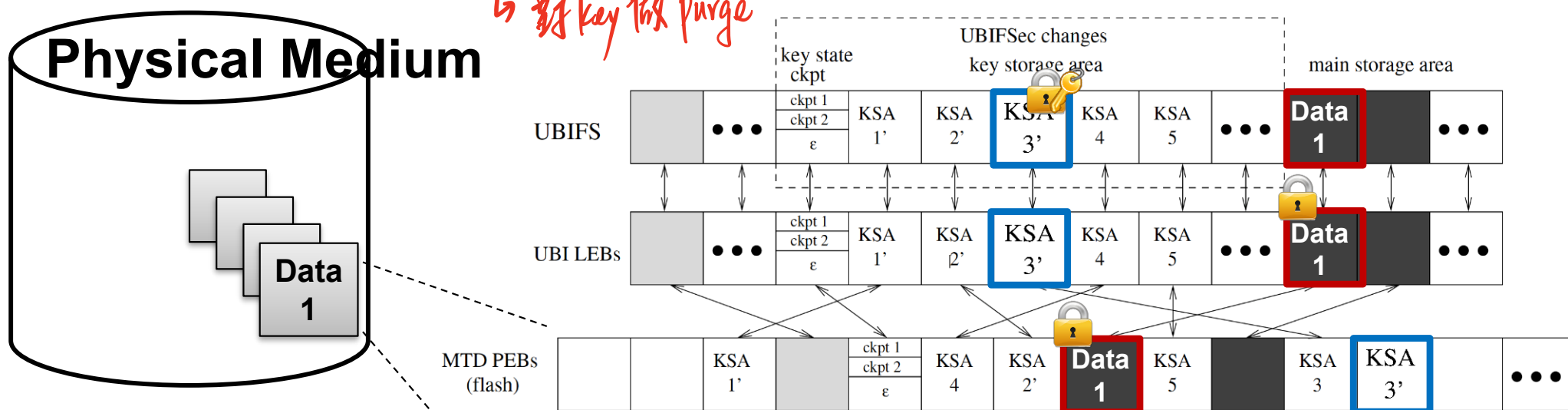
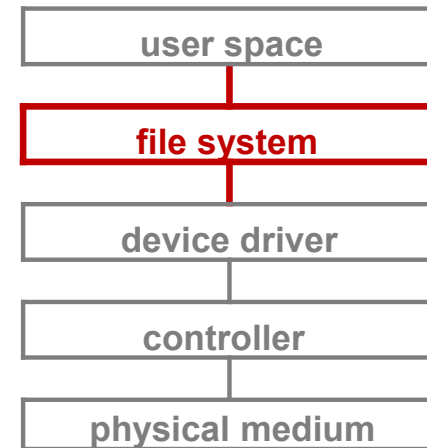
• Main Idea:

- To achieve the secure deletion by sanitizing encryption key

• Solution Steps:

- **Encrypt** each data node with a unique **key**
- Collect the keys in a (dense) key storage area (KSA)
- Periodically **purge KSA** to remove deleted keys

↪ 對 key 做 purge

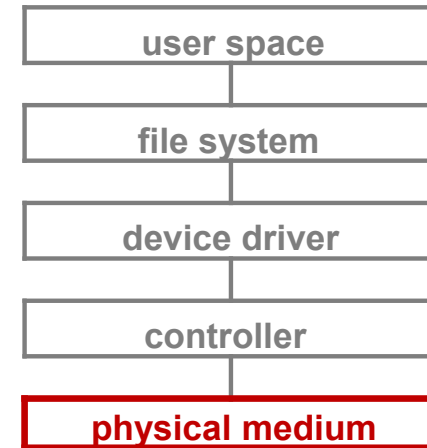




Scrubbing

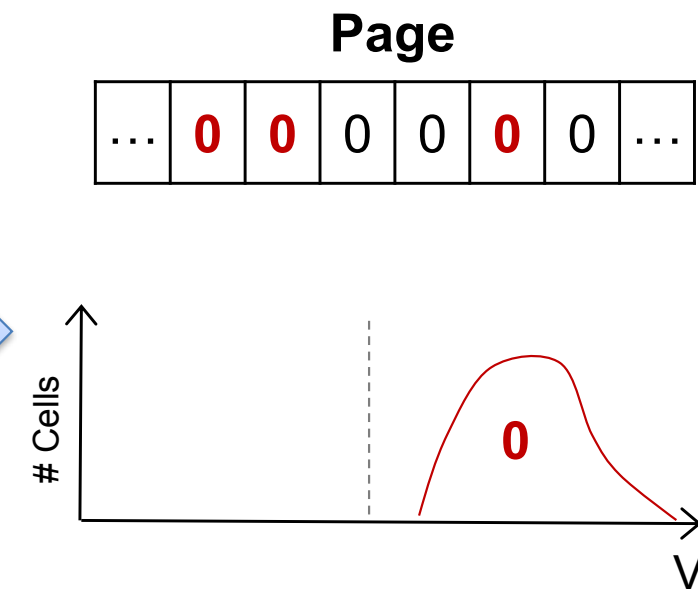
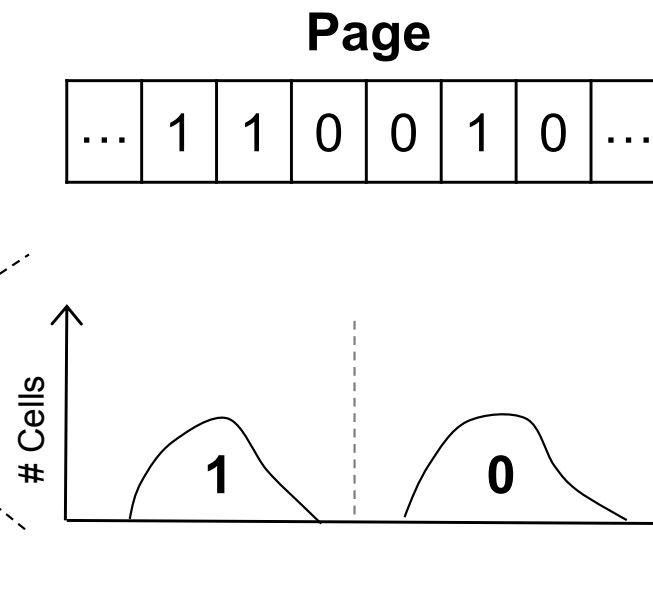
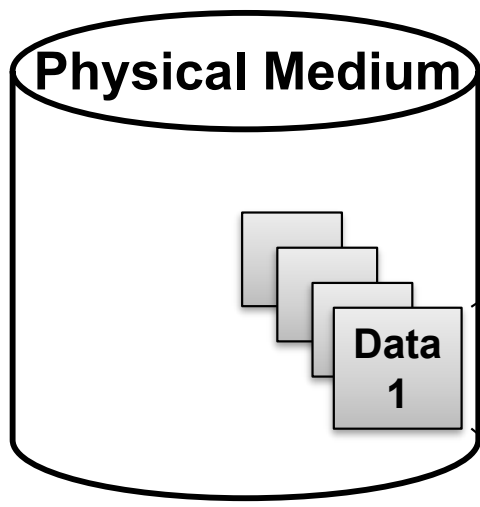
◆ Introduction :

- Scrubbing works by **re-charging** the page to turn all the remaining 1s into 0s
- **Per-page** sanitization



❌◆ Disadvantages:

- It results in the undefined behavior and has the possibility of introducing **read errors**





Limitation of Scrubbing

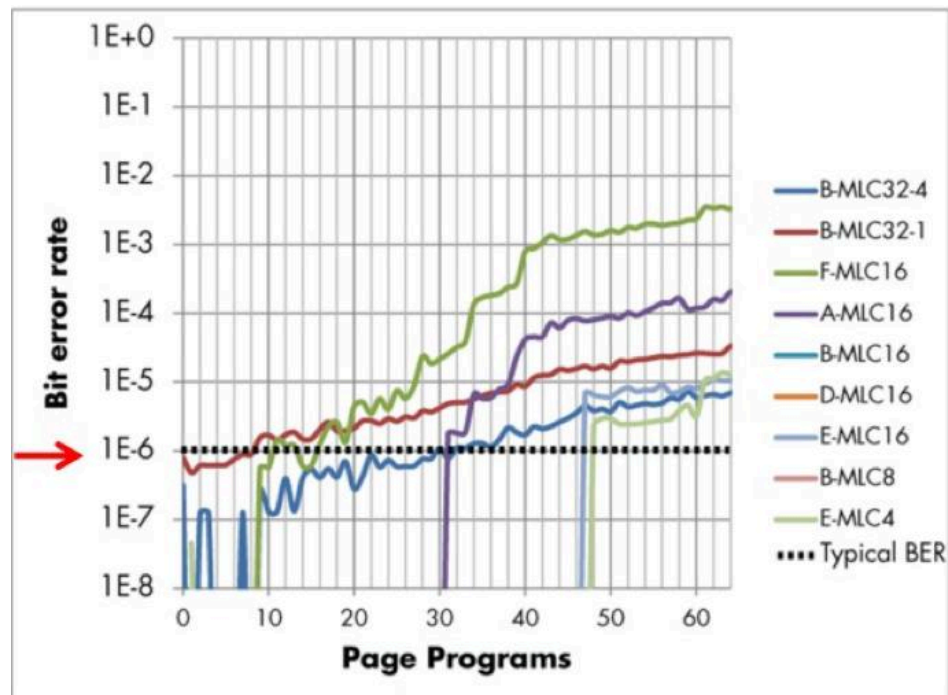
■ We can achieve the objective of data sanitization at the device side in the following ways

Secure deletion

- ① **Scrubbing** – Immediate & Per-file deletion
- ② Garbage Collection – If the condition is satisfied
- ③ Charge a little bit to make its bit pattern into disarrangement – Fast & Immediate

✓ Scrubbing has its limitation in MLC, which is called "**Scrub Budget**"

Typically "Safe" BER

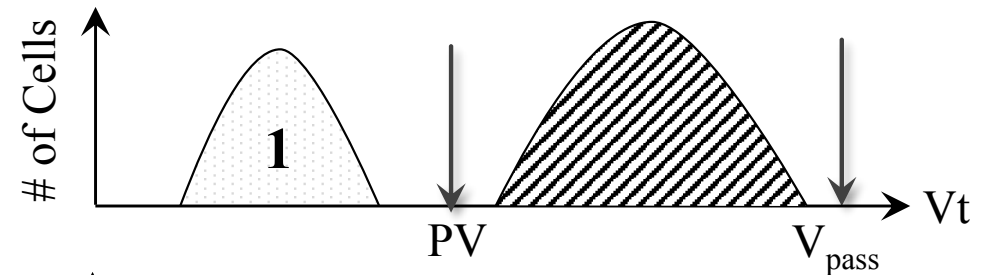


Disturbance of Scrubbing

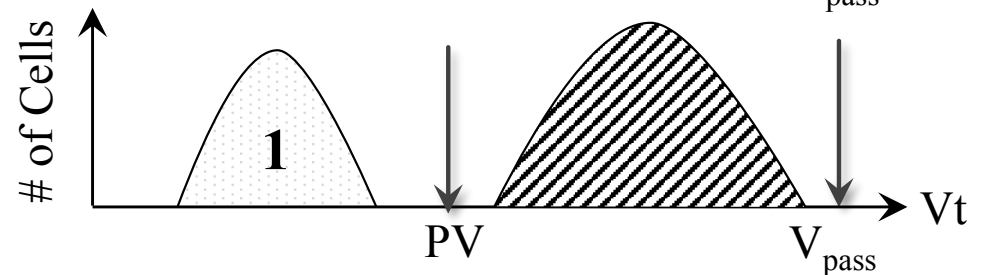
- Scrubbing works by **re-charging (re-programming)** the page to turn all the remaining 1s into 0s — effectively rewriting the memory to contain only zeros. It disturbs 4 adjacent pages
 - The adjacent pages will be **shifted to right slightly** to cause read errors

Page 1	Page 2	Page 3	Page 4
Page 5	Page 6	Page 7	Page 8
Page 9	Page 10	Page 11	Page 12
Page 13	Page 14	Page 15	Page 16

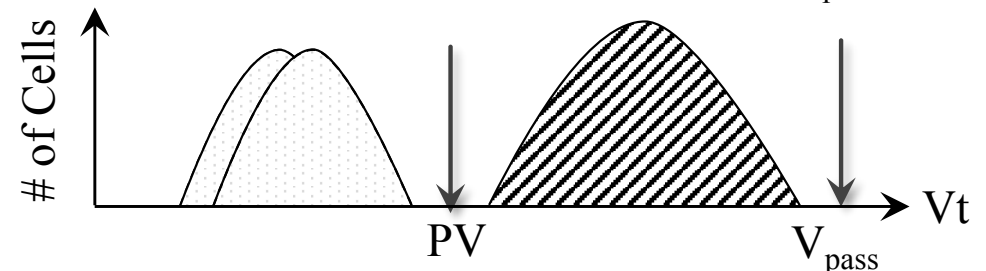
Page 10



Page 13



Page 14





Summary

- Both consumers and manufacturers pay attention to the issue of **data security** in recent years
- The existent sanitization protocols in SSD are **unreliable**
 - Because of the difference between HDD and SSD
- To create new methodology, there are several features to consider
 - Encryption vs. Unencryption
 - Cross-layer vs. Single-layer
 - Overwrite vs. Erase



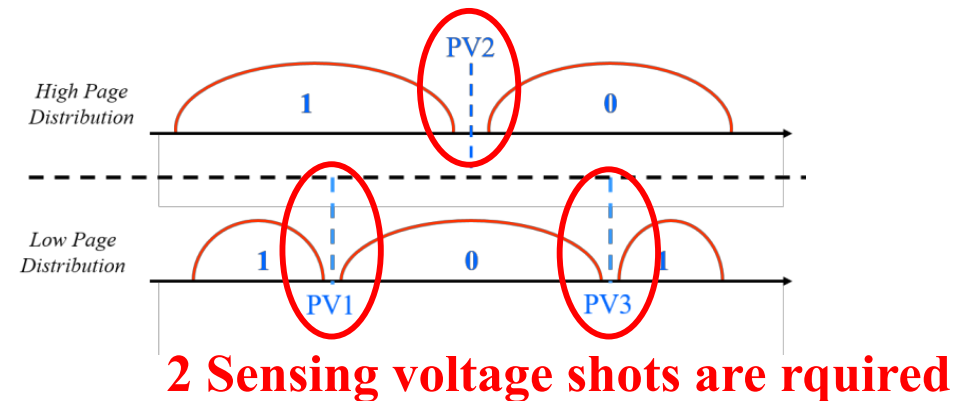
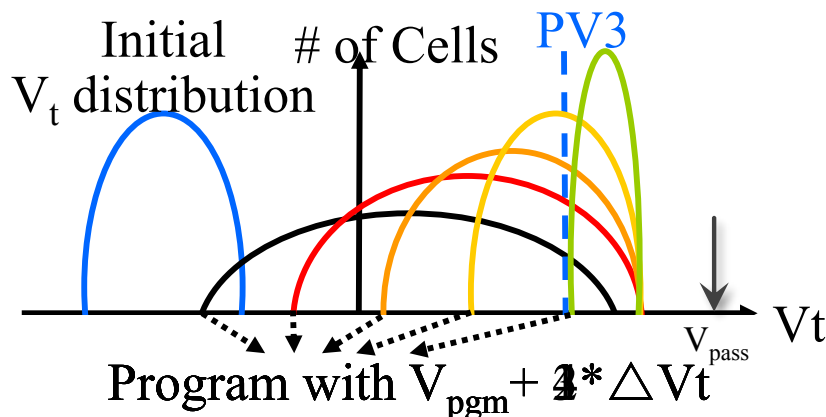


Outline

- Data Retention Issue
- Temperature Issue
- Data Security Issue
- **Exploiting Adverse Effects of NAND Flash Memory**
 - One-Shot Program Design (DAC' 18)
 - Fast Sanitization (ICCAD' 18)
 - Zero-Copy Overhead Sanitization (ICCAD' 19)
- Summary

How Do Defects of MLC Memory be Generated?

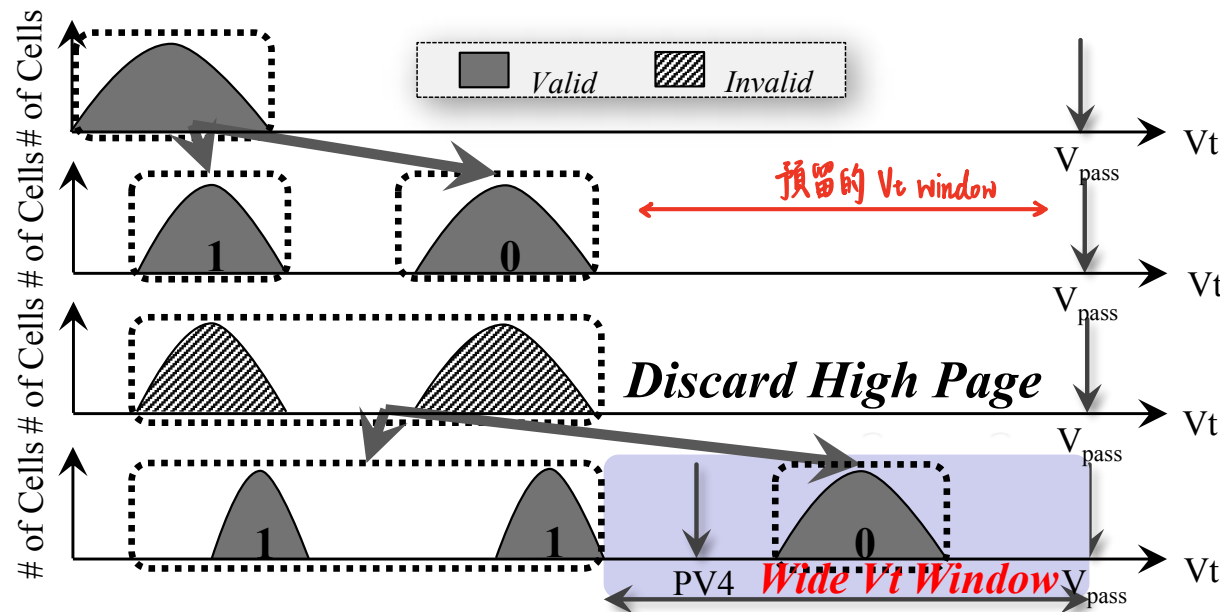
- Incremental step pulse programming (ISPP) method
 - Flash pages are programmed and verified in turn for multiple iterations
 - Memory cells' threshold voltage (V_t) distribution are gradually pushed to pass through their target program verify voltage
- Conventional programming designs result in the flash defects on low pages
 - Time-consuming delay on programming low page
 - More sensing voltages are required to read a low page data
 - Narrow V_t window results in the higher BER in low pages





Potential Solutions: One-shot programming Make Your NAND Flash Rewritable! $\Rightarrow$ MLC 當 SLC 用

- Strategically exploits the system knowledge, **data validity**, to provide an considerably large threshold voltage window similar to SLC memory via discarding some pages with out-of-date data [1, 2]

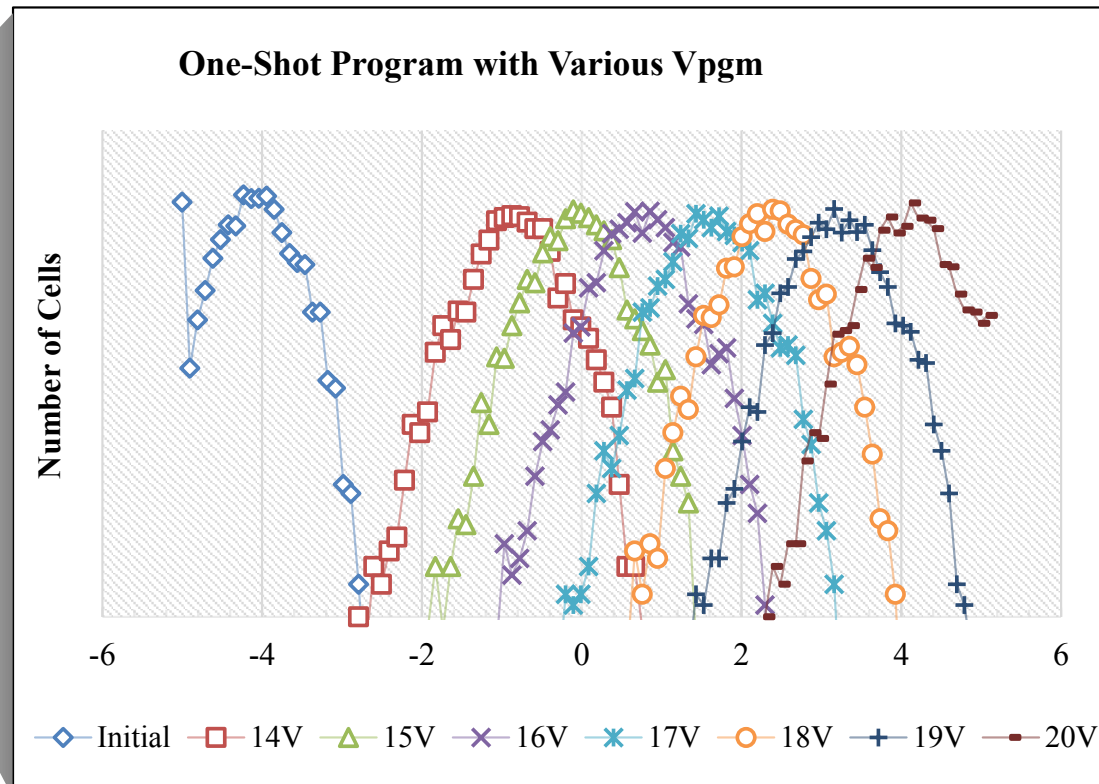
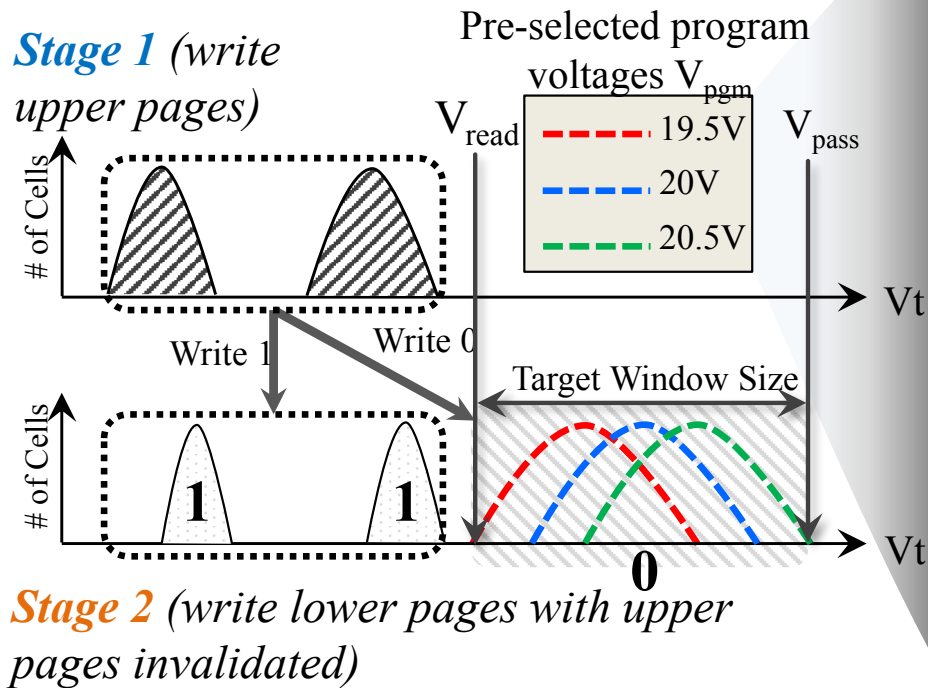


It improves the reliability with the widen V_t window
However, it still exploits ISPP method on programming procedures



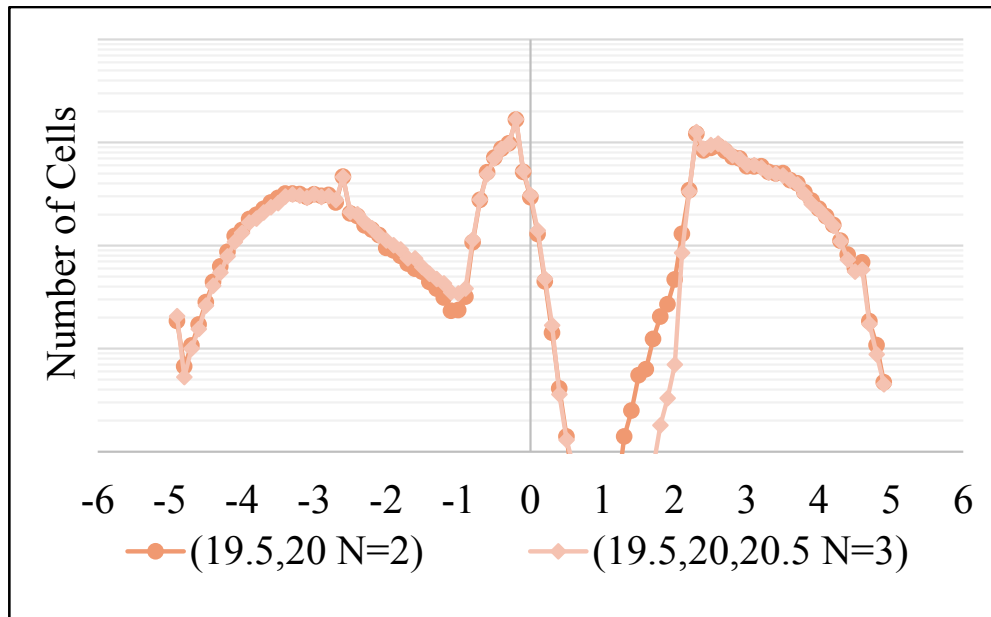
Key Concept: One-shot Programming

- Only use one of the pre-selected program voltage to program a low page
 - Use one program shot to push cells' V_t distribution to the desire state
 - Relax the needs of ISPP on programming a low page

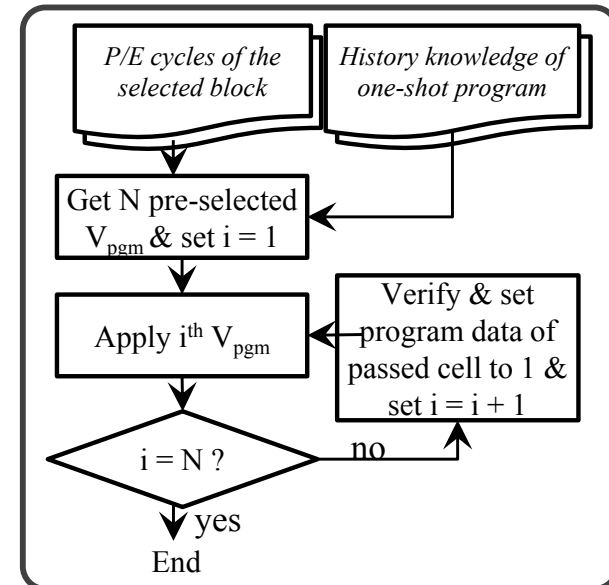


Prophetic Programming Strategy

- Dynamically change number of pre-selected program voltage shots according to block's P/E cycle and the history knowledge
 - As N gets larger, the RBER decreases with more overlaps and the program latency increases



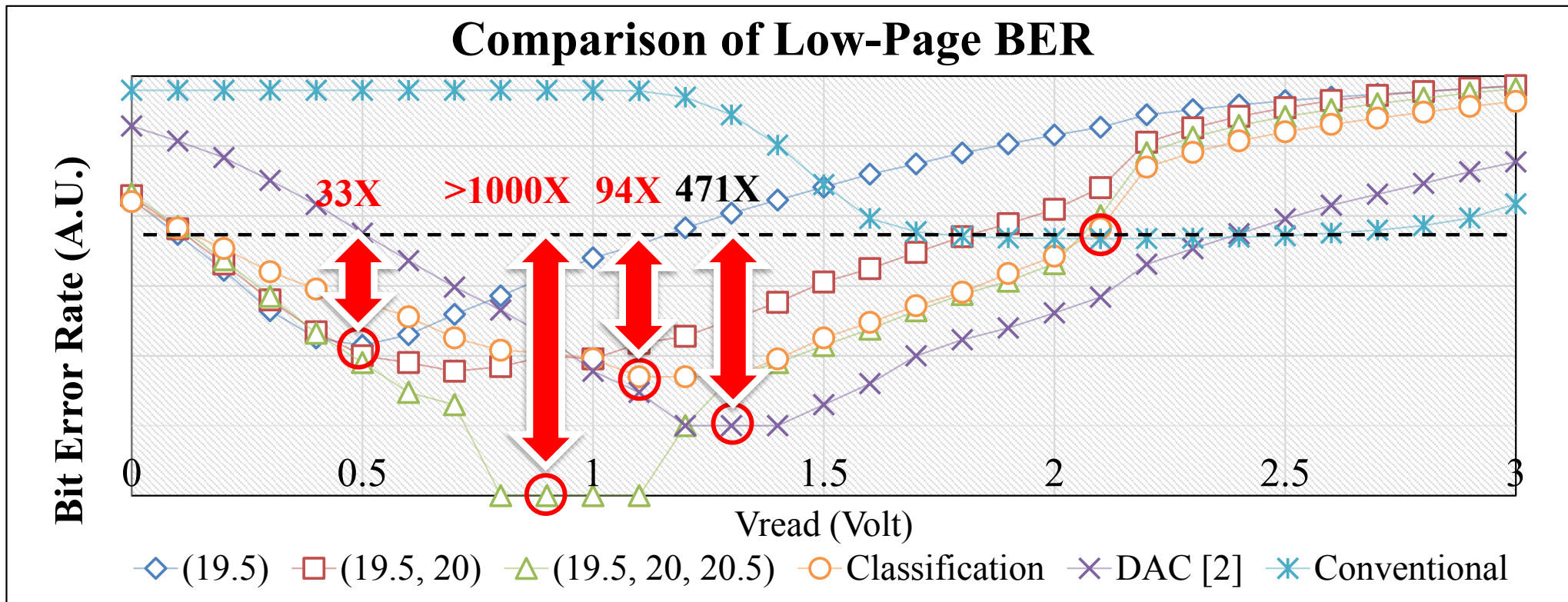
Program Flow of *Prophetic Programming*





Experimental Results: Low-Page BER

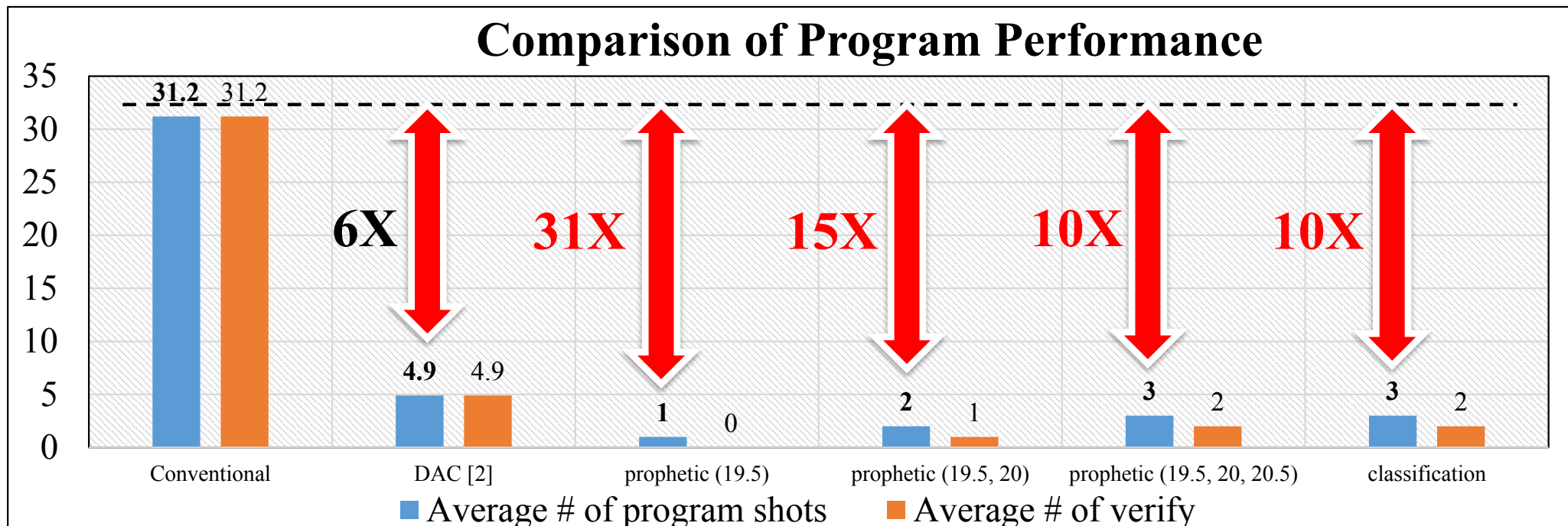
- As more pre-selected program voltages (e.g., 19.5V, 20V, and 20.5V) are included, **the RBER can be reduced by over 1000x** while the prophetic programming scheme is adopted
- Classification programming strategy **can deliver 94x reduction of RBER**





Experimental Results: Program Performance

- The proposed prophetic programming strategy can **achieves the best improvement of program speed by at most 31x**
 - As more pre-selected program voltages are included, it reveals the less improvement on program performance
 - The classification programming strategy can improve program performance by 10x





Outline

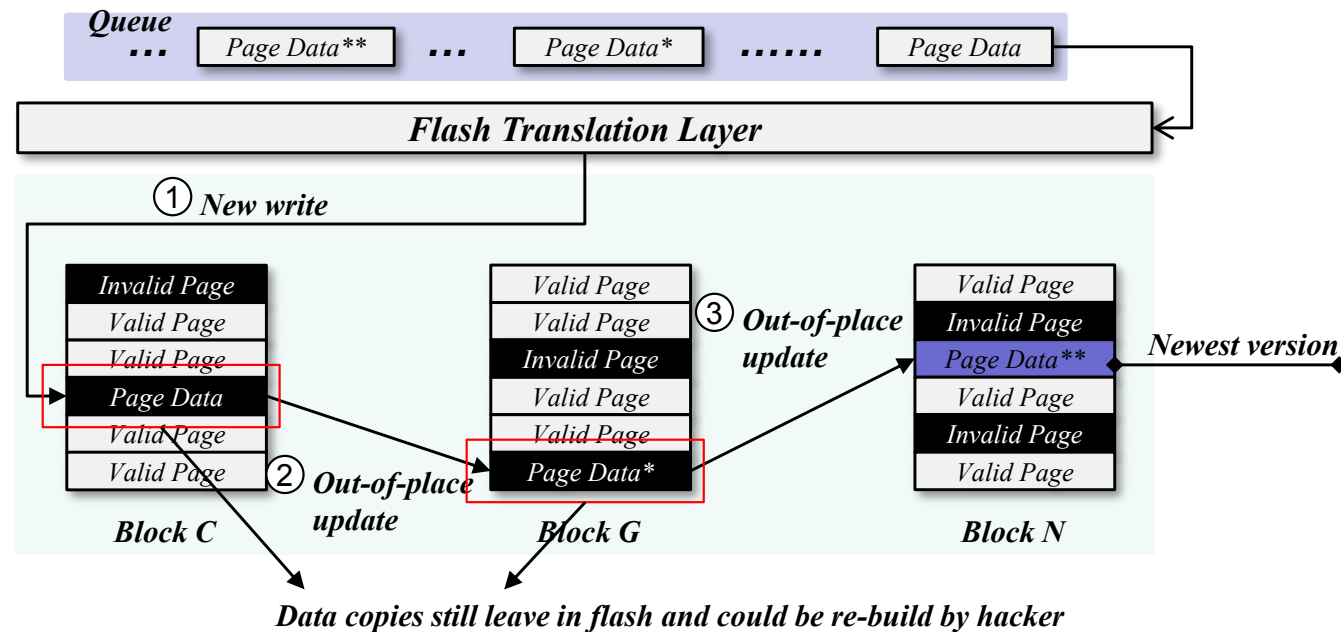
- Data Retention Issue
- Temperature Issue
- Data Security Issue
- **Exploiting Adverse Effects of NAND Flash Memory**
 - One-Shot Program Design (DAC' 18)
 - Fast Sanitization (ICCAD' 18)
 - Zero-Copy Overhead Sanitization (ICCAD' 19)
- Summary



Research Motivation – The Hidden Security Risk

- Flash translation layer(FTL) is adopted to handle the address translation
 - Logical address from host to physical address of flash
 - Interface between host and devices
- Write-once property
 - Out-of-place update
 - Produce several copies hidden from host

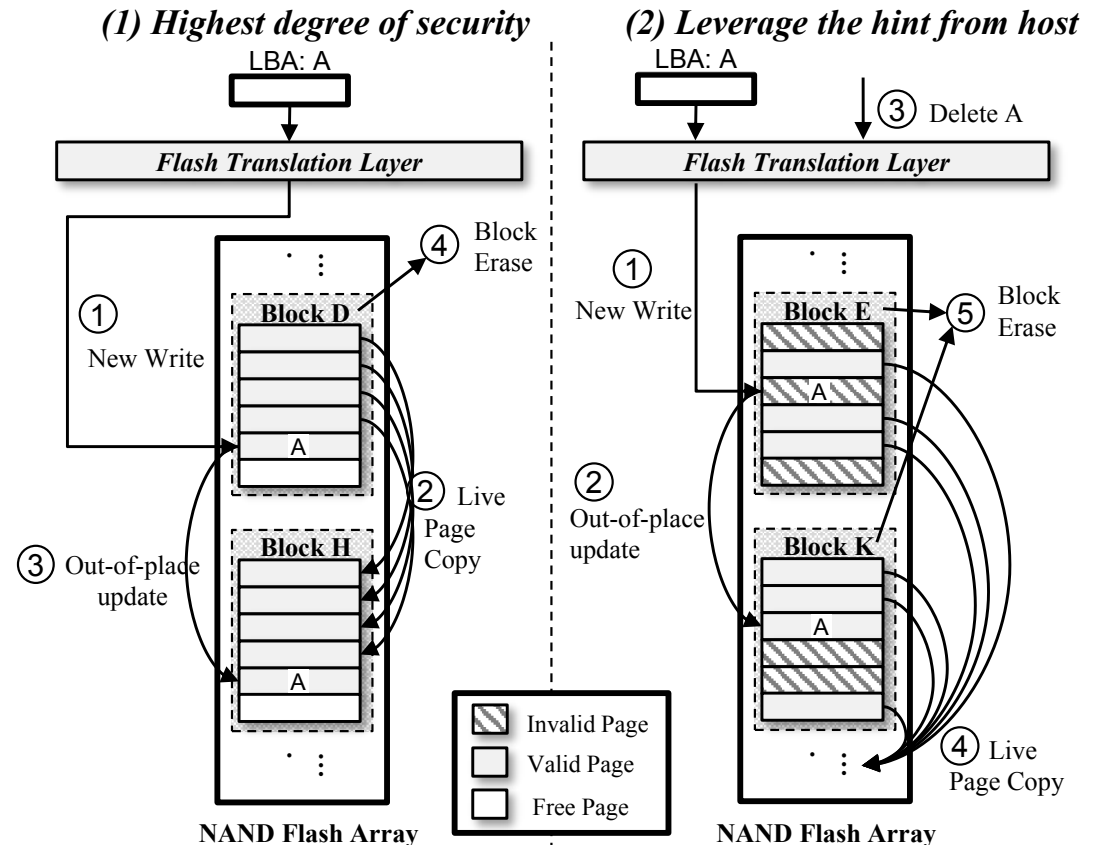
Data Sanitization Scheme is required



Definition

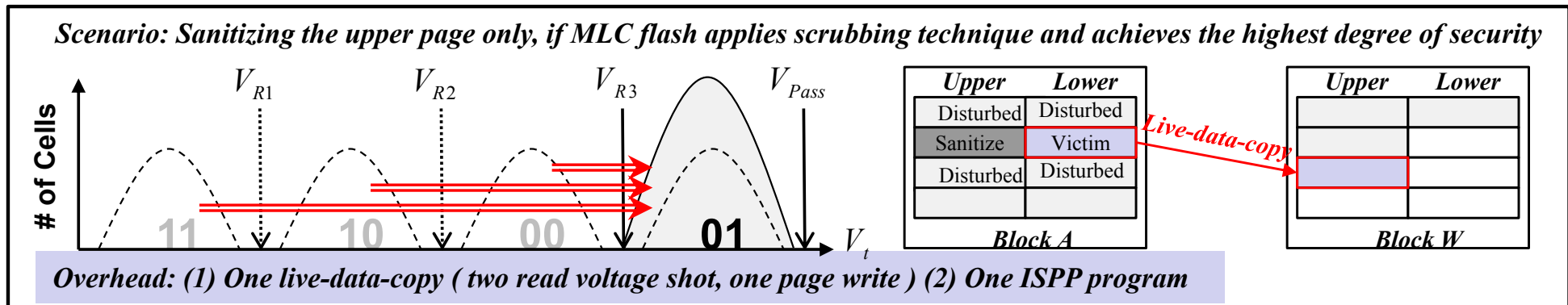
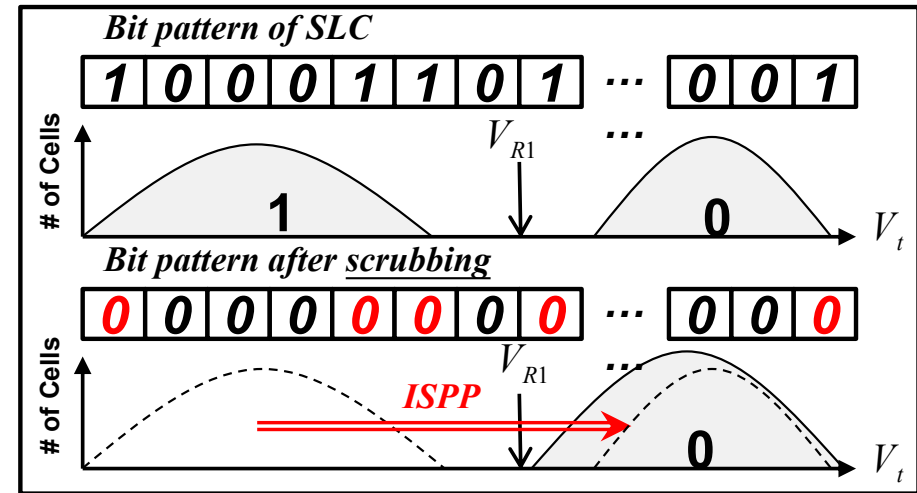
Research Motivation – Two Impractical Sanitization Methods

- Highest degree of security
 - 😊 Delete the old version ASAP
 - 😞 Several live-data-copy i.e., read/write operation
 - 😞 A time-consuming erase op. for each update (Lifetime↓)
- Leverage the hint from host
 - 😊 Better efficiency
 - 😞 Degree of security is sacrificed
 - 😞 Live-data-copy and erase operations are inevitable
 - 😞 Overhead on FTL management



Research Motivation – Re-Program Sanitization

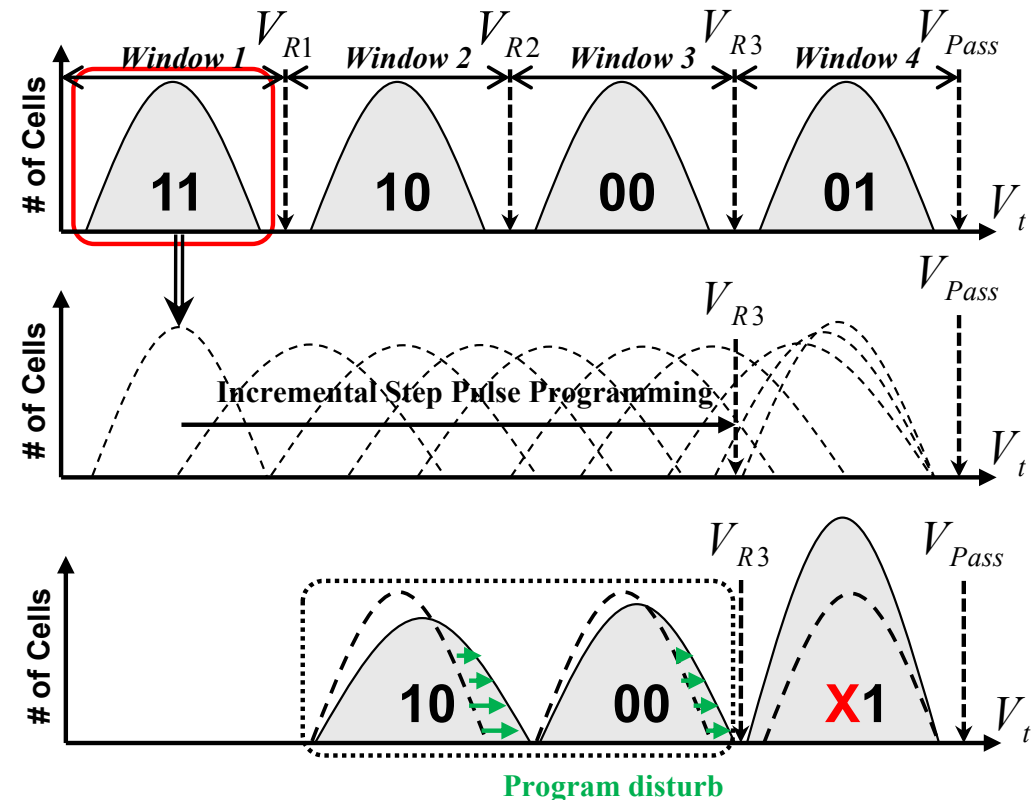
- Scrubbing* technique
 - Write all zero data on flash page
- 😊 – Fine-grained sanitization on SLC flash
- 😞 – ISPP is time-consuming
- 😞 – Induce serious program disturb issue
- 😞 – Inevitable live-data-copy on MLC flash



* Reliably Erasing Data From Flash-Based Solid State Drives (FAST'2011)

Design Concept of Fast Sanitization

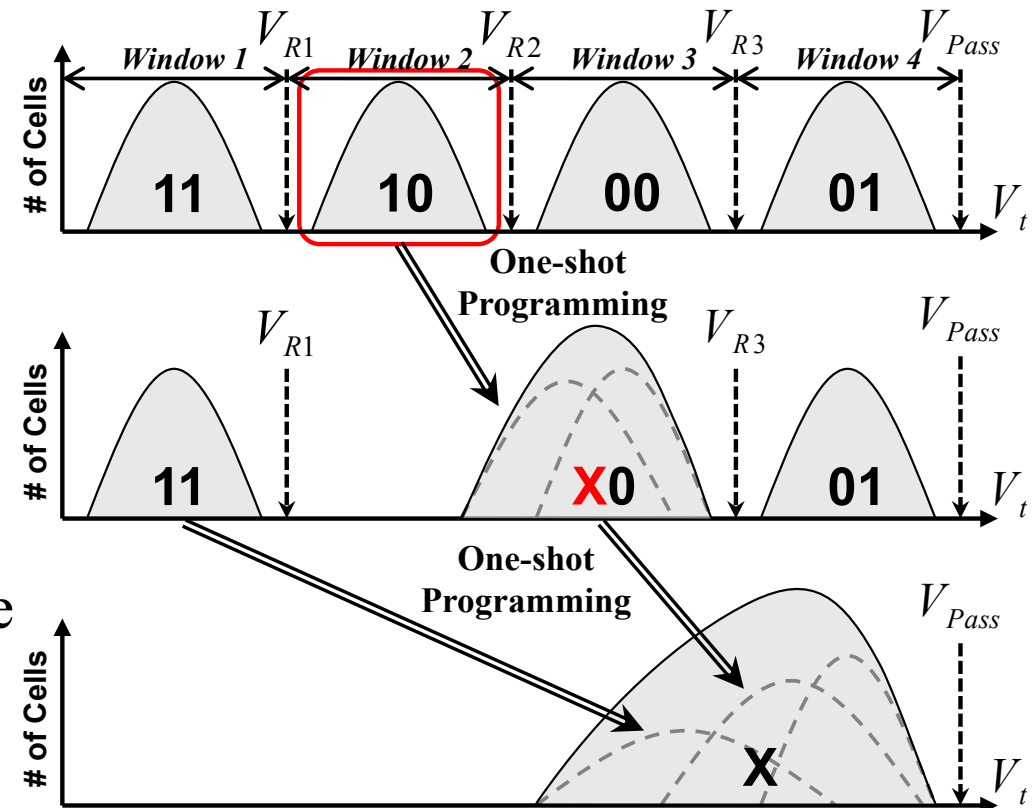
- Sanitizing the upper page data only
 - Observing from the bit pattern
 - Overlapping the window1 and 4
- Overhead
 - One verify shot
 - One ISPP program
- Cons and pros
 - 😊 – Validity of lower page is preserved
 - 😊 – Read latency is reduce by a halve
 - 😞 – ISPP is time-consuming
 - 😞 – Induce serious program disturb issue



One-Shot Sanitization – Sanitizing Upper Page Data

- Sanitizing the upper page data only

- Overlapping window 2 and 3
- A voltage of program shot is fine-tuned
- Validity of lower page is preserved
- Overhead
 - Two verify shots (V_{R1} and V_{R2}) $\Rightarrow$ 分辨率 0.1
 - One program shot

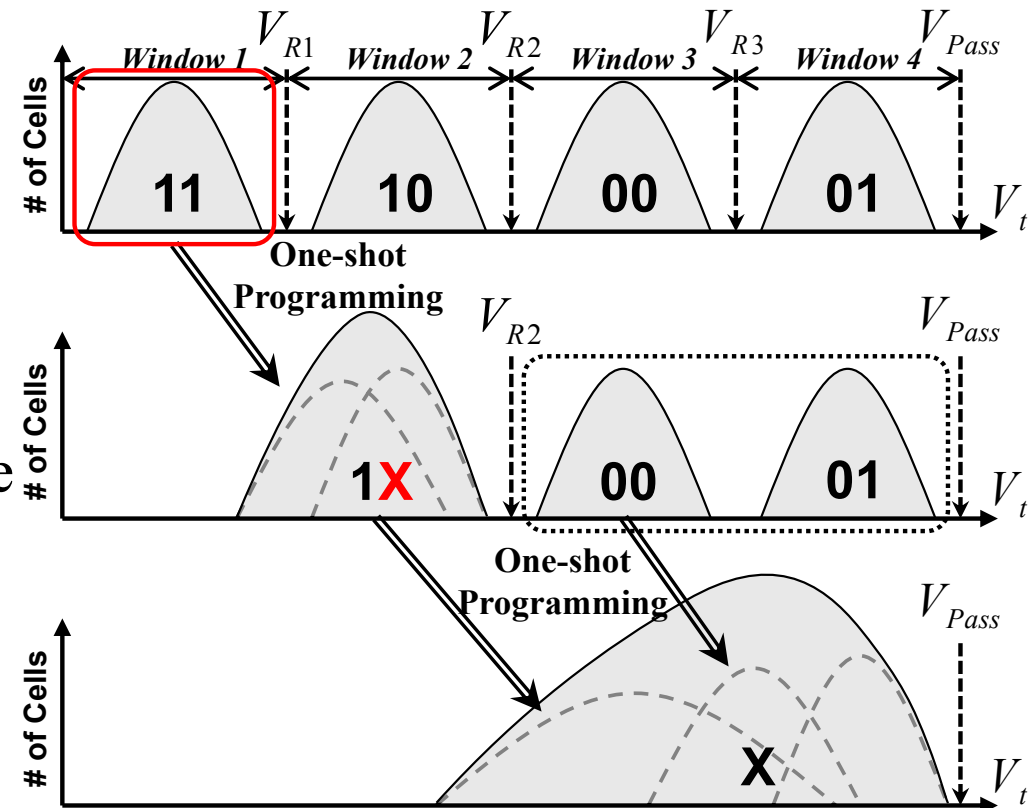


- Sanitizing the remaining lower page data

- One program shot to overlap all voltage threshold

One-Shot Sanitization – Sanitizing Lower Page Data

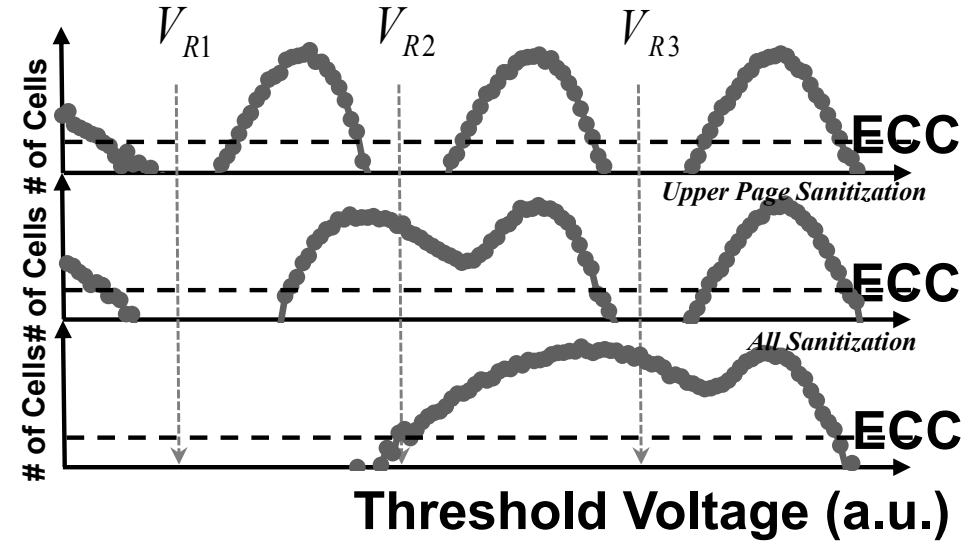
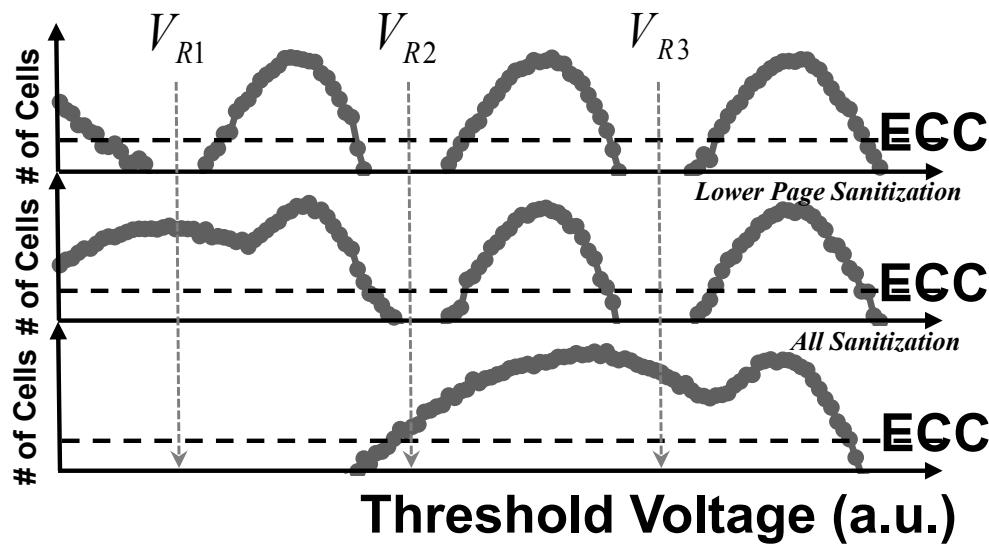
- Sanitizing the lower page data only
 - Overlapping window 1 and 2
 - Validity of upper page is preserved
 - Overhead
 - One verify shots (V_{R1})
 - One program shot
- Sanitizing the remaining lower page data
 - One program shot to overlap all voltage threshold





Experimental Results – Device-Level Results

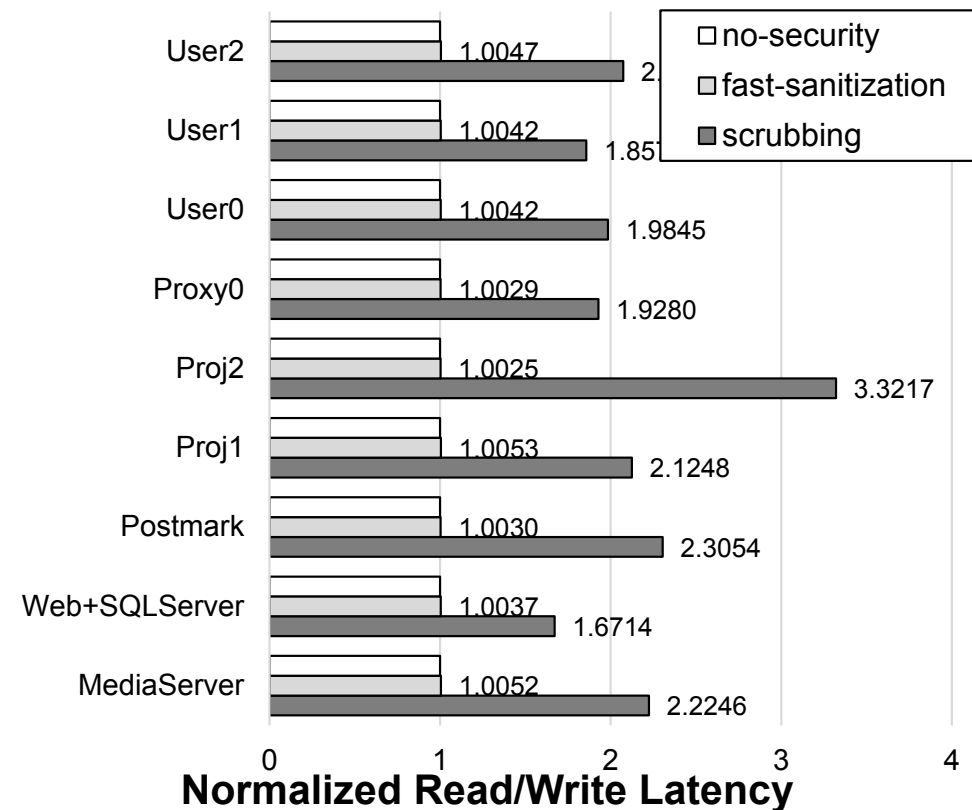
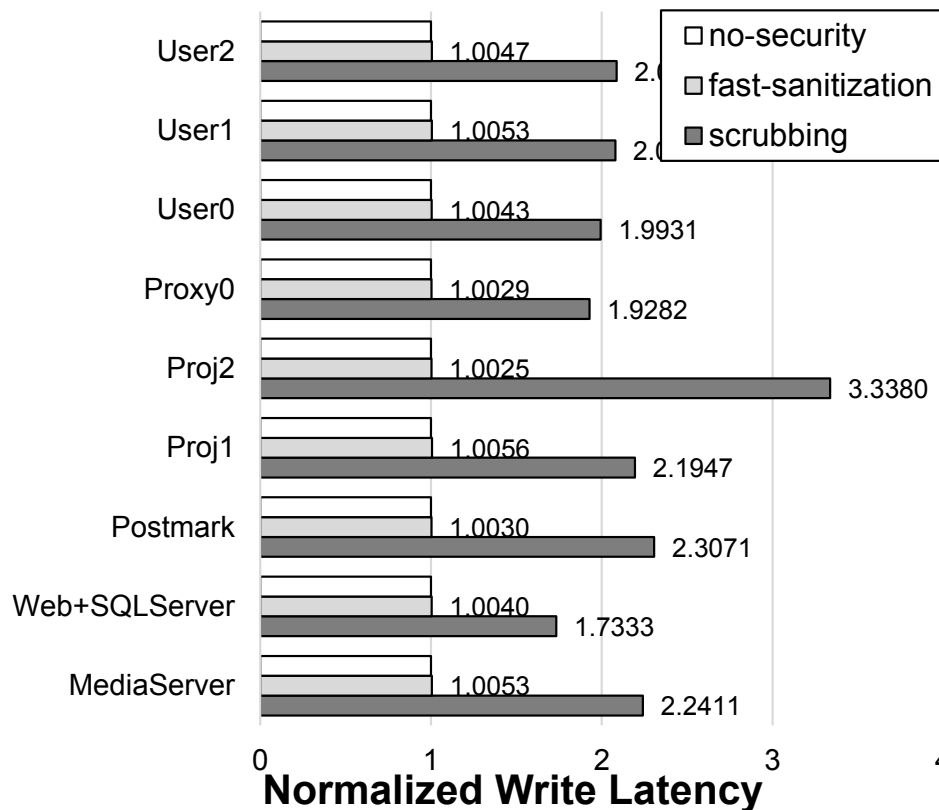
- The experimental results of device-level show the feasibility of fast sanitization with one-shot programming.





Experimental Results – System-Level Results (Performance)

- The results demonstrate the proposed scheme only induce less than 1% performance overhead
 - The scrubbing has a longer latency ranging from 1.73 to 3.34

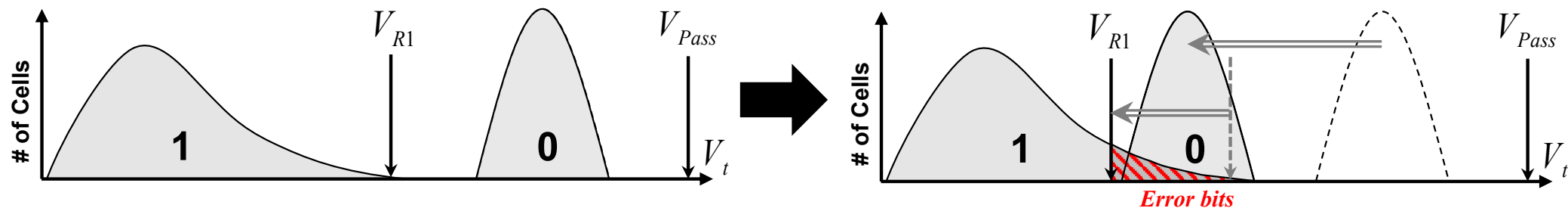


Outline

- Data Retention Issue
- Temperature Issue
- Data Security Issue
- **Exploiting Adverse Effects of NAND Flash**
 - One-Shot Program Design (DAC' 18)
 - Fast Sanitization (ICCAD' 18)
 - Zero-Copy Overhead Sanitization (ICCAD' 19)
- Summary

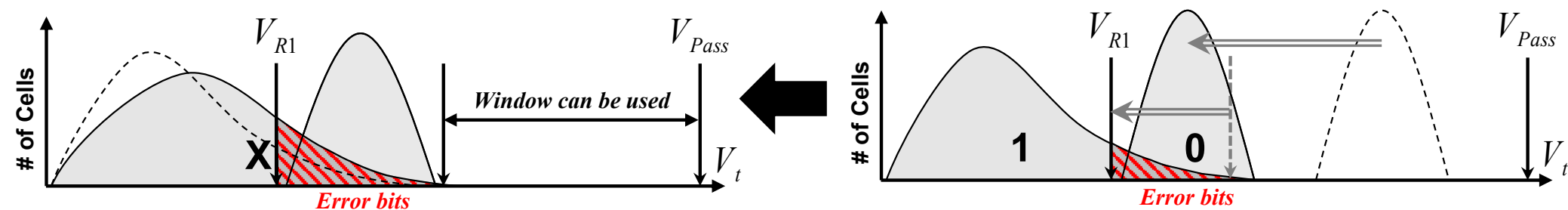
Observation

- In the past, lot of studies and works tell us to keep the V_t distribution of cells with the different data status as wide as possible for improving the reliability
 - Less error bits & program disturbances
- However, there exists the possibility to realize the efficient sanitization if we the properly control and utilize cells' V_t distribution and disturbance properties
 - The error bits arisen after squeezing two states of an SLC flash memory into a narrower window of V_t can still be correctable if the value of V_t shift is properly selected.



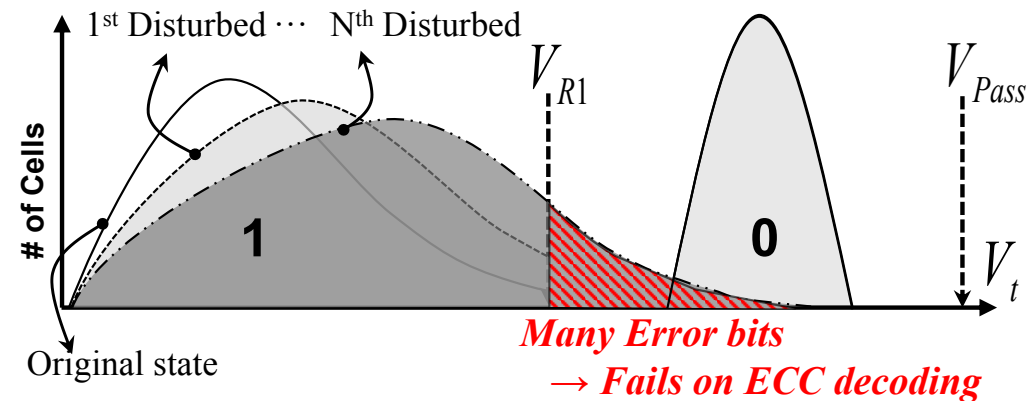
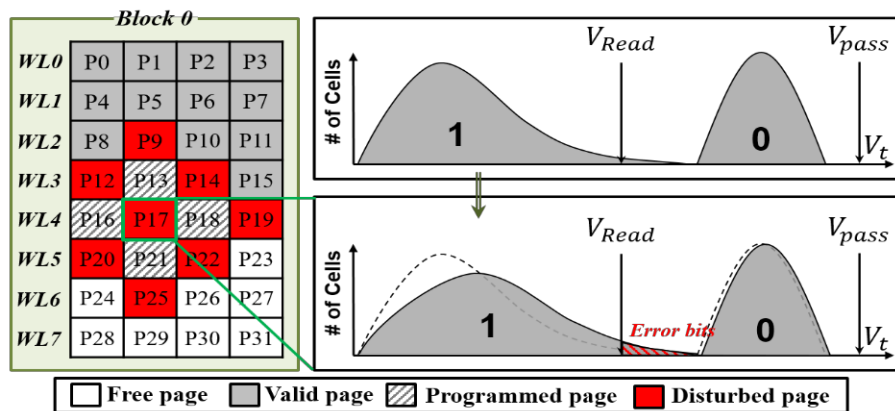
Research Observation

- In the past, lot of studies and works tell us to keep the V_t distribution of cells with the different data status as wide as possible for improving the reliability
 - Less error bits & program disturbances
- However, there exists the possibility to realize the efficient sanitization if we the properly control and utilize cells' V_t distribution and disturbance properties
 - The error bits arisen after squeezing two states of an SLC flash memory into a narrower window of V_t can still be correctable if the value of V_t shift is properly selected.
 - Under this case, programming adjacent pages can help in sanitizing written data in some SLC pages with a narrow window of V_t



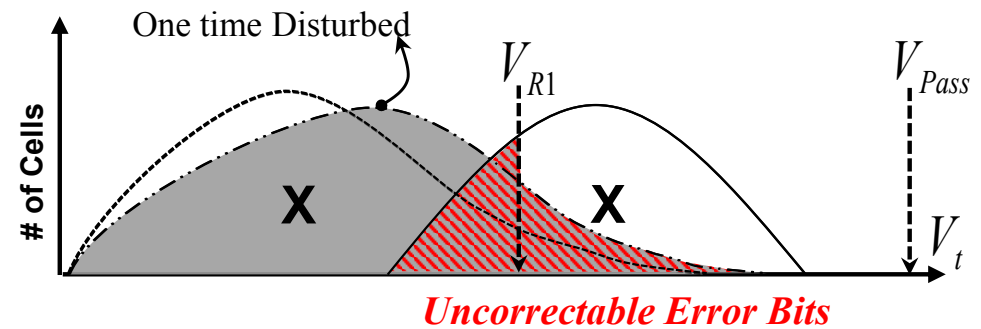
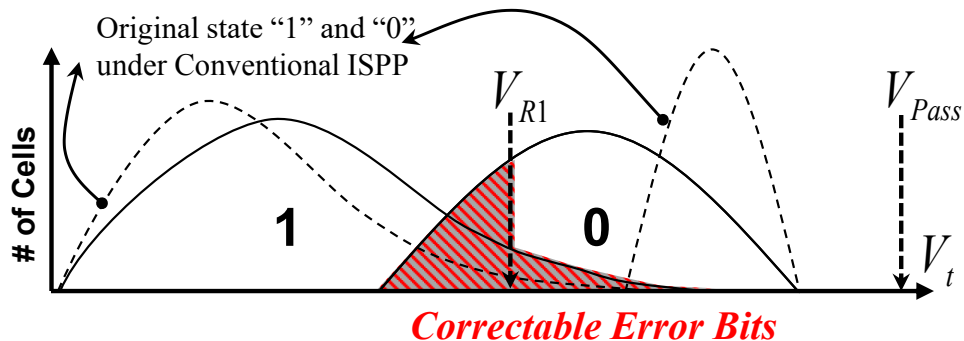
Design Concept – Disturbance-induced Error

- 3D flash memory suffers from more directions of program disturbance which could lead to more data error
 - Disturbance-induced error** refers to the data error caused by the program disturbance
- If there are too many disturbance-induced errors in a flash page, it fails on ECC decoding**



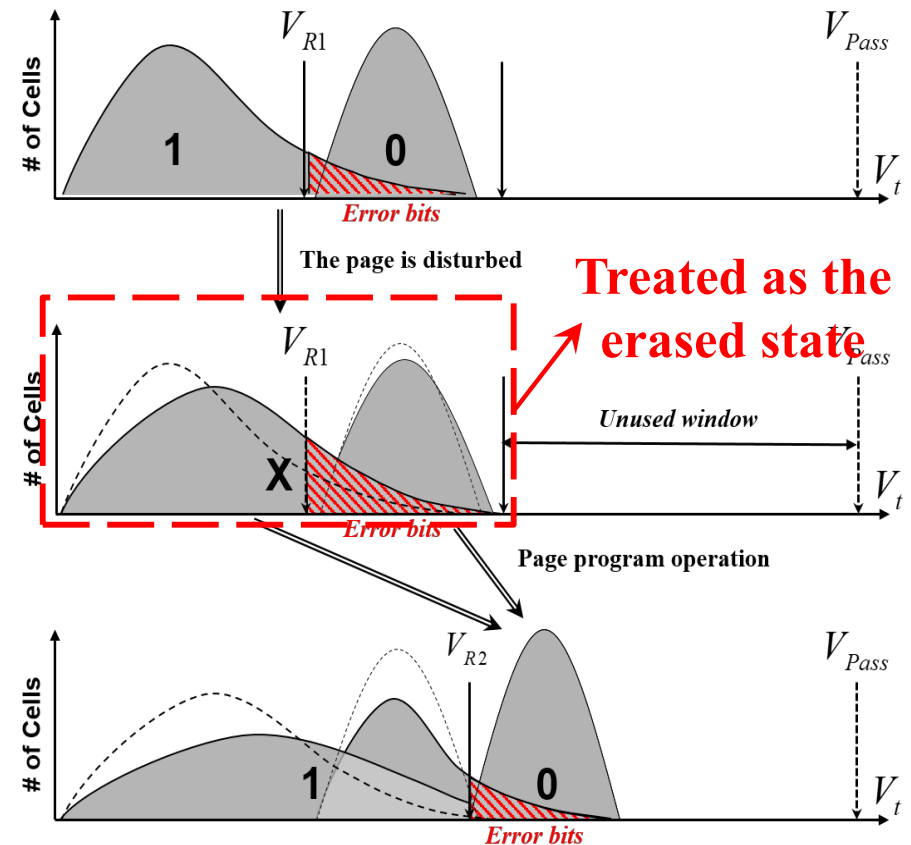
Instantaneous Sanitization

- We propose to deliberately create a certain number of disturbance-induced errors on a flash page in the very beginning stage while it is programmed
 - To **speed up the accumulation of the disturbance-induced errors** on a flash page.
 - To purposely let the two V_t distributions have a certain level of overlap
- With the purposely generated overlapping, the page data can be instantaneously sanitized **with being exactly disturbed one time!**



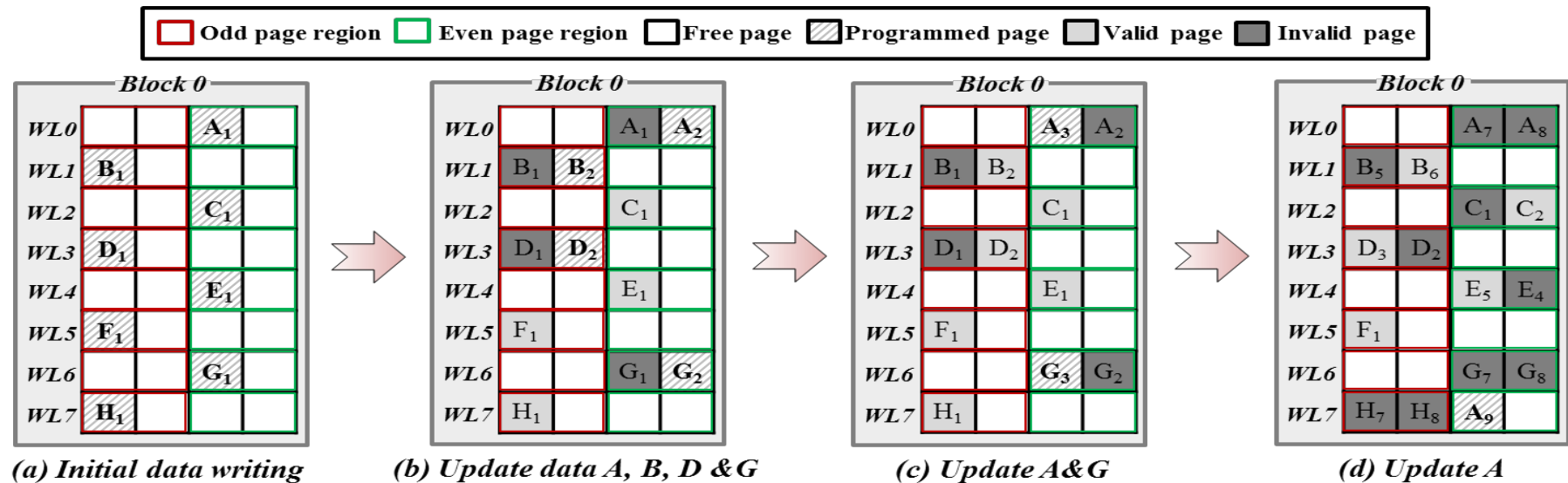
Recycling Programming

- **Recycling programming design**
 - To reduce the needs of frequently invoking GC process by reusing the sanitized pages
- Main idea
 - Squeeze two states of an SLC flash memory into a narrow V_t window so as to leave a wide range of unused V_t window
 - A sanitized page can be used to contain the newly written data without being erased, as if it is recycled and reused
 - There exists a limited number on applying the recycling programming design on a flash page.



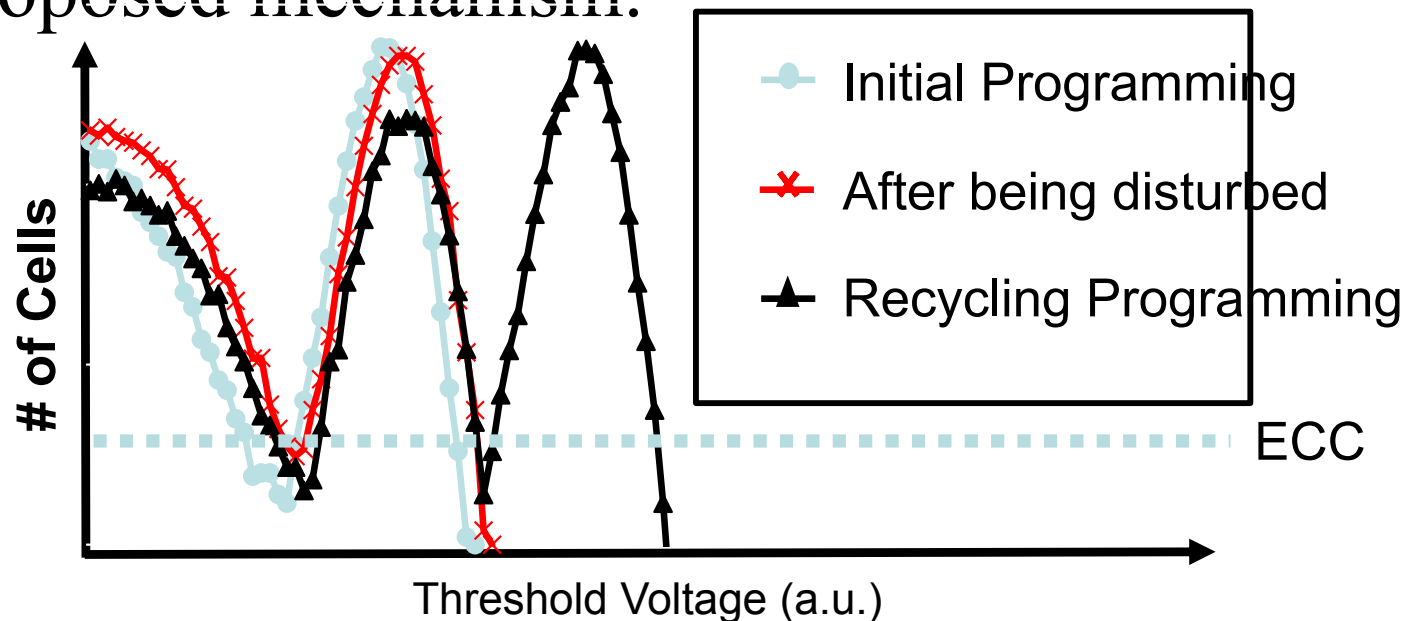
Interleaving Region Allocation Strategy

- The main function of IRA strategy are twofold:
 - Assigning and arranging page locations for storing the data which exist the risk of being sanitized in the nearly future
 - Preventing the pages which keep the valid data from being disturbed while the instantaneous sanitization is applied on their adjacent pages.



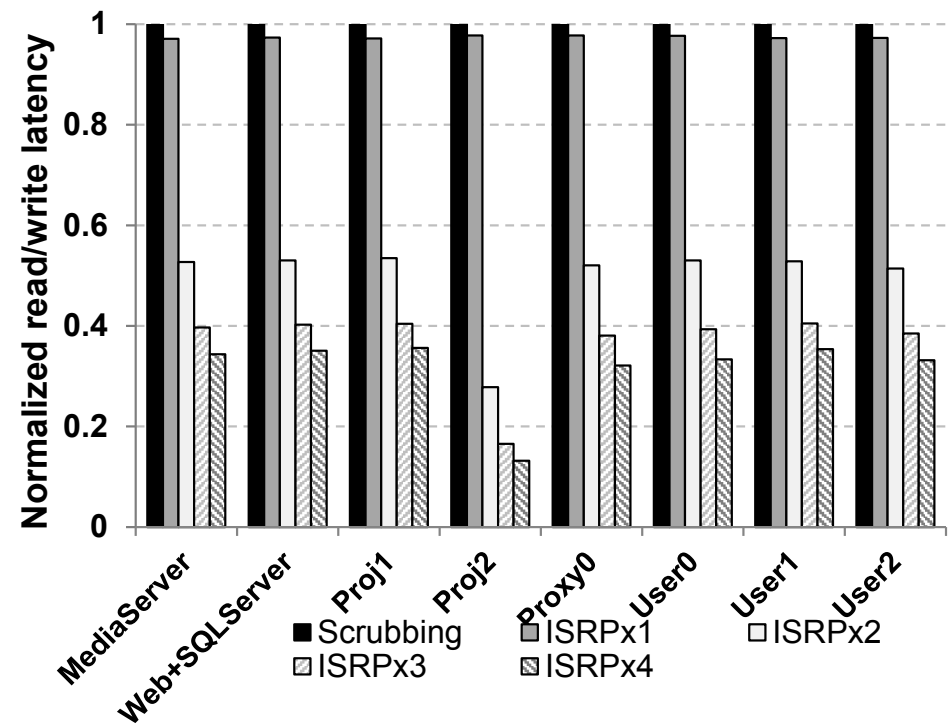
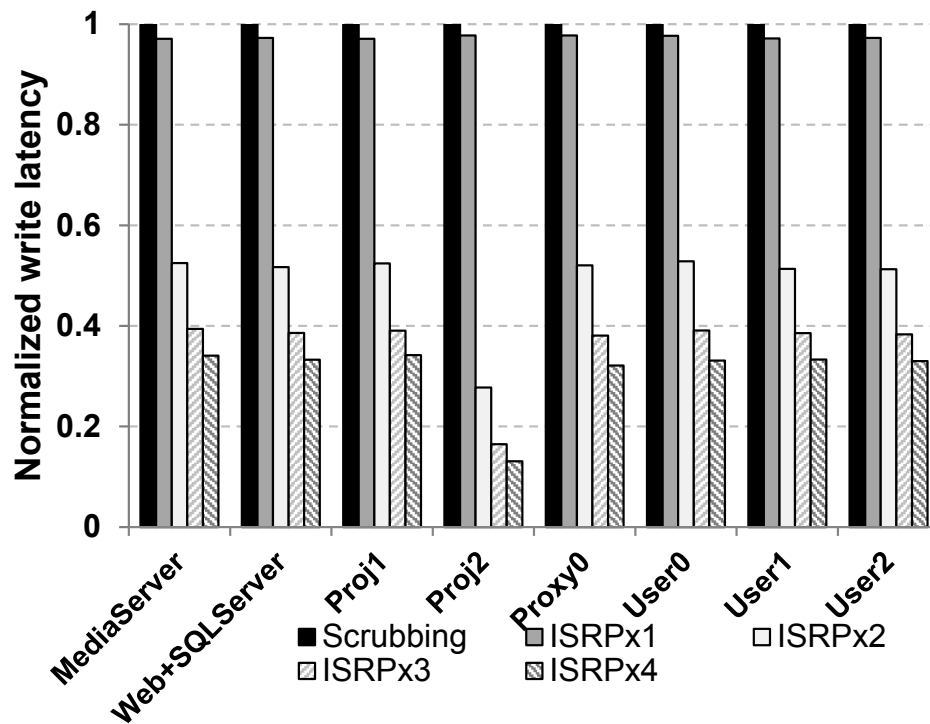
Experimental Results – Device-Level Results

- The experimental results of device-level show the capability and feasibility of instantaneous sanitization with proposed mechanism.



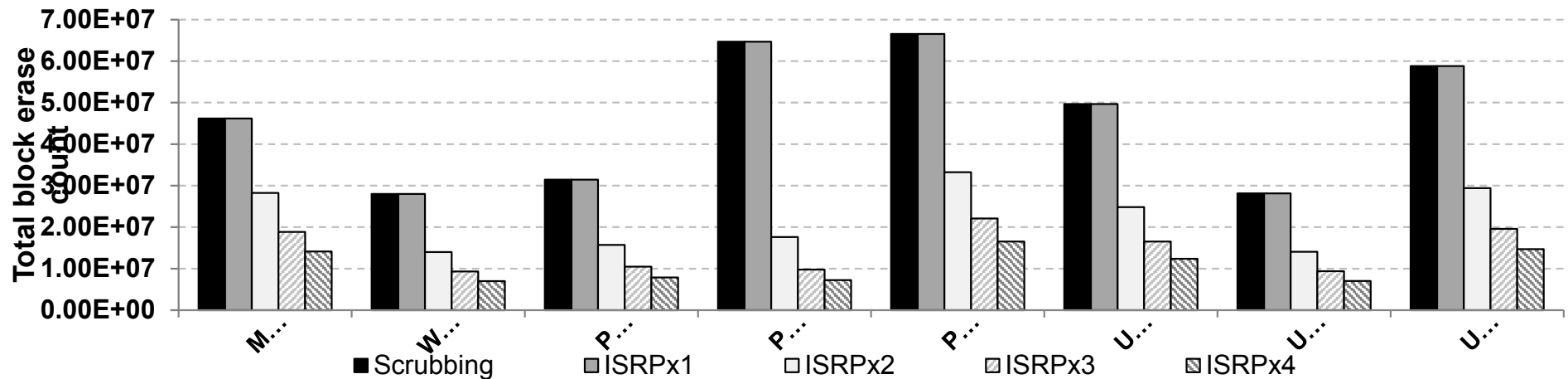
Experimental Results – System-Level Results (Performance)

- When the recycling programming design is adopted and each page can be programmed 4 times (i.e., ISRP $\times$ 4), the write response time improves excessively by 65.81% to 86.91% under the eight investigated traces.



Experimental Results – System-Level Results (Erase count)

- The number of P/E cycle of the proposed ISRP approach without adopting the recycling programming design (i.e., ISRP_1) is as same as that of the scrubbing approach under each trace.
- However, the proposed ISRP scheme can further reduces the number of block erase count ranging from 69.41% to 88.82% while adopting the 4-time recycling programming design





Outline

- Data Retention Issue
- Temperature Issue
- Data Security Issues
- Exploiting Adverse Effects of NAND Flash Memory
- **Summary**



Conclusion

- In this tutorial, we introduce flash memory characteristics and its management and design issues including:
 - Flash translation layer
 - Address mapping, garbage collection, and wear leveling
 - Error correction
 - Native flash file system
 - Hybrid flash application
- Some other current or emerging issues are discussed, including:
 - Data retention issue
 - Data security issue
 - Temperature issue

